bricks Programming Reference v3.5.1

- About this publication
- Contents
- Part 1. The bricks programming model
  - Chapter 1. Overview
  - Chapter 2. The EXEC CICS command environment
  - Chapter 3. The map DSL
- Part 4. EXEC CICS command reference
  - Chapter 4. Terminal I/O commands
  - Chapter 5. Program control commands
  - Chapter 6. System services
- Part 5. EXEC CICS file and queue commands
  - Chapter 7. KSDS file commands
  - Chapter 8. KSDS browse commands
  - Chapter 8a. File Control on `tmp_dir` Sequential Files
  - Chapter 9. Temporary storage and transient data commands
  - Chapter 10. Recovery and condition handling
  - Chapter 11. The Execute Interface Block (EIB)
  - Chapter 12. Response codes
  - Chapter 13. Commands not implemented
- Part 7. EXEC CICS WEB command reference
  - EXEC CICS WEB — server side (Phase 1)
  - EXEC CICS WEB — client side (Phase 2a)
  - EXEC CICS DOCUMENT — chunked body builder
  - EXEC CICS WEB — Phase 3b additions
- Part 3. The REXX language
  - Chapter 14. REXX program structure
  - Chapter 15. Variables and stems
  - Chapter 16. Control flow
  - Chapter 17. PARSE templates
  - Chapter 18. Conditions and SIGNAL ON
  - Chapter 19. Built-in functions
- Part 2. The COBOL language
  - Chapter 20. COBOL source format
  - Chapter 21. DATA DIVISION
  - Chapter 22. PROCEDURE DIVISION
  - Chapter 23. The EIB block in COBOL
  - Chapter 24. EXEC CICS in COBOL
  - Chapter 25. Copybooks
- Part 9. The PL/I language
  - Chapter 30. PL/I program structure
  - Chapter 31. Data declarations
  - Chapter 32. Expressions, conversions and decimal arithmetic
  - Chapter 33. Statements and control flow
  - Chapter 34. Built-in functions
  - Chapter 35. Runtime abends and the unit of work

    - Chapter 36. EXEC CICS in PL/I
    - Chapter 37. %INCLUDE
    - Chapter 38. Restrictions and deviations
- Part 6. EXEC SQL command reference
    - Chapter 26. Embedded SQL (COBOL, REXX and PL/I)
    - Chapter 27. Restrictions and deferred features
- Part 8. Sample programs
    - Chapter 28. Pre-installed sample transactions
    - Chapter 29. Worked examples
- Appendix A. Adapting to terminal size (mod 2 vs mod 4)
- Appendix B. Pitfalls and idioms
- Appendix C. Quick command reference card
    - EXEC CICS — terminal I/O (Chapter 4)
    - EXEC CICS — program control (Chapter 5)
    - EXEC CICS — system services (Chapter 6)
    - EXEC CICS — KSDS files (Chapter 7-8)
    - EXEC CICS — temporary storage / transient data (Chapter 9)
    - EXEC CICS — recovery + condition (Chapter 10)
    - EXEC SQL (Chapter 26)
    - EXEC CICS WEB — server side (Phase 1)
    - EXEC CICS WEB — client side (Phase 2)
    - EXEC CICS DOCUMENT
    - Standard copybooks
    - PL/I quick card (Part 9)
    - Standard PL/I include members
    - EIB fields (Chapter 11)
    - SQLCA fields (Chapter 26)

BRICKS
Transaction Server

Application Programming Reference

Order number: BX26-1500-01
Edition: First Edition
Date: 2026
Applies to: Bricks Transaction Server, version 1.5 and later

Document	Bricks Application Programming Reference
Audience	Application programmers writing REXX, COBOL or PL/I transactions for Bricks.
Companion	README.md — installation, configuration, and operations.
Operator manual	ISPF_editor.md — the in-3270 source editor reference.

About this publication

This reference describes the application-programming interface of the **Bricks Transaction Server**: the COBOL, REXX and PL/I languages it accepts, the `EXEC CICS` command set those programs may issue, the `EXEC SQL` embedded database surface, the `EXEC CICS WEB` command family for HTTP service, the BMS-flavoured map DSL used to build 3270 panels, the VSAM and temporary-storage queue surfaces, and the catalogue of sample programs shipped under `runtime/`.

Operational and installation topics — running `bricks`, editing `bricks.cnf`, signing on through CSSN, the CEMT master terminal, the `bricksload` stress tester, and the `/metrics` endpoint — are documented in the companion **README.md**.

Who should read this publication

This manual is for application programmers who write transactions that run under Bricks. Familiarity with the IBM CICS programming model (pseudo-conversational dispatch, the **EIB**, **COMMAREA**, BMS maps, KSDS files, temporary-storage queues, the SQLCA, and embedded SQL) is assumed. Where Bricks deviates from the IBM behaviour the difference is documented explicitly.

Conventions used in this publication

The following notational conventions appear throughout the manual.

Symbol	Meaning
`UPPERCASE`	A keyword. Coded literally.
`lowercase`	A value supplied by the programmer (a name or expression).
`[ ]`	An optional clause.
`{ a	b }`	
`...`	The preceding clause may repeat.

Command syntax is shown in the `EXEC CICS … END-EXEC` form; PL/I writes the same command terminated by `;` instead of `END-EXEC` (*Chapter 36. EXEC CICS in PL/I*). The bare-string form (`"VERB OPTIONS"` under `ADDRESS CICS`, REXX only) is described in *Chapter 2. The EXEC CICS command environment*; both forms dispatch identically.

Reference layout

Each command described in this manual follows the same five-section layout, in this order:

1. **Format** — the command syntax as a code block.
2. **Description** — what the command does and any important runtime behaviour.
3. **Options** — every keyword that follows the verb, with its meaning and any value constraints.
4. **Conditions** — the `EIBRESP` values (or `SQLCODE` values, for `EXEC SQL`) the command may return, together with the cause of each.
5. **Example** — a short, runnable fragment illustrating typical use.

> **Programming Note.** Where the same verb behaves differently in COBOL, REXX and PL/I, the difference is called out in a *Programming Note* like this one. Where the languages share identical behaviour, the example uses whichever form reads most clearly.

Contents

This publication is organised into nine parts, an appendix series, and a quick-reference card. Read **Part 1** first; the remaining parts are reference material consulted as needed.

Part 1. Bricks programming principles

- Chapter 1. Overview
- Chapter 2. The EXEC CICS command environment
- Chapter 3. The map DSL

Part 2. The COBOL language

- Chapter 20. COBOL source format
- Chapter 21. DATA DIVISION
- Chapter 22. PROCEDURE DIVISION
- Chapter 23. The EIB block in COBOL
- Chapter 24. EXEC CICS in COBOL
- Chapter 25. Copybooks
- Chapter 27. Restrictions and deferred features

Part 3. The REXX language

- Chapter 14. REXX program structure
- Chapter 15. Variables and stems
- Chapter 16. Control flow
- Chapter 17. PARSE templates
- Chapter 18. Conditions and SIGNAL ON
- Chapter 19. Built-in functions

Part 4. EXEC CICS command reference

- Chapter 4. Terminal I/O commands — `SEND MAP`, `RECEIVE MAP`, `CONVERSE`, `SEND TEXT`, `RECEIVE`
- Chapter 5. Program control commands — `RETURN`, `XCTL`, `LINK`, `ABEND`, `START`, `RETRIEVE`
- Chapter 6. System services — `ASSIGN`, `ASKTIME`, `FORMATTIME`, `QUERY SECURITY`, `VERIFY PASSWORD`, `INQUIRE SYSTEM`, `ADDRESS`, `GETMAIN/FREEMAIN`, `INQUIRE/SET` (FILE/TASK/TRANSACTION/TERMINAL/PROGRAM)
- Chapter 10. Recovery and condition handling — `SYNCPOINT`, `SYNCPOINT ROLLBACK`, `HANDLE CONDITION`, `IGNORE CONDITION`, `HANDLE AID`, `HANDLE ABEND`
- Chapter 11. The Execute Interface Block (EIB)
- Chapter 12. Response codes
- Chapter 13. Commands not implemented

Part 5. EXEC CICS file and queue commands

VSAM-style KSDS files and temporary-storage / transient-data queues.

- Chapter 7. KSDS file commands — `READ`, `WRITE`, `REWRITE`, `DELETE`
- Chapter 8. KSDS browse commands — `STARTBR`, `READNEXT`, `READPREV`, `RESETBR`, `ENDBR`
- Chapter 8a. File Control on `tmp_dir` Sequential Files — browse + positional `READ` against ESDS-style sequential files in the tmp_dir sandbox
- Chapter 9. Temporary storage and transient data commands — `READQ TS/TD`, `WRITEQ TS/TD`, `DELETEQ TS/TD`, the `tmp_dir` sandbox

Part 6. EXEC SQL command reference

- Chapter 26. Embedded SQL (COBOL, REXX and PL/I) — `SELECT INTO`, `INSERT`, `UPDATE`, `DELETE`, `COMMIT`, `ROLLBACK`, `CONNECT TO`, cursors (`DECLARE` / `OPEN` / `FETCH` / `CLOSE`), `WHENEVER`, null indicators, the SQLCA, the SQLCODE catalogue, SYNCPOINT integration.

Part 7. EXEC CICS WEB command reference

- EXEC CICS WEB — server side
- EXEC CICS WEB — client side
- EXEC CICS DOCUMENT — chunked body builder
- EXEC CICS WEB — Phase 3b additions

Part 8. Sample programs

- Chapter 28. Pre-installed sample transactions
- Chapter 29. Worked examples

Part 9. The PL/I language

- Chapter 30. PL/I program structure
- Chapter 31. Data declarations
- Chapter 32. Expressions, conversions and decimal arithmetic
- Chapter 33. Statements and control flow
- Chapter 34. Built-in functions
- Chapter 35. Runtime abends and the unit of work

- Chapter 36. EXEC CICS in PL/I
- Chapter 37. %INCLUDE
- Chapter 38. Restrictions and deviations

Appendixes

- Appendix A. Adapting to terminal size (mod 2 vs mod 4)
- Appendix B. Pitfalls and idioms
- Appendix C. Quick command reference card

Note on physical order. The chapter numbers in this contents list are the canonical numbers used throughout the publication. Because some parts have been re-ordered into the logical grouping above, a chapter you find on Part 4 of the contents may sit physically later in the file than a chapter listed in Part 5. Cross-references everywhere in the manual use the chapter number, not the file position.

Part 1. The bricks programming model

Chapter 1. Overview

A bricks **transaction** is a 4-character TRANSID listed in `runtime/transactions.conf`. Each transaction names a **program** — a REXX source file in `runtime/rexx/`, a COBOL source file in `runtime/cobol/` or a PL/I source file in `runtime/pli/` — and an optional access-control list:

```
HELO:rexx:hello.rexx
HELP:rexx:help.rexx:public
QAGE:rexx:qage.rexx:public,users,admin
HELC:cobol:hello.cob:public
GUST:cobol:gust.cob:public
CUSP:pli:cusp.pli:PUBLIC
```

A program that runs `EXEC SQL` against something other than the **first** database in `runtime/databases.conf` adds an explicit binding as the 5th colon-separated field:

```
BANK:cobol:bank.cob:public,users,admin:bank
```

See *Database binding* in Chapter 26 for the full contract; the short version is that an absent 5th field defaults to the first row of `databases.conf` and a misbinding manifests as `SQLCODE = -204` (relation does not exist) on every dispatch.

Organising programs into sub-directories

Once a deployment grows past a handful of programs, the flat `runtime/rexx/` / `runtime/cobol/` layout becomes hard to scan. You can group programs into sub-directories at any depth and reference them with a relative path from the language root:

```
runtime/cobol/billing/invoice.cob
runtime/cobol/billing/statement.cob
runtime/cobol/cards/visa/authz.cob
runtime/rexx/admin/userprov.rexx
```

```
INVC:cobol:billing/invoice.cob:public
STMT:cobol:billing/statement.cob:public
AUTH:cobol:cards/visa/authz.cob:admin
USRP:rexx:admin/userprov.rexx:admin
```

Forward slashes work on both POSIX and Windows. `CEDA PROGRAM` walks the language roots recursively and shows each file with its relative path (so `billing/invoice.cob` appears in the **FILE** column, matching what you write in `transactions.conf`). `CEMT INQUIRE TRANSACTION` shows the same form in the **PROGRAM** column, eliding a long middle (`billing/.../authz.cob`) so the leading application and the file basename always stay visible. Hidden entries (dotfile-prefixed, e.g. `.git/`, `.swp` files) are skipped during the catalogue walk; recursion is capped at 8 levels.

`CEDA TRANSACTION` accepts a relative subdirectory path in the **PROGRAM** form field. `..` traversal is rejected; so are empty path segments (`a//b.rexx`). Absolute POSIX paths are accepted unchanged for the rare case of running one-off programs from outside the runtime tree. A path containing a `:`, a newline or a whitespace-preceded `#` is **refused** — including a Windows-style `C:\app\prog.cob`, whose drive colon used

to write a `transactions.conf` line the parser read back as a different record, shifting the groups and database fields one slot to the right and leaving the transaction unloadable, unreachable, and bound to a nonexistent database.

`brickscompile` accepts either a single file or a directory. In directory mode it walks recursively (same depth cap + hidden-skip rules as `CEDA PROGRAM`) and reports per-file pass/fail; exit code is `0` only if every file parses cleanly.

Program caching

The first time a TRANSID is dispatched, its program is parsed and the AST is cached (see *Parsed-program cache* in the README); subsequent dispatches skip the parse. Each running task has its own heap and stack — there is no shared mutable state between concurrently running tasks of the same TRANSID.

Programs interact with three layers:

1. **The 3270 terminal**, through `EXEC CICS SEND MAP` / `RECEIVE MAP` (and the unstructured `SEND TEXT` / `RECEIVE` pairing). Maps are defined in the BMS-style DSL described in Chapter 3.
2. **Persistent data**, through KSDS file commands (Chapter 7) and TS-queue commands (Chapter 9). Both are backed by an embedded bbolt B+tree at `data/files.boltdb`.
3. **Other programs**, through `LINK` (synchronous, with COMMAREA), `XCTL` (transfer of control, no return), and `RETURN TRANSID` (pseudo-conversational chaining). The Execute Interface Block (EIB) exposes session and request state (Chapter 11).

REXX, COBOL and PL/I share **the same** `EXEC CICS` **dispatcher** (`cics/handler.go`). Every command documented in Part 4 is therefore available, with identical syntax and identical semantics, from all three languages (PL/I ends the command with `;` instead of `END-EXEC`, see Chapter 36).

A typical pseudo-conversational task has the shape:

```
ADDRESS CICS

EXEC CICS RECEIVE MAP('CUST1')              END-EXEC   /* read prior screen */
... validate / compute ...
EXEC CICS SEND    MAP('CUST1') FROM(SCR.) ERASE END-EXEC
EXEC CICS RETURN  TRANSID('CUST')           END-EXEC   /* chain back to self */
```

The dispatcher invokes the program once per ENTER. State that must survive between invocations rides in the COMMAREA on `RETURN`; the chained task receives it through `EIBCALEN` and the `DFHCOMMAREA` data area.

Chapter 2. The EXEC CICS command environment

Surface forms

Bricks accepts two equivalent surface forms inside an `ADDRESS CICS` scope.

1. IBM-canonical `EXEC CICS … END-EXEC` — the form CICS programmers recognize. Available from REXX and COBOL; PL/I writes the same command as `EXEC CICS ... ;` (Chapter 36).

```
ADDRESS CICS

EXEC CICS ASSIGN  USERID(USR) TERMID(TRM)        END-EXEC
EXEC CICS SEND    MAP('HELO1') FROM(SCR.) ERASE  END-EXEC
EXEC CICS RETURN                                  END-EXEC
```

In REXX, a small preprocessor (`rexx/preprocess.go`, called from `rexx.Parse`) rewrites each `EXEC CICS … END-EXEC` block into the equivalent quoted-string command before lexing. Multi-line bodies are collapsed into a single command string; trailing newlines preserve source line numbers in error messages. Comments and string literals are left alone.

In COBOL, the parser collects every token between `EXEC CICS` and `END-EXEC` and reconstructs the body verbatim for the same `cics.ParseCommand` REXX uses.

2. Bare string under `ADDRESS CICS` — terser; available **only in REXX**.

```
ADDRESS CICS
"ASSIGN USERID(USR) TERMID(TRM)"
"SEND MAP('HELO1') FROM(SCR.) ERASE"
"RETURN"
```

Both forms route through `cics.ParseCommand` and dispatch to the same handler. After every command the handler writes `EIBRESP`, `EIBRESP2`, and `RC` into the program's frame.

Argument-passing rules

Inside the parentheses of an option:

- A bare identifier is treated as a **variable name**. The handler reads the value at runtime, and (where the option is an output) writes the result back into that variable.
- A quoted string literal (`'...'` or `"..."`) is treated as a **literal value**. The handler does not write to literals; passing a literal to an output-only option is rejected with `INVREQ`.

This distinction matters most for length and key fields: `LENGTH(LEN)` both reads the requested length on input and writes the actual length back; `LENGTH(80)` only sets the input length.

Programming model summary

Concept	Bricks implementation
Task	One invocation of one TRANSID (`session.TxCB`).
Program	A parsed REXX, COBOL or PL/I source file, cached at L1/L2.
Pseudo-conversational chain	`EXEC CICS RETURN TRANSID(t) COMMAREA(d)`.
Conversational	Loop in the program; each `RECEIVE MAP` blocks on input.
Inter-program call	`LINK PROGRAM(name) COMMAREA(var)` (synchronous).
Storage shared across tasks	KSDS files, TS queues.
Storage shared within one task	REXX variables / COBOL WORKING-STORAGE / PL/I `STATIC` only.
Implicit task end	Program runs off the end, `STOP RUN` (COBOL), or `RETURN` from the PL/I MAIN procedure.
Forced task end	`EXEC CICS ABEND`.

Response-code handling (RESP, RESP2, DFHRESP)

Every EXEC CICS verb accepts the IBM-canonical RESP(var) and RESP2(var) clauses. After dispatch the runtime writes the numeric response into the named host variables, in addition to the always-set EIBRESP / EIBRESP2 frame fields. Programs may therefore write either style:

```
EXEC CICS READ FILE('CUST') INTO(REC) RIDFLD(KEY)
                            RESP(WS-RC) END-EXEC.
IF WS-RC = DFHRESP(NORMAL)   PERFORM USE-REC.
IF WS-RC = DFHRESP(NOTFND)   PERFORM CREATE-REC.

EXEC CICS WRITE FILE('LOG') FROM(BUF) RIDFLD(KEY) END-EXEC.
IF EIBRESP = DFHRESP(DUPREC) PERFORM HANDLE-DUP.
```

DFHRESP(NAME) is a compile-time function: the COBOL parser resolves it to the numeric constant from runtime/cobolcopy/DFHRESP.cpy (and from the matching table in cobol/parser.go). An unknown name is a parse-time error with a "see DFHRESP.cpy for the list" hint. PL/I has the same built-in, drawn from the same single table (cics/resp.go), so the numbers are identical in every language.

RESP and NOHANDLE make conditions non-fatal. As in IBM CICS, coding RESP(var) or NOHANDLE on a command means the program has taken ownership of the outcome: a CICS condition (NOTFND, MAPFAIL, INVREQ, PGMIDERR, ...) never ends the task, whatever the language. The response is in EIBRESP and the RESP variable, HANDLE CONDITION / HANDLE AID are suppressed for that one command, and the program continues. A few outcomes are not conditions and still end the task with RESP coded:

- the terminal connection is lost during SEND / RECEIVE;
- RETURN, XCTL and ABEND (they end the task by design);
- a LINKed program that fails while it runs (abend or runtime error); a LINK target that does not exist is the ordinary condition PGMIDERR;
- a blocking SEND in a scheduler-fired background task.

Without RESP and NOHANDLE, routine file conditions (NOTFND, DUPREC, ENDFILE) are reported in EIBRESP and the program continues, while an error-class condition (for example MAPFAIL for a map that is not installed) ends a REXX or COBOL task unless a HANDLE CONDITION trap catches it. A PL/I task never ends on a CICS condition (Chapter 36).

Length as an input bound

LENGTH(n) on READQ TS, RECEIVE INTO, LINK COMMAREA, and RETURN COMMAREA is treated as IBM's in/out parameter — the caller's value caps the bytes returned (or passed forward, for LINK/RETURN); the post-dispatch write-back stores the actual byte count. A LENGTH(80) on READQ TS against a 200-byte item now hands the program 80 bytes and reports 80, matching real CICS instead of overflowing the buffer.

On READ FILE, READNEXT, and READPREV (file-record family), LENGTH is OUTPUT-only — the store decides record size; there is no operator-supplied truncation bound. This is a bricks-defensible simplification for the KSDS-fixed-length model; real VSAM variable-length permits LENGTH-as-max-buffer. The write-back-then- read-as-input pattern (call 1 writes len(rec1) into LEN, call 2 re-reads LEN as a cap) would silently corrupt subsequent reads, so bricks ignores the input value entirely on these verbs.

Character data: the SBCS data plane

Everything a program can address is **one byte per character**, as on a mainframe: variables, `COMMAREA` / `DFHCOMMAREA`, `START FROM` / `RETRIEVE` data, TS and TD items, KSDS keys and records, `DOCUMENT` buffers, `ENQ` resource names and the text of `EXEC CICS` / `EXEC SQL` commands. The code page, *bricks SBCS*, is ISO-8859-1 (Latin-1) with a graphic plane in `X'80'-X'9F'` in place of the C1 controls:

X'8n'	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
X'80'	€	▉	▀	▄	▌	▐	▪	─	│	┌	┐	└	┘	├	┤	┬
X'90'	┴	┼	═	‖	•	←	→	↑	↓	≤	≥	≠	○	◊	¯	≡

So every `LENGTH` / `FLENGTH`, `EIBCALEN`, `KEYLENGTH`, `MAXLENGTH` and `LENGERR` decision counts characters, and all 256 byte values survive the variable interface untouched (`EIBAID`, `COMP` / `COMP-3` fields inside a `COMMAREA`, `LOW-VALUES` / `HIGH-VALUES`). `HIGH-VALUES` (`X'FF'`) shows as ÿ.

UTF-8 exists only outside the program. The CICS layer converts exactly once, where data crosses an edge. A character that has no SBCS byte (CJK, –, emoji, a real C1 control) becomes `?`; this is silent except for SQL results.

Edge	Into the program	Out of the program
Terminal	`RECEIVE MAP` / `RECEIVE INTO` / `CONVERSE` decode the 3270 input to SBCS	`SEND MAP` / `SEND TEXT` paint each byte as its glyph; plane glyphs travel as CP310 graphic-escape pairs (€ is native on the 114x pages)
`ARG(1)` (REXX)	the command-line tail is converted	-
KSDS, TS, `START` / `RETRIEVE`, `ENQ`, `LINK` / `XCTL` / `RETURN COMMAREA`, MRO	no conversion: bytes are bytes	no conversion
`tmp_dir` / `runtime_dir` files (`READQ TD`, `READNEXT` on a sequential file, `LINEIN`, IDCAMS `REPRO`)	each UTF-8 line becomes SBCS; a line that is not valid UTF-8 is taken as Latin-1	stored as UTF-8 on disk
Postgres (`EXEC SQL`)	results converted; `BYTEA` raw; a substituted ? sets `SQLWARN8` = `SQLWARN0` = `'W'` (`SQLCODE` stays 0); `SQLERRMC` is SBCS, 70 characters	statement text and host variables bound as UTF-8
HTTP (`EXEC CICS WEB`)	see the WEB conversion rules in Part 7	see Part 7
`DOCUMENT RETRIEVE`	SBCS by default; `CHARACTERSET('UTF-8')` delivers the UTF-8 form	-
`VERIFY PASSWORD`	-	the password is checked as UTF-8, matching what was typed at sign-on
`INQUIRE SYSTEM GMMTEXT`	the `bricks.cnf` text is converted	-
`WRITE OPERATOR`, logs, `CEMT INQUIRE ENQ`, `CECI` result pane, IDCAMS listing	-	shown as UTF-8; control bytes as `.` or ?

The `tmp_dir` / `runtime_dir` write path accepts TAB, LF, `X'20'-X'7E'` and `X'80'-X'FF'` (the plane and Latin-1); NUL, the other C0 controls, DEL and CR are rejected with `INVREQ` and a message such as `byte 0x7F at offset 3 is not printable`. Error messages that echo program data are pure ASCII and name non-ASCII characters by code point (`"K\u00c4SE"`).

Deviations from z/OS: the code page is Latin-1 plus the graphic plane, not an EBCDIC CCSID (037 / 1140); collation is byte order; EBCDIC hex constants (other than the AID bytes) need translating; TS queue and file names must still be ASCII; the WEB default outbound charset is UTF-8, not ISO-8859-1.

Chapter 3. The map DSL

Bricks ships its own line-oriented, BMS-flavoured map DSL (parsed by `mapdsl/`). Maps live as `*.map` files in the directory configured by `maps_dir` (default `runtime/map/`). Each file contains one or more `MAP … ENDMAP` blocks. Comments start with `*`.

Format

```
[MAPSET <name>]                          -- optional, namespace for the file
MAP <name> SIZE rowsxcols
  [FIELD AT r,c LEN n <attrs> "literal"]
  [INPUT <fieldname> AT r,c LEN n <attrs> [DEFAULT "value"]]
  [STOP AT r,c]
  [CURSOR AT {fieldname | r,c}]
ENDMAP
[MAP <name> SIZE rowsxcols               -- multiple MAP blocks per file
   ...
ENDMAP]
```

Statements

`MAPSET <name>` *(optional, must be the first non-comment line)* Names the mapset — the namespace every `MAP` block in this file belongs to. The catalog uses `MAPSET.NAME` as its key so two different files can declare a map named `MAPA` without colliding. When the line is omitted, the mapset defaults to the file's basename (uppercased, extension stripped): a file named `help1.map` has implicit mapset `HELP1`.

`MAP <name> SIZE rowsxcols` The map header. Names must be unique **within their mapset** (case-insensitive); typical sizes are `24x80` (mod 2) and `43x80` (mod 4). A single `.map` file may declare any number of MAP blocks back-to-back — useful when porting a BMS mapset that contained several DFHMDI macros under one DFHMSD.

`FIELD AT r,c LEN n <attrs> "literal"` A display-only field that paints the literal at row `r`, column `c`, length `n`. Useful for labels, headings, and panel chrome.

`INPUT <fieldname> AT r,c LEN n <attrs> [DEFAULT "value"]` A named input field. The name is how `SEND MAP FROM(STEM.)` and `RECEIVE MAP INTO(STEM.)` route data to / from this position (`STEM.<fieldname>`). Use `PROT` to make the field write-protected but still named — useful for output values painted by `SEND MAP FROM(STEM.)`.

`STOP AT r,c` An autoskip stop attribute that ends the preceding input field.

`CURSOR AT <fieldname>` *or* `CURSOR AT r,c` The home position for the cursor when the map is sent. The named form resolves to `(field.Row, field.Col + 1)` — i.e. one byte to the right of the leading 3270 attribute byte, which is the writable cell. Forward references are allowed; names are resolved after the whole map is parsed. When omitted, the renderer falls back to the first input field.

`ENDMAP` Terminates the map.

Converting legacy BMS sources

The `bricksconvert` CLI utility (under `cmd/bricksconvert`) converts IBM CICS **BMS map source** (`DFHMSD` / `DFHMDI` / `DFHMDF` macros) into this DSL — the recommended path for porting an existing CICS application's screens onto bricks without rewriting every panel by hand. See the BMS conversion section in the README for usage details.

Attributes

`PROT`, `UNPROT`, `BRIGHT`, `DIM`, `UNDERLINE`, `HIDDEN`, `NUMERIC`, `MDT`, `BLINK`, `REVERSE`, `COLOR=BLUE|RED|PINK|GREEN|TURQUOISE|YELLOW|WHITE`.

`UNDERLINE` is the canonical IBM BMS spelling (`HILIGHT=UNDERLINE`). `UNDERSCORE` is accepted as a legacy alias for hand-written bricks DSL maps that already use it; new maps should prefer `UNDERLINE`.

Character set

Map files are UTF-8. Every character inside a quoted literal (a `FIELD` label or an `INPUT ... DEFAULT`) occupies exactly **one 3270 cell**, so `LEN` counts characters, not bytes: `"ÄÖÅ"` fits `LEN 3`.

The parser enforces a character fence. A literal may contain:

- printable ASCII, including `[ ] { } ~ \ ` | ^`;
- Latin-1 letters and symbols `U+00A0-U+00FF` (Ä Ö Å ü é ß ...), **except** the currency sign ¤ (`U+00A4`), which the euro codepages 1140-1149 cannot show;
- the CP310 graphic-escape glyphs: block and box drawing ( –    –      ), • ≤ ≥ ≠ → ← ↑ ↓, the APL symbols, and €.

Rejected: control characters (TAB, CR, NUL, DEL, C1), invalid UTF-8, typographic dashes and quotes (– – “ ”), emoji, CJK, and other alphabets that the Latin EBCDIC codepages cannot show. Outside quotes the DSL is ASCII only. Comments (`*`) may contain anything, and a UTF-8 byte order mark at the start of the file is ignored.

The CP310 glyphs reach the terminal as a graphic-escape pair; a terminal without graphic-escape support shows a substitute instead.

A rejected character makes the whole file fail to load. The error names the line, the code point, and the column, e.g.

```
line 3: U+2014 not displayable on 3270 (col 27)
line 4: U+0009 control char in literal (col 30)
line 5: invalid UTF-8 byte X'E9' (col 28)
line 6: U+00DC outside quotes (col 33)
```

At runtime `SEND MAP` on such a map raises `MAPFAIL` with `SEND MAP: map "X" has bad char U+2014 at line 3`. A literal longer than its `LEN` is only a warning: it is truncated on the screen and counted as `Pass, N long` by `CEMT PERFORM RESCAN MAP`.

Catalogue lifecycle

`mapdsl.NewCatalog(dir)` parses every `*.map` file at startup and self-refreshes on subsequent edits. Each `Lookup(name)` stats the directory plus the source file backing the requested name; on `mtime` change the

directory is reparsed and swapped atomically. A failed re-parse keeps the prior catalogue in place — the operator can find the broken file with `CEMT PERFORM RESCAN MAP` (see the README).

Sending a map by mapset

`EXEC CICS SEND MAP('NAME') MAPSET('SET') FROM(stem)` resolves through the qualified `MAPSET.NAME` index, so:

```
EXEC CICS SEND MAP('MAPA') MAPSET('AM01') FROM(AM01) ERASE END-EXEC.
EXEC CICS SEND MAP('MAPA') MAPSET('AM16') FROM(AP16) ERASE END-EXEC.
EXEC CICS SEND MAP('MAPB') MAPSET('AM99') FROM(AM99) ERASE END-EXEC.
```

each picks the right map even when several files share the same bare map name. When `MAPSET(...)` is omitted, the catalog tries a bare-name lookup; if that name appears in more than one mapset the runtime returns the short diagnostic `SEND MAP: map "MAPA" is ambiguous; specify MAPSET` rather than silently picking one. Programs that only ever reference unique bare names (the typical case for the runtime/map shipped with bricks) keep working without any code change.

`RECEIVE MAP` accepts the same `MAPSET(...)` clause with identical resolution semantics.

Example

```
* CICS sign-on screen
MAP CSSN SIZE 24x80
  FIELD AT 1,28 LEN 24 PROT BRIGHT COLOR=TURQUOISE  "BRICKS SIGN-ON"
  FIELD AT 6,15 LEN 9  PROT                         "Userid:"
  INPUT USERID   AT 6,25 LEN 8 UNDERSCORE COLOR=GREEN
  STOP           AT 6,34
  FIELD AT 8,15 LEN 9  PROT                         "Password:"
  INPUT PASSWORD AT 8,25 LEN 8 HIDDEN UNDERSCORE COLOR=RED
  STOP           AT 8,34
  CURSOR         AT USERID
ENDMAP
```

Part 4. EXEC CICS command reference

> **Part 4 begins here.** This part documents the core non-file, non-SQL, non-WEB `EXEC CICS` verbs. File and queue commands are in **Part 5**; `EXEC SQL` is in **Part 6**; `EXEC CICS WEB` is in **Part 7**.

This part documents every `EXEC CICS` command bricks implements. Each command page follows the same layout: **Format**, **Description**, **Options**, **Conditions**, **Example**.

Commands are grouped by function:

- Chapter 4. Terminal I/O
- Chapter 5. Program control
- Chapter 6. System services
- Chapter 7. KSDS files
- Chapter 8. KSDS browse
- Chapter 9. Temporary storage

Cross-cutting reference:

- Chapter 10. Recovery and condition handling
- Chapter 11. The EIB
- Chapter 12. Response codes
- Chapter 13. Commands not implemented

Chapter 4. Terminal I/O commands

SEND MAP

Send a BMS-style map to the 3270 terminal and wait for the operator to press an AID key.

Format

```
EXEC CICS SEND MAP(name)
              [MAPSET(set)] [FROM(stem.)]
              [ERASE | ERASEAUP | DATAONLY] [MAPONLY]
              [CURSOR(position)]
              [FREEKB] [ALARM] [FRSET]
              [RESP(var)] [RESP2(var)]
END-EXEC
```

Description

Loads the named map from the catalogue, populates each named field with the value of `<stem>.<fieldname>` (when `FROM` is supplied), paints the screen, and waits for the operator to press an AID key (ENTER, PFn, PAn, CLEAR). On return, the captured response is stored on the TCB so a subsequent `RECEIVE MAP` can pull modified field values back into the program. `EIBAID` and `EIBCPOSN` are updated with the AID character and 1-based cursor position.

IBM-canonical clauses honoured: `MAPSET(set)` qualifies the lookup to disambiguate same-named maps across mapsets (see Chapter 3); `ERASEAUP` wipes unprotected fields only (approximated as `ERASE` in bricks); `MAPONLY` paints just the map's literals + INPUT defaults, ignoring any `FROM(stem.)`. `FREEKB`, `ALARM`, and `FRSET` are accepted for source compatibility — `FREEKB` and `FRSET` are subsumed by the fixed WCC the go3270 library writes; `ALARM` is parsed but not yet tunnelled to the wire.

If `FROM` is omitted, the map renders using its `INPUT DEFAULT` values. If `FROM` is a literal string, every named field on the map is filled with that literal (rare; supported for parity with BMS).

Three paint modes

Flag	Field set	Blocking?	When to use
`ERASE`	Every field in the map	Yes — waits for AID	Initial paint, full refresh.
(neither)	Only fields the program populated in `FROM(stem)`	Yes — waits for AID	In-place update inside a conversational loop (rare; usually `ERASE` is fine).
`DATAONLY`	Only fields the program populated in `FROM(stem)`	**No — returns immediately**	Background refresh while a different task is mid-input.

`ERASE` clears the screen first and emits **every** field in the map (with either the program's `FROM` value or the map default). Without `ERASE`, bricks does a **partial repaint**: only fields the program explicitly populated in `FROM(stem)` are sent; fields the program didn't touch stay on the terminal exactly as the operator left them. Use `ERASE` for the initial paint of a screen or a full refresh; omit it for in-place partial updates.

`DATAONLY` — background partial paint

`DATAONLY` is the bricks idiom for **background, fire-and-forget partial paint**, used to refresh part of the screen — typically a clock or other status fields — while a different task on the same terminal is blocking inside `SEND MAP` waiting for the operator's next keystroke. It layers two things on top of the plain no-`ERASE` partial-paint behaviour:

1. **No-block:** `DATAONLY` makes `SEND MAP` return immediately after writing the screen, without entering the AID-wait. The call is fire-and-forget — the operator's next AID is captured by whatever other `SEND MAP` or `RECEIVE MAP` happens to be blocking for it.
2. **Scheduler-safe:** when an `EXEC CICS START INTERVAL(...)` timer fires on a terminal whose current task is in input-wait, the bricks scheduler dispatches the queued task **inline** in the timer goroutine instead of poisoning the conn read deadline (which would abend the in-flight task with an `i/o timeout`). The inline-dispatched task is restricted to write-only verbs: `SEND MAP DATAONLY`, file I/O, TS queue, ASSIGN, START. Any blocking verb (`RECEIVE MAP`, `SEND MAP` without DATAONLY, `CONVERSE`) returns `INVREQ` in that context — the scheduler goroutine must not block on conn.Read. The inline-dispatched transaction must be a REXX or PL/I program; a background PL/I task has no COMMAREA and no SQL connection. Like any task, it commits its unit of work when it ends normally and backs it out when it abends, as IBM does (REXX background tasks used to never commit).

This together lets a transaction self-schedule a periodic refresh via:

```
EXEC CICS START TRANSID('CHAT') INTERVAL(000002) FROM('TICK') END-EXEC
EXEC CICS RETURN END-EXEC
```

When the START fires, the same transaction is dispatched a second time. The dispatched copy detects "I am a tick" by checking the `RETRIEVE` payload, paints only the fields it wants to refresh (`SEND MAP ... DATAONLY` — the operator's input field is not in the populated set, so it's never repainted), schedules the next tick, and returns. The main task on the same terminal stays blocked in its outer `SEND MAP` throughout. See `runtime/rexx/chat.rexx` for the worked example.

`DATAONLY` and `ERASE` are mutually exclusive — the runtime rejects the combination at parse time. Note: bricks does not implement the real-CICS `DATAONLY` attribute-byte semantics (application-supplied attribute bytes via `X'00'` sentinels); programs that need a synchronous partial repaint use the plain no-`ERASE` form, and programs that need a background refresh use `DATAONLY` with the START-INTERVAL pattern above.

Options

MAP(name) — *required* Name of a map in the catalogue (case-insensitive). MAPFAIL if absent.

FROM(stem.) Stem variable whose tails name the map's input fields. The stem's trailing dot is optional: `FROM(SCR)` and `FROM(SCR.)` are equivalent.

ERASE Clears the screen before painting AND emits every field in the map. Without this flag, only fields the program populated in `FROM(stem)` are sent — fields the program didn't touch stay on the terminal unchanged (partial repaint).

DATAONLY Fire-and-forget partial paint: SEND returns immediately without waiting for an AID. Implies no-ERASE partial-repaint semantics. Mutually exclusive with `ERASE`.

CURSOR(position) 1-based cursor position. Overrides the map's `CURSOR AT` clause.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Map sent, AID received.
MAPFAIL	36	Named map not found in the catalogue.
IOERR	17	The terminal write failed.

Example

```
SCR.GREETING = 'Hello, ' || USR
EXEC CICS SEND MAP('HELO1') FROM(SCR.) ERASE END-EXEC
```

Runtime field attribute overrides

A map's `COLOR=` clause is the *default* colour for each field. Programs can override the colour of any named field at runtime by setting a sibling element named `<FIELD>-C` in COBOL or `<stem>.<FIELD>_C` in REXX, before the `SEND MAP`. The override wins; an empty / unset value means "use map default."

COBOL

```
COPY DFHCOLOR.
01 SCR.
   05 STATUS    PIC X(10).
   05 STATUS-C  PIC X(9) VALUE SPACES.
...
IF BALANCE < 0
    MOVE DFHRED   TO STATUS-C
ELSE
    MOVE DFHGREEN TO STATUS-C
END-IF.
EXEC CICS SEND MAP('BALC') FROM(SCR) ERASE END-EXEC.
```

REXX

```
if balance < 0 then scr.status_c = 'RED'
               else scr.status_c = 'GREEN'
'EXEC CICS SEND MAP(BALC) FROM(SCR.) ERASE'
```

Accepted colour mnemonics: `BLUE RED PINK GREEN TURQUOISE YELLOW NEUTRAL DEFAULT` (also the `DFH`-prefixed canonical names from `DFHCOLOR.cpy`: `DFHBLUE DFHRED DFHPINK DFHGREEN DFHTURQ DFHYELLO DFHNEUTR DFHDFT`). Unknown strings degrade to the device default.

REXX symbol caveat. REXX evaluates stem tails by value, not literal: if your program has assigned a bare variable `STATUS_C` anywhere, then `scr.status_c = 'RED'` binds to the value of `STATUS_C`, not the literal tail `STATUS_C`. Defensive form is `scr.'STATUS_C' = 'RED'` (quoted tail).

Highlight (`-H`) and attribute byte (`-A`) overrides follow the same convention and are planned for a follow-up release.

See `runtime/cobol/balc.cob` and `runtime/rexx/balc.rexx` for complete worked examples.

RECEIVE MAP

Retrieve the operator's input from the most recent `SEND MAP`.

Format

```
EXEC CICS RECEIVE MAP(name)
                  [INTO(stem.)]
END-EXEC
```

Description

Pulls the response stored by the most recent `SEND MAP` into a stem, one tail per named field on the map (`<stem>.<fieldname>`). When `INTO` is omitted, the default stem is `MAP.`.

A `RECEIVE MAP` with no matching prior `SEND MAP` returns `MAPFAIL`.

Options

MAP(name) — *required* The same map name passed to the prior `SEND MAP`.

INTO(stem.) Destination stem. Default `MAP.`.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Fields retrieved.
MAPFAIL	36	No prior matching `SEND MAP`, or named map not found.

Example

```
EXEC CICS RECEIVE MAP('CUST1') INTO(IN.) END-EXEC
ACTION = IN.ACTION
CKEY   = IN.CUSTNO
```

CONVERSE

A single verb that fuses `SEND MAP` and `RECEIVE MAP` against the same map. Semantically identical to issuing the two halves back-to-back; provided because real-world CICS programs written between the 1970s and the 1990s used `CONVERSE` heavily, and porting them shouldn't require a SEND/RECEIVE rewrite.

Format

CONVERSE has two IBM-canonical forms — the 3270 mapped form and the raw-text TS form. Bricks supports both.

Mapped form (3270):

```
EXEC CICS CONVERSE MAP(name) [MAPSET(set)]
                   [FROM(area) | FROMMAP(area)]
                   [INTO(area)]
                   [ERASE] [CURSOR(pos)]
                   [RESP(var)] [RESP2(var)]
END-EXEC
```

Text form (TS):

```
EXEC CICS CONVERSE FROM(area) [FROMLENGTH(n)]
                  INTO(target) [TOLENGTH(var) | MAXLENGTH(n)]
                   [STRFIELD] [DEFRESP]
                   [RESP(var)] [RESP2(var)]
END-EXEC
```

In the mapped form: `FROM(area)` and `FROMMAP(area)` are accepted spellings for the outbound stem; either works, both feed the SEND half. `INTO(area)` is the inbound stem; it feeds the RECEIVE half. `MAPSET`, `ERASE`, `CURSOR` pass through to both halves.

In the text form (no `MAP`): bricks dispatches `SEND TEXT` then `RECEIVE INTO`. `FROMLENGTH(n)` caps the outbound bytes; `TOLENGTH(var)` / `MAXLENGTH(n)` cap the inbound buffer. `STRFIELD` (structured-field stream) and `DEFRESP` (definite- response) are accepted silently — bricks doesn't tunnel either wire bit today but program source ports cleanly.

Behaviour

CONVERSE issues `SEND MAP` first, then `RECEIVE MAP` against the same `MAP(name)`. If the SEND half returns anything other than `RESP-NORMAL`, the RECEIVE half is **skipped** and the SEND's response code is returned — matches real CICS semantics (no screen ever made it to the terminal, so there's no operator response to wait for).

Example

```
EXEC CICS CONVERSE MAP('TIM1') FROM(SCR) INTO(SCR)
                   ERASE
END-EXEC.
IF EIBAID = PF03 THEN
    EXEC CICS RETURN END-EXEC
END-IF.
```

This replaces the longer pair:

```
EXEC CICS SEND MAP('TIM1') FROM(SCR) ERASE END-EXEC.
EXEC CICS RECEIVE MAP('TIM1') INTO(SCR)     END-EXEC.
```

Live example: `runtime/cobol/timc.cob` uses CONVERSE for both the input screen and the reminder screen.

SEND TEXT

Free-form text output without a BMS map.

Format

```
EXEC CICS SEND TEXT FROM(area)
                    [LENGTH(n)]
                    [ERASE]
END-EXEC
```

Description

Real-CICS `SEND TEXT`. The body is treated as a flat row-major buffer of characters (3270 cells): every `cols` characters (default 80) start a new row, beginning at row 0. Only the first `cols-1` of each row's characters are painted, because column 0 holds the row field's attribute byte; the last character of each `cols`-wide slice is dropped rather than wrapped onto the next row. **3270 has no LF/CR**, so programs must pad each logical line to the column width and concatenate (REXX `LEFT(s,80)`, COBOL `PIC X(80)` group children). The body is SBCS program data, one byte per character (see Character data): each byte paints as its glyph, and control bytes are painted as `?`.

`LENGTH(n)` truncates the body to exactly `n` characters (a byte count, as in CICS, and in bricks SBCS bytes and characters are the same thing, so no character can be split). Characters past `rows*cols` are dropped. `ERASE` clears the screen before painting; without it, existing fields stay behind. Like `SEND MAP`, the call paints synchronously and waits for an AID, so the operator has time to read the screen before `RETURN`.

Works identically from REXX, COBOL and PL/I.

Options

FROM(area) — *required* The buffer to paint. Can be a REXX variable or a COBOL group item.

LENGTH(n) Truncate the body to exactly this many characters (bytes; one byte is one character in bricks SBCS).

ERASE Clear the screen first.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Body painted, AID received.
INVREQ	16	`FROM` missing.
IOERR	17	The terminal write failed.

Example

See Chapter 29. Worked examples, the `GETC` program, for the canonical multi-row `SEND TEXT` pattern.

RECEIVE

Retrieve the unedited terminal line typed by the operator at the blank prompt — typically used to pick up command-line arguments.

Format

```
EXEC CICS RECEIVE INTO(buffer)
                  [LENGTH(len)]
END-EXEC
```

Description

Returns the unedited terminal line the operator typed at the blank prompt (TRANSID prefix included), e.g. `EXAM 1 2 3`. Single-shot per task: a second `RECEIVE` in the same task, or any `RECEIVE` in a chained `RETURN TRANSID` task, returns `EOC` (RESP=6).

When `LENGTH(var)` is a bare variable, the actual byte count of the line is written back. The receiving COBOL field's `PIC X(n)` width handles padding/truncation via standard `MOVE` semantics; programs detect truncation by comparing `len` against `n`.

Options

INTO(buffer) — *required* Destination variable for the line.

LENGTH(len) When a bare variable, receives the actual length on return.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Line returned.
EOC	6	No input available (already consumed in this task chain).

Example

```
EXEC CICS RECEIVE INTO(BUF) LENGTH(LEN) END-EXEC   /* "GETC 100" */
PARSE VAR BUF TID CKEY .
```

```
EXEC CICS RECEIVE INTO(WS-INPUT) LENGTH(WS-LEN) END-EXEC
UNSTRING WS-INPUT DELIMITED BY ' '
   INTO WS-TID WS-A WS-B WS-C
END-UNSTRING.
```

Chapter 5. Program control commands

RETURN

End the current task; optionally chain to another TRANSID with a COMMAREA.

Format

```
EXEC CICS RETURN [TRANSID(id)]
                 [COMMAREA(data)]
END-EXEC
```

Description

Sets `tcb.NextTransid` and `tcb.Commarea` and ends the task. With no `TRANSID`, control falls back to the blank prompt. With a `TRANSID`, the dispatcher invokes that TRANSID next, with the supplied `COMMAREA` available to the new task as `DFHCOMMAREA` and its length as `EIBCALEN`.

> **COBOL program termination & the end-of-task syncpoint.** `EXEC CICS RETURN` is the canonical way a CICS program ends, and the only one that carries the pseudo-conversational `TRANSID` / `COMMAREA` handoff. `STOP RUN`, `GOBACK`, `EXIT PROGRAM`, and running off the end of the PROCEDURE DIVISION also end the task cleanly (bricks tolerates them LE-style), and bricks runs the **implicit end-of-task syncpoint** — which commits the task's `EXEC SQL` unit of work — for those paths too, so a program that mutates SQL and then `STOP RUN`s does **not** lose the update. `STOP RUN` is nonetheless the CICS anti-pattern: it carries no pseudo-conversational handoff and, in a CALLed sub-program, would terminate the whole run unit. Use `EXEC CICS RETURN` in CICS code (with `GOBACK`, not `STOP RUN`, as the unreachable post-`RETURN` terminator the compiler wants).

Options

TRANSID(id) The next TRANSID to dispatch.

COMMAREA(data) Bytes to flow into the next task.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Task ended.

Example

```
EXEC CICS RETURN TRANSID('CUST') COMMAREA(SAVEAREA) END-EXEC
```

XCTL

Transfer control to another program. The current program does not resume.

Format

```
EXEC CICS XCTL PROGRAM(name)
              [COMMAREA(data) [LENGTH(n)]]
              [RESP(var)] [RESP2(var)]
END-EXEC
```

`LENGTH(n)` on `COMMAREA` is the IBM-canonical input bound — caller's value caps the bytes passed to the receiving program. Without `LENGTH`, a COBOL data area travels with its full declared length (trailing blanks included), as on z/OS; this holds for `LINK`, `XCTL` and `RETURN COMMAREA` and for `START FROM`. A binary item (`COMP` / `COMP-3`) travels as its raw bytes, not as decimal text. The auto-injected `DFHCOMMAREA` (a program that declares none gets a `PIC X(2000)` default) passes only the `EIBCALEN` bytes it received, so `RETURN COMMAREA(DFHCOMMAREA)` hands the next task the same length. A program on the default `DFHCOMMAREA` therefore cannot pass on more than it received: to grow the state, declare your own area (a `WORKING-STORAGE` record, `RETURN COMMAREA(WS-STATE)`, or a `LINKAGE SECTION` `01 DFHCOMMAREA` of the full size). On a cold start (`EIBCALEN` 0) whatever was `MOVE`d into the default buffer travels with its trailing blanks trimmed. An unsubscripted `OCCURS` table passed as `COMMAREA` / `FROM` travels whole, all occurrences. A REXX variable travels as its value.

COMMAREA length rules on `LINK`, as on z/OS:

- With `LENGTH(n)` the callee sees `n` bytes and only those come back: the caller's bytes past `n` are left alone.
- The callee cannot change the length: the caller always gets back exactly the number of bytes it passed. A COBOL callee's view is overlaid on those bytes (fields it never touched keep the caller's data; a `MOVE` to the whole `DFHCOMMAREA` blank-pads as usual).
- A REXX callee's assignment to `DFHCOMMAREA` replaces the whole area, the counterpart of a COBOL `MOVE` to the whole 01: a shorter value is blank-padded (`DFHCOMMAREA = 'Q'` over `ABCDEFGH` gives `Q` + seven blanks), a longer one is cut. Use `OVERLAY()` or `SUBSTR()` to change only part of the area. `CHANNEL(...)` (the CICS-TS container alternative to COMMAREA) is rejected with a short "not supported in bricks (use COMMAREA)" message — single-region bricks doesn't model channels.

Description

Transfer of control. The named program runs in place of the current one; the COMMAREA, if supplied, becomes its `DFHCOMMAREA`. The current task's chain continues with the new program; `RETURN TRANSID` semantics carry through unchanged.

Options

PROGRAM(name) — *required* The target program. Resolved through `runtime/transactions.conf`.

COMMAREA(data) Bytes to flow to the target.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Control transferred.
PGMIDERR	27	Target program not in `transactions.conf`.
INVREQ	16	PROGRAM missing.

Example

```
EXEC CICS XCTL PROGRAM('VALIDATE') COMMAREA(SCR) END-EXEC
```

LINK

Synchronously call a sub-program with bidirectional COMMAREA.

Format

```
EXEC CICS LINK PROGRAM(name)
              [COMMAREA(var | 'literal')]
END-EXEC
```

Description

The target program runs in a fresh frame with its `DFHCOMMAREA` preloaded from the caller's `COMMAREA(var)` (or literal). When the sub-program exits, its final `DFHCOMMAREA` is written back to the caller's variable.

Caller state — `NextTransid`, `Commarea`, `LastResponse`, `LastMapName` — is saved before the LINK and restored on return, so the LINK is transparent to the caller's pseudo-conversational context.

The per-transaction ACL is rechecked on the target so a low-privilege caller cannot escalate by linking into an admin program.

REXX, COBOL and PL/I programs can LINK to each other freely; `DFHCOMMAREA` is marshalled as opaque bytes. A PL/I structure and a COBOL group with the same byte layout exchange packed and binary fields unchanged (see Storage layout and COMMAREA interop with COBOL).

Options

PROGRAM(name) — *required* The sub-program to call. Resolved through `runtime/transactions.conf`.

COMMAREA(var | 'literal') Bidirectional data area. A bare variable is read on entry and written on exit; a literal is read-only.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Sub-program returned cleanly.
PGMIDERR	27	Target not in `transactions.conf`, sub-program errored, or the caller is not authorised for the target.

Example

```
SAV = ''                                /* working area */
EXEC CICS LINK PROGRAM('CUSV') COMMAREA(SAV) END-EXEC
IF EIBRESP <> 0 THEN SIGNAL ERR
PARSE VAR SAV STATUS '|' MSG
```

ABEND

Abnormally terminate the current task.

Format

```
EXEC CICS ABEND [ABCODE(code)]
                [NODUMP] [CANCEL]
                [RESP(var)] [RESP2(var)]
END-EXEC
```

`CANCEL` bypasses any installed `HANDLE ABEND` trap — the task terminates rather than recovering. `NODUMP` is accepted silently (bricks has no dump facility).

Description

Ends the task with the supplied 4-character abend code (default `AAAA`). Bricks logs the abend on the operator console and reflects it in the `TxCB`'s status; pseudo-conversational chains are broken (no `NextTransid` is honoured after an `ABEND`).

Options

ABCODE(code) 4-character code to record. Defaults to `AAAA`.

NODUMP Accepted for parity with IBM CICS; no dumps are produced anyway.

Conditions

`ABEND` does not return; nothing follows it in the program flow.

Example

```
IF EIBRESP <> 0 THEN
   EXEC CICS ABEND ABCODE('FIO1') END-EXEC
```

START

Schedule a transaction to fire on this same terminal after a delay (or at a wall-clock time), optionally passing a byte payload the new task picks up via `RETRIEVE`. This is bricks's implementation of CICS's classic deferred-work pattern: a task queues "do X in 60 seconds" and ends; the terminal returns to the blank prompt; when the timer elapses, bricks dispatches X automatically.

Format

```
EXEC CICS START TRANSID(name)
                [INTERVAL(hhmmss) | TIME(hhmmss) |
                 AFTER HOURS(h) MINUTES(m) SECONDS(s) MILLISECS(ms) |
                 AT    HOURS(h) MINUTES(m) SECONDS(s) MILLISECS(ms)]
                [FROM(area) [LENGTH(n)]]
                [TERMID(tttt)]
                [REQID(handle)] [PROTECT]
                [RESP(var)] [RESP2(var)]
END-EXEC
```

`REQID(handle)` tags the START with a request-id handle. IBM uses this to disambiguate when cancelling or deferring; bricks's single-region scheduler accepts the option without action so program source ports cleanly. `PROTECT` declares the START as recoverable — bricks already enrolls every START in the task-end SYNCPOINT, so this is silent passthrough. `SYSID` and `QUEUE` — explicitly rejected with a short "not supported" message.

Options

TRANSID(name) *(required)* 4-character transaction id, must exist in `runtime/transactions.conf`. The dispatcher applies the normal per-transaction ACL at fire time, so the operator must still be authorised to invoke `name`.

INTERVAL(hhmmss) Delay before firing, expressed as up to 6 digits in `HHMMSS` form (left-padded with zeroes). `INTERVAL(30)` = 30 s; `INTERVAL(000130)` = 1 min 30 s; `INTERVAL(010000)` = 1 h. Omit (or pair with `INTERVAL(0)`) to fire on the next prompt-loop iteration.

TIME(hhmmss) Absolute wall-clock target, also `HHMMSS`. Respects the operator's `time_zone=` setting. If the target is earlier than now, bricks rolls forward to the same time tomorrow (real-CICS semantics). Mutually exclusive with `INTERVAL`.

AFTER / AT HOURS(h) MINUTES(m) SECONDS(s) MILLISECS(ms) The IBM-canonical compositional alternatives to `INTERVAL` / `TIME` — supply any subset of the four components. `AFTER` is relative, `AT` is the next wall-clock instant with that time-of-day, rolling forward to tomorrow when it has already passed (same rule as `TIME`). The components are additive **counts**, not clock digits, so `AFTER MINUTES(90)` is an hour and a half; IBM's ranges are `HOURS` 0–99, `MINUTES` 0–5999, `SECONDS` 0–359999 and `MILLISECS` 0–360000000, with an `AFTER` total capped at 100 hours and an `AT` total under 24 hours. `MILLISECS(ms)` is the bricks sub-second extension and also works on its own (`START TRANSID('X') MILLISECS(100)`); components coded with neither `AFTER` nor `AT` are read as `AFTER`. Mutually exclusive with `INTERVAL` / `TIME`.

FROM(area) [LENGTH(n)] Payload bytes the new task will pull back via `RETRIEVE`. `LENGTH` truncates `area` to `n` bytes; if omitted, the entire string value of `area` is queued. The bytes are copied at `START` time, so the issuing task may mutate `area` afterwards without disturbing the queued payload.

TERMID(tttt) Must equal the issuing terminal's `EIBTRMID` in this version. Cross-terminal STARTs return `RESP-INVREQ` with "only same-terminal STARTs are supported."

Scope and limitations (v1)

- **Same-terminal only.** A START always fires on the issuing TCB. No-`TERMID` background tasks and `TERMID(other)` are rejected; both require infrastructure (synthetic TCBs, cross-session authorisation) that's deferred.
- **In-memory queue.** Pending STARTs do **not** survive a bricks restart. If you need recovery-protected scheduling you'll have to wait for `START PROTECT` (out of scope).
- **No `SYSID` or `QUEUE` options.** Each returns `RESP-INVREQ` with the option name in the error string so a port can be triaged. (`REQID` is recorded on the queued entry and honoured by `CANCEL REQID`; `PROTECT` is accepted as a no-op; `HOURS` / `MINUTES` / `SECONDS` / `MILLISECS` are implemented — see `AFTER` / `AT` above.)
- **One pending payload per task.** When the START fires, the FROM bytes go into `tcb.RetrieveBuf`. The new task's first `RETRIEVE` drains it; a second `RETRIEVE` returns `RESP-ENDDATA` (29).

Conditions

- `RESP-NORMAL` on successful enqueue.
- `RESP-INVREQ` for missing TRANSID, cross-terminal TERMID, unsupported option, or malformed INTERVAL/TIME.

Example

```
       EXEC CICS START TRANSID('TIMC') INTERVAL('000030')
                      FROM(MSG)
       END-EXEC.
```

Schedules `TIMC` to fire on this terminal in 30 seconds with the contents of `MSG` queued as the payload. See `runtime/cobol/timc.cob` and `runtime/rexx/timr.rexx` for the worked examples.

RETRIEVE

Pull back the `FROM(...)` payload of the `START` that scheduled the *current* task. RETRIEVE is the receiving half of the START / RETRIEVE pair; the first thing a transaction fired by `START` typically does.

Format

```
EXEC CICS RETRIEVE INTO(var) [LENGTH(n)]
                   [RESP(var)] [RESP2(var)]
END-EXEC
```

`LENGTH(n)` is the IBM in/out parameter — caller's value caps the bytes returned, the post-call value is the actual byte count.

Options

INTO(var) *(required)* Variable / DATA area to receive the payload bytes. Must be a name reference, not a string literal.

LENGTH(lenvar) Optional. The variable named here is written with the actual byte length of the payload after a successful read, so the caller can size the consumer code correctly.

Behaviour

- Cold dispatch (operator typed the TRANSID at the prompt) → `RESP-ENDDATA` (29), nothing written to `INTO`.
- Scheduled re-entry → `RESP-NORMAL`, payload bytes copied into `INTO`, `tcb.RetrieveBuf` cleared.
- Second RETRIEVE in the same task → `RESP-ENDDATA` (the buffer is cleared on first read; v1 supports one pending payload per task).

Idiom

The canonical START / RETRIEVE program tests `EIBRESP` first to discover *which dispatch path* it's on, then branches:

```
       MOVE SPACES TO BUF.
       EXEC CICS RETRIEVE INTO(BUF) END-EXEC.

       IF EIBRESP = RESP-NORMAL THEN
           PERFORM SHOW-REMINDER       *> scheduled fire
           EXEC CICS RETURN END-EXEC
       END-IF.

       *> Cold path: prompt the operator, schedule next fire.
       PERFORM PROMPT-AND-SCHEDULE.
```

REXX equivalent in `runtime/rexx/timr.rexx`; COBOL in `runtime/cobol/timc.cob`.

Chapter 6. System services

ASSIGN

Read EIB, session, and environment fields into the program.

Format

```
EXEC CICS ASSIGN <FIELD>(target) [<FIELD>(target) ...]
END-EXEC
```

Description

Reads one or more system fields into program variables. Multiple options can be combined in a single `ASSIGN` call. Each `<FIELD>` is one of the keywords below; `target` is the variable that receives the value.

Options

Option	Returns
`USERID(t)`	Signed-on userid (empty until CSSN succeeds).
`TERMID(t)` / `EIBTRMID(t)`	Unique terminal id, T plus 4 digits (`T0001` …). Give COBOL and PL/I an 8-character target; a 4-character one keeps `T000`.

Option	Returns
`EIBAID(t)`	Single-byte AID character of the most recent `SEND MAP` / `RECEIVE MAP`. Compare with `C2X(EIBAID) = 'F3'` (REXX) or `EIBAID = X'F3'` (COBOL) to detect PF3, etc.
`EIBCPOSN(t)`	1-based cursor position from the most recent map response.
`EIBCALEN(t)`	Length of `DFHCOMMAREA` flowed in from the caller.
`CWALENG(t)`	Byte length of the region Common Work Area — the `wrkarea=` value from `bricks.cnf`. `0` when no CWA is configured. See EXEC CICS ADDRESS.
`TWALENG(t)`	Byte length of this task's Transaction Work Area — the `TWASIZE` 6th field of the transaction's `transactions.conf` row. `0` when no TWA is configured. See EXEC CICS ADDRESS.
`TCTUALENG(t)`	Always `0` (bricks does not model a TCTUA).
`SCREENHT(t)` / `SCREENWD(t)`	Negotiated terminal rows / columns.
`ALTSCRNHT(t)` / `ALTSCRNWD(t)`	Same values; bricks does not distinguish primary and alternate sizes.
`CONNECTED(t)` / `CONNTIME(t)`	Wall-clock connect timestamp `YYYY-MM-DD HH:MM:SS`.
`TLS(t)`	`yes` when the session is on the TLS listener, `no` otherwise.
`DATE(t)`	Today as `YYYYMMDD`. *(bricks-specific)*
`TIME(t)`	Now as `HHMMSS`. *(bricks-specific)*
`TODAYYR(t)` / `TODAYMO(t)` / `TODAYDY(t)`	Today's year / month / day individually. *(bricks-specific)*
`DAYCOUNT(t)`	Days since `1900-01-01` (the IBM ABSTIME epoch). Subtract two values for an exact day delta. **Behaviour change in round 2:** previously returned Unix-epoch days; bricks now matches real CICS.
`SYSID(t)`	Local CICS region ID (`BRKS` in bricks).
`APPLID(t)`	Region VTAM applid (`BRICKS01` in bricks).
`NETID(t)`	Network ID of the connected terminal (`TCPIP` in bricks; placeholder for a future config knob).
`ABCODE(t)`	Most recent abend code (the value `EIBABCODE` carries). Empty before any ABEND.
`EIBTASKN(t)`	Numeric task number — the integer body of `TxCB.ID`.
`STARTCODE(t)`	How the task was started. `TO` (terminal operator) is the bricks default; future work threads `S` (started by EXEC CICS START) through the scheduler.

The bricks-specific options exist primarily so the COBOL subset can do date math without REXX-style intrinsic functions; see `runtime/cobol/qagc.cob` for an example that computes age in years from a birthdate.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	All requested fields returned.
INVREQ	16	A field name is unknown, or its target is a literal rather than a variable.

Example

```
EXEC CICS ASSIGN USERID(USR)
                 TERMID(TRM)
                 SCREENHT(SCRH)
                 SCREENWD(SCRW)
END-EXEC
```

ASKTIME

Refresh the EIB date/time fields and, optionally, return the current time as a 15-digit absolute timestamp (`ABSTIME`).

Format

```
EXEC CICS ASKTIME [ABSTIME(target)]
END-EXEC
```

Description

`ASKTIME` re-reads the system clock and writes the current date and time into `EIBDATE` and `EIBTIME`. Both fields follow the IBM CICS formats:

- `EIBDATE` — packed-style decimal `0CYYDDD`, where `C` is `(century - 19)` and `YYDDD` is the two-digit year and Julian day-of-year. For 2026-05-12 (Julian day 132), `EIBDATE = 1026132`.
- `EIBTIME` — six-digit `HHMMSS`.

If `ABSTIME(target)` is given, `target` receives a 15-digit decimal string representing milliseconds since `1900-01-01 00:00:00.000`. Pass that value to `FORMATTIME` to break it back into formatted date / time components.

`ASKTIME` is the only way to get a fresh `ABSTIME`; the bricks-specific `ASSIGN DATE` / `TIME` shortcuts return formatted strings only.

Options

ABSTIME(target) Variable that receives the 15-character absolute time. Optional; when omitted, only `EIBDATE` and `EIBTIME` are refreshed.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Always (clock reads cannot fail).

Example

```
EXEC CICS ASKTIME ABSTIME(NOW) END-EXEC
EXEC CICS FORMATTIME ABSTIME(NOW)
                     YYYYMMDD(TODAY)
                     TIME(CLOCK) TIMESEP(':')
END-EXEC
SAY 'Stamped at' TODAY CLOCK
```

FORMATTIME

Decode an `ABSTIME` value into formatted date and time fields.

Format

```
EXEC CICS FORMATTIME ABSTIME(source)
                     [DATE(t)] [DATEFORM(fmt)] [DATESEP(c)]
                     [YYYYMMDD(t)] [MMDDYYYY(t)] [DDMMYYYY(t)]
                     [MMDDYY(t)]   [DDMMYY(t)]
                     [YYYYDDD(t)]  [YYDDD(t)]
                     [TIME(t)] [TIMESEP(c)]
                     [TIMEZONE(name)]
                     [YEAR(t)] [MONTHOFYEAR(t)] [DAYOFMONTH(t)]
                     [DAYOFWEEK(t)] [DAYCOUNT(t)]
                     [RESP(var)] [RESP2(var)]
END-EXEC
```

`TIMEZONE(name)` is the IBM-canonical per-call zone override — accepts IANA names (`America/New_York`, `Europe/Paris`) plus the `UTC` alias. Unknown zone → `INVREQ` with a short clean message. Without `TIMEZONE`, the handler's configured location (from the `time_zone` line of `bricks.cnf`) is used.

Description

`FORMATTIME` is the IBM-standard companion to `ASKTIME`. Given a 15-digit `ABSTIME` value (typically returned by `ASKTIME ABSTIME(...)`, but any 15-digit decimal millisecond stamp is accepted), it writes the requested representations into the named target variables.

Any combination of output options may be requested in a single call; options that are omitted cost nothing.

Options

ABSTIME(source) *(required)* The 15-digit decimal absolute time to decode.

DATE(t) with optional **DATEFORM(fmt)** and **DATESEP(c)** Generic date target. `DATEFORM` is one of `YYYYMMDD` (default), `MMDDYYYY`, `DDMMYYYY`, `MMDDYY`, `DDMMYY`. `DATESEP(c)` inserts a one-character separator between components (e.g. `'-'` → `2026-05-12`); omit `DATESEP` for a packed string with no separators.

YYYYMMDD / MMDDYYYY / DDMMYYYY / MMDDYY / DDMMYY Direct date format targets; each respects `DATESEP(c)` if given.

YYYYDDD(t) / YYDDD(t) Julian-style date with three-digit day-of-year. Honour `DATESEP`.

TIME(t) with optional **TIMESEP(c)** Six-digit `HHMMSS`, or with `TIMESEP(':')` formatted as `HH:MM:SS`.

YEAR(t) — four-digit year (`2026`). **MONTHOFYEAR(t)** — `1`..`12`. **DAYOFMONTH(t)** — `1`..`31`. **DAYOFWEEK(t)** — `0` (Sunday) .. `6` (Saturday), per IBM CICS. **DAYCOUNT(t)** — days since `1900-01-01` (signed, integer).

Conditions

Condition	EIBRESP	Cause
NORMAL	0	All requested fields written.
INVREQ	16	`ABSTIME` is missing, not 15 decimal digits, or out of range.

Example

```
EXEC CICS ASKTIME ABSTIME(WS-NOW) END-EXEC
EXEC CICS FORMATTIME ABSTIME(WS-NOW)
                     DATE(WS-DATE)  DATEFORM('DDMMYYYY')  DATESEP('/')
                     TIME(WS-TIME)  TIMESEP(':')
                     DAYOFWEEK(WS-DOW)
END-EXEC.
*  WS-DATE → "12/05/2026"     WS-TIME → "10:30:45"     WS-DOW → "2"
```

Security inquiry — `QUERY SECURITY` and `VERIFY PASSWORD`

Read-only programmatic introspection of the running user's authorisation. The verb lets an application program ask "may this caller perform action X on resource Y?" before it builds the menu or before it dispatches a destructive operation. Bricks ships both `QUERY SECURITY` (this section) and `VERIFY PASSWORD` (the following section, *Password verification*) in 2.7.x — the first asks the authorisation question, the second re-authenticates the caller's credentials.

Format

```
EXEC CICS QUERY SECURITY RESOURCE(name)
                         [RESCLASS(class)]
                         [READ(target)]
                         [UPDATE(target)]
                         [CONTROL(target)]
                         [ALTER(target)]
                         [RESP(rc) RESP2(rc2)]
END-EXEC.
```

Description

Consults the in-memory user record (loaded from `users.conf`) and the resource's group list (from the dispatch table). Writes one `DFHVALUE` integer into each of the four optional target host variables — the program then compares against the named constants in the `DFHVALUE` copybook.

The verb is stateless and has no side effects: nothing is logged, no lock is taken, no journal entry written. Calling it a thousand times in a tight loop is harmless.

Options

Option	Direction	Notes
`RESOURCE(name)`	input	TRANSID or program name being checked. Required.
`RESCLASS(class)`	input	`TCICSTRN` (transactions, default) or `PCICSPSB` (programs). Both class names consult the same transaction-table lookup — bricks has no separate program table, so a TRANSID and the program it dispatches share one ACL row. Anything else → `INVREQ` `QuerySecurityResp2BadResclass` (RESP2=2).
`READ(t)`	output	`DFHVALUE–READABLE` (50) when the caller may read, `DFHVALUE–NOTREADABLE` (51) otherwise.
`UPDATE(t)`	output	`DFHVALUE–UPDATABLE` (52) / `DFHVALUE–NOTUPDATABLE` (53).
`CONTROL(t)`	output	`DFHVALUE–CTRLABLE` (54) / `DFHVALUE–NOTCTRLABLE` (55).
`ALTER(t)`	output	`DFHVALUE–ALTERABLE` (56) / `DFHVALUE–NOTALTERABLE` (57).

A literal-string argument on any output axis (`READ('FOO')`) is silently skipped — matches the same "literal or empty → no-op" rule the `RESP(var)` / `RESP2(var)` clauses use everywhere.

The verb is a per-user inquiry, so the caller must be signed on. A `PUBLIC` resource is reachable at *dispatch* time without sign-on, but `QUERY SECURITY` answers it like any other resource: an unauthenticated terminal sees all four axes as the NOT-* code. After sign-on the verb delegates the READ axis to the same `txn.IsAllowed` rule the dispatch gate uses — including PUBLIC and `*` semantics — so a signed-on caller against a `PUBLIC` resource sees `READ=DFHVALUE-READABLE`.

Bricks-flavoured access mapping (explicit deviation)

Real CICS / RACF projects per-class permissions onto the four axes via a class-specific matrix. Bricks's existing ACL substrate is binary (you are in the resource's allow-list, or you are not), so the bricks projection narrows the IBM matrix:

Axis	Bricks rule
`READ`	Caller's userid shares any group with the resource's allow-list, OR the resource has no ACL (open), OR caller is in `ADMIN`.
`UPDATE`	Caller is in the `ADMIN` group.
`CONTROL`	Caller is in the `ADMIN` group.
`ALTER`	Caller is in the `ADMIN` group.

Document this rule in operator-facing notes alongside any program that gates a menu on `UPDATE` — "only members of `ADMIN` see the D key" reads more directly than the abstract DFHVALUE matrix.

Conditions

Condition	EIBRESP	EIBRESP2	Cause
NORMAL	0	0	Verb completed; up to four host variables written.
INVREQ	16	`QuerySecurityResp2NoResource` (1)	`RESOURCE(...)` is missing or resolves to the empty string.
INVREQ	16	`QuerySecurityResp2BadResclass` (2)	`RESCLASS(...)` is neither `TCICSTRN` nor `PCICSPSB`.
INVREQ	16	`QuerySecurityResp2NoChecker` (3)	The dispatcher did not wire a `SecurityChecker` (operator misconfiguration).

Unknown resource → `NORMAL` with every requested axis set to the NOT-* code. No `NOTFND` is emitted: the verb is a yes/no inquiry, and "I don't know that resource" collapses cleanly to "all axes denied". Programs that need to distinguish "denied" from "absent" should pair `QUERY SECURITY` with a separate `INQUIRE` of the transaction table (not yet implemented; tracked on the deferred list).

`NOTAUTH` (70) is the IBM-canonical "the caller is not authorised" response code; `QUERY SECURITY` never emits it (a denied caller sees `NORMAL` plus the `NOT*` DFHVALUE on each axis). The `VERIFY PASSWORD` verb in the following section is the verb that *does* return `NOTAUTH` for a bad-credentials outcome.

The admin group name is hard-coded to `ADMIN` in `main.go` (`securityAdminGroup`). If a deployment renames it, the `UPDATE` / `CONTROL` / `ALTER` axes return the NOT-* code for every user (the `READ` axis is unaffected because it consults the per-resource `Groups` list, not the admin group). A future `bricks.cnf` `admin_group=X` knob would lift this hard-coding.

DFHVALUE constants

The `runtime/cobolcopy/DFHVALUE.cpy` copybook declares each constant twice per the `cobol_copybook_dual_names` convention — once under the bricks-style short name and once under the `DFHVALUE-` IBM-traditional alias. Either form compiles:

Bricks mnemonic	IBM alias	Value
READABLE	DFHVALUE-READABLE	50
NOTREADABLE	DFHVALUE-NOTREADABLE	51
UPDATABLE	DFHVALUE-UPDATABLE	52
NOTUPDATABLE	DFHVALUE-NOTUPDATABLE	53
CTRLABLE	DFHVALUE-CTRLABLE	54
NOTCTRLABLE	DFHVALUE-NOTCTRLABLE	55
ALTERABLE	DFHVALUE-ALTERABLE	56
NOTALTERABLE	DFHVALUE-NOTALTERABLE	57

> **Storage format — bricks decimal-text deviation.** DFHVALUE constants ship as `PIC S9(8) VALUE` (DISPLAY) per the bricks decimal-text host-variable convention — see Chapter 8a for the same shape on `RBA`. Programs that declare target host vars as `PIC S9(8) COMP` and then `IF WS-CACR-UP = DFHVALUE-UPDATABLE` will silently mismatch: the comparison sees a 4-byte binary half-word on one side and the seven ASCII digits "0000052" on the other, and the `IF` always falls false. Declare the target as plain `PIC S9(8)` (DISPLAY) so the comparison is bit-equal to the constant; the `DFHVALUE.cpy` header block carries the long-form rationale and points back here.

COBOL example — gating a Delete menu entry

The shipped `runtime/cobol/gust.cob` sample demonstrates the typical pattern. In its `DISPATCH` paragraph, before the `EVALUATE ACTION` switch, an admin gate sits in front of the `D=Delete` branch:

```
       COPY DFHVALUE.
       ...
       01 WS-CACR-UP PIC S9(8) VALUE 0.
       ...
       IF VALID-ACTION = 'Y' AND ACTION = 'D' THEN
           EXEC CICS QUERY SECURITY RESOURCE('GUST')
                                    UPDATE(WS-CACR-UP) END-EXEC
           IF WS-CACR-UP NOT = DFHVALUE-UPDATABLE THEN
               MOVE 'Delete not permitted for this user.' TO MSG
               MOVE 'N' TO VALID-ACTION
           END-IF
       END-IF.
```

Non-admin callers see the message in `MSG` on the next menu SEND; admins fall through to the standard `ACT-DELETE` path.

REXX example

```
EXEC CICS QUERY SECURITY RESOURCE('GUST'),
                         READ(WS_RD),
                         UPDATE(WS_UP) END-EXEC
IF WS_UP = 52 THEN
    SHOW_DELETE = 'Y'
ELSE
    SHOW_DELETE = 'N'
```

REXX programs may compare against the bare integer (52 above) or import the same numeric values from a constants stem if the shop prefers symbolic names.

Password verification — `VERIFY PASSWORD`

Companion verb to `QUERY SECURITY`. Asks "is this plaintext password the right one for this userid?" against the same `users.conf`-backed authenticator `CSSN` uses for sign-on. Used by batch / SOAP entry points that need to re-authenticate before a privileged operation, and by terminal screens that want a "type your password again" gate in front of a destructive action — even though the session is already signed on.

Format

```
EXEC CICS VERIFY PASSWORD USERID(uid)
                          PASSWORD(pwd)
                          [CHANGETIME(t)]
                          [EXPIRYTIME(t)]
                          [DAYSLEFT(d)]
                          [INVALIDCOUNT(n)]
                          [LASTUSETIME(t)]
                          [RESP(rc) RESP2(rc2)]
END-EXEC.
```

Description

Re-authenticates the (userid, password) pair against the running `users.conf` and returns `NORMAL` (0) on success, `NOTAUTH` (70) on any failure. **The password is compared exactly** — byte for byte, case included. (The verifier used to try the as-typed, upper-cased and lower-cased forms against the single stored hash, so up to three different inputs satisfied this verb for one account; `VERIFY PASSWORD` inherited that from the shared authenticator. Exactly one spelling is accepted now. Nothing about the stored hashes changed — each was always built from the exact enrolled password.) The userid itself is still matched case-insensitively. The verb itself is stateless — no journal entry, no lock — and reads the same in-memory user record `CSSN` consults at sign-on. Calling it repeatedly is harmless aside from the bcrypt cost on each call (the FileStore.Authenticate path spends the same compute on unknown users as on known ones to defeat username-enumeration via response timing).

Options

Option	Direction	Notes
`USERID(uid)`	input	Userid to verify. Required. Trimmed of surrounding whitespace; empty → `INVREQ` RESP2=1.
`PASSWORD(pwd)`	input	Plaintext password. Required. Passed verbatim to the verifier (no trim — leading/trailing whitespace is part of the credential); empty or missing PASSWORD passes through to the verifier and folds onto the standard `NOTAUTH` (70) bad-credential branch. (A fast-reject on

Option	Direction	Notes
		empty PASSWORD was withdrawn in review pass 3 — see the Conditions table below for the rationale.)
`CHANGETIME(t)`	output	*Parsed but not populated* — see deviation below.
`EXPIRYTIME(t)`	output	*Parsed but not populated*.
`DAYSLEFT(d)`	output	*Parsed but not populated*.
`INVALIDCOUNT(n)`	output	*Parsed but not populated*.
`LASTUSETIME(t)`	output	*Parsed but not populated*.

Bricks deviation — non-distinguishing failure mode (explicit)

> **Loud deviation from IBM RACF.** Real CICS distinguishes "unknown userid" from "bad password" via separate RESP2 sub-codes. **Bricks intentionally does not.** Every authentication failure — unknown userid, wrong password, future account lockout, rate-limit refusal — collapses onto the single `NOTAUTH` (70) result with `RESP2=0`. The motivation is straightforward: this verb is safe to expose behind public-facing entry points (a WAPI re-auth endpoint, a SOAP credential check), and any RESP2-level distinction would let a hostile caller enumerate valid userids by inspecting the sub-code. The bricks `FileStore.Authenticate` path already pays the bcrypt cost on unknown users for the same timing-resistance reason, so the bricks path is uniform end-to- end. The contract is enforced in code by the `cics.ErrPasswordVerifyBadCredentials` sentinel: every `PasswordVerifier` implementation must collapse its failure modes onto this single sentinel before returning.

Bricks deviation — password-policy outputs not populated

The optional outputs `CHANGETIME` / `EXPIRYTIME` / `DAYSLEFT` / `INVALIDCOUNT` / `LASTUSETIME` parse cleanly (so source ported from a real z/OS shop continues to compile) but bricks does not populate them. Bricks's `users.conf` records bcrypt hash + group list and nothing else — there is no password-policy metadata to write back. If the named target is a host variable, it is left unchanged after the verb returns; if it is a literal, the option is silently dropped.

A program that relies on `DAYSLEFT` or `EXPIRYTIME` to drive an "expires in N days" warning must gate that branch on whether the variable is still at its pre-call value. The shipped `runtime/rexx/cssr.rexx` sample (below) does not invoke the optional outputs.

Conditions

Condition	EIBRESP	EIBRESP2	Cause
NORMAL	0	0	Credentials matched.
NOTAUTH	70	0	Bad credentials — wrong password OR unknown userid OR empty/missing PASSWORD OR any future failure mode. The sub-code is intentionally 0 to defeat enumeration (see deviation above).
INVREQ	16	`VerifyPasswordResp2NoUserid` (1)	`USERID(...)` missing or resolves to the empty string.
INVREQ	16	`VerifyPasswordResp2NoVerifier` (3)	The dispatcher did not wire a `PasswordVerifier` (operator misconfiguration).

Condition	EIBRESP	EIBRESP2	Cause
IOERR	17	0	The verifier reported a transient error (file-system fault, future store unreachable, ...). The shipped bricks `passwordAdapter` folds every observed `FileStore` error onto `NOTAUTH` today, but custom verifiers may return arbitrary errors that surface as `IOERR` — a deployer who plugs in an LDAP or RACF-passthrough verifier should expect this path to fire.

> **No `RESP2=2` row.** Sub-code 2 was withdrawn in review pass 3: it formerly meant "PASSWORD missing or empty" and was a fast-reject before the verifier ran. The wall-clock gap between "PASSWORD absent — fast `INVREQ`" and "PASSWORD wrong — ran-bcrypt-then-`NOTAUTH`" let an attacker time the verb and learn whether the verifier was reached. Empty / missing PASSWORD now falls through to the verifier and lands on the standard `NOTAUTH` branch. The numeric gap at 2 is preserved deliberately so an operator reading a `RESP2=3` log line from a deployed program still finds the same "no verifier configured" cause.
>
> **Refused in CECI.** The CECI debugging shell refuses `EXEC CICS VERIFY PASSWORD` outright because the operator's typed PASSWORD value would echo on the input pane (and could land in the CECI audit log if `ceci_audit` is ever re-enabled). Programs that need a "type your password again" gate must run as their own TRANSID (see `runtime/rexx/cssr.rexx` for the canonical sample).

REXX example — re-authenticate before a privileged action

The shipped `runtime/rexx/cssr.rexx` sample demonstrates the canonical "type your password again" gate (`CSSR` = "Sign-on Re-verify"; TRANSID registered in `runtime/transactions.conf` with `USERS,ADMIN` groups). Usage: type `CSSR` at the bricks blank prompt; the `CSSRPW` map then prompts for the password in a HIDDEN (non-echoing) writable field. PF3 cancels without verifying.

```
ADDRESS CICS
EXEC CICS ASSIGN USERID(USR) END-EXEC
MAP.USERID   = USR             /* preload the signed-on user */
MAP.PASSWORD = ''              /* clear any leftover value   */
EXEC CICS SEND    MAP('CSSRPW') FROM(MAP) ERASE END-EXEC
EXEC CICS RECEIVE MAP('CSSRPW') INTO(MAP) END-EXEC
IF EIBAID = '3' THEN DO        /* PF3 = cancel              */
  EXEC CICS SEND TEXT FROM('CSSR cancelled.') ERASE END-EXEC
  EXEC CICS RETURN END-EXEC
END
PWD = STRIP(MAP.PASSWORD)
EXEC CICS VERIFY PASSWORD USERID(USR) PASSWORD(PWD) RESP(RC) END-EXEC
IF RC = 0 THEN
  MSG = 'Password OK. Privileged step authorised.'
ELSE IF RC = 70 THEN
  MSG = 'Bad credentials. Privileged step refused.'
ELSE
  MSG = 'VERIFY PASSWORD failed RESP=' || RC || ' RESP2=' || EIBRESP2
EXEC CICS SEND TEXT FROM(MSG) ERASE END-EXEC
EXEC CICS RETURN END-EXEC
```

The `RESP(RC)` clause writes the result into a host variable inline so the `IF RC = 70` branch reads cleanly instead of fishing through `EIBRESP` after the fact.

Why the map, not a command-tail. A prior draft of CSSR read the password from the initial-input buffer (`CSSR <password>`). That pattern echoes the password on the 3270 input pane, drops it into any `CONS` shell view, and lands it in CECI's input echo if an operator ever reissues the command from there. The shipped sample uses a `CSSRPW` map whose `PASSWORD` writable field carries the `DARK` (HIDDEN) attribute, so the operator's keystrokes never render on screen. Production re-auth flows must follow the same pattern — see the 3270 EBCDIC 037 memory notes on writable-field attributes if you build your own map.

COBOL idiom — gating LINK with a typed-again password

The same pattern in COBOL. The `WS-PWD` field is the operator's re-typed password (collected on a `RECEIVE MAP` from a sensitive- action confirmation screen, not shown). On `NOTAUTH` the program falls through to a refuse-message path; on `NORMAL` it can `LINK` to the privileged sub-program.

```
       77  WS-VP-RC PIC S9(8) VALUE 0.
       77  WS-USR   PIC X(8).
       77  WS-PWD   PIC X(32).
       ...
       EXEC CICS ASSIGN USERID(WS-USR) END-EXEC
       EXEC CICS VERIFY PASSWORD USERID(WS-USR)
                                 PASSWORD(WS-PWD)
                                 RESP(WS-VP-RC) END-EXEC
       EVALUATE WS-VP-RC
           WHEN 0
               EXEC CICS LINK PROGRAM('ADMNDEL') END-EXEC
           WHEN 70
               MOVE 'Password does not match. Try again.' TO MSG
           WHEN OTHER
               MOVE 'VERIFY PASSWORD failed (config?).' TO MSG
       END-EVALUATE.
```

The `EXEC CICS ASSIGN USERID(WS-USR)` step is the canonical way to read the signed-on userid into a host variable — bricks does **not** populate an `EIBUSER` field, so a port from a z/OS shop that used `USERID(EIBUSER)` directly must be rewritten to go through `ASSIGN` first. The shipped `runtime/rexx/cssr.rexx` sample uses the REXX flavour of the same pattern.

A COBOL caller that intends to `EVALUATE WS-VP-RC WHEN 70` must declare `WS-VP-RC` as plain `PIC S9(8)` (DISPLAY) to match the bricks decimal-text host-variable convention — same rule as the DFHVALUE constants documented under `QUERY SECURITY` above.

System inquiry — `INQUIRE SYSTEM`

Recently added in 2.7.x. `INQUIRE SYSTEM` is the entry point for SIT-level inquiries — values the operator set in `bricks.cnf` that a program needs at run time. Bricks v1 honours **only** the `GMMTEXT` option (the IBM CICS canonical "Good Morning" sign-on text); the dispatch infrastructure is in place so future SIT options (`RELEASE`, `CICSSTATUS`, `MAXTASKS`, `JOBNAME`, `APPLID`, `SYSID`) extend the same verb without touching the Handler / Dispatcher field set.

Format

```
EXEC CICS INQUIRE SYSTEM
    [GMMTEXT(var)]
    [RESP(rc)] [RESP2(rc2)]
END-EXEC
```

Description

Reads SIT-level configuration into the program. v1 honours only the `GMMTEXT(var)` option, which writes the configured `gmtext=` value from `bricks.cnf` (default `"Welcome to bricks"`) into the named host variable. A bare `INQUIRE SYSTEM` (no options, or only RESP/RESP2) is a no-op success matching real CICS.

Options

Option	Bricks v1 behaviour
`GMMTEXT(var)`	Writes the configured GMTEXT into `var`. Full 256-byte value is delivered verbatim — no truncation at verb time. (The bricks LogonPrompt renderer truncates to `cols−1` at paint time, but the verb's host-variable surface preserves the full storage.)
Other (RELEASE, CICSSTATUS, MAXTASKS, JOBNAME, APPLID, SYSID, etc.)	Not yet supported in v1 — surfaces `INVREQ` with `EIBRESP2 = 1`.

Conditions

Condition	EIBRESP	EIBRESP2	Cause
NORMAL	0	0	Verb succeeded. Bare `INQUIRE SYSTEM` (no options) also returns NORMAL even when the dispatcher hasn't wired a `SystemInfoProvider`.
INVREQ	16	1	The operator named an option v1 doesn't honour yet. The diagnostic names the option.
INVREQ	16	2	`h.SystemInfo == nil` AND a substantive option was named — the dispatcher did not wire a provider. Mirrors the QUERY SECURITY / VERIFY PASSWORD `RESP2=3` loud-fail convention.

Reserved RESP2 sub-codes 3..9 are kept free for future INQUIRE form sub-errors (INQUIRE FILE, INQUIRE TASK, etc.). v1 programs that pin numeric RESP2 codes will continue to see only `1` or `2` for INQUIRE SYSTEM; future bricks releases will not silently re-bind these reservations.

Bricks deviations (loud)

Per the `bricks_cics_ibm_canonical` memory every deviation from IBM canonical INQUIRE SYSTEM is called out individually below.

> **Bricks deviation — `GMMTEXT` is the only honoured option in v1.**
>
> Of the 30+ IBM `INQUIRE SYSTEM` options (`RELEASE`, `CICSSTATUS`, `MAXTASKS`, `JOBNAME`, `APPLID`, `SYSID`, ...) bricks v1 honours only `GMMTEXT`. Any other named option surfaces `INVREQ` `RESP2=1` with the option name in the diagnostic, so a port from a z/OS shop fails loudly rather than silently returning zero / empty. Adding any deferred option is a ~5-line extension to the `cics.SystemInfoProvider` interface (one new method) plus a single case in `cics/inquire.go`'s `inquireSystem` handler — no Handler / Dispatcher / Frame changes.

Bricks deviation — EBCDIC-037 fail-fast at startup.

The `gmtext=` value is validated against the EBCDIC-037 printable range (`0x20..0x7E`) minus the `3270_ebcdic_037_printables` deny-set (`[ ] { } ~ \ ` | ^`) at config load. A `gmtext=` containing any non-printable byte (e.g. an embedded bell `\x07`, a control character, or a non-ASCII glyph) or any deny-set glyph is a startup error. Real CICS would silently substitute on the wire; bricks refuses to start so the operator notices.

Bricks deviation — empty `gmtext=` restores the default.

A bare `gmtext=` line (or one with only whitespace inside the quotes) DOES NOT blank the banner — it restores the `"Welcome to bricks"` default. Mirrors the `ntp_server=on` aliasing precedent and matches the bricks last-wins-per-key configuration policy. An operator who genuinely wants no banner cannot achieve that via `gmtext=`; the only way to suppress the sign-on banner entirely is to leave both `gmtext=` and the legacy welcome line out of the LogonPrompt path (currently not possible without a code change).

Bricks deviation — GMTEXT replaces the legacy welcomeLine on every welcome screen.

GMTEXT is painted at row 0 of the connect-time `ShowLogoSplash` AND at row 1 of the `LogonPrompt` (when `enforce_secure_login=yes`) AND on the CSSF LOGOFF confirmation splash. The legacy `Welcome to BRICKS HH:MM:SS` banner with timestamp only paints when `gmtext=` is empty or absent. `BlankPrompt` (the steady-state TRANSID cursor) is NOT a banner site — adding GMTEXT there would repaint on every return from a transaction, which surprises operators.

Real CICS paints both the GMTEXT and a "good morning" line on the sign-on prompt. Bricks stacks neither — the operator sees one centred banner instead of two, which the architect's C7 review flagged as the correct layout. When `gmtext=` is empty / whitespace (which the config load layer should have already restored to default, but the renderer composes safely either way) the legacy welcomeLine paints as fallback.

Sample REXX

The shipped `runtime/rexx/gmtq.rexx` (TRANSID `GMTQ`) demonstrates the recommended idiom — RESP(RC) for clean error branching, no comma-continuations inside the EXEC body (per the `bricks_rexx_exec_one_line` memory):

```
/* GMTQ -- read GMTEXT and echo it to the terminal. */
ADDRESS CICS

MSG = ''
EXEC CICS INQUIRE SYSTEM GMMTEXT(MSG) RESP(RC) END-EXEC

IF RC \= 0 THEN
  MSG = 'INQUIRE SYSTEM GMMTEXT failed; EIBRESP=' || RC || ' EIBRESP2=' || EIBRESP2

EXEC CICS SEND TEXT FROM(MSG) ERASE END-EXEC
EXEC CICS RETURN END-EXEC
```

A program that omits `RESP(RC)` still gets the Go-level error in `EIBRESP` via the existing captureRespVars pipeline (a `RESP` clause that names a host variable is the recommended shape for clean per-call error branching; the `EIBRESP` post-check shape also works).

SIT-knob — `gmtext=` in `bricks.cnf`

```
# bricks.cnf
gmtext="Sample bank -- production"   # max 256 bytes, EBCDIC-037 printable only
```

Defaults to `"Welcome to bricks"` when the line is absent or empty. See Configuration — `bricks.cnf` in the README for the full knob reference.

ADDRESS — `EXEC CICS ADDRESS`

> **Storage addressing.** `ADDRESS` hands a program a pointer to a CICS-owned storage area — the COMMAREA, the EIB, the region CWA, or the per-task TWA. It is the front half of the bricks COBOL pointer model: the handle `ADDRESS` writes into a `USAGE POINTER` item is later mapped onto a based LINKAGE 01 with `SET ADDRESS OF`. See Chapter 22 — pointers for the COBOL surface.

Format

```
EXEC CICS ADDRESS
    [COMMAREA(ptr)] [EIB(ptr)] [CWA(ptr)] [TWA(ptr)]
    [TCTUA(ptr)] [ACEE(ptr)]
    [RESP(rc)] [RESP2(rc2)]
END-EXEC
```

Description

Each named option's `ptr` operand (a `USAGE POINTER` item in COBOL) is set to an opaque **handle** addressing the requested area. A COBOL program then maps a based LINKAGE 01 onto those bytes with `SET ADDRESS OF L TO ptr`. The handles are not displayable machine addresses, but `SET ptr UP/DOWN BY n` arithmetic *is* supported — it adjusts the offset **within the pointer's current target buffer** (see the COBOL pointer model).

In **REXX** `ADDRESS` is accept-and-no-op: every operand is set to `0` (NULL) and `NORMAL` is returned. REXX has no pointers — the data a REXX program needs (COMMAREA, EIB fields) is already injected as variables — so the verb exists only for source compatibility.

Options

Option	Handle addresses	Notes
`COMMAREA(ptr)`	`DFHCOMMAREA`	Always a non-NULL handle (the based `DFHCOMMAREA` default buffer always exists). `EIBCALEN` tells you whether a caller actually flowed bytes in.
`EIB(ptr)`	`DFHEIB`	Always non-NULL. Maps the real `01 DFHEIB` block (children `EIBRESP`, `EIBAID`, `EIBCALEN`, …).
`CWA(ptr)`	region Common Work Area	NULL (handle `0`) **when** `wrkarea=0` in `bricks.cnf`. One slice shared by every task; writes are visible region-wide (unsynchronised). Length via `ASSIGN CWALENG`.
`TWA(ptr)`	per-task Transaction Work Area	NULL when the transaction's `TWASIZE` (`transactions.conf` 6th field) is `0`. Fresh per task. Length via `ASSIGN TWALENG`.
`TCTUA(ptr)`	terminal user area	**Always NULL** — not modelled in bricks.

Option	Handle addresses	Notes
`ACEE(ptr)`	security ACEE	**Always NULL** — not modelled in bricks.

`CSA` and `TCTTE` are **not** accepted (both obsolete as addressable areas in modern CICS); naming them — or any other option — returns `INVREQ`.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	The named areas were addressed. NULL is still NORMAL — a NULL handle for an unconfigured CWA / TWA (or for TCTUA / ACEE) is the documented outcome, not an error.
INVREQ	16	An option other than the accepted set was named (e.g. `CSA`, `TCTTE`). The diagnostic names the option.

Bricks deviations (loud)

> **Bricks deviation — TCTUA and ACEE are always NULL.** Neither the per-terminal user area nor the security ACEE is modelled. `ADDRESS TCTUA(p)` / `ADDRESS ACEE(p)` return `NORMAL` with `p` set to `0`; a program that dereferences either gets a clean NULL-pointer abend, not silent garbage.
>
> **Bricks deviation — `ADDRESS CWA` is NULL unless `wrkarea>0`.** A region with no configured CWA (the default `wrkarea=0`) yields a NULL CWA handle — exactly as real CICS does on a region with no CWA. Set `wrkarea=` in `bricks.cnf` to allocate one.
>
> **The CWA is unsynchronized — serialize updates with `ENQ`/`DEQ`.** The region CWA is a single byte slice shared by reference into every concurrent task; bricks does **not** auto-serialize access (neither does real CICS). Concurrent unsynchronized writes from different tasks are undefined — guard CWA updates with `EXEC CICS ENQ` / `DEQ`, as on real CICS.

Example (COBOL)

```
01 WS-CPTR  USAGE POINTER.
...
EXEC CICS ADDRESS COMMAREA(WS-CPTR) END-EXEC.
IF EIBCALEN = 0 THEN
    DISPLAY 'no commarea flowed in'
END-IF.
SET ADDRESS OF DFHCOMMAREA TO WS-CPTR.
```

The shipped `runtime/cobol/gmpt.cob` (TRANSID `GMPT`) exercises `ADDRESS COMMAREA` and `ADDRESS EIB` alongside GETMAIN.

GETMAIN / FREEMAIN — `EXEC CICS GETMAIN`, `EXEC CICS FREEMAIN`

> **Dynamic task storage.** `GETMAIN` allocates a buffer from the task heap and writes a pointer to it into a `USAGE POINTER` item; `FREEMAIN` releases it. Together with `SET ADDRESS OF` they let a COBOL program build and address records whose size or count isn't known at compile time.

Format

```
EXEC CICS GETMAIN SET(ptr)
    {FLENGTH(n) | LENGTH(n)}
    [INITIMG(byte)] [SHARED] [NOSUSPEND]
    [USERDATAKEY | CICSDATAKEY]
    [RESP(rc)] [RESP2(rc2)]
END-EXEC

EXEC CICS FREEMAIN {DATA(area) | DATAPOINTER(ptr)}
    [RESP(rc)] [RESP2(rc2)]
END-EXEC
```

Description

`GETMAIN` allocates `n` bytes (`FLENGTH` preferred, `LENGTH` accepted as a synonym), fills every byte with `INITIMG` (first byte of the operand; default `0x00`), and writes the buffer's handle into the `SET` pointer. `n ≤ 0` is `LENGERR`. A program then maps a based LINKAGE 01 onto the buffer with `SET ADDRESS OF L TO ptr` and reads / writes through the based item.

`FREEMAIN` resolves its operand to a GETMAIN handle and releases the buffer. `DATAPOINTER(ptr)` names a pointer variable holding the handle directly; `DATA(area)` names the based data area whose current address is the handle. Freeing an address that was **not** obtained by GETMAIN (e.g. `ADDRESS OF` a declared item, or a stale / zero handle) is `INVREQ`.

In **REXX** both verbs are accept-and-no-op: `GETMAIN` still validates the length (`LENGERR` on `≤ 0`) and writes a non-zero dummy handle so control flow is preserved, but no real bytes back it; `FREEMAIN` returns `NORMAL` with nothing to free.

Options

Option	Verb	Effect
`SET(ptr)`	GETMAIN	**Required.** Receives the buffer handle.
`FLENGTH(n)` / `LENGTH(n)`	GETMAIN	Allocation size in bytes; `FLENGTH` wins if both appear.
`INITIMG(byte)`	GETMAIN	Fill byte (first byte of the operand). Default `0x00`.
`SHARED`	GETMAIN	Accepted for source compatibility — **see the loud deviation below; NOT region-scoped in bricks.**
`NOSUSPEND`	GETMAIN	Accepted no-op (bricks never suspends a GETMAIN).
`USERDATAKEY` / `CICSDATAKEY`	GETMAIN	Accepted no-ops (bricks has no storage-key model).
`DATA(area)` / `DATAPOINTER(ptr)`	FREEMAIN	Identifies the buffer to release.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	GETMAIN allocated and wrote the handle; FREEMAIN released the buffer.
LENGERR	22	GETMAIN length ≤ 0.
NOSTG	42	GETMAIN length exceeds the 16 MiB region storage limit — no buffer is allocated.
INVREQ	16	GETMAIN with no `SET`; or FREEMAIN of an address not obtained by GETMAIN (including a zero / NULL handle).

Bricks deviations (loud)

> **Bricks deviation — `SHARED` GETMAIN is NOT region-scoped.** Real CICS `SHARED` storage survives the allocating task and is reachable region-wide. Bricks frees **every** task GETMAIN buffer (SHARED or not) when the task's storage is dropped at task end — there is no region-scoped storage pool yet. The `SHARED` flag is accepted so ports compile, but a program relying on cross-task SHARED storage will not see it persist. **Use a TS queue or the CWA (`ADDRESS CWA`, `wrkarea=`) for genuinely cross-task data.**
>
> **Bricks note — pointer arithmetic is per-buffer.** GETMAIN handles are opaque interned `(buffer, offset)` pairs. `SET ptr UP BY n` / `DOWN BY n` adjusts the offset **within the same buffer** the pointer already addresses; it does not walk across separate allocations. A past-end offset is allowed until dereferenced (then a clean abend); below `0` or on a NULL pointer the `SET` itself diagnoses cleanly. See the COBOL pointer model.

Example (COBOL)

```
01 WS-PTR  USAGE POINTER.
LINKAGE SECTION.
01 GM-AREA BASED.
   05 GM-TEXT PIC X(16).
PROCEDURE DIVISION.
    EXEC CICS GETMAIN SET(WS-PTR) FLENGTH(16) INITIMG(' ') END-EXEC.
    SET ADDRESS OF GM-AREA TO WS-PTR.
    MOVE 'POINTER-OK' TO GM-TEXT.        *> writes into the GETMAIN buffer
    EXEC CICS FREEMAIN DATAPOINTER(WS-PTR) END-EXEC.
```

The shipped `runtime/cobol/gmpt.cob` (TRANSID `GMPT`) runs exactly this round-trip and paints the read-back via `SEND TEXT`.

INQUIRE / SET resources — `FILE`, `TASK`, `TRANSACTION`, `TERMINAL`, `PROGRAM`

> **Resource inquiry.** Beyond `INQUIRE SYSTEM` (SIT values), bricks honours `INQUIRE` / `SET` for five resource classes, in both the direct and browse access shapes. Status attributes return the IBM happy-path defaults; sizes and counts bricks actually tracks are real.

Format

```
Direct:  EXEC CICS INQUIRE <class>(name) <attr(var)>... [RESP(rc)] END-EXEC

Browse:  EXEC CICS STARTBROWSE <class> [RESP(rc)] END-EXEC
         EXEC CICS INQUIRE <class>(area) NEXT <attr(var)>... RESP(rc) END-EXEC   (repeat)
         EXEC CICS ENDBROWSE <class> END-EXEC

Set:     EXEC CICS SET <class>(name) <attr(value)>... [RESP(rc)] END-EXEC
```

`<class>` is one of `FILE`, `TASK`, `TRANSACTION`, `TERMINAL`, `PROGRAM`.

Description

Direct reads one named resource's attributes into host variables. **Browse** freezes the resource-name list at `STARTBROWSE` and walks it one resource per `INQUIRE … NEXT`: the class option (`INQUIRE FILE(area) NEXT`) receives the current resource's name and the attribute options receive its values; the cursor advances each

call and returns `END` at exhaustion. One browse per class per task. **SET** validates the named resource exists and returns `NORMAL` — see the loud deviation: it does **not** persist a state change.

Attribute accept-lists per class

Class	`INQUIRE` attributes	Real vs. default
`FILE(name)`	`ENABLESTATUS OPENSTATUS RECORDSIZE KEYLENGTH KEYPOSITION RECORDFORMAT ACCESSMETHOD BASEDSNAME`	`RECORDSIZE/KEYLENGTH/BASEDSNAME` are **real** (store catalog). `ENABLESTATUS=ENABLED`, `OPENSTATUS=OPEN`, `KEYPOSITION=0`, `RECORDFORMAT=VARIABLE`, `ACCESSMETHOD=VSAM` are defaults.
`TASK(id)`	`TRANSACTION FACILITY USERID RUNSTATUS PRIORITY`	`TRANSACTION/FACILITY/USERID` are **real**. `RUNSTATUS=RUNNING`, `PRIORITY=1` are defaults. `id` is the numeric `EIBTASKN` form.
`TRANSACTION(name)`	`STATUS PROGRAM PRIORITY TWASIZE PROFILE`	`PROGRAM/TWASIZE` are **real**. `STATUS=ENABLED`, `PRIORITY=1`, `PROFILE=DFHCICST` are defaults.
`TERMINAL(id)`	`ACQSTATUS SERVSTATUS USERID NETNAME`	`USERID/NETNAME` are **real**. `ACQSTATUS=ACQUIRED`, `SERVSTATUS=INSERVICE` are defaults.
`PROGRAM(name)`	`STATUS LANGUAGE PROGTYPE RESCOUNT USECOUNT LENGTH`	`LANGUAGE` (`COBOL/REXX/PLI`) and `USECOUNT` are **real**. `STATUS=ENABLED`, `PROGTYPE=PROGRAM`, `RESCOUNT=0`, `LENGTH=0` are defaults.

`SET <class>(name)` accepts the same class option and the documented mutation attributes (e.g. `SET FILE('x') OPENSTATUS(CLOSED)`, `SET TRANSACTION('x') STATUS(DISABLED)`).

Conditions

Condition	EIBRESP	Where
NORMAL	0	Direct INQUIRE / SET succeeded; `INQUIRE … NEXT` returned a resource; STARTBROWSE / ENDBROWSE succeeded.
END	83	`INQUIRE <class> NEXT` after the last resource — the browse is exhausted.
NOTFND	13	Direct INQUIRE / SET named a FILE, TASK or TERMINAL that doesn't exist.
PGMIDERR	27	Direct INQUIRE / SET PROGRAM named a program that doesn't exist (RESP2=1, as IBM).
TRANSIDERR	28	Direct INQUIRE / SET TRANSACTION named a transaction that doesn't exist (RESP2=1, as IBM).
INVREQ	16	Unknown resource class; an unknown attribute keyword; a direct INQUIRE with no resource name and no `NEXT`; `ENDBROWSE` with no open browse.
ILLOGIC	21	`INQUIRE <class> NEXT` with no `STARTBROWSE` active for that class.

Bricks deviations (loud)

> **Bricks deviation — status attributes are IBM defaults, not live state.** Bricks has no persistent resource-state model. Every status field (`OPENSTATUS`, `ENABLESTATUS`, `RUNSTATUS`, `ACQSTATUS`, `SERVSTATUS`, transaction / program `STATUS`) returns the IBM happy-path default. A program must not rely on `INQUIRE` to discover that a file is CLOSED or a transaction DISABLED — that state is never modelled. The attributes bricks genuinely tracks (`RECORDSIZE`, `KEYLENGTH`, `LANGUAGE`, `USECOUNT`, `TWASIZE`, `PROGRAM`, `USERID`, `TRANSACTION`, …) are real.
>
> **Bricks deviation — `SET` is acknowledge-only.** `SET FILE(…) OPENSTATUS(CLOSED)`, `SET TRANSACTION(…) STATUS(DISABLED)`, etc. validate that the named resource exists (`PGMIDERR` for

PROGRAM, `TRANSIDERR` for TRANSACTION, `NOTFND` for FILE / TASK / TERMINAL if not) and return `NORMAL`, but **do not persist** the change — the very next `INQUIRE` would otherwise contradict the SET. This is deliberate: bricks fails honestly rather than faking a state transition it can't keep.

Example (REXX) — direct + browse

```
ADDRESS CICS

/* Direct: this task's own attributes. */
EXEC CICS ASSIGN EIBTASKN(MYTASK) END-EXEC
EXEC CICS INQUIRE TASK(MYTASK) RUNSTATUS(RS) TRANSACTION(TR) RESP(RC) END-EXEC

/* Browse: walk every FILE. RC goes non-zero (END) at exhaustion. */
EXEC CICS STARTBROWSE FILE RESP(RC) END-EXEC
DO WHILE RC = 0
  FN = ''
  EXEC CICS INQUIRE FILE(FN) NEXT OPENSTATUS(OS) RECORDSIZE(RZ) RESP(RC) END-EXEC
  IF RC = 0 THEN SAY FN OS RZ
END
EXEC CICS ENDBROWSE FILE END-EXEC
```

The shipped `runtime/rexx/inqr.rexx` (TRANSID `INQR`) runs this direct-plus-browse pattern and paints the result with `SEND TEXT`.

Part 5. EXEC CICS file and queue commands

> **Part 5 begins here.** This part covers the persistent and ephemeral data services — VSAM-style KSDS files and the temporary- storage / transient-data queue families. Both surfaces are per-task and bound by the same SYNCPOINT mechanism described in Chapter 10.

Chapter 7. KSDS file commands

> **VSAM in Bricks.** Bricks's KSDS file surface is the application- programmer's view of the persistent record store. Each FILE name in `EXEC CICS READ FILE(name)` corresponds to a B+tree bucket in the embedded `data/files.boltdb` database. A separate `DEFINE CLUSTER` step is **optional**: the first `EXEC CICS WRITE` creates the bucket if it isn't already there, and subsequent `READ` / `REWRITE` / `DELETE` / `STARTBR` / `READNEXT` / `READPREV` / `RESETBR` / `ENDBR` see it. Records are opaque byte payloads; the application chooses the layout (typically a pipe-delimited group of fields, the convention every sample uses).
>
> Operators who want the classic mainframe lifecycle — declare a cluster with `DEFINE CLUSTER`, bulk-load it from a sequential file with `REPRO`, drop it with `DELETE`, enumerate the catalogue with `LISTCAT`, dump records with `PRINT` — should use the **IDCA** transaction described under *VSAM file management* below. Files created through IDCAMS and files auto-created by `EXEC CICS WRITE` are byte-identical: same bucket, same `_catalog` row, same `CEMT INQUIRE FILE` view.

These commands operate on a **single record**, identified by a key, in a CICS FILE. Each FILE is a bbolt bucket inside `data/files.boltdb`; record bodies are opaque bytes (the application chooses the layout). See *How file storage works* in the README for the on-disk model.

READ

Read a record from a KSDS by key.

Format

```
EXEC CICS READ FILE(name)
              {INTO(target) | SET(target)}
              RIDFLD(key)
              [UPDATE]
              [LENGTH(len)]
END-EXEC
```

Description

B+tree exact-key lookup, O(log n). The record bytes (whatever the application stored) come back into the target. `UPDATE` records a per-session lock on the key, gating a subsequent `REWRITE` on the same FILE.

When `LENGTH(var)` is a bare variable, the actual record length is written back.

Options

FILE(name) — *required* The CICS FILE name.

INTO(target) / **SET(target)** — *one required* Destination for the record body. `INTO` and `SET` are accepted interchangeably here.

RIDFLD(key) — *required* The key to look up.

UPDATE Record a per-session lock so a subsequent `REWRITE` can update this record.

LENGTH(len) Receives the record's actual length when the option is a bare variable.

Bricks deviation from IBM canonical

`LENGTH(var)` on `READ FILE` is OUTPUT-only, not input-as-max- buffer. Real CICS VSAM variable-length reads `LENGTH(var)` as a cap on the bytes returned and writes back the actual size; bricks ignores the input value and only writes back. The write-back-then- read-as-input shape would otherwise corrupt subsequent reads in a loop (`LENGTH(LEN)` on call 1 stores 4; call 2 silently caps a 13-byte record to 4). The bricks KSDS store is fixed-size-per- record so there is no operator-meaningful truncation here.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Record returned.
NOTFND	13	Key not present.
INVREQ	16	Missing required option, or invalid file name.
IOERR	17	Underlying store error.

Example

```
EXEC CICS READ FILE('CUSTOMERS')
              INTO(REC) RIDFLD(CKEY)
END-EXEC
IF EIBRESP = 13 THEN MSG = 'Customer not found'
```

WRITE

Insert a new record into a KSDS.

Format

```
EXEC CICS WRITE FILE(name)
               FROM(data) [LENGTH(n)]
               RIDFLD(key)
               [MASSINSERT]
               [RESP(var)] [RESP2(var)]
END-EXEC
```

`LENGTH(n)` is the IBM in/out parameter — caller's value caps the bytes written, returned value is the actual bytes stored. `MASSINSERT` is accepted for source compatibility; bricks's KSDS engine is uniformly fast so the flag is a silent hint.

Description

B+tree insert. The bucket for the FILE is created implicitly on first WRITE — there is no `EXEC CICS DEFINE FILE` step. `DUPREC` if a record with the same key already exists.

The write commits in a single bbolt `Update` transaction with `fsync` on success.

Options

FILE(name) — *required* Target FILE. Created if absent.

FROM(data) — *required* The record bytes to store.

RIDFLD(key) — *required* The key under which to store the record.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Record written.
DUPREC	14	A record with this key already exists.
INVREQ	16	Missing required option, or invalid name.
IOERR	17	Underlying store error.

Example

```
REC = NM || '|' || AD || '|' || CY || '|' || PH
EXEC CICS WRITE FILE('CUSTOMERS') FROM(REC) RIDFLD(CKEY) END-EXEC
```

REWRITE

Replace the record locked by the most recent `READ … UPDATE`.

Format

```
EXEC CICS REWRITE FILE(name)
                  FROM(data)
END-EXEC
```

Description

Overwrites the value at the key locked by the most recent `READ FILE … UPDATE` on the same FILE. Releases the per-FCB update lock at end of transaction.

> **Bricks deviation from IBM canonical:** `READ UPDATE` + `REWRITE` do **not** survive the `CECI PF5` boundary. Real conversational CICS programs hold the UPDATE lock across the operator's pause; `CECI` in bricks treats each `PF5` press as an implicit `SYNCPOINT`, which releases non-`HOLD` locks. Recommended patterns: type both verbs on one `PF5` input (assembled as a single command), or wrap the pair with `ENQ resource HOLD … DEQ resource` so the lock persists across the press. Outside `CECI` (REXX, COBOL or PL/I programs running as a normal task) the lock survives until the next `SYNCPOINT` or task end, matching real CICS.

Options

FILE(name) — *required* **FROM(data)** — *required*

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Record updated.
INVREQ	16	No prior `READ … UPDATE`, or the lock has been released.
IOERR	17	Underlying store error.

Example

```
EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(CKEY) UPDATE END-EXEC
PARSE VAR REC NM '|' AD '|' CY '|' PH
PH = NEWPHONE                                      /* mutate */
REC = NM || '|' || AD || '|' || CY || '|' || PH
EXEC CICS REWRITE FILE('CUSTOMERS') FROM(REC) END-EXEC
```

DELETE

Remove a record from a KSDS.

Format

```
EXEC CICS DELETE FILE(name)
                [RIDFLD(key) [KEYLENGTH(n) GENERIC [NUMREC(var)]]]
                [RESP(var)] [RESP2(var)]
END-EXEC
```

Description

If `RIDFLD` is supplied, deletes that key. Otherwise deletes the key locked by the most recent `READ … UPDATE` on the same FILE.

IBM-canonical GENERIC delete-by-prefix: with `KEYLENGTH(n) GENERIC`, every record whose key starts with the first `n` bytes of `RIDFLD` is deleted. The count goes back through `NUMREC(var)` when supplied; `RC = NOTFND` when no record matches.

Options

FILE(name) — *required*

RIDFLD(key) Optional. When omitted, the most recent READ-UPDATE key is used.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Record deleted.
NOTFND	13	Key not present.

Condition	EIBRESP	Cause
INVREQ	16	No RIDFLD and no prior READ-UPDATE; or invalid name.
IOERR	17	Underlying store error.

Example

```
EXEC CICS DELETE FILE('CUSTOMERS') RIDFLD(CKEY) END-EXEC
```

VSAM file management — the IDCA (IDCAMS) transaction

`IDCA` is the BRICKS equivalent of mainframe **IDCAMS / AMSERV** — an Access Method Services screen for declaring, loading, dropping, listing, and printing VSAM-style KSDS clusters without writing an application program. It is a built-in transaction (no entry in `runtime/transactions.conf` is required) and is **admin-only**: only signed-on users in the `admin` group see the screen.

Invocation

At the `>` prompt on any 3270 terminal, type:

```
IDCA
```

ENTER drops you into a SYSIN / SYSPRINT screen. Type one or more control statements in the SYSIN area, press ENTER to run them, and the SYSPRINT area shows classic `IDC0001I FUNCTION COMPLETED…`-style messages plus the highest condition code. `PF5` clears the input; `PF7` / `PF8` scroll the listing; `PF3` exits.

Supported commands

Command	Syntax
DEFINE CLUSTER	`DEFINE CLUSTER (NAME(name) [RECORDSIZE(avg max)] [KEYS(len off)] [INDEXED] [REPLACE])`
REPRO (load)	`REPRO INFILE(seq-file) OUTFILE(cluster) [KEYS(len off)] [SKIP(n)] [COUNT(n)]`
REPRO (unload)	`REPRO INFILE(cluster) OUTFILE(seq-file) [COUNT(n)]`
DELETE	`DELETE name [CLUSTER]`
LISTCAT	`LISTCAT [LEVEL(prefix)] [NAMES | ALL]`
PRINT	`PRINT INFILE(cluster) [CHARACTER | HEX | DUMP] [COUNT(n)] [FROMKEY(k)] [TOKEY(k)]`

Tokenisation is case-insensitive for keywords; values inside parens (file names, keys) are taken verbatim. A trailing `-` on a line is a continuation marker (JCL convention). Lines starting with `*` and text bracketed by `/* … */` are comments.

Sequential file convention

`REPRO INFILE(…)` and `REPRO OUTFILE(…)` name files **relative to** `runtime_dir` (`./runtime/` by default — see the `runtime_dir` knob in `bricks.cnf`). Subdirectories are allowed, so an operator can keep loads under `loads/`, dumps under `dumps/`, ad-hoc files under `tmp/`, etc.:

```
REPRO INFILE(tmp/orders.in)         OUTFILE(ORDERS)         KEYS(7 0)
REPRO INFILE(loads/jan/customers.in) OUTFILE(CUSTOMERS)     KEYS(7 0)
REPRO INFILE(ORDERS)                OUTFILE(dumps/orders.dat)
```

Bricks treats a name with a `.` in it as a sequential file and a name without one as a cluster, so subdirectory-bearing paths still pick LOAD vs UNLOAD direction correctly as long as the seq side has an extension. Each line of a sequential file becomes one record. The `KEYS(len off)` clause carves the key out of those bytes (length first, then offset — matching real IDCAMS); omit it and Bricks defaults to the cluster's declared `KeyMax` from `DEFINE CLUSTER`.

Parent directories are not auto-created. Writing to `OUTFILE(dumps/cust.dat)` when `runtime/dumps/` doesn't exist fails cleanly — `mkdir -p runtime/dumps` on the host first.

Security

Operator-supplied paths are sandboxed to `runtime_dir`. The resolver applies four independent traversal-defence layers before any file is opened: a per-segment lexical filter (alphanumeric + `_-.` only, rejects `..`, rejects leading-dot, rejects absolute paths and backslashes), a `filepath.Clean` round-trip check, an absolute-prefix invariant against `runtime_dir`, and a `filepath.EvalSymlinks` check that rejects symlinks whose target leaves the sandbox. Attempts like `INFILE(../etc/passwd)`, `INFILE(/etc/passwd)`, or `INFILE(symlink)` where the symlink points outside the root all return `IDC3009I` with condition code 8 and never reach `os.Open`. The IDCA transaction is also admin-gated at the screen level; the path sandbox is defence in depth on top of that.

Worked example

```
DEFINE CLUSTER (NAME(CUSTOMERS) RECORDSIZE(80 80) KEYS(7 0))
REPRO INFILE(tmp/customers.in) OUTFILE(CUSTOMERS) KEYS(7 0)
LISTCAT LEVEL(CUST)
PRINT INFILE(CUSTOMERS) CHARACTER COUNT(5)
```

`LISTCAT` produces a CEMT-style fixed-column listing:

```
CATALOG LISTING -- 1 ENTRY(S)
  NAME                 RECORDS  KEYMAX  RECMAX  LAST-MOD
  CUSTOMERS                 42       7      80  28May26 15:04
IDC0001I FUNCTION COMPLETED, CONDITION CODE IS 0
```

Condition codes

Same scale as classic IDCAMS:

Code	Meaning
0	success
4	warning (e.g. duplicate records bypassed on REPRO, empty LISTCAT)
8	error (file not found, file locked by another terminal, DEFINE without REPLACE on an existing cluster)
12	severe (parse error, I/O failure)

The highest code any single statement produced is reported on the final `IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS nn` line.

Lock behaviour and concurrency

`DELETE` and `REPRO` (load mode) check the file's lock state before proceeding. A file held under `EXEC CICS READ … UPDATE` by another terminal is reported as `IDC3009I ** name LOCKED BY term=Txxxx, NOT DELETED` with condition code 8 — the same refusal the CEDA VSAM purge screen produces. Locks held by the operator's own terminal are ignored (so a user can DELETE a file they themselves are mid-update on, if they really want to).

Every IDCAMS command produces one line in the audit log (`log/<timestamp>.log`):

```
idcams=DEFINE  target=CUSTOMERS term=T0001 user=admin status=OK detail="keymax=7 recmax=80"
idcams=REPRO   target=CUSTOMERS term=T0001 user=admin status=OK detail="source=customers.in reco
rds=42 skipped=0 dup=0"
idcams=DELETE  target=CUSTOMERS term=T0001 user=admin status=OK
```

Relationship to CEDA VSAM

`CEDA VSAM` and `IDCA` are two views of the same catalogue:

- **CEDA VSAM** — interactive table of every cluster with one-keystroke purge (`P` selector + secondary type-the-name confirmation). Use when you want to browse and purge a handful of files visually.
- **IDCA** — command-driven AMS. Use when you want to script a reproducible cluster lifecycle, declare metadata up front, or bulk-load from a sequential file.

Both go through the same audit logging and same lock checks; both operate on the same `data/files.boltdb` buckets that `EXEC CICS WRITE` populates from REXX, COBOL and PL/I.

Offline IDCAMS — the `idcams` CLI

`idcams` is the **standalone** counterpart to the IDCA transaction. Same engine, same control statements, same audit log format — but it opens `data/files.boltdb` directly from the shell, so the bricks region must be **stopped** while it runs. This is the classic mainframe batch-IDCAMS model: shut the region down, run a deck of DEFINE / REPRO / DELETE statements, bring the region back up.

```
idcams [-c bricks.cnf] [-d datadir] [-r runtimedir] -sysin deck.idcams
idcams [-c bricks.cnf] -e "DEFINE CLUSTER (NAME(X))"
idcams                                # reads SYSIN from stdin
```

Flags:

Flag	Purpose
`-c <path>`	path to `bricks.cnf` (default: `./bricks.cnf` then `../bricks.cnf`)
`-d <dir>`	data directory holding `files.boltdb` (overrides `bricks.cnf`)
`-r <dir>`	`runtime_dir` sandbox root for REPRO INFILE/OUTFILE (overrides `bricks.cnf`)
`-sysin <file>`	read IDCAMS control statements from this file
`-e <stmt>`	execute one inline statement and exit
`-logdir <dir>`	audit-log directory (default: `bricks.cnf` `log_location`)
`-q`	suppress the SYSIN echo on stdout (listing only)

Exit code is the highest IDCAMS condition code (0=OK, 4=warning, 8=error, 12=severe), so the CLI plugs into shell pipelines and CI scripts directly. A REPRO that bypassed duplicate records exits 4; a

LISTCAT on an empty catalogue exits 4; a DELETE of a missing file exits 8.

Worked example:

```
$ idcams -sysin reload.idcams
idcams — data=data  tmp=runtime/tmp  sysin=reload.idcams
------------------------------------------------------------------------
IDCAMS  SYSTEM SERVICES                                    BRICKS / KICKS
                                                       TIME: 14:21:08

   DEFINE CLUSTER (NAME(CUSTOMERS) RECORDSIZE(80 80) KEYS(7 0) REPLACE)
IDC0508I CLUSTER CUSTOMERS DEFINED  (KEYMAX=7, RECMAX=80)
IDC0001I FUNCTION COMPLETED, CONDITION CODE IS 0

   REPRO INFILE(customers.in) OUTFILE(CUSTOMERS) KEYS(7 0)
IDC0005I NUMBER OF RECORDS PROCESSED WAS 12450
IDC0001I FUNCTION COMPLETED, CONDITION CODE IS 0

IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0
$ echo $?
0
```

If the bricks region is still running, the CLI detects the bbolt lock contention and exits with code 16 plus a hint to use the IDCA transaction instead — no hang, no partial write, no chance of corrupting an in-flight region's view of the catalogue.

Chapter 8. KSDS browse commands

A **browse** walks a CICS FILE in B+tree key order. The browse runs inside a bbolt MVCC read transaction, so the cursor sees a stable point-in-time snapshot — concurrent `WRITE` / `REWRITE` / `DELETE` on the same FILE do not disturb an in-progress browse.

The browse is **per-task** and tied to one FILE. A program may have multiple browses open on different FILEs at once; a second `STARTBR` on the same FILE replaces the first (`STARTBR` is implicitly idempotent in CICS).

The dispatcher releases any cursor the program forgot to `ENDBR` via a `defer handler.CloseBrowses()` at task end.

> **Bricks deviation from IBM canonical:** browse cursors survive `SYNCPOINT`. Real CICS closes VSAM RPL browses at every `SYNCPOINT`; bricks browses are bbolt MVCC snapshots and remain valid until the matching `ENDBR` (or the dispatcher's task-end defer). Closing them at `SYNCPOINT` would be theater — the MVCC snapshot is already isolated from concurrent writers. This is what lets a `STARTBR` on `CECI PF5` #1 drive `READNEXT` on `PF5` #2 (each press is a per-PF5 implicit `SYNCPOINT`).

STARTBR

Open a browse cursor on a KSDS file.

Format

```
EXEC CICS STARTBR FILE(name)
                  [RIDFLD(start)]
                  [GTEQ | EQUAL]
                  [GENERIC]
                  [KEYLENGTH(n)]
                  [REQID(handle)]
                  [RESP(var)] [RESP2(var)]
END-EXEC
```

`REQID(handle)` is the IBM-canonical concurrent-browse handle: a program may hold several independent browses against the same file by tagging each STARTBR / READNEXT / READPREV / ENDBR / RESETBR with a distinct integer handle. Without `REQID` the file's single default cursor is used — backward-compatible with every existing bricks program.

Description

Opens a B+tree browse cursor on the file. With no `RIDFLD`, positions on the first key. With `RIDFLD` and `GTEQ` (the IBM default and bricks default), positions on the first key ≥ start. With `EQUAL`, requires an exact match (`NOTFND` if absent). With `GENERIC` and `KEYLENGTH(n)`, positions on (and walks only through) keys whose first `n` bytes match the first `n` bytes of `RIDFLD`.

Options

FILE(name) — *required*

RIDFLD(start) Starting key. Default: first key in the file.

GTEQ | **EQUAL** Comparison rule. Default `GTEQ`.

GENERIC Restrict the walk to keys whose prefix matches.

KEYLENGTH(n) With `GENERIC`, the prefix length to match.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Cursor opened.
NOTFND	13	`EQUAL` requested but no such key.
INVREQ	16	Missing required option, invalid name, etc.
IOERR	17	Underlying store error.

Example

```
EXEC CICS STARTBR FILE('CUSTOMERS')
                  RIDFLD('NY-')
                  GENERIC KEYLENGTH(3)
END-EXEC
```

READNEXT

Step the open browse cursor forward by one record.

Format

```
EXEC CICS READNEXT FILE(name)
                  {INTO(target) | SET(target)}
                  [RIDFLD(keyvar)]
                  [LENGTH(len)]
END-EXEC
```

Description

Advances the cursor and returns the next record (or the first record if this is the first `READNEXT` after `STARTBR`). When `RIDFLD` is a bare variable, the matching key is written back; when `LENGTH` is a bare variable, the actual record length is written back.

Returns `ENDFILE` past the last key (or past the end of the `GENERIC` prefix). Records that were deleted by a concurrent transaction between `STARTBR` and the read are skipped automatically (bounded forward loop, no goroutine-stack risk).

Options

FILE(name) — *required*

INTO(target) / **SET(target)** — *one required*

RIDFLD(keyvar) Receives the key of the record returned.

LENGTH(len) Receives the actual record length.

Bricks deviation from IBM canonical

`LENGTH(var)` on `READNEXT` (browse-family) is OUTPUT-only, not input-as-max-buffer. Real CICS reads the caller's `LENGTH(var)` as a cap on the bytes returned and writes back the actual size; bricks ignores the input value and only writes back. The write-back-then- read-as-input shape would otherwise corrupt subsequent reads in a loop (`LENGTH(LEN)` on call 1 stores 4; call 2 caps a 13-byte record to 4). The bricks KSDS store is fixed-size-per-record so there is no operator-meaningful truncation here. A `LENGTH(20)` literal on a 13-byte record returns the full 13 bytes (the literal is silently ignored).

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Record returned.
ENDFILE	20	Past the last key (or out of `GENERIC` prefix).
INVREQ	16	No `STARTBR` is open on this FILE.
IOERR	17	Underlying store error.

Example

See `STARTBR` example, then:

```
DO FOREVER
  EXEC CICS READNEXT FILE('CUSTOMERS') INTO(REC) RIDFLD(K) END-EXEC
  IF EIBRESP = 20 THEN LEAVE
  SAY K ':' REC
END
```

READPREV

Step the open browse cursor backward by one record.

Format

```
EXEC CICS READPREV FILE(name)
                  {INTO(target) | SET(target)}
                  [RIDFLD(keyvar)]
                  [LENGTH(len)]
END-EXEC
```

Description

Same writeback rules as `READNEXT`. Returns `ENDFILE` before the first key (or before the start of the `GENERIC` prefix). Useful for paginating backward through a key range.

Options

As for `READNEXT`.

Bricks deviation from IBM canonical

Same shape as `READNEXT`: `LENGTH(var)` is OUTPUT-only, not input-as-max-buffer. The write-back-then-read-as-input pattern would corrupt subsequent reads in a backward walk; bricks ignores the operator-supplied input value and only writes back the actual record size. Real VSAM variable-length permits LENGTH-as-max- buffer; the bricks KSDS-fixed model has no operator-meaningful truncation here.

Conditions

As for `READNEXT`, with `ENDFILE` meaning *before the first key*.

Example

```
EXEC CICS READPREV FILE('CUSTOMERS') INTO(REC) RIDFLD(K) END-EXEC
```

RESETBR

Reposition the cursor of an open browse without closing the underlying read transaction.

Format

```
EXEC CICS RESETBR FILE(name)
                 RIDFLD(start)
                 [GTEQ | EQUAL]
                 [GENERIC]
                 [KEYLENGTH(n)]
END-EXEC
```

Description

Cheaper than `ENDBR + STARTBR` when the program wants to jump within the same browse session — the read snapshot is preserved.

Options

As for `STARTBR`. `RIDFLD` is required.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Cursor repositioned.
INVREQ	16	No STARTBR is open on this FILE.
NOTFND	13	EQUAL requested but no such key.

Example

```
EXEC CICS RESETBR FILE('CUSTOMERS') RIDFLD('00500') END-EXEC
```

ENDBR

Close an open browse cursor.

Format

```
EXEC CICS ENDBR FILE(name)
END-EXEC
```

Description

Releases the cursor and the underlying bbolt read transaction. The dispatcher releases any cursor the program forgot to `ENDBR` at task end, but explicit `ENDBR` is recommended.

Options

FILE(name) — *required*

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Cursor closed.
INVREQ	16	No STARTBR is open on this FILE.

Example

```
EXEC CICS ENDBR FILE('CUSTOMERS') END-EXEC
```

Chapter 8a. File Control on `tmp_dir` Sequential Files

The browse verbs in Chapter 8 (`STARTBR` / `READNEXT` / `READPREV` / `RESETBR` / `ENDBR`) and the direct positional `READ FILE … RIDFLD` also operate on **sequential files** living in the `tmp_dir` sandbox — the same files `READQ TD` / `WRITEQ TD` and the REXX `LINEIN` / `LINEOUT` family read and write. The routing is transparent and existence-based: if `tmp_dir/<name>` exists, the browse verbs use the line-record / RBA-keyed sequential backend; otherwise they fall through to the KSDS bbolt store (Chapter 8). When the same name is present in BOTH back-ends, the sequential file wins on `STARTBR`, and bricks logs a `WARN` line at startup so the operator knows the routing.

Use this chapter when the program needs **non-FIFO** access to a sequential file — random-access reads by byte offset, backward walks, or jumping to a known position with `RESETBR`. Pure FIFO consumption stays cheapest via `READQ TD` (Chapter 9).

RBA semantics

The Relative Byte Address (RBA) is the **decimal byte offset of the start of a record**, expressed as plain ASCII text — consistent with `READQ TS NUMITEMS` and the rest of the bricks numeric conventions. A three-line file `"line1\nline2\nline3\n"` (each line is 5 bytes plus the LF terminator) has records at RBA 0, 6, and 12.

Bricks deviation from IBM canonical: real CICS ESDS exposes RBA as a 4-byte fullword (`PIC S9(8) COMP`). Bricks renders it as decimal text so REXX programs and COBOL `PIC X(n)` host variables both see a value they can arithmetic on without datatype conversion. The deviation is called out here and in `README.md` so a porter from a z/OS shop isn't surprised.

`STARTBR` (and `RESETBR`) with an RBA that falls **mid-line** rounds backward to the prior LF+1 so the next `READNEXT` returns the **containing** record rather than a garbled split. Mid-line `RIDFLD` on a direct `READ FILE … RIDFLD(rba)` rounds the same way. This matches the operator's mental model: "the byte I named lives inside record X; give me X."

Record boundary

Records terminate with a single LF (`0x0A`). The LF is excluded from the `INTO(area)` payload. A trailing line without LF is returned once; the next `READNEXT` returns `ENDFILE`. The byte-level encoding is the same as Chapter 9's TD queues — see "The `tmp_dir` sandbox" for the encoding rule (records are SBCS, the file is UTF-8 on disk, and `RBA` stays a byte offset into that file).

Bricks deviation: real ESDS uses an RDW (Record Descriptor Word) or fixed-length records, not LF-terminated lines. Bricks aligns the sequential format with the rest of the tmp_dir contract (REXX `LINEIN`, COBOL `READQ TD`) so one file is portable across all three I/O surfaces.

Verb subset

Verb	Supported?	Notes
`STARTBR`	yes	`RIDFLD` optional — defaults to BOF (= RBA 0). `EQUAL`, `GENERIC`, `KEYLENGTH` rejected with INVREQ.
`READNEXT`	yes	Returns next line; RIDFLD writeback = decimal RBA.
`READPREV`	yes	Walks backward; same RBA conventions.
`RESETBR`	yes	RIDFLD parsed as decimal RBA; backward-rounds same as STARTBR.
`ENDBR`	yes	Releases the FD.
`READ FILE … RIDFLD(rba)`	yes	One-shot positional read; LENGTH semantics same as READNEXT.
`UNLOCK FILE`	yes	No-op accepted (bricks has no record locks).
`WRITE FILE`	**no** — INVREQ "use WRITEQ TD"	
`REWRITE FILE`	**no** — INVREQ "use WRITEQ TD"	
`DELETE FILE`	**no** — INVREQ "use DELETEQ TD"	

LENGTH semantics (divergence from KSDS!)

On KSDS browse verbs (Chapter 8), `LENGTH(var)` is **output-only** — the store decides record size and writes it back to `var`. The recent KSDS fix made this explicit: treating LENGTH-on-READNEXT as an INPUT bound triggered the write-back-then-read-as-input bug that truncated record 2 to the length of record 1.

On `tmp_dir` sequential files, `LENGTH(var)` is **INPUT bound + OUTPUT actual** — the program's declared buffer capacity is honoured. If the record exceeds the buffer, the verb returns `LENGERR`, truncates the payload to the buffer size, and writes the **actual** (untruncated) record length back to `var`. The "actual not truncated" rule is the canonical real-CICS LENGERR semantic so a program can see how much was lost.

```
LEN = 80                              /* buffer cap */
EXEC CICS READNEXT FILE('orders.txt') INTO(REC) RIDFLD(K) LENGTH(LEN) END-EXEC
IF EIBRESP = 22 THEN                  /* LENGERR */
   SAY 'Record was' LEN 'bytes; lost' (LEN - 80) 'bytes after truncation'
```

RESP / RESP2 mapping

Condition	EIBRESP	Cause
NORMAL	0	Record returned (or `ENDBR` succeeded).
INVREQ	16	`EQUAL` / `GENERIC` / `KEYLENGTH` on sequential file; bad RBA (negative, parse failure); `WRITE` / `REWRITE` / `DELETE` against a tmp_dir name; missing `STARTBR` cursor.
LENGERR	22	Record exceeded `LENGTH(var)` buffer; payload truncated. Actual length writeback.

Condition	EIBRESP	Cause
NOTFND	13	File not present in tmp_dir at `READ FILE … RIDFLD(rba)`, or `STARTBR` opened OK but the RBA landed past EOF on the next `READNEXT`. Real-CICS `FILENOTFOUND` (RESP=12) is not separately surfaced; programs already test for NOTFND.
ENDFILE	20	`READNEXT` past end, or `READPREV` past beginning. (Both directions reuse one constant — matches real CICS.)
IOERR	17	Filesystem error (rare; the sandbox is local-only).

Interaction with `WRITEQ TD` mid-browse

Bricks's sequential backend is single-FD-per-cursor. When a concurrent task issues `WRITEQ TD` against the **same file** while this task holds a `STARTBR` open, the appended bytes ARE picked up: on each `READNEXT` whose internal position has reached the size we captured at `STARTBR`, the cursor re-`Stat`s the file descriptor. If the file has grown, the new bytes flow into the next `READNEXT`; otherwise, `ENDFILE` is returned as normal.

This is a documented deviation — real CICS holds a transient view of the dataset at `STARTBR` time and does not see mid-browse writes. Bricks chose the "moving target" rule so a long-running report transaction can stream a file that's still being written by a feeder task.

Real-CICS-veteran gotchas

- **RBA-is-decimal-text** — not a 4-byte `COMP` fullword. A REXX variable receiving RBA is a plain string; arithmetic works directly.
- **Record boundary is LF**, not a fixed-length record or RDW. A binary-mode browse is out of scope today.
- **STARTBR without RIDFLD** = BOF. Real CICS requires `RIDFLD`.
- **Browses survive** `SYNCPOINT` — same deviation already documented for KSDS (Chapter 8). Released by `ENDBR` or by the dispatcher's defer at task end.
- **Mid-browse writes are visible** via Stat-refresh-on-EOF. Real CICS doesn't do this.

Chapter 9. Temporary storage and transient data commands

Bricks supports two related queue families:

- **Temporary Storage (TS)** — ordered, persistent sequences of opaque byte items inside the bbolt database. Each queue is a sub-bucket whose keys are 8-byte big-endian item numbers (1, 2, 3 …) and whose values are the item payloads. Item-addressed; reads and writes are O(log N). Use for ephemeral state, scratch storage, producer/consumer chaining inside the database.

- **Transient Data extra-partition (TD)** — sequential text files in a sandboxed on-disk directory (`tmp_dir`). Line-oriented; read-once-and-advance / append-only semantics. Use for staging imports from text files into VSAM and exports out of it. The REXX `LINEIN` / `LINEOUT` / `STREAM` family hits the same backend, so a file written by COBOL is readable by REXX and vice-versa.

The encoding contract for TD files is strict: records are bricks SBCS (`0x09` TAB, `0x20–0x7E`, `0x80–0xFF`), stored as UTF-8 on disk; lines are terminated by a single LF (`0x0A`); CR (`0x0D`) and the other controls are rejected on write with `INVREQ`. The sandbox enforces a flat namespace under `tmp_dir` — no sub-directories, no leading dot, no `..`, no slashes in the queue name. See The `tmp_dir` sandbox below for the full rules.

READQ TS

Read an item from a temporary storage queue.

Format

```
EXEC CICS READQ TS QUEUE(name)
                   INTO(target)
                   [ITEM(n) | NEXT]
                   [LENGTH(len)]
                   [NUMITEMS(num)]
END-EXEC
```

`QNAME` is accepted as a synonym for `QUEUE`.

Description

`READQ TS` runs in one of two modes:

1. **Cursor mode** — advance the **per-task implicit cursor**: the first cursor-less READQ on a queue returns item 1, the second returns item 2, and so on. The cursor is keyed on the running TxCB and released when the task ends — a fresh invocation of the same TRANSID starts at item 1 again.
2. **Explicit-item mode** — `ITEM(n)` reads that exact item without touching the cursor. Useful for random-access reads where the program knows the item number in advance.

The mode is selected by IBM-canonical strict semantics. Cursor mode fires when `NEXT` is present OR when `ITEM` is absent. Explicit-item mode fires when `ITEM(n)` is present without `NEXT`; the variable's current value names the item to read and must be a positive integer. The four operator patterns are:

```
READQ TS QUEUE(q) INTO(d)                  — cursor mode; item number not returned
READQ TS QUEUE(q) INTO(d) NEXT             — cursor mode; same as above
READQ TS QUEUE(q) INTO(d) NEXT ITEM(N)     — cursor mode; writes item number to N
READQ TS QUEUE(q) INTO(d) ITEM(N)          — explicit mode; N must be a positive integer input
```

> **Bricks deviation from IBM canonical:** the per-task implicit cursor survives `SYNCPOINT`. Real CICS clears the read cursor on recoverable TSQs at every SYNCPOINT; bricks does not. The bricks design treats `SYNCPOINT` as a unit-of-work commit boundary that does not touch cursor state, browse cursors, TD handles, WEB sessions, or DOCUMENT tokens. This is what lets a `CECI` operator walk a queue across `PF5` presses (each press is a per-PF5 implicit `SYNCPOINT`), and what lets a long-running REXX program issue periodic `SYNCPOINT`s without rewinding its read loop.

Options

QUEUE(name) / **QNAME(name)** — *required*

INTO(target) — *required*

ITEM(n) | **NEXT** `NEXT` (or `ITEM` absent) selects cursor mode. `ITEM(n)` without `NEXT` is **strict input**: the variable must hold a positive integer or `INVREQ` fires with a short error pointing the operator at `NEXT`. `NEXT ITEM(var)` is the cursor-loop form — the resolved item number is written back into `var` on success.

LENGTH(len) Receives the actual item length.

NUMITEMS(num) Receives the queue's current high-water item count.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Item returned.
ITEMERR	26	`ITEM(n)` out of range, or implicit cursor past the last item.
QIDERR	44	Queue does not exist.
INVREQ	16	Missing required option, invalid name, or `ITEM(var)` (no NEXT) when `var` is unset / non-numeric.

Example

```
DO FOREVER
  EXEC CICS READQ TS QUEUE(QNM) INTO(REC) NEXT END-EXEC
  IF EIBRESP = 26 THEN LEAVE                /* end of queue */
  SAY REC
END
```

WRITEQ TS

Append a new item to a queue, or rewrite an existing item.

Format

```
EXEC CICS WRITEQ TS QUEUE(name)
                    FROM(data)
                    [ITEM(n) REWRITE]
                    [NUMITEMS(var)]
                    [MAIN | AUXILIARY]
                    [RESP(var)] [RESP2(var)]
END-EXEC
```

`QNAME` is accepted as a synonym for `QUEUE`. `NUMITEMS(var)` receives the queue's current item count after the write — IBM canonical; lets programs drive paging without a follow-up READQ. `MAIN` and `AUXILIARY` are storage-class hints from real CICS; bricks uses a uniform bbolt-backed store so both are accepted silently.

Description

In **append** mode (no `ITEM REWRITE`), the item is added at the next sequence number; an in-memory high-water-mark counter assigns the number without scanning. When `ITEM(var)` is a bare variable, the assigned item number is written back.

In **rewrite** mode (both `ITEM(n)` and `REWRITE` present), item `n` is replaced.

The write commits in a single bbolt transaction so a crash mid-write leaves either the prior state or the new one — never a partial.

Options

QUEUE(name) / **QNAME(name)** — *required* **FROM(data)** — *required* **ITEM(n) REWRITE** Both required for rewrite mode; either alone is rejected.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Item written.
ITEMERR	26	`ITEM(n) REWRITE` and item n does not exist.
INVREQ	16	Missing required option, invalid name, or `ITEM`/`REWRITE` not paired.
IOERR	17	Underlying store error.

Example

```
EXEC CICS WRITEQ TS QUEUE('AUDIT') FROM(MSG) ITEM(I) END-EXEC
SAY 'Wrote item' I
```

DELETEQ TS

Delete an entire temporary storage queue.

Format

```
EXEC CICS DELETEQ TS QUEUE(name) END-EXEC
```

Description

Drops the queue's sub-bucket and resets the in-memory counters and cursors. Subsequent `WRITEQ` will recreate it starting at item 1.

Options

QUEUE(name) / **QNAME(name)** — *required*

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Queue deleted.
QIDERR	44	Queue does not exist.

Example

```
EXEC CICS DELETEQ TS QUEUE('AUDIT') END-EXEC
```

READQ TD

Read the next line from a sequential text file in `tmp_dir`.

> **Need non-FIFO access?** `READQ TD` is the cheapest read for plain forward-sequential consumption. When the program needs random access by byte offset, backward walks, or repositioning by RBA on the same file, switch to the browse verbs against the same `tmp_dir` name — see Chapter 8a. File Control on `tmp_dir` Sequential Files. Both verbs read the same on-disk bytes, so a file written via `WRITEQ TD` is immediately browsable via `STARTBR`.

Format

```
EXEC CICS READQ TD QUEUE(name)
                   INTO(target)
                   [LENGTH(len)]
END-EXEC
```

`QNAME` is accepted as a synonym for `QUEUE`. `name` is a flat filename under `tmp_dir`; sub-directories and traversal are rejected with `INVREQ`.

Description

Reads the next LF-terminated line from `tmp_dir/name`, strips the terminating LF, and copies the payload into `target`. The file is auto-opened in read mode on the first call per task and the read cursor advances item-by-item across subsequent calls. A program that WRITEs then READs the same queue causes the handler to close the write handle and reopen for read; the read cursor restarts at line

1. The handle is closed automatically at task end.

Options

QUEUE(name) / **QNAME(name)** — *required*

INTO(target) — *required*

LENGTH(len) Receives the line's byte count (after the LF strip).

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Line returned.
QZERO	12	End-of-file. `target` is untouched; bricks closes the handle so a later READQ on the same queue rewinds to line 1.
QIDERR	44	File does not exist in `tmp_dir`.
INVREQ	16	Missing `INTO` / invalid name / sandbox rejection.
IOERR	17	Underlying file-system error.

Example

```
PERFORM IMPORT-ONE UNTIL DONE-FLAG = 'Y'.
...
IMPORT-ONE.
    MOVE SPACES TO REC.
    EXEC CICS READQ TD QUEUE('orders.sample.txt') INTO(REC) END-EXEC.
    IF EIBRESP = 12 THEN
        MOVE 'Y' TO DONE-FLAG
    END-IF.
    IF EIBRESP = 0 THEN
        PERFORM HANDLE-RECORD
    END-IF.
```

WRITEQ TD

Append a line to a sequential text file in `tmp_dir`.

Format

```
EXEC CICS WRITEQ TD QUEUE(name)
                    FROM(data)
                    [LENGTH(len)]
END-EXEC
```

`QNAME` is accepted as a synonym for `QUEUE`.

Description

Appends `data` plus a single LF to `tmp_dir/name`. The file is auto-opened in append mode (creating it if absent) on the first call per task. Trailing ASCII spaces on `data` are right-stripped before write — COBOL `PIC X(n)` values arrive padded to the right and the canonical text-file convention is to strip them. The payload is validated byte-by-byte: anything outside `0x09` (TAB), `0x0A` (LF), `0x20–0x7E` or `0x80–0xFF` (printable bricks SBCS, see Character data) causes the write to fail with `INVREQ` and a byte-offset diagnostic; the line is then written as UTF-8. CR (`0x0D`) is explicitly rejected — bricks emits Unix-style LF-only files.

Options

QUEUE(name) / **QNAME(name)** — *required*

FROM(data) — *required*

LENGTH(len) Accepted for syntactic compatibility; the actual byte count is `LENGTH(data)` after rstrip.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Line appended.
INVREQ	16	Missing `FROM` / invalid name / sandbox rejection / control byte / CR in payload.
IOERR	17	Underlying file-system error.

Example

```
DO I = 1 TO REC.0
   EXEC CICS WRITEQ TD QUEUE('export.txt') FROM(REC.I) END-EXEC
END
```

DELETEQ TD

Delete a sequential text file in `tmp_dir`.

Format

```
EXEC CICS DELETEQ TD QUEUE(name) END-EXEC
```

Description

Closes any handle the task has open on `name`, then unlinks the file. Already-gone is treated as success (matches the `DELETE FILE` forgiving semantics in bricks). Use to reset an export staging file before a fresh batch.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	File deleted (or already absent).
INVREQ	16	Invalid name / sandbox rejection.
IOERR	17	Underlying file-system error.

Example

```
EXEC CICS DELETEQ TD QUEUE('export.txt') END-EXEC.
```

The `tmp_dir` sandbox

`tmp_dir` is configured in `bricks.cnf`:

```
tmp_dir = runtime/tmp
```

Defaults to `runtime/tmp` (under `runtime_dir`) when the line is omitted. The directory is created at startup if missing.

Name rules. A queue name must match `[A-Za-z0-9._-]{1,255}` — no leading dot, no `..`, no slash, no backslash, no NUL. Bricks also runs `filepath.Rel` against every resolved path as defense-in-depth, so symlink shenanigans bounce too.

Encoding rules. Programs read and write bricks SBCS (see Character data); the file on disk is UTF-8. The write path accepts TAB, LF, `0x20-0x7E` and `0x80-0xFF`, rejects every other control byte with `INVREQ` on the first violation (naming the offending offset and hex value), and writes the UTF-8 form. The read path converts each line back; a line that is not valid UTF-8 (a Latin-1 file from another tool) is taken byte for byte.

`CHARIN` reads characters the same way: a byte that is not valid UTF-8 is kept as its Latin-1 character, so a Latin-1 file reads alike through `LINEIN` and `CHARIN`; only a character outside the code page becomes `?`. `CHARS()` counts file bytes, so it is an upper bound on the characters left. Lines terminate with LF only; CR is rejected on write. The read path splits on LF and preserves any CR bytes it finds verbatim — bricks does **not** silently strip them. If you need to import a file authored on Windows, pre-strip the CRs:

```
tr -d '\r' < windows.csv > runtime/tmp/clean.csv
```

Cross-language interop. Both COBOL (`READQ TD` / `WRITEQ TD`) and REXX (`LINEIN` / `LINEOUT` / `STREAM`) hit the same `*cics.TmpStore` backend. A file produced by one language is readable by the other; the only constraint is that both sides agree on the column format inside each line (canonical bricks convention: pipe-delimited).

Task-end cleanup. Every handle the program opens is closed automatically when the task ends. A program that forgets to call `STREAM CLOSE` or `DELETEQ TD` does not leak descriptors.

Chapter 10. Recovery and condition handling

This chapter covers two related families:

- **Unit-of-work commands** — `SYNCPOINT` and `SYNCPOINT ROLLBACK` group multiple file / TS mutations into one atomic step that can be committed or undone.
- **Non-local control commands** — `HANDLE CONDITION`, `IGNORE CONDITION`, `HANDLE AID`, and `HANDLE ABEND` arm program labels that the dispatcher branches to on a non-`NORMAL` response, on a particular AID key, or when the task abends. Together with the per-call `EIBRESP` test (Chapter 12) and REXX `SIGNAL ON ERROR` (Chapter 18) they cover the full range of CICS error-handling idioms.

How the unit of work works

Each task owns a **journal** of pending undo entries that lives on its `TxCB`. Every successful `WRITE` / `REWRITE` / `DELETE` and every `WRITEQ TS` / `DELETEQ TS` appends an inverse operation to the journal *immediately after* the underlying bbolt write commits.

- `SYNCPOINT` clears the journal — the work becomes irrevocably committed, and a new unit of work begins.
- `SYNCPOINT ROLLBACK` walks the journal in reverse and applies each inverse op (re-writing pre-images, re-creating deleted records, restoring queue contents, etc.), then clears the journal.
- `RETURN` performs an **implicit** `SYNCPOINT` before the task ends.
- `ABEND` performs an **implicit** `SYNCPOINT ROLLBACK` *unless* a `HANDLE ABEND` exit catches it; the exit may then choose whether to commit or roll back explicitly.

In-flight uncommitted writes are visible to concurrent tasks (this matches IBM CICS under `READ NO UPDATE`); the rollback model exists to recover *the current task* from its own partially-applied work, not to provide multi-task isolation.

How the condition / AID / abend traps work

After every `EXEC CICS` command, the dispatcher writes `EIBRESP` and then consults three optional per-task tables:

Table	Populated by	Consulted	Branches when
Condition map	`HANDLE CONDITION` / `IGNORE CONDITION`	After every command	`EIBRESP ≠ NORMAL` and a label is armed for that condition (or for `ERROR`).
AID map	`HANDLE AID`	After every command that updates `EIBAID` (`SEND MAP`, `RECEIVE MAP`, `RECEIVE`)	The new `EIBAID` matches an armed key (`PF1`–`PF24`, `PA1`–`PA3`, `ENTER`, `CLEAR`, or the catch-all `ANYKEY`).
Abend exit	`HANDLE ABEND`	When the task abends	An exit is armed and not cancelled.

When a trap fires, control transfers to the named label as if the program had `GO TO`'d (COBOL) or `SIGNAL`'d (REXX) it directly. The trap *stays armed* across the branch — re-arming after each fire is not required (and the `IGNORE` / bare-name disarm forms exist for when the program wants the trap to stop firing).

SYNCPOINT

Commit the current unit of work and begin a new one.

Format

```
EXEC CICS SYNCPOINT END-EXEC
```

Description

Clears the task's undo journal. All file / TS writes performed since the last `SYNCPOINT` (or since the task began) are made permanent; they can no longer be rolled back. A fresh, empty journal is started and subsequent mutations accumulate against it.

`SYNCPOINT` is implicit on `EXEC CICS RETURN` — programs that run to completion never need to call it explicitly. The explicit form is useful in long-running tasks that want to checkpoint partial progress, or in programs that mix recoverable phases with phases where rollback is no longer meaningful.

Options

None.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Journal cleared.

`SYNCPOINT` cannot fail in bricks because the underlying bbolt writes have already committed by the time the journal entry was appended.

Example

```
*  Phase 1: build the audit row, commit immediately so a later
*  validation failure does not undo the audit trail.
EXEC CICS WRITEQ TS QUEUE('AUDIT') FROM(WS-LOG) END-EXEC.
EXEC CICS SYNCPOINT END-EXEC.

*  Phase 2: real business work; rolled back if validation fails.
EXEC CICS REWRITE FILE('CUSTOMERS') FROM(WS-CUST) END-EXEC.
IF WS-INVALID
   EXEC CICS SYNCPOINT ROLLBACK END-EXEC
   EXEC CICS ABEND ABCODE('VAL1') END-EXEC
END-IF.
```

SYNCPOINT ROLLBACK

Discard the current unit of work and begin a new one.

Format

```
EXEC CICS SYNCPOINT ROLLBACK END-EXEC
```

Description

Walks the task's undo journal in reverse and applies each inverse operation:

- `WRITE` is undone by deleting the new key.
- `REWRITE` is undone by re-writing the captured pre-image.
- `DELETE` is undone by re-writing the deleted record.
- `WRITEQ TS` (append) is undone by deleting the appended item.
- `WRITEQ TS ... REWRITE` is undone by re-writing the pre-image.
- `DELETEQ TS` is undone by restoring every prior item from a snapshot taken before the delete.

After the walk the journal is cleared and a new unit of work begins.

`ROLLBACK` is implicit when `EXEC CICS ABEND` runs **without** a `HANDLE ABEND` exit. When an exit *does* intercept the abend, the exit code decides whether to call `SYNCPOINT ROLLBACK` (typical) or `SYNCPOINT` (commit partial work and continue).

Options

`ROLLBACK` is the only option — it is what distinguishes this form from plain `SYNCPOINT`.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	All journal entries successfully reverted.
ROLLEDBACK	82	One or more inverse ops failed (the underlying bbolt error is logged). The journal is still cleared; some writes may remain partially undone.

Example

```
ADDRESS CICS

EXEC CICS WRITE FILE('ACCT') RIDFLD(DEBIT_ID)  FROM(DEBIT_REC)  END-EXEC
EXEC CICS WRITE FILE('ACCT') RIDFLD(CREDIT_ID) FROM(CREDIT_REC) END-EXEC

IF DEBIT_TOTAL <> CREDIT_TOTAL THEN DO
   EXEC CICS SYNCPOINT ROLLBACK END-EXEC      /* both writes undone */
   EXEC CICS ABEND ABCODE('IMBL') END-EXEC
END

EXEC CICS SYNCPOINT END-EXEC                  /* commit both writes */
```

HANDLE CONDITION

Arm program labels to receive control on named EXEC CICS conditions.

Format

```
EXEC CICS HANDLE CONDITION cond1[(label)] cond2[(label)] ...
END-EXEC
```

Description

Builds a per-task map from condition name to label. After every EXEC CICS command, when `EIBRESP ≠ NORMAL`, the dispatcher consults the map:

1. If the *exact* condition name is armed, control branches to the named label.
2. Otherwise, if the catch-all `ERROR` is armed, control branches to that label.
3. Otherwise, the response code is left in `EIBRESP` for the program to test, exactly as if `HANDLE CONDITION` had never been issued.

The `(label)` form arms the trap; the bare condition name (no parens) **disarms** it. Arming a previously-armed condition replaces the old label.

`HANDLE CONDITION` is the legacy CICS error-handling style. Modern programs that use `RESP(rc)` style (or REXX `SIGNAL ON ERROR`) do not need it; the two styles can coexist within the same program because both ultimately read `EIBRESP`. A command that codes `RESP(var)` or `NOHANDLE` never fires an armed trap: the suppression lasts for that one command, as in IBM CICS.

> **Programming Note.** PL/I has no statement labels, so a PL/I handler target is a parameterless internal procedure of MAIN, and the task ends when it returns. See HANDLE CONDITION, HANDLE AID and HANDLE ABEND.

Options

condN(label) Arm `condN` (one of the names in Chapter 12. Response codes, e.g. `NOTFND`, `DUPREC`, `MAPFAIL`, `INVREQ`, `IOERR`, `LENGERR`, `QIDERR`, `ITEMERR`, `ENDFILE`, `PGMIDERR`, `EOC`) so it branches to `label`.

condN *(no parens)* Disarm `condN`.

ERROR(label) Arm a catch-all that fires for any non-`NORMAL` response not matched by a more specific arm.

A single `HANDLE CONDITION` may combine any number of arm and disarm options.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Map updated.
INVREQ	16	Unknown condition name.

Example

```
ADDRESS CICS

EXEC CICS HANDLE CONDITION NOTFND(NOREC)
                           DUPREC(DUP)
                           ERROR(OOPS)
END-EXEC

EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(CKEY) END-EXEC
SAY 'Found:' REC
EXIT

NOREC: SAY 'No customer with key' CKEY; EXIT 4
DUP:   SAY 'Duplicate key on insert';  EXIT 8
OOPS:  SAY 'Other CICS error, RC=' EIBRESP; EXIT 12
```

IGNORE CONDITION

Suppress the `HANDLE CONDITION` trap for one or more conditions.

Format

```
EXEC CICS IGNORE CONDITION cond1 cond2 ...
END-EXEC
```

Description

Marks each named condition as *ignored*: even if `HANDLE CONDITION` previously armed it, the trap will no longer fire and the program must test `EIBRESP` itself. `IGNORE CONDITION` stays in effect until a later `HANDLE CONDITION cond(label)` re-arms the same condition.

The distinction between *unarmed* (never armed, or disarmed by the bare-name form) and *ignored* matters only when `ERROR(label)` is armed: an unarmed condition still falls through to `ERROR`, but an ignored one does not.

Options

Each operand is a bare condition name — no parentheses, no label. Multiple conditions may be combined in a single call.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Map updated.
INVREQ	16	Unknown condition name.

Example

```
EXEC CICS HANDLE CONDITION ERROR(BADIO) END-EXEC.

*  We *expect* NOTFND on this READ — don't trip the ERROR trap.
EXEC CICS IGNORE CONDITION NOTFND END-EXEC.

EXEC CICS READ FILE('CUSTOMERS') INTO(WS-REC) RIDFLD(WS-KEY) END-EXEC.
IF EIBRESP = 13
   PERFORM CREATE-NEW-CUSTOMER
END-IF.
```

HANDLE AID

Arm program labels to receive control on a particular attention key.

Format

```
EXEC CICS HANDLE AID key1[(label)] key2[(label)] ...
END-EXEC
```

Description

Builds a per-task map from AID byte to label. After any command that updates `EIBAID` (`SEND MAP`, `RECEIVE MAP`, `RECEIVE`), the dispatcher consults the map and branches if the new `EIBAID` matches an armed key.

The bare key name (no parens) disarms a previously-armed key. `ANYKEY` is a catch-all matched by any AID for which no specific arm exists.

Options

keyN(label) / **keyN** `keyN` is one of `ENTER`, `CLEAR`, `PA1`–`PA3`, `PF1`–`PF24`, or `ANYKEY`. Parenthesised arms the trap; bare disarms it.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Map updated.
INVREQ	16	Unknown AID name.

Example

```
ADDRESS CICS

EXEC CICS HANDLE AID PF3(QUIT) PF12(QUIT) CLEAR(QUIT) END-EXEC

DO FOREVER
   EXEC CICS RECEIVE MAP('CUST1') INTO(SCR.) END-EXEC
   ... process ...
   EXEC CICS SEND MAP('CUST1') FROM(SCR.) ERASE END-EXEC
END

QUIT:
   EXEC CICS RETURN END-EXEC
```

HANDLE ABEND

Arm a program-level abend exit.

Format

```
EXEC CICS HANDLE ABEND { LABEL(label) | PROGRAM(name) | CANCEL | RESET }
END-EXEC
```

Description

Registers an exit that runs when the task abends (either via `EXEC CICS ABEND` or via an unhandled runtime error). The exit *replaces* the implicit `SYNCPOINT ROLLBACK` that an uncaught abend would perform — it is now the exit's responsibility to call `SYNCPOINT` or `SYNCPOINT ROLLBACK` explicitly.

`EIBABCODE` is set to the abend code (default `AAAA`) before the exit receives control.

Options

LABEL(label) Branch to `label` within the current program when the task abends. This is the form REXX programs almost always use.

PROGRAM(name) `XCTL` to the named program when the task abends. Currently accepted by the parser; the runtime treats `PROGRAM(name)` as equivalent to `LABEL(name)` (i.e. it branches inside the current program rather than transferring control).

CANCEL Disarm the current exit and restore the previous one (one level of nesting is preserved).

RESET Re-enable an exit previously suspended by `CANCEL`.

Exactly one of `LABEL`, `PROGRAM`, `CANCEL`, `RESET` must be given.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Exit registered or restored.
INVREQ	16	Conflicting / missing options, or RESET with no prior exit.

Example

```
ADDRESS CICS

EXEC CICS HANDLE ABEND LABEL(CLEANUP) END-EXEC

EXEC CICS WRITE FILE('LEDGER') RIDFLD(K) FROM(REC) END-EXEC
EXEC CICS WRITE FILE('AUDIT')  RIDFLD(K) FROM(REC) END-EXEC

/* something later may ABEND */
EXIT

CLEANUP:
   SAY 'Caught abend' EIBABCODE
   EXEC CICS SYNCPOINT ROLLBACK END-EXEC      /* undo both writes */
   EXEC CICS RETURN END-EXEC
```

Chapter 11. The Execute Interface Block (EIB)

The EIB is a per-task scratch area populated by the dispatcher before the program runs and by each `EXEC CICS` command on return. In bricks the EIB is exposed as a set of well-known variable names in the program's frame; both REXX and COBOL auto-inject the names that aren't already declared. PL/I predeclares them as fields of the `DFHEIBLK` structure, with fixed PL/I types (see The EIB and other predeclared names).

Field	Set by	Meaning
`EIBAID`	`SEND MAP` / `RECEIVE MAP`	Single-byte AID character of the most recent map response. Use `C2X(EIBAID)` (REXX) or compare with `X'F3'` (COBOL) to detect PF/PA/CLEAR/ENTER.
`EIBCPOSN`	`SEND MAP` / `RECEIVE MAP`	1-based cursor position of the most recent map response.
`EIBCALEN`	dispatcher (per-task entry)	Length of `DFHCOMMAREA` flowed in from the prior task or `LINK` caller. Zero when no COMMAREA was passed.
`EIBTRMID`	dispatcher	This terminal's id. REXX gets the full id (`T0001`); the COBOL (`PIC X(4)`) and PL/I (`CHAR(4)`) fields hold 4 characters as in the IBM EIB, so they read `T000`. `ASSIGN TERMID(...)` into an 8-character area returns the full id.
`EIBRESP`	every `EXEC CICS`	The response code of the most recent command (see Chapter 12).
`EIBRESP2`	every `EXEC CICS`	The secondary response code of the most recent command (currently always 0).
`RC`	every `EXEC CICS`	Mirror of `EIBRESP`, for REXX `IF RC <> 0` style.
`DFHCOMMAREA`	dispatcher / `LINK` / `RETURN`	The COMMAREA bytes; in COBOL, auto-injected as `PIC X(2000)` if not declared.

REXX

`EIBAID` and friends are ordinary REXX variables that the handlers write through the `cics.Frame` interface. A REXX program tests them exactly like any other variable:

```
IF EIBRESP <> 0 THEN SAY 'Command failed, RC=' EIBRESP
IF C2X(EIBAID) = 'F3' THEN EXEC CICS RETURN END-EXEC
```

`SIGNAL ON ERROR` (Chapter 18) is the alternative to per-call `IF EIBRESP <>`.

COBOL

The same names are auto-injected as `PIC` items if the program doesn't declare them (`cobol.ensureSystemItems`). Test them with ordinary `IF` clauses:

```
IF EIBRESP NOT = 0
   DISPLAY 'Command failed'
END-IF.

IF EIBAID = X'F3'
   EXEC CICS RETURN END-EXEC
END-IF.
```

There is no COBOL equivalent of `SIGNAL ON ERROR` yet.

PL/I:

```
EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(CKEY) RESP(RC);
SELECT (RC);
   WHEN (DFHRESP(NORMAL)) ;
   WHEN (DFHRESP(NOTFND)) MSG = 'Customer not found';
   OTHERWISE              MSG = 'I/O error, RESP=' || CHAR(RC);
END;
```

`RC` is an ordinary `FIXED BIN(31)` variable the PL/I program declares; only REXX and COBOL get an automatic `RC`.

Chapter 12. Response codes

After every `EXEC CICS` command the handler writes `EIBRESP`, `EIBRESP2`, and `RC` into the program's frame. The full set of constants is in `cics/resp.go`; the values a typical bricks program tests for are:

Constant	Value	Meaning
`NORMAL`	0	Success.
`ERROR`	1	Generic error.
`EOC`	6	End-of-chain (second `RECEIVE` in same task).
`NOTFND`	13	Record / key not found.
`DUPREC`	14	`WRITE` collided with an existing key.
`DUPKEY`	15	Duplicate key on a non-unique index (reserved).
`INVREQ`	16	Invalid request: bad option combination, missing data store, invalid name.
`IOERR`	17	Underlying store / network IO failed.
`NOSPACE`	18	Out of storage (reserved).
`NOTOPEN`	19	File not open (reserved).
`ENDFILE`	20	Browse walked past the last key (or out of `GENERIC` prefix).
`LENGERR`	22	Length mismatch.
`QZERO`	23	Queue is empty (reserved).
`ITEMERR`	26	TS item out of range; typical at end-of-queue.

Constant	Value	Meaning
PGMIDERR	27	LINK / XCTL target not in `transactions.conf`, or sub-program errored, or caller is not authorised for the target.
MAPFAIL	36	SEND MAP / RECEIVE MAP could not find or render the named map.
QIDERR	44	TS queue id invalid or unknown.

Idiomatic error handling

REXX, per-call test:

```
EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(K) END-EXEC
SELECT
   WHEN EIBRESP = 0  THEN NOP
   WHEN EIBRESP = 13 THEN MSG = 'Customer' K 'not found'
   OTHERWISE              MSG = 'I/O error, RC=' EIBRESP
END
```

REXX, signal-on:

```
SIGNAL ON ERROR NAME CICSERR
EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(K) END-EXEC
...
EXIT
CICSERR:
   SAY 'CICS error at line' SIGL ', RC=' RC
   EXIT
```

COBOL:

```
EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(CKEY) END-EXEC
EVALUATE EIBRESP
   WHEN 0   CONTINUE
   WHEN 13  MOVE 'Customer not found' TO MSG
   WHEN OTHER MOVE 'I/O error' TO MSG
END-EVALUATE.
```

Chapter 13. Commands not implemented

The following commands are not implemented and the parser does not recognize them at all:

Command	Use
GETMAIN, FREEMAIN	Dynamic storage. REXX has dynamic variables; COBOL has WORKING-STORAGE.
LOAD, RELEASE	Explicit program-fetch lifecycle. Bricks caches program text on first dispatch.
WAIT EVENT, POST	ECB-driven inter-task signalling — bricks's scheduler model does not expose ECBs.

> **Recently added in 2.7.x.** QUERY SECURITY and VERIFY PASSWORD both graduated from this deferred list — programs can now ask the four-axis authorisation question and re-authenticate a userid / password pair through the verbs described in Chapter 6 — Security inquiry.

DELAY

```
EXEC CICS DELAY
            {INTERVAL(hhmmss) | TIME(hhmmss) |
             FOR HOURS(h) MINUTES(m) SECONDS(s) MILLISECS(ms) |
             UNTIL HOURS(h) MINUTES(m) SECONDS(s)}
            [RESP(rc) RESP2(rc2)]
END-EXEC.
```

Suspends the issuing task until the given duration elapses (`INTERVAL` / `FOR`) or until the named time-of-day arrives (`TIME` / `UNTIL`). A bare `DELAY` with no clauses is a `NORMAL` yield. `INTERVAL` and `TIME` accept the same `HHMMSS` digit forms as `START`. `FOR` / `UNTIL` are the IBM-canonical compositional alternatives — supply any subset of `HOURS`, `MINUTES`, `SECONDS`, `MILLISECS`; the millisecond clause is the only sub-second form available.

Bricks sleeps the calling goroutine; PA1 break-out continues to work because the connection's prompt loop is unaffected. A long `DELAY` on a terminal that disconnects mid-sleep ends when the conn read deadline poisons the next dispatch — the task is not held alive past the connection.

CANCEL

```
EXEC CICS CANCEL
            {REQID(handle) | TRANSID(name) [TERMID(t)]}
            [RESP(rc) RESP2(rc2)]
END-EXEC.
```

Drops a pending `START` from the in-region scheduler. `REQID(handle)` matches against the 1–8-character handle the originating `START` supplied; bricks records the handle on the queued entry. `TRANSID(name)` drops the soonest pending START to that transid on `TERMID` (default = the issuing terminal). `RESP=NOTFND` (13) when no match.

The scheduler is in-memory; a bricks restart drops every pending entry, after which any `CANCEL` returns `NOTFND`.

ENQ / DEQ

```
EXEC CICS ENQ RESOURCE(name) [LENGTH(n)] [HOLD] [NOSUSPEND]
            [RESP(rc) RESP2(rc2)]
END-EXEC.

EXEC CICS DEQ RESOURCE(name) [LENGTH(n)]
            [RESP(rc) RESP2(rc2)]
END-EXEC.
```

Named-resource locking with region scope. `ENQ` blocks until the resource is acquired; with `NOSUSPEND`, an already-held resource returns `RESP=ENQBUSY` (55) immediately. A re-entrant `ENQ` by the same task on the same resource is a `NORMAL` no-op. The default lock scope is the next `SYNCPOINT` (commit or rollback) or task end; `HOLD` extends the scope past `SYNCPOINT` so the lock persists until an explicit `DEQ` or task end.

`LENGTH(n)` is accepted for source compatibility but bricks uses the entire resolved `RESOURCE` value as the key (uppercased, trimmed).

A non-`NOSUSPEND` `ENQ` that has waited five minutes for the holder to release returns `ENQBUSY` rather than blocking indefinitely. That cap converts a dropped-owner edge case into a clean recoverable status; legitimate same-region holders are never close to that limit.

Operator visibility — `CEMT INQUIRE ENQ`

`CEMT INQUIRE ENQ` (letter `E` on the INQUIRE menu) pages every enqueue currently held and every task suspended behind one. It answers the question a hung terminal raises: who holds the lock, and for how long.

Column	Meaning
`RESOURCE`	The uppercased, trimmed `RESOURCE` key. Names longer than 24 characters are truncated with a trailing `+`; the head is kept so the value still matches the `ENQ RESOURCE(...)` literal in your source.
`OWNER`	The owning task — the task number, else `TERM:<termid>` for a lock taken outside a task, else `ANON`.
`TRANID / TERM`	Joined from the live task table. A lock that outlives its task shows `–` rather than a stale name.
`STATE`	`OWNED` or `WAITING`. One `OWNED` row heads each resource, followed by its waiters, longest-waiting first.
`HOLD`	`YES` when the lock was taken with `HOLD` and so survives `SYNCPOINT` — usually the answer to "why is this lock still here?". – on a waiting row.
`AGE`	How long this task has held the lock, or has been waiting for it. A re-entrant `ENQ` does not reset it.

The panel is read-only and open to every operator, not just `ADMIN` — an enqueue is a resource, not internal control-block detail, and every column is already visible elsewhere via `INQUIRE TRANSACTION` and `INQUIRE TERMINAL`. `ENTER` re-snapshots, `PF7` / `PF8` page, `PF3` exits.

On an idle region the panel shows `No enqueues held.` — and that screen is live too: `ENTER` re-snapshots it, so an operator can sit on it and watch for a lock to appear, at which point the table takes over. `PF3` / `CLEAR` backs out.

IBM's `ENQTYPE` and `ENQSCOPE` columns are deliberately omitted rather than filled with a constant: bricks has only `EXECENQ` enqueues, and its locks are process-scoped, so a blank `ENQSCOPE` (which means region scope in real CICS) would assert something bricks cannot guarantee. `RETAINED` likewise never appears as a `STATE`, because there are no shunted units of work to retain a lock.

Because the snapshot is retaken on every `ENTER`, a refresh landing between a holder's `DEQ` and the woken waiter's acquisition can show a lone `WAITING` row with no `OWNED` row above it. That window is real and brief, not a rendering fault.

WRITE OPERATOR

```
EXEC CICS WRITE OPERATOR
            TEXT(area) [LENGTH(n)]
            [ROUTECODES(codes)] [NUMROUTES(n)]
            [EVENTUAL | ACTION | CRITICAL | IMMEDIATE]
            [CONSNAME(name)]
            [RESP(rc) RESP2(rc2)]
END-EXEC.
```

Writes `TEXT(area)` to the bricks console with the `SYS` tag. The issuing `TERMID` is included so the operator can correlate the message with the task that wrote it. `LENGTH(n)` bounds the bytes written. `ROUTECODES`, `NUMROUTES`, `CONSNAME`, `EVENTUAL`, `ACTION`, `CRITICAL`, and `IMMEDIATE` are accepted for source compatibility but fold into a single console write — bricks has no multi-console fan-out.

The `REPLY` form (`REPLY(var) MAXLENGTH(n) REPLYLENGTH(var) [TIMEOUT(n)]`) is rejected with `RESP=INVREQ`. Bricks has no operator-console input port to source the reply from, so accepting it silently would hang the

task. A program that needs operator acknowledgement must reach the operator through `SEND MAP` on a configured operator terminal.

SEND CONTROL

```
EXEC CICS SEND CONTROL [ERASE | ERASEAUP]
            [FREEKB] [ALARM] [FRSET]
            [CURSOR(pos)] [PRINT] [LDC(name)] [STRFIELD]
            [RESP(rc) RESP2(rc2)]
END-EXEC.
```

3270 control order without a data payload — used between a `SEND MAP DATAONLY` pre-paint and the next operator interaction to clear the screen, unlock the keyboard, or reset MDTs without re-painting any field content. `ERASE` and `ERASEAUP` clear the screen; the other flags are accepted but their wire-bits are not tunnelled by the underlying go3270 layer (`FREEKB` and `FRSET` are part of every paint's fixed WCC, so they have no separate effect; `ALARM` / `CURSOR` / `PRINT` / `STRFIELD` / `LDC` are silently accepted so program source compiles). A flag-less `SEND CONTROL` is a `NORMAL` no-op.

Recently added

- `CONVERSE` — see Chapter 4. Syntactic sugar for `SEND MAP` + `RECEIVE MAP`.
- `START` / `RETRIEVE` — see Chapter 5. Same-terminal scheduling with payload pass-through. Cross-terminal and no-`TERMID` (headless) STARTs are still deferred; the v1 surface rejects them with `RESP-INVREQ` and a clear message.
- `DELAY`, `CANCEL`, `ENQ` / `DEQ`, `WRITE OPERATOR`, `SEND CONTROL` — round-4 verbs documented above.
- `WEB *` (server-side, Phase 1) — see "EXEC CICS WEB" below. The client-side surface (`WEB OPEN / SEND / RECEIVE / CONVERSE / CLOSE`) and the DOCUMENT API land in Phase 2; URIMAP / TLS / mTLS in Phase 3.

Part 7. EXEC CICS WEB command reference

> **Part 7 begins here.** Bricks ships a complete `EXEC CICS WEB` family — both the **inbound** (server-side, the WAPI listener) and the **outbound** (client-side, programs calling external HTTP services). The `DOCUMENT` API for assembling response bodies from many fragments is documented here too, alongside the URIMAP catalogue, inbound mTLS, the `CONVERSE WEB` alias, and the dedicated copybooks.

EXEC CICS WEB — server side (Phase 1)

bricks ships an inbound HTTP listener that turns each matched request into a transaction dispatch via the `EXEC CICS WEB *` verbs. Enable it in `bricks.cnf` with `enable_wapi=yes`. Define routes in `runtime/web_routes.conf` — the URIMAP layer; the matched TRANSID is looked up in `transactions.conf` exactly as a 3270 operator's typed dispatch would. Both files reload on mtime change, so adding a new route requires no restart.

```
# method  path-pattern         transid  [groups]   [response_timeout]
GET       /api/customer/{id}   WAPI     public
POST      /api/customer        WAPC     admin
GET       /api/orders/{id}     OAPI     users
DELETE    /admin/cache/{key}   CACR     admin                5s
```

The matching `transactions.conf` rows route TRANSID → program (REXX or COBOL) — same format used by 3270 dispatch:

```
WAPI:rexx:wapi.rexx:public,users,admin
WAPC:cobol:wapic.cob:admin
OAPI:rexx:orders.rexx:users,admin
CACR:cobol:cacheclr.cob:admin
```

A single program is therefore reachable from **both** the 3270 prompt (the operator types its TRANSID) and the HTTP endpoint (a client hits the matching URL). The transaction distinguishes the front door by which family of verbs it calls — `EXEC CICS WEB *` returns INVREQ on a 3270 task, and `EXEC CICS SEND MAP` is meaningless on a web task — but most programs naturally use one set or the other.

A `{name}` placeholder in the path is exposed to the transaction via `WEB READ QUERYPARM('name')`, sharing one namespace with the inbound `?name=…` query string (path captures win on conflict).

A path capture is percent-decoded before the program sees it. The request path is matched in its *escaped* form, so a percent-encoded `/` can never split a segment and fool a single-segment placeholder — but the captured value is then decoded, because it shares a namespace with the already-decoded query string. `GET /api/customer/Smith%20John` against `/api/customer/{id}` therefore gives `WEB READ QUERYPARM('id')` the value `Smith John`, the same text `?id=Smith%20John` would have produced. (Before this, the capture arrived raw, so one program could see `id="Smith%20John"` next to `q="Smith John"` for the very same name and the record lookup found nothing.) A capture whose escape sequence is malformed makes the route not match, and the request 404s.

Authentication — the same `users.conf` that CSSN uses

A web client identifies itself using **HTTP Basic authentication**, and bricks verifies the credentials against the exact same `users.conf` and bcrypt hash that the `CSSN` sign-on transaction uses on the 3270. The credential carrier is different (an HTTP header instead of a 3270 password field); everything downstream is identical.

Layer	3270 / `CSSN`	WAPI / HTTP
Credential store	`runtime/users.conf`	`runtime/users.conf` (same file)
Hashing	bcrypt	bcrypt
Verification call	`auth.FileStore.Authenticate(user, pass)`	`auth.FileStore.Authenticate(user, pass)` (same call)
Credential carrier	Hidden `PASSWORD` field on the CSSN map, posted by SEND→RECEIVE	`Authorization: Basic base64(user:pass)` header
Operator-visible state	`sess.Authenticated=true`; UCB attached for the whole 3270 session until `CSSF LOGOFF`	One-shot per request — `sess.Authenticated=true` for the dispatch only
What the transaction sees	`EXEC CICS ASSIGN USERID(...)` returns the operator	Same — `EXEC CICS ASSIGN USERID(...)` returns the bcrypt-verified user

The dispatch flow per request:

1. **No `Authorization` header** — if the route's `groups` column contains the literal token `public`, the request is accepted as anonymous (`UserID=""`, `Authenticated=false`); otherwise the response is **401 Unauthorized** with `WWW-Authenticate: Basic realm="bricks"` so a browser shows the standard credential prompt.
2. `Authorization: Basic base64(user:pass)` — the credential is base64-decoded, split on `:`, then `auth.FileStore.Authenticate` runs (same path CSSN uses), comparing the password **exactly** — byte for byte, case included. On success the user's `users.conf` groups are matched against the route's groups: **any one** shared group admits the caller.
3. **Authenticated AND a group matches** — dispatch proceeds with `sess.Authenticated=true`, `sess.UserID=<username>`, `sess.Groups=<users.conf groups>`. The dispatcher's `IsAllowed` gate runs again against the TRANSID's `transactions.conf` groups, so **both** layers must permit the caller before the program runs.
4. **Authenticated but no group match** — **403 Forbidden**.
5. **Authentication fails** (wrong password, unknown user, malformed header) — **401 Unauthorized**.

ACL policy is **opt-in**: a `web_routes.conf` row with no `groups` column denies every request, log warning at boot. Add the literal `public` to permit anonymous access; add specific group names (`users`, `admin`, etc.) to require Basic-Auth against `users.conf` for a user belonging to that group.

Two properties of that column are easy to get wrong:

- **The list is a UNION, not an intersection.** `admin,users` admits anyone in *either* group, and `public` anywhere in the list makes the whole route unauthenticated — `admin,public` is exactly as open as `public` alone. There is **no way to spell "public AND admin"**; if the route is meant to be restricted, leave `public` out. bricks warns at load when a row mixes them.
- **A row has at most 5 fields, and a 6th is a hard error.** It used to be silently discarded, so `GET /admin ADMN public 5s admin` loaded as `Groups=[PUBLIC]` and served the admin route to

unauthenticated callers with a `200`. Such a row is now refused (and the route 404s, which is the safe direction to fail in). Every group goes in **one** comma-separated field with no spaces.

A user `alice` listed in `users.conf` as `alice:<bcrypt-hash>:users,admin` can run the **same** TRANSID from both front doors with the same credential:

```
curl -u alice:alice-password http://localhost:8080/api/orders/42
# → 200 + JSON body, sess.UserID=alice

curl -u alice:wrong-password http://localhost:8080/api/orders/42
# → 401 Unauthorized

curl http://localhost:8080/api/orders/42
# → 401 Unauthorized (no Authorization header)
```

…and at a 3270 emulator she signs on with `CSSN` → `alice` / `alice-password` and types the same TRANSID. The program sees the same `EIBTRMID / USERID / EIBCALEN` set and serves both callers identically. Most programs don't even need to know which front door they came from.

What's intentionally NOT yet supported:

- **Bearer tokens / cookies / sessions** — every request is stateless and re-authenticates against `users.conf`. There is no equivalent of the 3270 "stay signed on until CSSF LOGOFF" state across HTTP requests. The token-mint endpoint and `Authorization: Bearer …` flow land in Phase 3.
- **OAuth / JWT / OIDC** — Phase 3+ work.
- **Per-request authentication via the `CSSN` transaction itself** — `CSSN` is a 3270-only screen artefact; pointing a route at it doesn't produce the browser UX a credential prompt should have. The Phase 3 token endpoint will be the right shape for that.

Server-side verb set

The Phase 1 verb set:

Verb	Use
`WEB EXTRACT METHOD(var) PATH(var) …`	Request metadata: METHOD / SCHEME / HOST / PORT / PATH / QUERYSTRING / HTTPVERSION / CLIENTADDR / SERVERADDR. Each option is the name of a target variable.
`WEB READ HTTPHEADER(name) VALUE(var) [LENGTH(var)]`	Read one inbound header. `NOTFND` when absent.
`WEB STARTBROWSE HTTPHEADER` / `WEB READNEXT HTTPHEADER NAME(var) VALUE(var) [NAMELENGTH(var)] [VALUELENGTH(var)]` / `WEB ENDBROWSE HTTPHEADER`	Walk every inbound header in sorted order. `ENDFILE` when exhausted.
`WEB READ QUERYPARM(name) VALUE(var)`	Read query parameter OR routing-table `{name}` capture. Both are **percent-decoded** — `%20` reaches the program as a space. `NOTFND` when absent.
`WEB STARTBROWSE QUERYPARM` / `WEB READNEXT QUERYPARM …` / `WEB ENDBROWSE QUERYPARM`	Iterate every parameter / capture.
`WEB READ FORMFIELD(name) VALUE(var)`	Read one `application/x-www-form-urlencoded` field. Body parsed lazily on first call. A body that is not valid form encoding (a bad escape such as `%zz`, or a `;` separator) raises `INVREQ`, on every call for that request.

Verb	Use
`WEB STARTBROWSE FORMFIELD` / `WEB READNEXT FORMFIELD …` / `WEB ENDBROWSE FORMFIELD`	Iterate form fields. `STARTBROWSE` raises `INVREQ` on a malformed body, as `READ FORMFIELD` does.
`WEB RECEIVE INTO(var) [MAXLENGTH(n)] [LENGTH(var)] [TYPE(var)] [MEDIATYPE(var)] [BODYCHARSET(var)] [SRVCONVERT | NOSRVCONVERT | SERVERCONV(cvda) | CHARACTERSET(s) | INTOCCSID(n) | INTOCODEPAGE(s)]`	Read the request body, converted per the rules below. `MAXLENGTH`, `LENGTH` and `LENGERR` count the converted characters.
`WEB WRITE HTTPHEADER(name) VALUE(value)`	Set / append an outbound response header.
`WEB SEND FROM(buf) [LENGTH(n)] [MEDIATYPE(s)] [STATUSCODE(n)] [CHARACTERSET(s)] [SRVCONVERT | NOSRVCONVERT | SERVERCONV(cvda)] [HOSTCODEPAGE(s)]`	Emit response status + body. First `SEND` wins for status/MEDIATYPE and for the body conversion; subsequent `SEND` calls in the same task append to the body (matches CICS).
`WEB PARSE URL(s) SCHEMENAME(var) HOST(var) PORT(var) PATH(var) QUERYSTRING(var) [HOSTLENGTH(var)] …`	Direction-agnostic URL splitter.
`WEB CONVERTTIME DATESTRING(s) ABSTIME(var)`	RFC 1123 / 850 / asctime → milliseconds since 1970-01-01 UTC.

Body code page conversion

Program data is bricks SBCS (see Character data); HTTP bodies are usually UTF-8. The same rules apply to the server verbs (`WEB RECEIVE`, `WEB RETRIEVE DOCTOKEN`, `WEB SEND`) and the client verbs (`WEB SEND` / `CONVERSE` request bodies, `WEB RECEIVE` / `CONVERSE` responses). A body is *text-like* when its media type is `text/*`, `application/json`, `xml`, `javascript`, `ecmascript`, `x-javascript`, `x-www-form-urlencoded`, `xhtml+xml`, or any `+json` / `+xml` type; anything else (images, `octet-stream`, `multipart/*`) always passes through unchanged.

Into the program:

Content-Type of the received body	Program gets
text-like, `charset=utf-8`	converted to SBCS (? for characters outside it)
text-like, no charset	converted when the bytes are valid UTF-8, else taken as Latin-1 unchanged
text-like, `iso-8859-1` / `us-ascii` / `cp819` / `windows-1252`	unchanged (already SBCS)
other charset, not text-like, missing or malformed	unchanged
`NOSRVCONVERT`, `SERVERCONV(NOSRVCONVERT)`, `NOCLICONVERT`, `CLIENTCONV(NOCLICONVERT)`, `INTOCCSID(1208)`, `INTOCODEPAGE('UTF-8')` (on `WEB RECEIVE` also `CHARACTERSET('UTF-8')`)	unchanged (the raw UTF-8 bytes)

`BODYCHARSET(var)` returns the charset in effect (`utf-8`, `iso-8859-1`, the declared label, or blank for a binary body).

Out of the program: the target charset is `CHARACTERSET` or `CLNTCODEPAGE` (server and client alike), else the `charset` parameter of `MEDIATYPE`, else **UTF-8** (IBM's default is ISO-8859-1; bricks deviates so a browser shows ä without extra options). A UTF-8 target converts the body; a Latin-1 target sends the SBCS bytes as they are. A converted `MEDIATYPE` without a charset gets one appended (`application/json; charset=utf-8`); `CHARACTERSET` (server: also `CLNTCODEPAGE`) overrides a contradicting parameter. `NOSRVCONVERT` / `SERVERCONV(NOSRVCONVERT)` (client: `NOCLICONVERT` / `CLIENTCONV(NOCLICONVERT)`) send bytes unchanged and

leave `Content-Type` alone. Without `MEDIATYPE` (and no `Content-Type` written via `WEB WRITE HTTPHEADER`) the body is treated as text and converted, and no header is added.

> **Binary bodies:** always code `MEDIATYPE` (for example `MEDIATYPE('image/png')` or `'application/octet-stream'`) or `NOSRVCONVERT` / `NOCLICONVERT` when the body is not text. Without either, the default "treat as text" rule converts every byte from `X'80'` up to a two- or three-byte UTF-8 sequence and the binary data is corrupted. `LENGTH` cuts the SBCS characters before conversion. `SRVCONVERT`, `CLICONVERT` and `HOSTCODEPAGE` are accepted (they describe the default).

Header values, query parameters, form fields, `WEB EXTRACT` and `WEB PARSE URL` results reach the program as SBCS with lengths in characters; lookup names (`QUERYPARM('größe')`) are matched in UTF-8. `WEB WRITE HTTPHEADER` sends its bytes unchanged.

Malformed form bodies are an ordinary condition. `INVREQ` from `WEB READ FORMFIELD` / `WEB STARTBROWSE FORMFIELD` on a body that will not decode is a plain condition: code `RESP(...)` and answer the client with a 400. (Earlier releases ended the task with an internal error even when `RESP` was coded.)

```
EXEC CICS WEB READ FORMFIELD('title') VALUE(WS-TITLE)
          RESP(WS-RESP) END-EXEC
IF WS-RESP = DFHRESP(INVREQ)
   PERFORM SEND-400
END-IF
```

The full REXX sample is `runtime/rexx/wapi.rexx`; in COBOL the helpful copybooks are `DFHWBSC` (status-code constants), `DFHWBUH` (common header-name literals), `DFHWBMT` (MIME-type literals), and `DFHWBMETH` (HTTP method literals).

COBOL server-side sample — `todo.cob` (transaction `TODO`)

`runtime/cobol/todo.cob` is the first server-side COBOL WAPI sample: a multi-user, multi-list to-do REST API. One program serves every `/api/todo/...` route in `runtime/web_routes.conf` (transaction row `TODO:cobol:todo.cob:PUBLIC`; needs `enable_wapi=yes`) and works out the request from the method plus the shape of the path (`EVALUATE WS-SHAPE ALSO WS-METHOD`). What it shows:

- **VSAM storage** in three KSDS files created on first `WRITE` — `TODOUSR` (key user), `TODOLST` (user + list), `TODOITM` (user + list + item). Each user only ever sees their own data.
- **JSON both ways.** Every response object is built by `JSON GENERATE` from a small flat record (lower-case names via `NAME OF`, the item's `done` flag via `CONVERTING ... TO JSON BOOLEAN`); arrays are joined with `STRING ... WITH POINTER`. A JSON request body is read with `JSON PARSE`; form-encoded bodies still work through `WEB READ FORMFIELD`.
- **Units of work.** All input is validated before the first file update; an unexpected `RESP` after an update does `SYNCPOINT ROLLBACK` and answers 500. A record lock collision answers 409.
- `EXIT PARAGRAPH` for early returns from validation paragraphs.
- **Auth** by `X-Todo-User` / `X-Todo-Key` headers (accounts are made with `POST /api/todo/users`); the routes themselves are `public`.
- **Latin-1 titles.** Item titles take Latin-1 letters and the graphic-plane glyphs (`café`, `5 €`, ▉); the web edge turns the UTF-8 body into one byte per character, so the 80-character limit counts characters. User names, ids and passwords stay ASCII.

A browser front end lives in `runtime/web/todo/`. When the static web server is on (`webserver_port` > 0), the WAPI listener also serves the static doc root for paths no route matches, so the app opens at `http://host:`

`<wapi_port>/todo/` on the same origin as the API. `scripts/todo_api_test.bash [BASE]` runs an end-to-end test against a running instance (needs `curl` and `jq`).

EXEC CICS WEB — client side (Phase 2a)

The same `WEB` verb family also drives **outbound** HTTP — a bricks transaction calling a remote HTTP service. No `bricks.cnf` toggle is required; bricks builds one shared `*http.Client` at startup (tuned by `web_client_timeout` / `web_client_max_idle_conns` / `web_client_tls_skip_verify`) and every task reuses it. The host's CA bundle validates outbound HTTPS certificates by default — point at any normal public HTTPS endpoint and it just works.

The client uses a per-task **session token**: `WEB OPEN` returns a 32-character hex token; subsequent `WEB SEND` / `WEB RECEIVE` / `WEB CONVERSE` / `WEB CLOSE` all thread that token via `SESSTOKEN(...)`. A token still open at task end is auto-closed (dispatcher defer, mirrors `CloseBrowses` / `CloseAllTD`).

The Phase 2a verb set:

Verb	Use
`WEB OPEN HOST(s) [PORT(n)] [SCHEME(s)] SESSTOKEN(var)`	Open a logical client session. PORT defaults to 80 (HTTP) / 443 (HTTPS). SCHEME defaults to `http`. The token is written into the variable named by SESSTOKEN.
`WEB CONVERSE SESSTOKEN(t) METHOD(s) PATH(s) [FROM(buf)] [LENGTH(n)] [QUERYSTRING(s)] [MEDIATYPE(s)] INTO(var) [STATUSCODE(var)] [STATUSTEXT(var)] [MEDIATYPE(var)] [TYPE(var)] [MAXLENGTH(n)] [LENGTH(var)]`	One-shot send + receive. Builds the request, fires it through the shared client, writes the response body into INTO; LENGERR when the body exceeds MAXLENGTH (default 1 MiB).
`WEB SEND SESSTOKEN(t) METHOD(s) PATH(s) [FROM(buf)] [QUERYSTRING(s)] [MEDIATYPE(s)]`	Stage a request on the session (no round-trip yet).
`WEB RECEIVE SESSTOKEN(t) INTO(var) [STATUSCODE(var)] [STATUSTEXT(var)] [MEDIATYPE(var)] [MAXLENGTH(n)]`	Fire the staged request; populate the named output variables from the response.
`WEB CLOSE SESSTOKEN(t)`	Release the session and the underlying response body.
`WEB WRITE HTTPHEADER(name) VALUE(v) SESSTOKEN(t)`	Set an outbound *request* header on the session. Repeated WRITE calls for the same name produce multi-valued headers (matches `http.Header.Add`). Must precede the next SEND / CONVERSE -- headers committed at request build time.
`WEB READ HTTPHEADER(name) VALUE(var) [LENGTH(var)] SESSTOKEN(t)`	Read one response header by name from the most-recent SEND/CONVERSE/RECEIVE on the session. Case-insensitive name match. `NOTFND` when absent; `INVREQ` when no response has landed yet.
`WEB STARTBROWSE HTTPHEADER SESSTOKEN(t)` / `WEB READNEXT HTTPHEADER NAME(var) VALUE(var) [NAMELENGTH(var)] [VALUELENGTH(var)] SESSTOKEN(t)` / `WEB ENDBROWSE HTTPHEADER SESSTOKEN(t)`	Iterate every response header in sorted order. `ENDFILE` when exhausted; `INVREQ` when no STARTBROWSE has run on the session.
`WEB EXTRACT SESSTOKEN(t) [SCHEME(var)] [HOST(var)] [PORT(var)] [PATH(var)] [HOSTLENGTH(var)] [PORTNUMBER(var)]`	Inspect the session's resolved endpoint *after* OPEN. Distinct from `WEB EXTRACT URIMAP(name)` (no token) and from the server-side request-meta form (no token, different option set).

Bricks issues `Accept: */*` and `User-Agent: bricks-cics/1.0` on every outbound request by default; both can be overridden with `WEB WRITE HTTPHEADER('User-Agent') VALUE(...) SESSTOKEN(t)` before the next SEND / CONVERSE on that session. The `runtime/cobol/whdr.cob` sample (TRANSID `WHDR`) walks through the WRITE-then-READ flow end-to-end.

RESP codes: `NORMAL`, `NOTFND` (unknown SESSTOKEN), `INVREQ` (missing required option, web client not configured), `IOERR` (network / TLS / timeout), `LENGERR` (body over MAXLENGTH).

COBOL copybook for the client surface

`runtime/cobolcopy/DFHWBSI.cpy` carries the standard session-token + body buffer templates:

```
       01 DFH-WB-SESS     PIC X(36).
       01 DFH-WB-BODY     PIC X(8192).
       01 DFH-WB-URL      PIC X(2048).
```

Pair it with the existing `DFHWBSC` (status codes) and `DFHWBMETH` (HTTP method literals) so a CICS-style program reads idiomatically:

```
       COPY DFHRESP.
       COPY DFHWBSC.
       COPY DFHWBMETH.
       COPY DFHWBSI.

       EXEC CICS WEB OPEN HOST('api.github.com') PORT(443)
                          SCHEME('HTTPS')
                          SESSTOKEN(DFH-WB-SESS) END-EXEC.

       EXEC CICS WEB CONVERSE SESSTOKEN(DFH-WB-SESS)
                              METHOD(WB-GET)
                              PATH('/zen')
                              INTO(DFH-WB-BODY)
                              STATUSCODE(STAT) END-EXEC.

       EXEC CICS WEB CLOSE SESSTOKEN(DFH-WB-SESS) END-EXEC.

       IF STAT = DFHRESP-WB-OK ...
```

URIMAP — symbolic outbound endpoints (Phase 3a)

A URIMAP entry in `runtime/web_routes.conf` defines a named upstream service that outbound transactions reference by symbol. Both layers of `web_routes.conf` (routes for inbound, URIMAPs for outbound) live in the same file and hot-reload together.

```
# Format: URIMAP NAME scheme://host[:port][/path-prefix]
URIMAP   GITHUB    https://api.github.com
URIMAP   WEATHER   https://api.openweathermap.org/data/2.5
URIMAP   INTRANET  https://api.internal:8443/v1
```

Names are 1..8 uppercase characters (CICS resource-name limit), drawn from `A-Z`, `0-9` and the national characters `$ @ #` plus `_`. The charset is enforced at parse and at the `CEDA URIMAP` write path, so a name such as `A;B` — which used to load and then never match what a program asked for — is refused with `URIMAP name: only A-Z 0-9 $ @ # allowed`. The path-prefix is optional; when set, it becomes the leading segment of every PATH the program later supplies on `WEB SEND` / `WEB CONVERSE`. So `URIMAP WEATHER ...data/2.5` paired with `WEB CONVERSE PATH('/weather?city=NYC')` issues against `https://api.openweathermap.org/data/2.5/weather?city=NYC`.

The URIMAP-aware verb shapes:

Verb	Use
`WEB OPEN URIMAP(name) SESSTOKEN(var)`	Open a client session against the named endpoint. SCHEME / HOST / PORT come from the URIMAP row; explicit `HOST(...)` / `PORT(...)` / `SCHEME(...)` on the same command override per-field (matches IBM CICS). `NOTFND` when the name is unknown.
`WEB SEND URIMAP(name) METHOD(s) PATH(s) [FROM(buf)] [QUERYSTRING(s)] [MEDIATYPE(s)] INTO(var) [STATUSCODE(var)] [STATUSTEXT(var)] [LENGTH(var)] [MAXLENGTH(n)]`	One-shot OPEN + CONVERSE + CLOSE: no `SESSTOKEN` required. Resolves the URIMAP, opens a fresh session, sends the request, populates INTO and the named output variables from the response, and releases the session. Use for fire-and-forget outbound calls where session-token plumbing has no value. (Phase 3b.)
`WEB EXTRACT URIMAP(name) [SCHEME(var)] [HOST(var)] [PORT(var)] [PATH(var)] [HOSTLENGTH(var)] [PORTNUMBER(var)]`	Inspect a URIMAP definition without opening a session. Used by programs that branch on the resolved endpoint or need to display it to an operator. `NOTFND` when the name is unknown.

URIMAP rows load even when `enable_wapi=no` — outbound-only deployments are supported without exposing any inbound listener.

The `runtime/cobolcopy/DFHWBUM.cpy` copybook ships the conventional WORKING-STORAGE variables:

```
       01 DFH-WB-UM-NAME    PIC X(8).
       01 DFH-WB-UM-SCHEME  PIC X(8).
       01 DFH-WB-UM-HOST    PIC X(64).
       01 DFH-WB-UM-PORT    PIC X(5).
       01 DFH-WB-UM-PATH    PIC X(128).
```

Outbound TLS verification (Phase 3a)

By default the outbound HTTPS client verifies upstream certificates against the host system's trust store — the OS's `/etc/ssl/certs/...` bundle on Linux, the Keychain on macOS, Windows trust store on Windows. Three `bricks.cnf` knobs adjust:

Knob	Use
`web_client_ca_bundle`	Path to a PEM file of trusted CAs. When set, bricks uses ONLY these for verification — the system trust store is ignored. Useful for internal CAs that aren't installed system-wide.
`web_client_cert` + `web_client_key`	PEM client-cert and key for outbound mTLS. Presented to upstream services that require the caller to authenticate with a certificate. Both must be set together; one without the other is a startup error.
`web_client_tls_skip_verify`	`yes` accepts ANY upstream certificate — test-only escape hatch for self-signed endpoints. Logs a loud startup `WARNING`; never use in production.

Operator visibility (Phase 3a)

- `CEMT INQUIRE URIMAP` — pages the URIMAP rows loaded from `web_routes.conf` (NAME / SCHEME / HOST / PORT / PATH-PREFIX).
- `CEMT INQUIRE WEB` — pages the inbound HTTP requests currently in dispatch (METHOD / URI / TERM / TRANSID / USER / AGE). When no requests are in flight, the screen still shows a one-row summary with the lifetime request / error totals.

Phase 2a sample — `wzen.cob`

`runtime/cobol/wzen.cob` (TRANSID `WZEN`) is the shipped end-to-end demonstration. It fetches GitHub's `/zen` endpoint over HTTPS — a single-line plain-text response, no JSON parsing needed — and renders the result on the `WZEN1` 3270 map. The program demonstrates the full client lifecycle, branches cleanly on each `EIBRESP`, and surfaces network / HTTP failures with a distinct error path so the operator sees what went wrong on the 3270 screen rather than getting an opaque abend.

Drive it from a 3270 emulator: sign on (`CSSN` → `admin`), type `WZEN`, press ENTER. The first line under the header shows the actual URL fetched (`GET https://api.github.com/zen`) so the demo is self-documenting; the body appears on row 8.

Phase 2b sample — `whdr.cob` (custom request + response headers)

`runtime/cobol/whdr.cob` (TRANSID `WHDR`) is the canonical end-to-end demonstration of `WEB WRITE HTTPHEADER` and `WEB READ HTTPHEADER` on a client session. It opens the `GITHUB` URIMAP, sets two outbound headers (`User-Agent: bricks-whdr/1.0`, `Accept: text/plain`), issues `GET /zen`, then reads three response headers — two that always land on a GitHub response (`Content-Type`, `Server`) and one deliberately-absent header to exercise the `NOTFND` branch. The 3270 screen shows the request headers sent, the response headers received, and the body excerpt.

Drive it the same way as `WZEN`: sign on with `CSSN`, type `WHDR`, press ENTER. Pair the screen with `CEMT INQUIRE WEB` in a second 3270 session to watch the per-route counters move each time you rerun the transaction.

EXEC CICS DOCUMENT — chunked body builder

The `DOCUMENT` API is the buffer-builder that pairs with `WEB SEND DOCTOKEN(t)` for programs assembling large responses from many small fragments. Classic CICS pattern for emitting HTML / XML / JSON in pieces; the bricks implementation also serves as the chunk store for `WEB RETRIEVE DOCTOKEN(var)` when programs want the inbound body as a document handle instead of a flat `INTO(buf)`.

Verb	Use
`DOCUMENT CREATE DOCTOKEN(var) [SYMBOLLIST(s)] [LISTLENGTH(n)] [DELIMITER(s)]`	Allocate a fresh empty document; return its token in the named variable. Optional `SYMBOLLIST` seeds a `name=value&...` symbol table that subsequent INSERT SYMBOL(...) substitutes from.
`DOCUMENT INSERT DOCTOKEN(t) [FROM(buf) [LENGTH(n)] | TEXT(buf) | BINARY(buf) | SYMBOL(name) | DOCUMENT(other-token)] [AT(position)]`	Append (or splice with `AT(position)`) one chunk to the document. `SYMBOL(name)` substitutes from the bound symbol list (`NOTFND` when unknown); `DOCUMENT(other-token)` splices another document verbatim (`NOTFND` when unknown); `TEXT` and `BINARY` are codepage-aware in real CICS but collapse to `FROM` in bricks (single-codepage). `LENGERR` when the resulting buffer would exceed 4 MiB.
`DOCUMENT SET DOCTOKEN(t) SYMBOLLIST(s) [LISTLENGTH(n)] [DELIMITER(d)] [UNESCAPED]`	(Re-)bind the symbol list a subsequent INSERT SYMBOL substitutes from. Pass an empty `SYMBOLLIST` to clear.
`DOCUMENT RETRIEVE DOCTOKEN(t) INTO(buf) LENGTH(var) [CHARACTERSET(s)] [DATAONLY | HOSTCODEPAGE]`	Read the assembled body back into `INTO`; `LENGTH` receives the delivered length. The document is SBCS; `CHARACTERSET('UTF-8')` (or `'1208'`) delivers it as UTF-8 instead. Other `CHARACTERSET` values, `DATAONLY` and `HOSTCODEPAGE` are accepted and change nothing.
`DOCUMENT DELETE DOCTOKEN(t)`	Release the document. `NOTFND` when the token isn't known. Forgotten documents are released at task end automatically.

Verb	Use
`WEB SEND DOCTOKEN(t) [STATUSCODE(n)] [MEDIATYPE(s)]`	Emit the named document as the outbound response body. The DOCUMENT INSERT chain has already built it; SEND just splices the bytes onto BodyOut. `NOTFND` when the token isn't known.
`WEB RETRIEVE DOCTOKEN(var)`	Inbound-body-as-document. Allocates a fresh document, copies the request body into it, and returns the token. Alternative to `WEB RECEIVE INTO(buf)` for programs that prefer the document API.

The standard COBOL declaration:

```
       COPY DFHDCDOC.   *> DFH-DOC-TOK (PIC X(36)) + delimiter constants

       EXEC CICS DOCUMENT CREATE DOCTOKEN(DFH-DOC-TOK) END-EXEC.
       EXEC CICS DOCUMENT INSERT DOCTOKEN(DFH-DOC-TOK)
                                 FROM('<html><body>') END-EXEC.
       EXEC CICS DOCUMENT INSERT DOCTOKEN(DFH-DOC-TOK)
                                 FROM(BODY-CHUNK) END-EXEC.
       EXEC CICS DOCUMENT INSERT DOCTOKEN(DFH-DOC-TOK)
                                 FROM('</body></html>') END-EXEC.
       EXEC CICS WEB SEND DOCTOKEN(DFH-DOC-TOK)
                          MEDIATYPE('text/html')
                          STATUSCODE(200) END-EXEC.
```

RESP codes: `NORMAL`, `NOTFND` (unknown DOCTOKEN, unknown SYMBOL, unknown spliced DOCUMENT), `INVREQ` (missing required option), `LENGERR` (INSERT would exceed 4 MiB), `IOERR` (token generation failed).

EXEC CICS WEB — Phase 3b additions

Phase 3b closes out the master Phase-3 surface. Each addition slots onto the foundations laid by 3a (URIMAP catalogue, active- request tracking, outbound TLS extras) without breaking existing verb shapes.

`CONVERSE WEB` — alias for `WEB CONVERSE`

Some shops write the keywords in either order; bricks normalises both at the parser, so the two forms below dispatch identical:

```
       EXEC CICS WEB CONVERSE SESSTOKEN(T) METHOD('GET')
                              PATH('/x') INTO(B) END-EXEC.
       EXEC CICS CONVERSE WEB SESSTOKEN(T) METHOD('GET')
                              PATH('/x') INTO(B) END-EXEC.
```

`WEB EXTRACT TCPIPSERVICE` — inspect the receiving listener

When `WEB EXTRACT` carries the `TCPIPSERVICE` keyword (or the legacy `EXTRACT TCPIP` form without `WEB`), the verb reports the listener that received the inbound request:

Output	Source
`TCPIPSERVICE(var)`	Symbolic listener name — `WAPI` for the plain port, `WAPITLS` for the TLS port.
`PORTNUMBER(var)`	Numeric port the listener is bound to.
`IPADDRESS(var)`	The server's bind address (mirrors `bricks.cnf::dns_name`).
`CLIENT(var)`	Caller's IP address (mirrors `WEB EXTRACT CLIENTADDR`).
`AUTHENTICATE(var)`	`BASIC` when the request carried `Authorization: Basic …`, blank otherwise.

The legacy form drops the `WEB` keyword entirely — the parser recognises `EXTRACT TCPIP CLIENT(…) AUTHENTICATE(…)` and routes it through the same handler. Useful for programs ported from pre-CICS-TS web-services code.

Inbound mTLS + `WEB EXTRACT CERTIFICATE`

Set `web_inbound_client_ca=ca.pem` to require every TLS client to present a certificate signed by `ca.pem`. The TLS handshake rejects unauthorised clients before any verb runs — no transaction is dispatched at all. Inside the dispatched program, `WEB EXTRACT CERTIFICATE` reads the verified peer cert:

Output	Source on the peer cert
`COMMONNAME(var)`	Subject CN.
`ORGANISATION(var)`	Subject O.
`COUNTRY(var)`	Subject C.
`SERIALNUM(var)`	Hex serial number.
`ISSUER(var)`	Issuer CN.
`ISSUERORG(var)`	Issuer O.

`EIBRESP = NOTFND` on plain (non-TLS) requests and on TLS requests that carried no client cert — so one TRANSID can serve both authenticated and anonymous callers and branch on the resp.

```
       COPY DFHRESP.
       COPY DFHWBCC.   *> CN / ORG / CO / SERIAL / ISSUER buffers
       EXEC CICS WEB EXTRACT CERTIFICATE
                      COMMONNAME(DFH-WB-CN)
                      ORGANISATION(DFH-WB-ORG)
                      COUNTRY(DFH-WB-CO)
                      SERIALNUM(DFH-WB-SERIAL)
                      ISSUER(DFH-WB-ISSUER)
       END-EXEC.
       EVALUATE EIBRESP
           WHEN DFHRESP(NORMAL)
              *> Verified cert -- use the fields.
           WHEN DFHRESP(NOTFND)
              *> Plain HTTP or TLS without a client cert.
       END-EVALUATE.
```

HTTP/2 on the inbound TLS listener

Go's `net/http` server auto-negotiates HTTP/2 over the TLS listener via ALPN whenever `tls.Config.NextProtos` is left unset — which bricks does on the WAPI TLS port. No knob is required; clients that advertise `h2` in their ALPN list get HTTP/2, clients that don't fall back to HTTP/1.1. Verify with `curl --http2 -v https://localhost:443/...` and look for `ALPN, server accepted to use h2` in the trace output. Cleartext HTTP/2 (`h2c`) is not enabled — the plain port stays on HTTP/1.1 to avoid a transitive dependency on `golang.org/x/net/http2/h2c`.

`CEDA URIMAP` — live edit the URIMAP catalogue

The Phase 3a `CEMT INQUIRE URIMAP` browser becomes operator-editable in 3b. Type `CEDA URIMAP` (or `CEDA M`) from the 3270 prompt for the list view:

```
S  NAME       SCHEME    HOST                              PORT   PATH
_  GITHUB     https     api.github.com                    443    -
_  WEATHER    https     api.openweathermap.org            443    /data/2.5
_  INTRANET   https     api.internal                      8443   /v1
```

- Type `A` in the selector column for **Alter** — opens a form pre-loaded with the row's endpoint URL so a one-character edit is enough to repoint the URIMAP.
- Type `D` in the selector column for **Delete** — a Y/N confirmation overlay opens; the row is removed only when the operator types `Y`.
- Press **PF6** for **Define new** — opens an empty form for the NAME (1..8 chars) and ENDPOINT URL (`scheme://host[:port][/path-prefix]`).

Every committed change is written atomically through a temp + rename to `web_routes.conf`, with an mtime guard that refuses the write if the file was modified externally between when the screen loaded and when the operator pressed PF5. Each successful mutation logs an audit line in the bricks log:

```
ceda=URIMAP op=DEFINE user=ADMIN term=T0001 target=NEWAPI detail=https://internal.svc:8443/v2
```

so the audit trail of who changed which row when is greppable.

The new in-process surface that backs the screen lives on `*web.Table`: `AddURIMap`, `AlterURIMap`, `DeleteURIMap`, and `Mtime()` for the optimistic-concurrency guard. Programs in REXX or COBOL **do not call these directly** — they're the CEDA implementation, not part of the EXEC CICS surface.

Part 3. The REXX language

> **Part 3 begins here.** This part describes the REXX dialect bricks accepts and the small preprocessor that rewrites `EXEC CICS` blocks into the same parsed command shape COBOL produces. Every command in **Part 4** (`EXEC CICS`), **Part 6** (`EXEC SQL`) and **Part 7** (`EXEC CICS WEB`) is available verbatim from REXX with identical semantics.

Chapter 14. REXX program structure

A bricks REXX program is a flat sequence of statements with optional labels and procedures. Execution begins at the first statement; the program ends when execution falls off the bottom, or hits `EXIT`. A program file is plain UTF-8; see REXX character set for what a literal may hold. There is no fixed column discipline — newlines and whitespace are token separators.

```
/* HELO — minimum REXX program */
ADDRESS CICS
EXEC CICS SEND MAP('HELO1') ERASE END-EXEC
EXEC CICS RETURN END-EXEC
```

Comments

Block comments `/* ... */` may span multiple lines; nesting is not supported. A REXX program traditionally opens with a one-line header comment so the operator gets a description in CEDA / CEMT when applicable.

Statements

Statements are separated by newlines or `;`. The parser does not require a terminator after the last statement of a block. Labels have the form `NAME:` and may appear at any statement position.

String literals

A string is written in single or double quotes (`'…'` / `"…"`); a doubled quote inside is an escaped quote (`'it''s'`). A quoted string immediately followed by an `X` (hexadecimal) or `B` (binary) suffix — with no intervening blank — is a **hex / binary literal** whose value is the decoded bytes:

Literal	Value (bytes)
`'F3'X`	one byte `0xF3`
`'F1F2'X`	`0xF1 0xF2`
`'1'X`	`0x01` (an odd hex-digit count is left-padded with `0`)
`'0110'B`	`0x06` (left-padded to a full byte, then bit-packed)
`''X` / `''B`	the empty string

The suffix is recognised only when the `X` / `B` is a standalone letter: `'03'XY` is the string `'03'` abutted to the symbol `XY` (it concatenates with no blank; see Concatenation), *not* a hex literal. Invalid digits (`'GG'X`, `'2'B`) raise a SYNTAX error. Decoded literals are ordinary byte strings — `C2X('F3'X)` is `'F3'`, and `X2C` / `D2C` / `B2X` round-trip them.

REXX character set

Every REXX value — variables, stem tails, literals, built-in function arguments and results, and the text of `EXEC CICS` / `EXEC SQL` commands — is a string of bricks SBCS bytes: **one byte per character**, as on a mainframe. The code page is ISO-8859-1 (Latin-1) with a graphic plane in `X'80'-X'9F'` that holds, in byte order, € ▉ ▀ ▄ ▌ ▐ ■ – | ┌ ┐ └ ┘ ├ ┤ ┬ ┴ ┼ = ‖ • ← → ↑ ↓ ≤ ≥ ≠ ○ ◊ ¯ ≡. So `LENGTH('ÄÖÅ')` is `3`, `SUBSTR`, `LEFT`, `POS` and `PARSE` positions count characters, `C2X('Ä')` is `'C4'`, `D2C(196)` is `'Ä'` and `LENGTH('▉')` is `1` (`C2X` gives `'81'`).

Source text. Program files are UTF-8. The parser converts each string literal to SBCS once, when the program is loaded:

Where	Allowed
Inside `'…'` / `"…"`	ASCII, Latin-1 `U+00A0`-`U+00FF`, and the 32 plane glyphs above
Hex / binary literal	Raw bytes, never converted: `'C4'X` is Ä, `'C384'X` is two characters
Comments	Any character (the file must be valid UTF-8)
Symbols and operators	ASCII only

A UTF-8 byte order mark at the start of the file is ignored. Anything else is a load error that names the line, the column (a byte offset) and the character:

```
line 1 col 6: character U+4E2D in string literal is not in the bricks single-byte character set
line 1 col 9: invalid UTF-8 byte X'E9' in string literal (source must be UTF-8)
line 2 col 2: character U+00C4 is only allowed inside a string literal or comment
```

Columns count bytes of the UTF-8 file, starting at 1: in `SAY 'ÄÄ中'` each `Ä` takes two bytes, so the error is reported at col 10, not 8. An unterminated string is reported at the column of its opening quote (`line 2 col 5: unterminated string`).

Typographic dashes and quotes (– “), emoji and CJK are therefore allowed in comments only.

Edges. Conversion to and from UTF-8 happens only where data leaves or enters the program:

- `SAY` writes UTF-8 (`SAY D2C(255)` shows ÿ).
- `JSONGET` / `JSONCOUNT` hand the document to a UTF-8 JSON parser and convert string results back; a character outside the code page (for example `"\u4e2d"`) becomes `?`.
- `INTERPRET` runs a REXX value, which is SBCS already: it is **not** decoded a second time, so `INTERPRET "x='Ä'"` gives `LENGTH(x) = 1`.
- `EXEC CICS` / `EXEC SQL` text, `ARG()` and every value the CICS layer stores in a variable (`RECEIVE MAP`, `READQ`, `COMMAREA`, `EIBAID` …) are SBCS; the terminal, Postgres and HTTP edges convert.
- `LINEOUT` / `CHAROUT` pass the SBCS bytes to the `tmp_dir` stream layer, which stores the file as UTF-8 on disk.

Case. `UPPER()` / `LOWER()` (bricks extensions) fold the full Latin-1 alphabet: `UPPER('äöå')` is `'ÄÖÅ'`; ß, ÿ, µ, ÷, × and the graphic plane are unchanged. Everything IBM defines folds `a-z` only (per the TRL): `PARSE UPPER`, `ARG UPPER`, one-argument `TRANSLATE(s)`, and symbol / compound-tail folding. With `K = 'äb'`, `S.K` refers to `S.äB`.

Blanks and classes. Words, the = comparison's blank stripping and numbers recognise ASCII whitespace only; `X'A0'` (no-break space) is an ordinary character. `DATATYPE(s, 'A')` accepts ASCII letters and digits only, and `SYMBOL('Ä')` is `'BAD'`.

Error messages that quote program data are pure ASCII; non-ASCII characters appear as escapes, e.g. `not numeric: "\u00c4"`.

Differences from z/OS. The code page is Latin-1, not EBCDIC: `C2X('A')` is `'41'` (z/OS: `'C1'`), comparisons and `XRANGE` follow Latin-1 byte order, and `UPPER()` / `LOWER()` do not exist in IBM REXX.

Procedures

A label followed by `PROCEDURE [EXPOSE list]` defines a procedure with its own variable scope. `EXPOSE` re-routes named variables to the caller's frame recursively. Both simple names and stem roots (trailing dot) are accepted in the EXPOSE list. A procedure is dispatched by name from both the `CALL` statement and the `NAME(args)` function form; a `PROCEDURE` whose name matches a built-in shadows it (internal routines are resolved first, per standard REXX).

```
CALL GREET 'Alice'
EXIT

GREET: PROCEDURE EXPOSE LANG.
   PARSE ARG NAME
   SAY LANG.HELLO NAME
   RETURN
```

A procedure returns to its caller on `RETURN [expr]` or by falling off the bottom. The optional `expr` is assigned to the caller's special variable `RESULT`; a bare `RETURN` or a fall-off-end drops `RESULT` so the caller's existing value is not silently masked. Recursion is permitted (the only ceiling is the interpreter's `MaxSteps` runaway guard).

A procedure may be invoked either as a `CALL` statement or as a **function** in an expression — `Y = DOUBLE(21)`. In the function form the routine MUST `RETURN` a value; a bare `RETURN` or a fall-off-end raises a SYNTAX error ("function did not return data"). `RETURN ''` counts as data. (As a `CALL` statement, a value-less return is fine and simply leaves `RESULT` unset.)

The `NUMERIC` settings, the `ADDRESS` environment, and the `SIGNAL ON` / `CALL ON` condition traps are saved when a procedure is entered and restored on `RETURN`, so a routine may change them locally without leaking the change back to its caller.

```
SAY DOUBLE(21)               /* 42 — internal routine called as a function */
EXIT

DOUBLE: PROCEDURE
   PARSE ARG N
   RETURN N * 2
```

`ADDRESS`

`ADDRESS <env>` switches the active command handler. Bare strings inside an `ADDRESS` scope are commands routed to that handler; bricks ships a `CICS` handler. `ADDRESS` with no argument clears the active environment, after which a bare string is a syntax error.

```
ADDRESS CICS
"SEND MAP('CUST1') FROM(SCR.) ERASE"
"RETURN"
```

A command's return code is assigned to the special variable `RC`. A non-zero `RC` fires `SIGNAL ON ERROR` (if armed) — see Chapter 18.

Special variables

Name	Set by	Cleared by
`RC`	Every command (bare string) executed under an `ADDRESS` env, every `EXEC CICS` / `EXEC SQL`.	Survives until the next command updates it.
`RESULT`	`CALL fn …` when the called procedure does `RETURN expr`.	A bare `RETURN`, a fall-off-end, or `DROP RESULT`.
`SIGL`	Any direct `SIGNAL` and every condition trap (set to the source line of the failing statement).	Survives the trap; persists across normal flow.
`SQLCODE` / `SQLSTATE` / `SQLERRMC`	Every `EXEC SQL` verb.	Next `EXEC SQL`.

These names are otherwise ordinary writable variables; nothing stops a program from assigning them, though doing so is bad hygiene.

The `EXEC CICS` / `EXEC SQL` preprocessor

Bricks parses `EXEC CICS verb operand-list END-EXEC` (and the `EXEC SQL` form) as a single command at the same statement position a bare string would occupy under `ADDRESS CICS`. The two forms compile to the same dispatch path, so

```
EXEC CICS SEND MAP('HELO1') ERASE END-EXEC
```

and

```
ADDRESS CICS
"SEND MAP('HELO1') ERASE"
```

are equivalent. The `EXEC` form is preferred in shipped programs because it doesn't depend on the current `ADDRESS` environment and survives an explicit `ADDRESS` switch.

Chapter 15. Variables and stems

Naming rules

- Names are case-insensitive; internally everything is uppercased before lookup.
- A name begins with an ASCII letter or _ and may continue with letters, digits, _, or `!`, `?`, `#`, `@`, `$`. Stem-tail segments are separated by `.`.
- The literal `RC`, `SIGL`, `RESULT`, `SQLCODE`, `SQLSTATE`, `SQLERRMC` are reserved for runtime-managed values (see Chapter 14 — Special variables).

Simple variables and NOVALUE

An unset simple variable resolves to its own uppercased name — the classic REXX `NOVALUE` convention. `SAY X` when `X` has never been assigned prints `X`. If `SIGNAL ON NOVALUE` is armed (see Chapter 18) the reference traps instead of resolving to the name.

Assignment is `var = expr`; the right-hand side is evaluated once and stored verbatim (as a string — REXX has only one runtime type).

Stems

A stem is a variable name that ends in `.`. A bare assignment to the stem (`STEM. = value`) installs a default that any tail inherits when it has no explicit value. Per-tail values override the default:

```
STEM. = 'unset'
STEM.42 = 'forty-two'
SAY STEM.1    /* "unset"     -- default       */
SAY STEM.42   /* "forty-two" -- explicit tail */
```

Compound-symbol tail substitution

Non-numeric tail segments are resolved at every reference. With `J = 3`, the symbol `A.J` reads or writes `A.3`. The rules:

Tail segment shape	Behaviour
`STEM.42` (numeric)	Literal — the digit run becomes the tail directly.
`STEM.NAME` where `NAME` is set	Substituted — value of `NAME` becomes the tail.
`STEM.NAME` where `NAME` is **not** set (NOVALUE)	Literal — the bare uppercased name becomes the tail.
`STEM.I.J` (multi-segment)	Each segment substituted independently — with `I=1, J=2`, references `STEM.1.2`.

> The NOVALUE-stays-literal rule is the source of bricks's most common porting bug; see Appendix B for the standard pitfall.

`DROP`

`DROP name [name …]` removes one or more variables from the current frame. A trailing dot on a stem name drops the default AND every tail — useful for resetting an accumulator between paginated reads:

```
DROP RECS.            /* clear the whole stem    */
DROP A B C            /* drop three simples      */
DROP STEM.42          /* drop just one tail      */
```

`PROCEDURE` and `EXPOSE`

Inside a `PROCEDURE`, the parent frame's variables are invisible unless named on the `EXPOSE` list. `EXPOSE` accepts both simple names and stem roots:

```
GREET: PROCEDURE EXPOSE LANG. SUFFIX
   ...
```

`LANG.` exposes the parent's entire `LANG.` stem (default + all tails); `SUFFIX` exposes a single simple variable. Exposure is transitive — a procedure that exposes a name from a caller which also exposed it from its caller sees the outermost binding.

Chapter 16. Control flow

`IF` / `THEN` / `ELSE`

```
IF expr THEN stmt-or-block [ELSE stmt-or-block]
```

The `THEN` keyword is mandatory. Both arms accept either a single statement or a `DO ... END` block. Strict boolean evaluation: the condition must evaluate to `0` or `1`; anything else raises `SYNTAX` (so `IF X` where `X = 'foo'` is an error, not a truthy test).

```
IF AID = 'F3' THEN DO
   EXEC CICS RETURN END-EXEC
   EXIT
END
ELSE
   CALL REFRESH
```

`SELECT` / `WHEN` / `OTHERWISE` / `END`

```
SELECT
   WHEN expr THEN stmt
   WHEN expr THEN stmt
   ...
   OTHERWISE stmt-or-block
END
```

`WHEN` arms are tested in order; the first match's body runs and the rest are skipped. `OTHERWISE` is optional but if no `WHEN` matched and `OTHERWISE` is absent, the runtime raises `SYNTAX`. Use `OTHERWISE NOP` for an explicit no-op default.

`DO` family

Form	Behaviour
`DO ... END`	Group several statements as one body (most useful inside `IF` / `WHEN`).
`DO N ... END`	Repeat the body `N` times. `N` is any expression that evaluates to a non-negative integer.
`DO ctrl = start TO end [BY step] ... END`	Counted loop. `BY` defaults to 1. The loop runs while `ctrl` is between `start` and `end` (inclusive). `DO ctrl=1 TO 5 BY 0` is rejected at runtime (infinite-loop guard).
`DO WHILE expr ... END`	Pre-test loop — body runs while `expr` is truthy.
`DO UNTIL expr ... END`	Post-test loop — body runs at least once, then re-tests.
`DO FOREVER ... END`	Unbounded; exit only via `LEAVE` / `RETURN` / `EXIT` / `SIGNAL`.
`DO ctrl OVER stem. ... END`	Iterate `ctrl` over every set tail of `stem.`. Numeric tails are visited in numeric order, then non-numeric tails in lexicographic order. The default value (`stem. = …`) is not visited.

The control variable in `DO ctrl =` / `DO ctrl OVER` is set on each iteration. `LEAVE [ctrl]` and `ITERATE [ctrl]` target either the innermost loop (no argument) or the loop whose control variable matches the named argument:

```
DO I = 1 TO 100
   DO J = 1 TO 100
      IF GRID.I.J = '*' THEN ITERATE I  /* skip rest of inner */
   END
END
```

CALL / RETURN / EXIT

- `CALL name [arg1, arg2, ...]` — call a procedure or a built-in function in statement position. Arguments are expressions, evaluated left-to-right and made visible to the callee via `ARG(n)`. A `CALL` to a built-in throws away the returned value; a user procedure's `RETURN expr` populates `RESULT`. The same routine can also be called as a function in expression position (`x = name(args)`) — see Chapter 14.
- `RETURN [expr]` — return from a procedure (or terminate a top-level program). When `expr` is present and the caller was `CALL`, the value lands in `RESULT`; without `expr`, `RESULT` is dropped.
- `EXIT [expr]` — terminate the entire program immediately. The optional `expr` becomes the process exit code (clamped to a byte by the host OS); without it the program exits 0.

SIGNAL

- `SIGNAL label` — non-local jump to a label. Active `DO` / `SELECT` / `PROCEDURE` frames are unwound first. `SIGL` is set to the source line of the `SIGNAL` statement.
- `SIGNAL ON cond [NAME label]` — arm a condition trap (see Chapter 18).
- `SIGNAL OFF cond` — disarm.

INTERPRET

`INTERPRET expr` evaluates `expr` to a string and executes that string as REXX source in the current frame. Labels and procedures inside the snippet are scoped to that one evaluation (they do not leak into the surrounding program).

NUMERIC

Form	Effect
`NUMERIC DIGITS n`	Sets the precision used to round arithmetic results. Default 9.
`NUMERIC FUZZ n`	Sets the tolerance applied to numeric comparisons. Default 0.
`NUMERIC FORM SCIENTIFIC` / `NUMERIC FORM ENGINEERING`	Accepted syntactically but bricks always renders numbers in plain decimal — neither form changes output today.

Arithmetic is `float64` internally, then rounded to `NUMERIC DIGITS` significant figures before storing. The `%` and `//` operators (integer divide and modulo) bypass that rounding.

NOP, PUSH, PULL, QUEUE, OPTIONS, TRACE

- `NOP` — explicit no-op (clearer than an empty `DO ... END` in a `WHEN`).
- `TRACE [opt]` — parsed and accepted but ignored; bricks does not run an interpretive tracer.
- `OPTIONS …` — accepted and ignored (no recognised options).
- `PUSH` / `PULL` / `QUEUE` / `PARSE PULL` — accepted; the associated terminal data queue is always empty in bricks. Read forms return the empty string; write forms are silent no-ops. Use `EXEC CICS RECEIVE` for terminal input.

Bare strings

A bare string at statement position is a command sent to the current `ADDRESS` environment (see Chapter 14). Outside an active environment it is a syntax error.

```
ADDRESS CICS
"SEND MAP('CUST1') FROM(SCR.) ERASE"
"RETURN"
```

Chapter 17. PARSE templates

```
PARSE [UPPER] {VAR var | VALUE expr WITH | ARG | PULL} template
```

The source decides where the input string comes from; the template decides how it's chopped up and which pieces land in which variables.

Sources

Source	Input string
`VAR var`	Current value of the named variable. A compound name resolves its tail — `PARSE VAR airports.CODE …` parses the value of `airports.<CODE>`, not the literal name `AIRPORTS.CODE`.
`VALUE expr WITH`	The result of evaluating `expr`. The literal `WITH` keyword separates the source expression from the template.
`ARG [, ARG, …]`	The arguments the program (or procedure) was called with. Each comma-separated segment of the template is one argument, in order — `PARSE ARG H, W` peels `ARG(1)` into the first sub-template and `ARG(2)` into the second.
`PULL`	The terminal-input queue, which in bricks is always empty. Use `EXEC CICS RECEIVE` for real input.

`PARSE UPPER` uppercases the source string before splitting it across the template — convenient for case-insensitive command parsing.

Template elements

Element	Effect
Bare variable name	In a *run* of bare variables, all but the last get one blank-delimited word each (leading blanks skipped); the last variable gets the **verbatim** remainder, preserving internal and trailing blanks. A *single* variable with no following anchor receives the whole segment verbatim (leading blanks included). The word separator is the space character only. A dynamic compound target (`A.I`) resolves its tail at assignment time.
`.` (period)	Placeholder — consumes one token without binding it. Useful as a "skip" in a variable run.
`'literal'`	String anchor. Matches the next occurrence of `literal` in the source and splits there. The literal itself is discarded.
n (positive integer)	Absolute column marker — jumps to column n (1-based) in the source. Variables before the marker get everything from the previous column up to column n−1.
+n / −n	Relative column marker — moves the cursor n characters forward or backward from the current position.

Worked examples

```
/* Whitespace tokens. TID = first word, CKEY = second word,
 * '.' discards the rest of the line.                        */
PARSE VAR LINE TID CKEY .

/* Pipe-delimited fields.                                    */
PARSE VAR REC NM '|' AD '|' CY '|' PH

/* Fixed columns.                                            */
PARSE VAR ROW 1 NAME 21 ADDR 51 PHONE

/* Multi-arg PARSE — peels two call arguments.               */
GREET: PROCEDURE
   PARSE ARG NAME, GREETING
   SAY GREETING NAME
   RETURN

/* PARSE UPPER on user command, case-insensitive switch.     */
PARSE UPPER VAR LINE VERB REST
SELECT
   WHEN VERB = 'LIST' THEN CALL LIST_HANDLER REST
   WHEN VERB = 'ADD'  THEN CALL ADD_HANDLER  REST
   OTHERWISE SAY 'unknown verb' VERB
END

/* Compound tail resolves on both sides.                     */
I = 2
A.2 = 'hello world'
PARSE VAR A.I W1 W2            /* W1='hello', W2='world'      */

/* Whitespace preserved: the final variable keeps the run of */
/* blanks between arg1 and arg2 (older PARSE collapsed them). */
LINE = 'CMD   arg1  arg2'
PARSE VAR LINE VERB REST       /* VERB='CMD', REST='arg1  arg2' */
```

Not supported

Feature	Status
`PARSE LOWER`	Not parsed (only `UPPER` is). Lowercase the source explicitly with `LOWER(VAR)` then `PARSE VAR`.
`PARSE SOURCE`	Not wired — returns empty.
`PARSE VERSION`	Not wired — returns empty.
`PARSE NUMERIC`	Not wired.
Regex / pattern matching in templates	REXX has none; use `POS` / `LASTPOS` + `SUBSTR` for ad-hoc splits.

Chapter 18. Conditions and SIGNAL ON

```
SIGNAL ON {ERROR | NOVALUE | SYNTAX | HALT} [NAME label]
SIGNAL OFF cond
SIGNAL label
```

`SIGNAL ON cond [NAME label]` arms a trap. The optional `NAME` clause picks an explicit handler label; without it, bricks defaults the handler label to the condition name (so `SIGNAL ON ERROR` jumps to a label named

ERROR:).

When a trapped condition fires:

- SIGL is set to the source line of the failing statement.
- Control transfers to the handler label, unwinding active DO / SELECT / PROCEDURE frames first.
- The trap remains armed for the next occurrence.

SIGNAL OFF cond disarms.

Recognised conditions

Condition	Fires on
ERROR	An EXEC CICS or EXEC SQL command returns a non-zero RC (i.e. the bare-string command path under ADDRESS CICS).
NOVALUE	A reference to a simple or compound variable that has never been assigned.
SYNTAX	Any other interpreter error — bad numeric, divide by zero, unknown function, out-of-range subscript, ...
HALT	External halt request. Parsed and reserved; bricks has no fan-in for an external halt today, so the condition never fires.

FAILURE, NOTREADY, and LOSTDIGITS from standard REXX are not recognised — SIGNAL ON FAILURE parses but never fires.

Example

```
SIGNAL ON ERROR  NAME CICSERR
SIGNAL ON SYNTAX NAME OOPS

ADDRESS CICS
EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(CKEY) END-EXEC
EXIT

CICSERR:
   SAY 'EXEC CICS failed at line' SIGL ', RC=' RC
   EXIT 12

OOPS:
   SAY 'Interpreter error at line' SIGL
   EXIT 16
```

Without an armed trap, the legacy "test EIBRESP after every verb" pattern still works. For per-condition (rather than blanket) traps, use EXEC CICS HANDLE CONDITION (Chapter 10); the two styles can coexist.

Chapter 19. Built-in functions

The bricks REXX interpreter ships with the functions listed below. A CALL to any of these in statement position throws away the return value; the same name in expression position returns the value. Built-in lookups are case-insensitive. Per standard REXX, a user-defined PROCEDURE of the same spelling **takes precedence** over the built-in (the internal routine shadows it) — for both the CALL statement and the NAME(args) function form.

Length / index

Signature	Returns
`LENGTH(s)`	Byte length of `s`.
`POS(needle, hay [, start])`	1-based byte position of the first occurrence of `needle` in `hay` starting at `start` (default 1), or 0 if not found.
`LASTPOS(needle, hay [, end])`	1-based byte position of the last occurrence at or before `end` (default = end-of-string), or 0.
`WORDS(s)`	Number of whitespace-delimited words in `s`.
`WORDPOS(phrase, s [, start])`	1-based word index where `phrase` first appears as a sub-sequence of words, or 0.
`WORDINDEX(s, n)`	1-based character position of the start of word `n`, or 0 if `n` is past the end.
`WORDLENGTH(s, n)`	Length of word `n`, or 0.
`COUNTSTR(needle, hay)`	Non-overlapping occurrence count.

Substring / words

Signature	Returns
`SUBSTR(s, start [, length [, pad]])`	Substring starting at 1-based `start`. With `length`, the result is exactly `length` characters (right-padded with `pad`, default space). Negative `length` raises `SYNTAX`.
`LEFT(s, n [, pad])`	Leftmost `n` characters, padded on the right with `pad` (default space).
`RIGHT(s, n [, pad])`	Rightmost `n` characters, padded on the left with `pad`.
`SUBWORD(s, n [, length])`	Words `n` through `n+length−1` (default: to end). Single space between words in the result.
`WORD(s, n)`	Word `n`, or empty string.
`DELSTR(s, n [, length])`	`s` with `length` characters from position `n` removed (default: through end).
`DELWORD(s, n [, length])`	`s` with `length` words from word `n` removed.
`INSERT(new, target [, n [, length [, pad]]])`	Insert `new` into `target` after position `n` (default 0 = prepend), padded to `length` first.
`OVERLAY(new, target [, n [, length [, pad]]])`	Overwrite `target` at position `n` with `new`, extending `target` if needed.
`CHANGESTR(needle, hay, repl [, count])`	Replace every occurrence (or the first `count`) of `needle` with `repl`.

Whitespace / case

Signature	Returns
`STRIP(s [, opt [, ch]])`	Trim characters `ch` (default space) from `L` (left), `T` (right), or `B` (both, default).
`SPACE(s [, n [, pad]])`	Collapse internal whitespace; insert `n` copies of `pad` (default 1 space) between words. `n=0` removes whitespace entirely.
`CENTER(s, n [, pad])` / `CENTRE(s, n [, pad])`	Centre `s` in a field of width `n`; pad with `pad` (default space). Truncates equally from both ends when `s` is longer than `n`.

Signature	Returns
`JUSTIFY(s, n [, pad])`	Justify the words of `s` to both margins of width `n`: gaps are widened with `pad`, the extra going to the leftmost gaps first. If the single-blank form is longer than `n`, it is truncated (and a trailing blank dropped) first: `JUSTIFY('abcdef', 3)` is `'abc'`, `JUSTIFY('a b c', 2)` is `'a '`.
`UPPER(s)` / `LOWER(s)`	Latin-1 case conversion (ä <-> Ä); see REXX character set.
`REVERSE(s)`	Byte-reverse `s`.
`COPIES(s, n)`	`s` repeated `n` times.
`ABBREV(s, prefix [, minlen])`	Returns `1` if `prefix` is a prefix of `s` with at least `minlen` characters; else `0`.

Translation / bits

Signature	Returns
`TRANSLATE(s [, out [, in [, pad]]])`	Per-character map from `in` to `out`. With no `out`/`in`, uppercases. With only `out`, `in` defaults to `XRANGE()` (the full byte range), giving a per-position translation table.
`VERIFY(s, ref [, opt [, start]])`	1-based position of the first character in `s` (from `start`) that is not in `ref` (`opt='N'`, default) or that IS in `ref` (`opt='M'`); 0 if none.
`COMPARE(s1, s2 [, pad])`	1-based position of the first differing byte after pad-equalising lengths with `pad` (default space); 0 if equal.
`XRANGE([from [, to]])`	All bytes from `from` (default `'\x00'`) to `to` (default `'\xFF'`) inclusive.
`BITAND(s1 [, s2 [, pad]])` / `BITOR(...)` / `BITXOR(...)`	Bytewise logical operation; lengths equalised with `pad`.

Conversion

Signature	Returns
`C2X(s)`	Uppercase hex of every byte. Canonical way to compare `EIBAID` to a PF-key code: `IF C2X(EIBAID) = 'F7' THEN …` (PF7).
`X2C(hex)`	Decode `hex` to a byte string. An odd digit count gets a leading `0` (`X2C('F')` is `'0F'X`); `X2C('')` is `''`. Blanks are allowed between byte pairs only (`'4865 6C6C6F'`), not leading or trailing.
`D2X(n [, width])`	Decimal integer to uppercase hex, optionally zero-padded to `width`.
`X2D(hex [, n])`	Hex to decimal. With `n`, treats the value as an `n`-digit signed integer (two's complement when the top bit is set).
`C2D(s [, n])`	Byte string to decimal. With `n`, signed two's complement over `n` bytes.
`D2C(n [, len])`	Integer to byte string, optionally `len` bytes wide.
`B2X(bits)`	Bit string (`0`/1 chars) to hex.
`X2B(hex)`	Hex to bit string.

Numeric

Signature	Returns
`ABS(n)`	Absolute value.
`MAX(n, ...)`	Maximum of all arguments.
`MIN(n, ...)`	Minimum of all arguments.

Signature	Returns
`INT(n)`	Integer part (truncate toward zero).
`TRUNC(n [, places])`	Truncate to `places` decimal positions (default 0).
`MOD(a, b)`	Modulo. Same value as `a // b`.
`SIGN(n)`	−1 if n<0, 0 if n=0, +1 if n>0.
`FORMAT(n [, before [, after [, expp [, expt]]]])`	Numeric formatting with `before` characters in the integer part and `after` decimal places. `expp` / `expt` accepted syntactically but ignored.
`DIGITS()` / `FUZZ()` / `FORM()`	Return the current `NUMERIC DIGITS` / `FUZZ` / `FORM` settings.

Type / data

Signature	Returns
`DATATYPE(s)`	`'NUM'` if s parses as a number; `'CHAR'` otherwise.
`DATATYPE(s, 'N')`	`1` / `0` — is s numeric?
`DATATYPE(s, 'W')`	`1` / `0` — is s a whole number?
`DATATYPE(s, 'A')`	`1` / `0` — is s alphanumeric (letters, digits, no spaces)?

Date / time

Signature	Returns
`DATE()` / `DATE('N')`	Today as `dd Mmm yyyy`.
`DATE('S')`	Today as `yyyymmdd`.
`DATE('E')`	Today as `dd/mm/yy`.
`DATE('U')`	Today as `mm/dd/yy`.
`DATE('O')`	Today as `yy/mm/dd`.
`DATE('B')`	Days since `0001-01-01`.
`DATE('B', 'yyyymmdd', 'S')`	Basedate of a given date — subtract two basedates for an exact day delta.
`TIME()` / `TIME('N')`	`hh:mm:ss`.
`TIME('L')`	`hh:mm:ss.uuuuuu` with microseconds.
`TIME('S')`	Seconds since midnight.

Variables / arguments

Signature	Returns
`VALUE(name)`	Read the value of the variable named at runtime — useful for indirect reads (`VALUE('SCR.ROW'
`VALUE(name, newval)`	Set the variable named at runtime; returns the prior value. The two-argument form usually appears under `CALL VALUE 'SCR.ROW'
`ARG()`	Number of arguments the current procedure was called with.
`ARG(n)`	The nth argument (1-based), or empty if out of range.

Signature	Returns
`SYMBOL(name)`	Classifies a symbol: `'VAR'` if it names a variable that has a value, `'LIT'` if it is a valid symbol with no value assigned (or a constant symbol such as `77`, `.5` or `3D`, which can never be assigned), `'BAD'` if it is not a valid REXX symbol at all. Evaluation is two-step and stops there: the argument expression is evaluated once and the RESULT is the symbol name, so with `J = 3` `SYMBOL('J')` tests `J` while `SYMBOL(J)` tests the name `3`. Compound tails are derived exactly as an ordinary reference derives them, so `SYMBOL('A.J')` tests `A.3`. The answer is always three uppercase characters.
`ADDRESS()`	The name of the current host-command environment, uppercased — `CICS` unless an `ADDRESS` instruction changed it, and restored to the caller's value when a `PROCEDURE` returns. Note that `ADDRESS()` written alone as a whole statement is the ADDRESS *instruction*, not this function; use it inside an expression (`SAY ADDRESS()`).
`RANDOM([min [, max]])`	Random integer in `[min, max]` (default `[0, 999]`).
`ERRORTEXT(code)`	Echoes the numeric code as a string (placeholder — bricks does not maintain the standard REXX error-text table).

Stream I/O

The stream functions read and write text files in the `tmp_dir` sandbox — the same backend that powers COBOL's `EXEC CICS READQ TD` / `WRITEQ TD` / `DELETEQ TD` (see Chapter 9). The sandbox is strict: printable bricks SBCS characters only (see REXX character set), LF-terminated, no traversal. Files are stored as UTF-8 on disk.

Function	Form	Behaviour
`LINEIN(name)`	next line	Auto-opens `name` for read on first call; returns the next line (LF stripped). Empty string at EOF.
`LINEIN(name, 1)`	rewind	Closes and reopens `name`, then returns line 1. Only `1` is honoured; any other line number is treated as the "next line" form.
`LINEOUT(name)`	close	Closes the named stream. Returns `0`.
`LINEOUT(name, line)`	append	Auto-opens `name` for append on first call; appends `line
`LINES(name)`	EOF probe	Returns `1` while more data is available, `0` at EOF.
`CHARIN(name, start, n)`	character read	Reads `n` characters (default 1). `start = 1` rewinds first; other `start` values are ignored.
`CHAROUT(name, s)`	character write	Appends `s`. Same character/LF validation as `LINEOUT`.
`CHARS(name)`	bytes remaining	Returns the file size minus the current read position, in bytes of the UTF-8 file: an upper bound on the characters left.
`STREAM(name, 'S')`	state	Returns `'READY'`, `'NOTREADY'`, or `'ERROR'`.
`STREAM(name, 'D')`	description	Returns a short status string (`'OK'` / parse-time reason).
`STREAM(name, 'C', cmd)`	command	`OPEN READ` / `OPEN WRITE` / `OPEN APPEND` / `CLOSE` / `QUERY EXISTS` / `QUERY SIZE` / `DELETE`.

```
/* Append a row, then read every row back. */
CALL LINEOUT 'export.txt', 'C0000099|WIDGET-Z|7|9.95'
CALL LINEOUT 'export.txt'                   /* close the writer    */
DO WHILE LINES('export.txt') > 0
   SAY LINEIN('export.txt')
END
CALL STREAM 'export.txt', 'C', 'CLOSE'
```

Every handle a program opens is closed automatically at task end; an interpreter that forgets to call `CLOSE` does not leak descriptors. A sandbox violation (`'../etc/passwd'`, `'sub/x'`, leading-dot) causes the next `LINEIN` / `LINEOUT` to raise an error and `STREAM('S')` to report `'ERROR'`.

JSON

For programs that consume JSON — typically a REST response read with `EXEC CICS WEB CONVERSE` — two functions extract values by a dotted path. They parse the document on each call (fine for the small payloads a 3270 program renders).

Signature	Returns
`JSONGET(json, path)`	The **scalar** at `path` as a string. `path` is dot-separated; an **integer** segment indexes a JSON array (0-based). Returns `''` for a missing key, an out-of-range index, a JSON `null`, a value that is itself an object/array, or a parse error. JSON numbers render like REXX numbers (`671`, not `671.0`); booleans become `1`/`0`.
`JSONCOUNT(json, path)`	The number of elements in the **array** at `path` (the empty path `''` walks the whole document). `'0'` for a missing path, a non-array value, or a parse error.

```
/* RESP holds the WEB CONVERSE body */
N = JSONCOUNT(RESP, 'data')                /* array length            */
DO I = 0 TO N - 1
  P   = 'data.' || I
  FLT = JSONGET(RESP, P || '.flight.iata')  /* "BA178"                */
  DEP = JSONGET(RESP, P || '.departure.iata')
  ARR = JSONGET(RESP, P || '.arrival.iata') /* path-aware: dep != arr  */
END
EMSG = JSONGET(RESP, 'error.message')      /* '' if no error object   */
```

Limitations (acceptable for typical REST payloads): a key containing a literal `.` is unreachable; array indices are non-negative only (no wildcards, no negative indices); JSON integers beyond 2^53 lose precision (they decode through a float). There is no JSON *builder* — assemble output JSON with string concatenation, as the `WAPI` sample does. (COBOL programs have `JSON GENERATE` / `JSON PARSE` instead — see Chapter 22.)

> **Worked example.** The **SABR** reservation transaction (`runtime/rexx/book.rexx`) uses `JSONGET` / `JSONCOUNT` to parse live aviationstack flight data fetched with `EXEC CICS WEB CONVERSE` and paint departures / arrivals boards on the 3270 — see its `DO_FLIFO` handler, and the **Live flight info (FLIFO)** section of SABRE.md for the operator-facing `DO<apt>/D` commands it drives.

Operators

Listed lowest to highest precedence. All operators are left-associative except `**` which is right-associative.

Precedence	Operators	Notes
1 (lowest)	`|`	Logical OR. Strict boolean: both sides must evaluate to `0` or `1`. Short-circuit.
2	`&` / `&&`	Logical AND. Same strict-boolean rules. `&&` is the strict-AND alias. Short-circuit.
3	`=` `==` `\=` `\==` `<>` `><` `<` `>` `<=` `>=`	Comparison. The strict variants (`==`, `\==`) compare byte-for-byte; the plain forms compare as numbers when both sides parse as numeric (subject to `NUMERIC FUZZ`) and as trimmed strings otherwise. `\=` and `<>` and `><` are all "not equal".
4	`||`, blank, abuttal	Concatenation. `A || B` joins with no separator; `A B` (blanks between the terms) joins with exactly one blank; `A'/'B` (terms abutted, nothing between them) joins with no separator. See Concatenation below.
5	`+` `-`	Addition / subtraction.

Precedence	Operators	Notes
6	`* / % //`	Multiplication, real division, integer division (`%` — truncates toward zero), modulo (`//` — sign of the dividend).
7	unary `+ - \`	Arithmetic sign and logical NOT.
8 (highest)	`**`	Exponentiation. Integer exponents are exact (via repeated squaring); fractional exponents use float64.

Comparison precedence note: REXX has no separate equality vs. ordering precedence, so `A < B = C` parses as `(A < B) = C` — comparing the boolean result of `A < B` against `C`.

Concatenation

Two terms written next to each other with no operator between them are concatenated, as in IBM REXX. What separates them decides the result:

Written	Result	Rule
`'a' 'b'`, `'a'  'b'`	`a b`	One or more blanks: exactly **one** blank.
`'a'"b"`, `A'/'D`, `'LEN='LENGTH(X)`	`ab`, `abc/def`, `LEN=3`	Abuttal (nothing between): **no** blank.
`A/*c*/D`	`abcdef`	Only a comment between: abuttal.
`'a' /*c*/'b'`	`a b`	A blank anywhere between: one blank.
`'a',` (line end) `'b'`	`a b`	A continuation comma counts as a blank.
`'41'X'42'X`, `(1+2)'a'`, `STEM.I'x'`	`AB`, `3a`, `Sx`	Every kind of term abuts the same way.

Arithmetic binds tighter than concatenation: `1+2'a'` is `3a`.

A `(` after a term starts a function call in two cases:

- **Symbol, then** `(` — `LENGTH(X)` calls `LENGTH`. Internal routines (`NAME: PROCEDURE`) are searched first, then the built-ins; the name is case-insensitive.
- **Literal string immediately followed by** `(` — `'LENGTH'('abc')` is `3` (IBM TRL). The name is used exactly as written, *not* uppercased, and only built-in functions are searched: internal routines are skipped. This is the IBM idiom for reaching a built-in that an internal routine of the same name shadows. `'length'('abc')` and `'x'('y')` therefore fail with `unknown function`. With a blank between them, `'x' ('y')` is an ordinary blank concatenation, `x y`.

After any other term, `(` simply abuts: `(A)(D)` is `abcdef`.

Difference from z/OS: IBM REXX treats a symbol followed by a **blank** and then `(` as a blank concatenation (`X (Y)` is the value of `X`, a blank, and the value of `Y`). bricks treats it as a function call, exactly like `X(Y)`, so existing programs written as `LENGTH (S)` keep working. To concatenate in that position, write `X || ' ' || (Y)`.

Restrictions / unsupported

Feature	Status
`TRACE` (interpretive tracer)	Parsed and accepted; bricks does not emit trace output.
`OPTIONS` settings (`NOVALUE`, etc.)	Parsed and accepted; have no effect.
External function libraries / RXFUNCADD	No dynamic linking — every routine must be a `PROCEDURE` in the program or a built-in.

Feature	Status
`ADDRESS COMMAND` / `ADDRESS SYSTEM` / `ADDRESS TSO`	Not wired — only `ADDRESS CICS` is recognised.
EXECIO and other host-environment I/O	Not provided — use the stream functions (next section) or `EXEC CICS READQ TD` / `WRITEQ TD` for file I/O.
`PARSE PULL` / terminal queue (`PUSH` / `QUEUE`)	Statements are parsed; the queue is always empty. Use `EXEC CICS RECEIVE` for terminal input.
`NUMERIC FORM SCIENTIFIC` / `ENGINEERING`	Statement is parsed; rendering is always plain decimal.
Strict IEEE arithmetic	Backed by float64 with `NUMERIC DIGITS` rounding, so very large factorials and the like will round before they overflow.

Part 2. The COBOL language

> **Part 2 begins here.** Bricks ships a free-form COBOL interpreter (`cobol/`) that sits beside REXX as a second front end on the same `EXEC CICS` / `EXEC SQL` / `EXEC CICS WEB` surface. The chapters that follow describe COBOL source format, the DATA DIVISION, PROCEDURE DIVISION, the EIB binding, the `COPY` directive (including the standard `DFHAID`, `DFHRESP`, `SQLCA`, and `DFHWB*` copybooks), and the restrictions. Command-level behaviour is identical between COBOL, REXX and PL/I (**Part 9**) — see **Parts 4**, **6**, and **7** for the verbs.

Chapter 20. COBOL source format

Bricks ships a free-form COBOL interpreter (`cobol/`) that sits beside REXX as a second front end on the same `EXEC CICS` surface. The language-specific layer is `cobol/frame.go`, which adapts COBOL's group-item world to the REXX-style `STEM.TAIL` lookup the CICS handlers use.

Source rules

- **Free-form.** No column 1-72 ruling; no Area A / Area B.
- **Hyphens** are allowed in identifiers (`CUST-RECORD`).
- **Comments** use modern `*>` anywhere on a line, or legacy `*` in column 1.
- **Strings** use `'...'` or `"..."` with quote-doubling for embedded quotes (`'don''t'`, `"say ""hi"""`).
- **Numeric literals** are decimal digit runs with an optional decimal point (`42`, `1.5`, `0.050`) and an optional sign written against the digits (`-1`, `+5`, `-1.5`). They work anywhere a number is accepted: `VALUE`, `MOVE`, `COMPUTE`, the arithmetic verbs, and conditions (`IF B / 10 = 4.5`). A fractional literal is placed by the receiving PIC's `V` (see `VALUE` on a scaled item).
- **Hex literals** like `X'F3'` and `X"7C"` decode to their byte value — used for `IF EIBAID = X'F3'` to check PF3 / PF12 / etc. without `C2X(EIBAID) = 'F3'` round-trips.
- **Identifiers** start with a letter or _, continue with letters, digits, _, and -. Names are case-insensitive (uppercased at parse time).
- **Periods** are scope terminators, not per-statement. The parser requires a period after each division header, section header, paragraph header, and top-level data item; periods inside an explicit `END-IF` / `END-EVALUATE` / etc. body are tolerated as inner punctuation, not as scope terminators.

COBOL character set

COBOL program data is **one byte per character**, as on a mainframe. Every byte of a `PIC X` item is one character of the bricks single-byte character set (bricks SBCS): ISO-8859-1 (Latin-1), with a graphic plane in `X'80'-X'9F'` in place of the C1 controls:

X'8n'	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
X'80'	€	▉	▀	▄	▌	▐	▪	─	│	┌	┐	└	┘	├	┤	┬
X'90'	┴	┼	═	‖	•	←	→	↑	↓	≤	≥	≠	○	◊	¯	≡

UTF-8 exists only at the edges (terminal, web, SQL, files, logs); the CICS layer converts there, once. Inside the program:

- **Source** files are UTF-8; a leading byte order mark is ignored. Each character of a quoted literal is converted to its single byte at compile time: `'Grüße'` is 5 bytes, `FUNCTION LENGTH('Grüße')` is 5, and

`'▉'` is `X'81'`. A character outside the table (CJK, –, emoji, a C1 control, `U+FFFD`) or invalid UTF-8 is a compile error, never a silent `?`:

```
line 12 col 22: U+4E2D in literal is not a bricks SBCS character
line 13 col 18: invalid UTF-8 byte X'C4' in literal
line 14 col 20: character U+00C4 not allowed outside quotes (ASCII only)
```

Names, keywords and PIC strings are ASCII only. Comments may hold any Unicode text (they must still be valid UTF-8). Error columns count characters, not bytes, and every error message is pure ASCII.

- **Hex literals** are raw bytes and are never converted: `X'C4'` is `Ä` (on z/OS, EBCDIC `D`). AID constants (`X'7D'`, `X'F3'`, `DFHAID`) keep working because `EIBAID` holds the raw AID byte; other EBCDIC hex constants need translating when porting.
- **Byte = character** everywhere: `MOVE`, reference modification, `REDEFINES`, group moves, `STRING` / `UNSTRING`, `INSPECT`, `FUNCTION LENGTH` / `POS` / `REVERSE`, COMMAREA offsets and record keys all count characters.
- `DISPLAY` converts to UTF-8 on the way out.
- **Case:** `FUNCTION UPPER-CASE` / `LOWER-CASE` fold a-z and the Latin-1 letters (ä ↔ Ä, þ ↔ Þ), length preserved; ß, ÿ, µ, ª, º, ×, ÷ and the graphic plane are unchanged. `INSPECT CONVERTING` is a plain byte table, so list the Latin-1 letters explicitly (see the `UPPR` sample, `runtime/cobol/uppr.cob`).
- **Class tests:** `ALPHABETIC`, `ALPHABETIC-UPPER` and `ALPHABETIC-LOWER` accept ASCII letters and space only (IBM): `'Müller' IS ALPHABETIC` is false.
- **Collation** is byte order: `'Ä' > 'Z'`, and `'A' THRU 'Z'` does not include `Ä`. EBCDIC collation is not emulated.
- **Figuratives:** `HIGH-VALUES` is `X'FF'` (shown as `ÿ` by `DISPLAY`, on the terminal and in JSON); `LOW-VALUES` is `X'00'`.
- **Numeric MOVE** of text: a Latin-1 letter in the sender is a data exception like an ASCII letter (target zeroed, logged); non-letter symbols such as `€`, `£` or NBSP are dropped like `$`.
- `FUNCTION TRIM` trims ASCII blanks only; NBSP `X'A0'` is data. `FUNCTION NUMVAL` rejects Latin-1 junk with an ASCII error.
- `FUNCTION ORD` / `CHAR` / `BYTE-LENGTH`: `ORD('Ä')` = 197 (byte value + 1), `CHAR(197)` = `Ä`, `BYTE-LENGTH` = `LENGTH`.
- **JSON** text in a `PIC X` item is SBCS. `JSON GENERATE` writes every byte from `X'80'` up raw (the web edge sends it as UTF-8) and uses `\u00hh` only for controls below `X'20'`; `COUNT IN` counts characters. `JSON PARSE` stores a `\uXXXX` escape as its single byte (`\u00e9` → é, `\u20ac` → €) or `?` when the code point is not in the table.
- **Not supported:** `PIC N`, `USAGE NATIONAL`, `PIC U`, `FUNCTION NATIONAL-OF` / `DISPLAY-OF`.

Deviations from z/OS: the code page is Latin-1 plus the graphic plane, not an EBCDIC CCSID (037 / 1140); hex literals denote Latin-1 bytes; collation is Latin-1 byte order; JSON text is SBCS, not UTF-8 (CCSID 1208).

Divisions

The four divisions parse in their canonical order:

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. HELC.

       ENVIRONMENT DIVISION.        *> optional, must be empty

       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 SCR.
          05 INFOLINE PIC X(78).
          05 GREETING PIC X(20).

       PROCEDURE DIVISION.
       MAIN.
           MOVE 'Hello' TO GREETING.
           EXEC CICS SEND MAP('HELO1') FROM(SCR) ERASE END-EXEC.
           EXEC CICS RETURN END-EXEC.
           STOP RUN.
```

An optional `LINKAGE SECTION` may follow `WORKING-STORAGE` (see LINKAGE SECTION); `DFHCOMMAREA` is auto-injected as `PIC X(2000)` if the program doesn't declare it (see `cobol.ensureSystemItems`), so a sub-program can `MOVE DFHCOMMAREA TO key` immediately. The dispatcher strips trailing space when reading the COBOL frame's `DFHCOMMAREA` back out, so a fixed-width buffer round-trips cleanly across an inter-language `EXEC CICS LINK`.

Chapter 21. DATA DIVISION

The DATA DIVISION declares every data item the program touches. Bricks expects a single `WORKING-STORAGE SECTION` followed by an optional `LINKAGE SECTION` (`DFHCOMMAREA` is auto-injected when the program omits it).

Level numbers

Level	Meaning
`01`	Top-level data item. Owns a self-contained byte buffer.
`02` – `49`	Subordinate items inside a group. Children share their parent's buffer at a computed offset.
`77`	Standalone elementary item (no children). Functionally equivalent to a flat `01`.
`88`	Condition-name attached to the preceding non-88 item (see below).

The level numbers `66` (RENAMES) and `78` (constant) are **not** parsed.

PIC clauses — unedited

Unedited PICs hold raw data: digits, letters, or padded bytes.

Form	Class	Notes
`X(n)` or `XXXX…`	Alphanumeric	n bytes; left-justified, space-padded on `MOVE`. Up to 1 MiB.

Form	Class	Notes
`A(n)` or `AAA…`	Alphabetic	Same storage as `X(n)`; runtime class-check rejects non-alpha on `INSPECT`-style tests.
`9(n)` or `999…`	Numeric (unsigned)	n decimal digits; right-justified, zero-padded. Capped at 18 digits (int64 limit).
`S9(n)`	Numeric (signed)	Byte 0 holds the sign character (`' '` for positive, – for negative); bytes 1..n-1 hold the digits. Cap 18 digits.
`9(n)V9(m)`	Numeric with implicit decimal	The `V` is positional — no byte is consumed. Total `n+m` digits stored; arithmetic uses the fixed-point `decimal` type so scale survives `MOVE` / `COMPUTE` / `DISPLAY`.
`S9(n)V9(m)`	Signed scaled numeric	Same as above plus a sign byte.

`(n)` is a repetition count; `9(3)` and `999` are equivalent.

PIC clauses — edited numeric

Edited PICs hold a formatted representation of a numeric value suitable for display. Five edit characters are recognised:

Edit char	Role
`Z`	Replace a leading zero with a space (zero suppression).
`.`	Insert a literal decimal point at that column.
`,`	Insert a thousands separator at that column.
`$`	Insert a `$`. A *single* `$` is fixed (always emitted, must be the leftmost character). Two or more in a row *float* — the leftmost is the symbol and slides right to the position immediately before the first significant digit.
`*`	Check protection. Replaces a leading zero with `*` instead of a space — used on cheques to prevent forgery.

Constraints:

- At most one suppression family per PIC: `Z`, floating `$`, and `*` are mutually exclusive (a single fixed `$` may coexist with `Z` — e.g. `$ZZ9.99`).
- `Z` and `*` are forbidden in the fractional half (after the `.` insertion).
- The literal width of the mask is the PIC length; total digit positions (Z + floating-$ + * + 9) are capped at 18.

Worked examples:

Source	PIC mask	Stored bytes
`42`	`ZZ9`	`" 42"`
`0`	`ZZ9`	`" 0"`
`1.5`	`ZZZ.99`	`" 1.50"`
`1234.56`	`Z,ZZ9.99`	`"1,234.56"`
`12345.67`	`$$$,$$9.99`	`"$12,345.67"`
`0.45`	`$$$,$$$.99`	`" $.45"`
`12.34`	`**,**9.99`	`"****12.34"`
`42`	`$9999`	`"$0042"`

When a scaled numeric source is moved through an edited receiver the source's V-scale is honoured: `MOVE N TO N-Z` where `N` is `PIC 9(5)V99 VALUE 12345.67` and `N-Z` is `PIC ZZ,ZZ9.99` stores `"12,345.67"`.

The remaining ANSI edit characters — `+`, `-`, `CR`, `DB`, `B` (blank), `0` (zero insertion), `/` (slash), and the `BLANK WHEN ZERO` clause — are not yet supported.

`VALUE`

`VALUE` declares the initial content of an elementary data item. Accepted forms:

Form	Example
String literal	`VALUE 'ENTER ID'.` or `VALUE "OK".`
Numeric literal	`VALUE 42.` or `VALUE 0.` (signed +/− allowed).
Figurative	`VALUE SPACES.`, `VALUE ZEROS.`, `VALUE HIGH-VALUES.`, `VALUE LOW-VALUES.`, `VALUE QUOTES.`

The figurative aliases `SPACE` / `ZERO` / `ZEROES` / `HIGH-VALUE` / `LOW-VALUE` / `QUOTE` are normalised to their plural form at parse time. `VALUE` on a group item is rejected — initialise the children individually.

`VALUE` on a scaled (`V`) item

A **numeric** `VALUE` carries the value, and the PIC's `V` places it — exactly as `MOVE` and `COMPUTE` do with the same literal:

```
01 A PIC S9(4)V99 VALUE 1.5.      *> 1.50, stored 000150
01 B PIC S9(4)V99 VALUE 2.        *> 2.00, stored 000200
01 C PIC S9(4)   VALUE 1.5.       *> 1, truncated toward zero
```

Do **not** write the stored digit run and expect it to be read as scaled: `PIC S9(4)V99 VALUE 150` is one hundred fifty, not 1.50.

A **quoted** `VALUE` is byte data and takes the alphanumeric path, so `VALUE '150'` on that same item is 1.50 — the digits are laid into the field's bytes rather than scaled. This mirrors `MOVE '150'`, so a literal means the same thing wherever it is written.

Group items

A `01` (or any subordinate level >01) item with one or more children becomes a group. The group's storage is the concatenation of its children's storage; the group itself owns no extra bytes beyond what its children occupy.

```
01 PARENT.
   05 CHILD PIC X(8).
   05 OTHER PIC X(4).
```

`MOVE source TO PARENT` fans out to every elementary child; `EXEC CICS SEND MAP FROM(PARENT)` walks the children by name to look up map-field values.

`OCCURS`

`OCCURS n [TIMES]` declares an array of `n` copies of an item. The cap is 4096 occurrences per item. Reference one slot with a 1-based subscript expression: `K(I)`, `K(5)`, `K(I + 1)`. Subscript bounds are

checked at runtime. `OCCURS` works on elementaries and on group items; nested `OCCURS` (an array of arrays) is **rejected at parse time**.

```
01 TABLE-AREA.
   05 ROW OCCURS 15 TIMES.
      10 K  PIC X(8).
      10 N  PIC X(28).
01 I  PIC 9(4).
...
MOVE 'KEY-A' TO K(1).
MOVE 'KEY-B' TO K(I).
```

`REDEFINES`

`REDEFINES` overlays a second description on storage that an earlier item already owns. No new bytes are allocated — the redefining item and the redefined item are two names for the same buffer slice, and a `MOVE` through either name is visible through the other immediately. The PIC of whichever name the statement references decides how the bytes are interpreted.

```
level data-name-1 REDEFINES data-name-2 [PIC ...] [USAGE ...].
```

The clause must be the first thing after the data name. Rules bricks enforces at parse time (each mirrors the IBM Enterprise COBOL rule unless flagged):

- `data-name-1` and `data-name-2` must carry the **same level number** (01–49 or 77; never 88).
- `data-name-2` must be the **immediately preceding** item at that level under the same parent. Only other redefinitions of the same item (with their subordinates and 88s) may sit between them.
- Multiple redefinitions all name the **original** item: `B REDEFINES A` then `C REDEFINES A` — never `C REDEFINES B`.
- The redefining item must **fit**: its storage may not exceed the redefined item's. This holds at level 01 too — stricter than IBM, which lets a non-EXTERNAL 01 redefiner grow the area (deferred; see Chapter 27). A *shorter* redefiner is fine and only overlays the leading bytes.
- Neither item may carry `OCCURS` on its own entry, and neither may live inside an `OCCURS` group. **Subordinates of the redefining item may carry** `OCCURS` — that subordinate-table shape is the main reason the clause exists.
- No `VALUE` anywhere in the redefining item's subtree (88-level condition VALUEs are fine). A `VALUE` on the *redefined* item is visible through the overlay at startup.
- `FILLER` may redefine (`05 FILLER REDEFINES D8.`); nothing can redefine a `FILLER` (it has no referenceable name). Likewise the auto-injected system items (`EIB*`, `RC`, `SQLCA*`, `JSON-CODE`, `JSON-STATUS`, `DFHCOMMAREA`) cannot be redefinition targets — they don't exist yet when the DATA DIVISION parses.

Dual view — one set of bytes, two shapes:

```
01 WS-DATE      PIC 9(8) VALUE 20260611.
01 WS-DATE-R REDEFINES WS-DATE.
   05 WD-YYYY   PIC 9(4).
   05 WD-MM     PIC 9(2).
   05 WD-DD     PIC 9(2).
...
DISPLAY WD-MM.              *> 06
MOVE 12 TO WD-MM.
DISPLAY WS-DATE.            *> 20261211
```

Row table — name the rows for the map, subscript them for the loop. This replaces the unrolled N-branch `EVALUATE` fan-out that older sample programs carry (see `esdc.cob`):

```
01 SCR.
   05 INFOLINE  PIC X(78).
   05 ROWS-AREA.
      10 ROW1   PIC X(76).
      10 ROW2   PIC X(76).
      10 ROW3   PIC X(76).
   05 R-TAB REDEFINES ROWS-AREA.
      10 ROWV   PIC X(76) OCCURS 3.
...
MOVE WS-LINE TO ROWV(WS-SLOT).
EXEC CICS SEND MAP('ESDS') FROM(SCR) ERASE END-EXEC.
```

`SEND MAP FROM(SCR)` still finds `ROW1`–`ROW3`: map routing resolves qualified names through storage, and the redefinition shares their bytes, so whatever the loop wrote through `ROWV(n)` is what the map paints.

Storage caveat — signed DISPLAY fields. bricks stores `PIC S9(n)` as **n+1 bytes**: byte 0 is a leading separate sign (`' '` positive, `'-'` negative), bytes 1..n the digits — in effect `SIGN IS LEADING SEPARATE`, with space standing in for +. IBM's default is n bytes with the sign overpunched into the trailing digit. A redefinition sees bricks's layout, not IBM's: the full alphanumeric view of `PIC S9(4)` is `X(5)`, and it reads `-0123`, sign first. Ported IBM copybooks that redefine signed DISPLAY fields must be widened by one byte. `PIC 9(n)`, `COMP` (big-endian two's complement, 2/4/8 bytes) and `COMP-3` (BCD, C/D/F sign nibble) overlays are byte-for- byte IBM-faithful — though a `PIC X` view of `COMP` bytes shows raw binary, exactly as it would on the mainframe.

`FILLER`

`FILLER` is a reserved name for anonymous storage. Multiple `FILLER` children under the same parent are allowed (they don't collide on the duplicate-name check) and they're never registered in the lookup tables — they can only be addressed through the parent.

`USAGE` — DISPLAY vs. COMP vs. COMP-3

Numeric items have three storage layouts:

USAGE	Synonyms	Storage
`DISPLAY` (default)	—	One byte per digit; signed PICs reserve byte 0 for the sign character (`' '` / `'-'`). The historical bricks layout.
`COMP`	`COMPUTATIONAL`, `COMP-4`, `BINARY`	Big-endian two's-complement signed integer packed into 2, 4, or 8 bytes based on the total digit count.
`COMP-3`	`COMPUTATIONAL-3`, `PACKED-DECIMAL`	Packed decimal: two BCD digits per byte (high nibble first), with the last byte's low nibble carrying the sign.
`POINTER`	`USAGE IS POINTER`	An opaque 4-byte pointer handle, declared **without** a `PIC` clause: `01 WS-PTR USAGE POINTER.`. Holds a CICS storage handle from `EXEC CICS ADDRESS` / `GETMAIN` or an `ADDRESS OF` value. See the COBOL pointer model in Chapter 22.

The `USAGE` keyword is optional — `PIC S9(8) COMP.` and `PIC S9(8) USAGE IS COMP.` both work. `USAGE` may appear before or after the `PIC` clause within the same item.

COMP byte sizing follows IBM's halfword / fullword / doubleword convention:

Total digits (`PIC 9(n)` or `S9(n)V9(m)` → n+m)	Bytes
1 – 4	2 (halfword)
5 – 9	4 (fullword)
10 – 18	8 (doubleword)

`USAGE COMP` requires a numeric PIC (`9` or `S9`); declaring it on `PIC X(n)` is rejected at parse time. The signed/unsigned flag does *not* change the byte count — both `PIC 9(4) COMP` and `PIC S9(4) COMP` occupy two bytes. Bricks always stores the value as a two's-complement signed int internally.

Scale survives the format transition: a value moved between a `PIC 9(5)V99` DISPLAY field and a `PIC S9(8)V99 COMP` field preserves the implicit decimal point. Within the interpreter every numeric load goes through the fixed-point `decimal` engine; DISPLAY decodes digits, COMP decodes binary bytes, but both produce the same `decimal{val, scale}` tuple.

COMP-3 byte sizing uses the packed-decimal formula `ceil((digits + 1) / 2)` — digits plus the trailing sign nibble, rounded up:

Total digits	Bytes
1	1
4	3
9	5
15	8
18	10

The sign nibble follows IBM Enterprise COBOL convention: `0xC` for signed positive, `0xD` for signed negative, `0xF` for unsigned. On read, sign nibbles `0xB` and `0xD` are treated as negative; everything else is positive. Like `COMP`, `COMP-3` requires a numeric PIC and the digit cap is 18 (the int64 fixed-point engine's ceiling) — `PIC S9(19)` and wider are parser-rejected.

The canonical mainframe shape `PIC S9(15) COMP-3` is the standard CICS `EIBABSTIME` carrier:

```
01  SYS-DT  PIC S9(15) COMP-3.
EXEC CICS ASKTIME ABSTIME(SYS-DT) END-EXEC.
EXEC CICS FORMATTIME ABSTIME(SYS-DT) YYYYMMDD(YYYYMMDD) END-EXEC.
```

`ASKTIME` writes the 15-digit ms-since-1900 decimal string into `SYS-DT`; bricks packs it into 8 BCD bytes transparently. `FORMATTIME` reads the packed bytes back into the same decimal string and parses it.

Other USAGE variants — `COMP-1` (single-precision float) and `COMP-2` (double-precision float) are explicitly rejected at parse time with a "not yet supported" message. `SYNC`, `INDEXED BY`, and `OCCURS DEPENDING ON` are also unsupported. (`REDEFINES` IS supported — see the `REDEFINES` section above.)

Level 88 condition-names

A level-88 line declares a boolean condition tied to the **preceding non-88 data item**. The condition is true when the parent's current value matches one of the listed values or falls within a `THRU` range. Reference as a bare name in any PROCEDURE DIVISION condition (see Chapter 22 — 88-level condition names for the runtime behaviour).

```
01  RESP-CODE  PIC 9(3).
    88 RESP-OK        VALUE 0.
    88 RESP-MISSING   VALUE 13.
    88 RESP-RECOVER   VALUES 12, 26, 80.
    88 RESP-FATAL     VALUE 100 THRU 999.
```

`VALUES` (plural) and `VALUE` (singular) are interchangeable. Multiple values are separated by spaces or commas. `THRU` (synonym `THROUGH`) declares an inclusive range; ranges and single values can be mixed: `88 OK VALUES 0, 1 THRU 5, 9.`

A condition-name lives in its own namespace, separate from data items. Declaring an 88 whose name collides with an existing data item is rejected at parse time.

LINKAGE SECTION

The `LINKAGE SECTION` declares records that are addressed through a pointer rather than allocated their own storage. Bricks supports both the classic auto-injected `DFHCOMMAREA` and operator-declared **based** records redirected with `SET ADDRESS OF`.

`DFHCOMMAREA`**.** Bricks auto-injects `DFHCOMMAREA PIC X(2000)` when the program doesn't declare it, so a sub-program can `MOVE DFHCOMMAREA TO key` immediately. A `LINK` hands back exactly as many bytes as the caller passed (IBM: the callee cannot change the COMMAREA length): the caller's bytes, overlaid with the callee's view of `DFHCOMMAREA`. Trailing blanks and `X'00'` bytes are kept, so a binary field at the end of the area survives. The default buffer only carries the `EIBCALEN` bytes the caller passed, so a program using it cannot hand back or forward more than it received. `DFHCOMMAREA` is itself a **based** 01 with a default buffer: a program can `MOVE … TO DFHCOMMAREA` out of the box, *and* redirect it onto another area's bytes with `SET ADDRESS OF DFHCOMMAREA TO ptr`.

`DFHEIB`**.** The Execute Interface Block is a real `01 DFHEIB` based block; its children keep the bare IBM names (`EIBRESP`, `EIBAID`, `EIBCALEN`, `EIBTRMID`, `EIBCPOSN`, …) so a program references them unqualified exactly as on z/OS. `EXEC CICS ADDRESS EIB(ptr)` hands back a handle addressing it.

Based records — `BASED`**.** A LINKAGE 01 declared `BASED` owns no storage of its own:

```
LINKAGE SECTION.
01 CUST-REC BASED.
   05 CUST-NO   PIC X(8).
   05 CUST-NAME PIC X(30).
```

Until the program points it somewhere with `SET ADDRESS OF`, `CUST-REC` is NULL — any read or write through it is a clean NULL-pointer abend (`ASRA`-style), never a crash. After `SET ADDRESS OF CUST-REC TO ptr` (where `ptr` came from `GETMAIN`, `ADDRESS`, or `ADDRESS OF` another item), every reference to `CUST-NO` / `CUST-NAME` reads and writes the addressed bytes. See the COBOL pointer model in Chapter 22 for the full surface.

Cross-group name collisions — `OF` / `IN` qualification

A child name that appears under more than one group is fine; the parser accepts the declaration and tracks the collision in `Program.AmbiguousNames`. Unqualified access to such a name is a runtime error ("ambiguous reference") — disambiguate with `OF` (or its synonym `IN`):

```
01 SCR.
   05 CUSTNO PIC X(8).
01 DET.
   05 CUSTNO PIC X(8).
...
MOVE 'A1234567' TO CUSTNO OF SCR.
MOVE 'B7654321' TO CUSTNO OF DET.
DISPLAY CUSTNO OF SCR.
```

Single-step qualification (`X OF Y`) picks the matching child anywhere in `Y`'s subtree; multi-step (`X OF Y OF Z`) chains through nested groups. Bare names that are unique across the program continue to work without qualification — the pre-7 `runtime/cobol/gust.cob` convention of prefixed child names (`DCUSTNO`, `DNAME`, `DMSG`) still compiles and runs, it just isn't required any more.

A sibling-duplicate (two children of the SAME parent sharing a name) is still a parse-time error — no qualifier can disambiguate between siblings of one group.

Type coercion on `MOVE`

Source class	Target class	Behaviour
Alphanumeric → Alphanumeric	`X` / `A` → `X` / `A`	Copy, right-pad with spaces if target is wider; right-truncate if source is wider.
Numeric → Numeric	`9` / `S9` → `9` / `S9`	Load through the fixed-point `decimal` engine, rescale to the target's `V` position, store. Truncates high-order if the integer part doesn't fit (silent unless `ON SIZE ERROR` is set on the surrounding verb).
Numeric → Edited	`9` / `S9` → `Z` / `$` / `*` / ...	Read the source as a `decimal` (honouring `V`-scale), render the value through the edited mask.
Edited → Numeric	`Z` / `$` / `*` / ... → `9` / `S9`	De-edit on read by stripping insertion characters (`,`, `.`, `$`, `*`, spaces) and re-parsing the digits.
Alphanumeric → Numeric	`X` → `9`	Best-effort `loadNumeric` — succeeds if the byte content parses as digits after trimming. A letter raises a runtime data exception.
Numeric → Alphanumeric	`9` / `S9` → `X`	Source stringified, then alphanumeric-padded.
Anything → Group	* → group item	Fans out to every elementary child (recursively).

A **numeric literal** source follows the Numeric → Numeric row: it is rescaled to the target's `V` position, so `MOVE 2 TO PIC S9(4)V99` stores 2.00 and `MOVE 1.5 TO PIC S9(4)` truncates to 1. `MOVE`, `COMPUTE` and a numeric `VALUE` clause all agree on the same literal. A **quoted** literal is alphanumeric and takes the byte path instead (`MOVE '150'` lays those digits into the field's bytes).

Figurative expansion

`SPACES` → `' '` repeated to fill the target; `ZEROS` → `'0'` for numeric targets, `\x00` for alphanumeric; `HIGH-VALUES` → `\xFF`; `LOW-VALUES` → `\x00`; `QUOTES` → `'\''`. Group targets compute the fill length from the sum of the elementary children's storage.

Chapter 22. PROCEDURE DIVISION

The PROCEDURE DIVISION is a flat list of paragraphs. Each paragraph is a label (`PARAGRAPH-NAME.`) followed by one or more statements. Paragraphs are visible to `PERFORM` and `GO TO`.

Statement reference

Data movement

Verb	Syntax	Notes
`MOVE`	`MOVE src TO tgt1, tgt2, ...`	One or more targets; fan-out to group children supported. Type coercion table in Chapter 21 — Type coercion.
`INITIALIZE`	Not yet supported.	Initialise individual items with `MOVE`.

Arithmetic

Every arithmetic verb accepts the optional clauses `ROUNDED`, `ON SIZE ERROR ... [END-VERB]`, and `GIVING` (where listed). See "ROUNDED and ON SIZE ERROR" below.

Verb	Syntax
`COMPUTE`	`COMPUTE tgt [ROUNDED] = expr [ON SIZE ERROR ...] [END-COMPUTE]`
`ADD`	`ADD a TO b [GIVING c] [ROUNDED] [ON SIZE ERROR ...] [END-ADD]`
`SUBTRACT`	`SUBTRACT a FROM b [GIVING c] [ROUNDED] [ON SIZE ERROR ...] [END-SUBTRACT]`
`MULTIPLY`	`MULTIPLY a BY b [GIVING c] [ROUNDED] [ON SIZE ERROR ...] [END-MULTIPLY]`
`DIVIDE` (INTO)	`DIVIDE a INTO b [GIVING c] [ROUNDED] [ON SIZE ERROR ...] [END-DIVIDE]`
`DIVIDE` (BY)	`DIVIDE a BY b GIVING c [ROUNDED] [ON SIZE ERROR ...] [END-DIVIDE]`

`REMAINDER` is not yet supported on `DIVIDE`. Arithmetic expressions support `+`, `-`, `*`, `/`, `**`, parentheses, and unary sign.

Signed numeric literals

A sign written **immediately** against the digits is part of the literal, exactly as on IBM, so a signed literal is legal anywhere a plain one is:

```
MOVE -1 TO WS-IND.
MOVE +5 TO WS-N.
ADD -1 TO WS-COUNT.
DISPLAY -1.
COMPUTE WS-X = -1.5.
IF WS-N = -100 THEN ...
01 WS-LIMIT PIC S9(4) VALUE -150.
```

The spacing is what distinguishes a literal from an operator: `-1` is a signed literal, `- 1` is the minus operator applied to `1`. Inside an arithmetic expression both readings work out the same (`COMPUTE X = A - 1` and `COMPUTE X = A -1` are each unambiguous), but in a list-taking statement they differ — `DISPLAY A -1` displays two items, while `DISPLAY A - 1` is a parse error, because `DISPLAY` takes a list of operands, not an expression.

A redundant `+` is not stored, so `DISPLAY +5` prints `5`. Moving a negative literal into an unsigned `PIC 9` target drops the sign, as IBM specifies.

Control flow

Verb	Syntax	Notes
`IF`	`IF cond [THEN] stmts [ELSE stmts] [END-IF]`	The period after the last unscoped statement closes the `IF` when `END-IF` is omitted.
`EVALUATE`	`EVALUATE subject [ALSO subject ...] {WHEN object [ALSO object ...]} ... stmts [WHEN OTHER stmts] [END-EVALUATE]`	Full COBOL-85 form: `TRUE` / `FALSE` / condition / value subjects, `ANY`, `NOT`, `THRU` ranges, stacked `WHEN`s. See EVALUATE.

`PERFORM`	`PERFORM para [THRU para2]`
	`PERFORM [para] [WITH TEST BEFORE|AFTER] UNTIL cond`
	`PERFORM [para] N TIMES`
	`PERFORM [para] VARYING idx FROM x BY y UNTIL cond [AFTER ...]`
	…any of the above with the paragraph name omitted and an in-line body closed by `END-PERFORM`
		Out-of-line (named paragraph) and in-line (`END-PERFORM`) forms both work. `WITH TEST` is accepted but inert on the `N TIMES` and no-clause shapes.

`GO TO`	`GO TO para`	Calculated `GO TO ... DEPENDING ON` not supported.
`CONTINUE`	`CONTINUE`	Explicit no-op (used in empty `WHEN OTHER` etc.).
`STOP RUN`	`STOP RUN` (or just `STOP`)	Terminate the task.
`GOBACK`	`GOBACK`	Synonym for `EXIT PROGRAM`.

`EXIT`	`EXIT [PROGRAM]`
	`EXIT PERFORM [CYCLE]`
	`EXIT PARAGRAPH`
		`EXIT` alone is a no-op; `EXIT PROGRAM` returns to the caller. `EXIT PERFORM` leaves the innermost **in-line** `PERFORM`, `EXIT PERFORM CYCLE` ends only the current iteration. `EXIT PARAGRAPH` ends the current paragraph (see EXIT PARAGRAPH). `EXIT SECTION` is a parse error — bricks has no sections.

Strings

Verb	Syntax
`STRING`	`STRING src ... DELIMITED [BY] (SIZE | delim) ... INTO tgt [[WITH] POINTER ptr] [[ON] OVERFLOW stmts] [NOT [ON] OVERFLOW stmts] [END-STRING]` — see STRING and UNSTRING
`UNSTRING`	`UNSTRING src [DELIMITED [BY] [ALL] d1 [OR [ALL] d2] ...] INTO {t [DELIMITER [IN] d] [COUNT [IN] c]} ... [[WITH] POINTER ptr] [TALLYING [IN] n] [[ON] OVERFLOW stmts] [NOT [ON] OVERFLOW stmts] [END-UNSTRING]`
`INSPECT TALLYING`	`INSPECT subject TALLYING counter FOR (ALL | LEADING | CHARACTERS) [needle] [BEFORE/AFTER INITIAL delim] [, ...]`
`INSPECT REPLACING`	`INSPECT subject REPLACING (ALL | LEADING | FIRST | CHARACTERS) [needle] BY replacement [BEFORE/AFTER INITIAL delim] [, ...]`
`INSPECT CONVERTING`	`INSPECT subject CONVERTING from TO to [BEFORE/AFTER INITIAL delim]`

JSON

Verb	Syntax
`JSON GENERATE`	`JSON GENERATE tgt FROM rec [COUNT [IN] n] [NAME [OF] item [IS] {'lit' | OMITTED} ...] [SUPPRESS ...] [CONVERTING ...] [[ON] EXCEPTION stmts] [NOT [ON] EXCEPTION stmts] [END-JSON]` — see JSON GENERATE and JSON PARSE
`JSON PARSE`	`JSON PARSE src INTO rec [WITH DETAIL] [NAME ...] [SUPPRESS item ...] [CONVERTING ...] [[ON] EXCEPTION stmts] [NOT [ON] EXCEPTION stmts] [END-JSON]`

I/O and external

Verb	Syntax	Notes
`DISPLAY`	`DISPLAY item1 item2 ...`	Concatenates items with no separator; newline after the list.
`ACCEPT`	Not yet supported.	Use `EXEC CICS RECEIVE` for terminal input.
`EXEC CICS`	`EXEC CICS verb operand-list END-EXEC`	Same verb set as REXX. See Parts 4–5 and 7.
`EXEC SQL`	`EXEC SQL stmt END-EXEC`	See Part 6.
`CALL`, `SEARCH`, `ALTER`	Not yet supported.	

Every target name above (the second operand of `MOVE`, the target of `COMPUTE` / `ADD` / `SUBTRACT` / `MULTIPLY` / `DIVIDE` / `GIVING` / `STRING INTO` / `UNSTRING INTO` / `INSPECT`, and the subject of `EVALUATE` / `IF`) accepts qualification via `OF` / `IN` (see the Data Division section on globally non-unique names) and subscripts via `(idx)` for OCCURS-resident items.

Reference modification (string slicing)

`identifier(start:length)` addresses a byte window inside a data item. Positions are **1-indexed**, `length` is a count of bytes, and both operands may be any integer arithmetic expression — literals, data-names, or `WS-POS + 1`. Omitting the length (`identifier(start:)`) means "from `start` to the end of the item", padding included.

Slices work on both sides of a statement:

```
01 WS-LINE  PIC X(80) VALUE 'HELLO WORLD'.
01 WS-PART  PIC X(5).
01 WS-POS   PIC 9(2)  VALUE 7.

    MOVE WS-LINE(1:5)      TO WS-PART        *> read  -> 'HELLO'
    MOVE WS-LINE(WS-POS:5) TO WS-PART        *> read  -> 'WORLD'
    MOVE '*'               TO WS-LINE(6:1)   *> write -> 'HELLO*WORLD'
    IF WS-LINE(6:1) = SPACE THEN ...          *> test one byte
```

A write touches **only** the named window; the bytes on either side keep their contents. That makes `MOVE ch TO BUF(POS:1)` the natural way to render a screen row one character at a time, without an `OCCURS` table or `REDEFINES` overlay.

Three rules worth knowing:

- **A slice is always category alphanumeric**, whatever the item's own PIC says. `NUM-FLD(1:1)` compares as bytes, so `IF NUM-FLD(1:1) = '1'` is the correct test and `= 1` is not. On a write, the value

is left-justified and space-padded to the window width — never right-justified or zero-filled, even into a `PIC 9` item.
- **A `USAGE COMP` / `COMP-3` item is sliced through its decimal rendering**, not its raw bytes: `PIC 9(4) COMP VALUE 1234`, sliced `(1:2)`, yields `'12'`.
- **Out-of-range windows are errors.** A read whose length overruns the item is space-padded, but a start below 1 or past the end of the item aborts the transaction, as does any write whose window would run past the item's last byte — those bytes belong to the next field in the same `01` and silently overwriting them corrupts it.

ROUNDED and ON SIZE ERROR

`ROUNDED` applied to `COMPUTE` and the four arithmetic verbs rounds the final value half-away-from-zero to the destination's PIC scale instead of truncating toward zero (the unROUNDED default).

`ON SIZE ERROR` runs its statement list when the calculation overflows the int64 fixed-point engine, the result's integer-part digits don't fit in the target PIC, or a `DIVIDE` divisor is zero; when present, the destination is left untouched on a size-error firing. Without an `ON SIZE ERROR` branch the value is silently truncated/wrapped as before so existing programs aren't disrupted.

PERFORM N TIMES and PERFORM VARYING

```
PERFORM FILL-ROW 15 TIMES.

PERFORM FILL-ROW
    VARYING I FROM 1 BY 1 UNTIL I > 15.
```

`PERFORM ... N TIMES` runs the named paragraph exactly N times; N can be a numeric literal or a numeric data item. `PERFORM ... VARYING idx FROM x BY y UNTIL cond` initialises `idx` to `x`, runs the body while `cond` is false, then increments `idx` by `y` between iterations.

In-line PERFORM

Leave the paragraph name out and the loop body sits between the `PERFORM` and a mandatory `END-PERFORM`:

```
PERFORM UNTIL WS-DONE = 'Y'
    COMPUTE WS-N = WS-N + 1
    IF WS-N > 9 THEN
        MOVE 'Y' TO WS-DONE
    END-IF
END-PERFORM.

PERFORM 10 TIMES
    COMPUTE WS-TOT = WS-TOT + 1
END-PERFORM.

PERFORM VARYING I FROM 1 BY 1 UNTIL I > 15
    MOVE SPACES TO ROW-TEXT(I)
END-PERFORM.
```

All four loop shapes work in-line — no clause (the body runs once), `UNTIL`, `N TIMES`, and `VARYING`. In-line and out-of-line `PERFORM` nest freely in either direction.

`END-PERFORM` is **required** on the in-line form: unlike `IF`, a period does not close it. A period inside the body ends the sentence and is reported as `in-line PERFORM needs END-PERFORM`.

PERFORM THRU

```
PERFORM WORK-A THRU WORK-EXIT.
```

Runs every paragraph from `WORK-A` through `WORK-EXIT` in source order, then returns. `THROUGH` is accepted as a synonym. This enables the classic early-return idiom — a `GO TO` aimed at a paragraph **inside** the range transfers control there and the range still returns to the caller when its last paragraph ends:

```
PERFORM CHECK-IT THRU CHECK-EXIT.
...
CHECK-IT.
    IF WS-AMT = 0 THEN
        GO TO CHECK-EXIT
    END-IF.
    COMPUTE WS-FEE = WS-AMT * 2.
CHECK-EXIT.
    EXIT.
```

A `GO TO` aimed **outside** the range still abandons the `PERFORM` and restarts at the target, which is bricks' general `GO TO` behaviour (see Periods and GO TO). A backwards range (`PERFORM Z THRU A`) is rejected at run time.

WITH TEST BEFORE / AFTER

```
PERFORM WITH TEST AFTER UNTIL WS-RESP = DFHRESP(NORMAL)
    EXEC CICS READ FILE('CUSTOMER') INTO(WS-REC)
              RIDFLD(WS-KEY) RESP(WS-RESP) END-EXEC
END-PERFORM.
```

`WITH TEST BEFORE` is the default: the condition is tested at the top, so a condition that is already true runs the body zero times. `WITH TEST AFTER` tests at the bottom, so the body always runs at least once. `WITH` is optional (`TEST AFTER` alone is accepted). It applies to the `UNTIL` and `VARYING` shapes; on `N TIMES` and the no-clause form it is accepted but has no effect.

On a post-tested `VARYING` the flow is body → test → augment, so the index is left at its last in-range value rather than one step past the limit.

EXIT PERFORM

```
PERFORM VARYING I FROM 1 BY 1 UNTIL I > 100
    IF TBL-KEY(I) = WS-WANTED THEN
        MOVE I TO WS-HIT
        EXIT PERFORM
    END-IF
    IF TBL-KEY(I) = SPACES THEN
        EXIT PERFORM CYCLE
    END-IF
    COMPUTE WS-SCANNED = WS-SCANNED + 1
END-PERFORM.
```

`EXIT PERFORM` breaks out of the innermost in-line `PERFORM`; `EXIT PERFORM CYCLE` ends only the current iteration and continues with the next one (still reaching a `VARYING` augment, so the loop advances rather than spinning).

Both are scoped to the **in-line** `PERFORM` that lexically contains them, matching IBM. An `EXIT PERFORM` written in a performed *paragraph* is a program error, not a way to break the caller's loop, and is reported as `EXIT PERFORM outside an in-line PERFORM`.

EXIT PARAGRAPH

`EXIT PARAGRAPH` (COBOL 2002, IBM Enterprise COBOL 6) ends the current paragraph at once. Control goes wherever the end of that paragraph would have gone:

- under `PERFORM para` (also `TIMES` / `UNTIL` / `VARYING`) — back to the performer, or on to the next iteration;
- under `PERFORM a THRU b` — on to the next paragraph of the range;
- when execution simply fell into the paragraph — on to the next paragraph.

It works at any nesting depth — inside `IF`, `EVALUATE`, an `ON OVERFLOW` phrase or an in-line `PERFORM`. Inside an in-line `PERFORM` it leaves the whole paragraph, not just the loop (use `EXIT PERFORM` for that).

```
VALIDATE-INPUT.
    IF WS-NAME = SPACES
        MOVE 'NAME REQUIRED' TO WS-MSG
        EXIT PARAGRAPH
    END-IF
    PERFORM CHECK-DUPLICATE.
```

With `WS-NAME` blank, the `PERFORM VALIDATE-INPUT` returns with `WS-MSG` = `NAME REQUIRED` and `CHECK-DUPLICATE` is not run.

`EXIT PARAGRAPH.` written as a sentence of its own is fine. (Earlier bricks releases read it as `EXIT.` followed by a new paragraph named `PARAGRAPH`, so the program compiled and silently did the wrong thing.)

`EXIT SECTION` is rejected at parse time with `EXIT SECTION needs SECTIONs, not supported in bricks COBOL`: bricks has no PROCEDURE DIVISION sections, so there is nothing for it to leave.

PERFORM VARYING ... AFTER

```
PERFORM VARYING I FROM 1 BY 1 UNTIL I > 4
          AFTER J FROM 1 BY 1 UNTIL J > 3
    COMPUTE SLOT((I - 1) * 3 + J) = I * J
END-PERFORM.
```

Each `AFTER` adds an inner dimension. Inner indices are re-initialised to their `FROM` value on every iteration of the dimension outside them, so the example above runs the body 12 times. `AFTER` may be repeated for further dimensions. A plain `EXIT PERFORM` unwinds the whole nest, not just the innermost dimension.

> ⚠ bricks rejects nested `OCCURS`, so a two-dimensional walk addresses a flat table with a computed subscript as shown above rather than `SLOT(I, J)`.

OCCURS arrays

```
01 TABLE-AREA.
   05 ROW OCCURS 15 TIMES.
      10 K  PIC X(8).
      10 N  PIC X(28).
01 I  PIC 9(4).
...
MOVE 'KEY-A' TO K(1).
MOVE 'KEY-B' TO K(I).
PERFORM SHOW-ROW VARYING I FROM 1 BY 1 UNTIL I > 15.
```

`OCCURS n TIMES` declares `n` copies of an item; references add a parenthesised 1-based subscript expression that picks one. The subscript can be any numeric expression — a literal, a data item, or arithmetic — and is bounds-checked at runtime. Subscripts work on group items (`ROW(5)`) and on elementaries inside the OCCURS group (`K(5)`) alike. The cap is 4096 occurrences per item; nested OCCURS (an OCCURS group inside another OCCURS group) is rejected at parse time.

Periods

Periods are scope-terminators, not per-statement: a statement inside an `IF ... END-IF` body has no period; only the unscoped statement at the paragraph level does.

Postfix NOT

IBM-style postfix NOT (`X NOT = Y`, `X NOT > Y`) is supported in `IF` / `EVALUATE` / `PERFORM UNTIL` conditions.

Relational operators

Both symbolic and English-spelled relational operators are accepted in `IF`, `EVALUATE WHEN`, and `PERFORM UNTIL` conditions. The two styles are interchangeable; legacy CICS applications typically use the English spelling.

The optional words `IS`, `THAN`, and `TO` are noise — present or omitted without changing meaning. `NOT` may prefix any operator and appears before or after `IS` (`IF X IS NOT EQUAL TO Y`).

Meaning	Symbolic	English-spelled forms (noise words bracketed)
equal	`=`	`[IS] EQUAL [TO]`
not equal	`<>`, `NOT =`	`[IS] NOT EQUAL [TO]`
greater	`>`	`[IS] GREATER [THAN]`
not greater (≤)	`NOT >`	`[IS] NOT GREATER [THAN]`
less	`<`	`[IS] LESS [THAN]`
not less (≥)	`NOT <`	`[IS] NOT LESS [THAN]`
greater or equal	`>=`	`[IS] GREATER [THAN] OR EQUAL [TO]`
less or equal	`<=`	`[IS] LESS [THAN] OR EQUAL [TO]`

The four lines below are equivalent — all four compile to the same comparison and run identically:

```
IF WK-MTH = 7         DISPLAY 'JULY'.
IF WK-MTH EQUAL 7     DISPLAY 'JULY'.
IF WK-MTH EQUAL TO 7  DISPLAY 'JULY'.
IF WK-MTH IS EQUAL TO 7  DISPLAY 'JULY'.
```

Combined conditions and parentheses

Conditions combine with `NOT`, `AND` and `OR` (precedence `NOT` > `AND` > `OR`) and can be grouped with parentheses anywhere a condition is allowed (`IF`, `PERFORM UNTIL`, `EVALUATE` objects):

```
IF A = 0 OR (A = 1 AND B = 2) ...
IF NOT (WS-CODE = 'X' OR WS-CODE = 'Y') ...
IF ((A + 1) * 2 = B) OR (C = 1) ...
```

`NOT` must be followed by a condition: `IF NOT NOT A = 1` is rejected with `NOT must be followed by a condition`.

Division in a condition follows IBM's intermediate-result rule (Enterprise COBOL Programming Guide, Appendix A): a quotient keeps `(d2 − d1)` or `dmax` decimal places, whichever is greater. `d2` and `d1` are the decimal places of the dividend and the divisor; `dmax` is the largest number of decimal places of any operand in the relation, both sides counted, divisors and exponents excluded. The quotient is truncated, not rounded:

Condition	Values	Compares	Result
`IF 5 / 2 > 2`	—	`2 > 2`	false (as on z/OS)
`IF A / 2 > 2`	`A PIC 9(3)` = 5	`2 > 2`	false
`IF B / 10 = 4.5`	`B PIC 9(3)` = 45	`4.5 = 4.5`	true (the literal gives dmax 1)

An `EVALUATE` subject and each object are evaluated on their own operands (their own `dmax`): `EVALUATE B / 10 WHEN 4.5` does **not** match for `B = 45`, because `B / 10` alone has no decimal places and gives 4. `COMPUTE`, subscripts and reference modifiers are not affected — `COMPUTE` still divides at the receiving field's scale.

A parenthesis that opens an arithmetic operand still works: `IF (A + 1) > B` and `IF (A + 1) * 2 = B OR (C = 1)` read the first group as arithmetic because a relational or arithmetic operator follows it.

Abbreviated combined relation conditions

After a relation, later relations in the same condition may omit the subject, or the subject and the operator; the last stated subject and operator are filled in (IBM rules):

Written	Means
`A = 1 OR 2 OR 3`	`A = 1 OR A = 2 OR A = 3`
`A > 1 AND < 9`	`A > 1 AND A < 9`
`A NOT = 1 AND 2`	`A NOT = 1 AND A NOT = 2`
`A > B AND NOT < C OR D`	`(A > B AND A NOT < C) OR A NOT < D`
`A > 5 AND NOT 9`	`A > 5 AND NOT (A > 9)`
`NOT A = B OR C`	`NOT (A = B) OR A = C`

A `NOT` directly in front of a relational operator belongs to the operator and is carried along with it; any other `NOT` is the logical NOT. A level-88 condition-name after `AND` / `OR` is a condition, not an implied object (`IF A =`

`1 OR WS-DONE`). As in Enterprise COBOL, an omitted subject may not open a parenthesised group: `A = B AND (< C OR D)` is rejected with `a relational operator cannot start a parenthesised condition`. A group that starts with a full relation may use the short forms inside it: `IF B = 0 AND (A = 4 OR 5 OR 6).`

> **Changed behaviour.** Earlier bricks releases accepted these forms but evaluated them wrongly: `IF A = 1 OR 2 OR 3` was true whatever the value of `A`, and `IF C NOT = 'L' AND 'H'` dropped the `NOT` on the second part, so it was true for `C = 'H'`. Both now follow the IBM rules above (`C NOT = 'L' AND C NOT = 'H'` is false for `C = 'H'`). Check old programs that relied on the previous results.

Class condition tests

`X IS [NOT] {NUMERIC | ALPHABETIC | ALPHABETIC-UPPER | ALPHABETIC-LOWER}` tests the current **byte contents** of a data item against a character class — not its static `PIC` class. A `PIC 9(4)` field holding `0000` is NUMERIC; the same field after a partial move that injects letters is not.

`IS` is optional noise; `NOT` negates as in any other condition.

```
IF WS-INPUT IS NUMERIC
   PERFORM CALC
ELSE
   DISPLAY 'NON-NUMERIC INPUT'.
IF WS-NAME IS ALPHABETIC
   PERFORM ACCEPT-NAME.
```

`ALPHABETIC` accepts uppercase letters, lowercase letters, and space. `ALPHABETIC-UPPER` and `ALPHABETIC-LOWER` restrict to their case (plus space). For `PIC S9` signed numerics the byte 0 sign position is allowed to hold `' '`, `'+'`, or `'-'`.

Sign condition tests

`X IS [NOT] {POSITIVE | NEGATIVE | ZERO}` tests the numeric sign of a numeric expression. The operand must be a numeric data item (`PIC 9...` or `PIC S9...`) or a numeric expression; a sign test on a non-numeric operand is a runtime error rather than a silent false.

```
IF WS-BALANCE IS NEGATIVE
   DISPLAY 'OVERDRAWN'.
IF AMOUNT IS ZERO
   PERFORM SKIP-ENTRY.
```

Signed zero is neither POSITIVE nor NEGATIVE; an unsigned PIC (`PIC 9...` without `S`) can never be NEGATIVE — that short- circuits to false without inspecting the value.

88-level condition names

A level-88 line in DATA DIVISION declares a boolean condition tied to the **preceding non-88 data item**. The condition is true when the parent's current value matches one of the listed values or falls within a `THRU` range. Reference as a bare name in any PROCEDURE DIVISION condition.

```
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01  RESP-CODE  PIC 9(3).
           88 RESP-OK        VALUE 0.
           88 RESP-MISSING   VALUE 13.
           88 RESP-RECOVER   VALUES 12, 26, 80.
           88 RESP-FATAL     VALUE 100 THRU 999.
       PROCEDURE DIVISION.
       MAIN.
           EXEC CICS READ FILE('CUSTOMER')
                          INTO(REC) RIDFLD(KEY)
                          RESP(RESP-CODE) END-EXEC.
           IF RESP-OK
              CONTINUE
           ELSE IF RESP-RECOVER
              PERFORM RETRY
           ELSE IF RESP-FATAL
              PERFORM ABEND.
```

`VALUES` (plural) and `VALUE` (singular) are interchangeable. Multiple values may be separated by commas or by spaces. `THRU` (synonym `THROUGH`) declares an inclusive range; ranges and single values can be mixed in one declaration: `88 OK VALUES 0, 1 THRU 5, 9.`

Comparison style follows the parent's PIC class — numeric parents compare numerically, alphanumeric parents compare byte-by-byte (after rstripping trailing spaces). `NOT` and the AND/OR connectives compose normally: `IF NOT RESP-OK OR RESP-FATAL`.

A condition-name lives in its own namespace, separate from data items — `01 STATUS PIC X.` and `88 STATUS-OK VALUE 'Y'.` are two different lookups. Declaring an 88 whose name collides with an existing data item is rejected at parse time.

Intrinsic functions

The shipped intrinsic library is:

Function	Args	Result
`FUNCTION UPPER-CASE(s)`	1 alphanumeric	s uppercased: a-z and the Latin-1 letters (ä → Ä); ß / ÿ unchanged.
`FUNCTION LOWER-CASE(s)`	1 alphanumeric	s lowercased: A-Z and the Latin-1 letters (Ä → ä).
`FUNCTION TRIM(s)`	1 alphanumeric	s with leading and trailing ASCII blanks removed (both ends; one-arg form). NBSP `X'A0'` is kept.
`FUNCTION REVERSE(s)`	1 alphanumeric	s reversed character by character.
`FUNCTION LENGTH(s)`	1 alphanumeric	numeric character count of the operand's storage (PIC X(8) → 8; `'Grüße'` → 5).
`FUNCTION BYTE-LENGTH(s)`	1 alphanumeric	same as `LENGTH` (one byte per character).
`FUNCTION ORD(c)`	1 one-character	ordinal position of c: byte value + 1 (`ORD('A')` = 66, `ORD('Ä')` = 197).
`FUNCTION CHAR(n)`	1 integer 1-256	the character whose ordinal position is n (`CHAR(197)` = Ä).

Function	Args	Result
`FUNCTION NUMVAL(s)`	1 alphanumeric	numeric value of s after stripping leading / trailing whitespace; errors on non-numeric content (does not silently coerce to zero).
`FUNCTION POS(needle, haystack)`	2 alphanumeric	1-based character position of needle in haystack, or 0 when not found. Mirrors REXX's `POS()`. Empty needle returns 0.

See COBOL character set for how characters map to bytes.

Numeric-returning intrinsics (`LENGTH`, `BYTE-LENGTH`, `NUMVAL`, `POS`, `ORD`) participate in `IF` / `EVALUATE` / arithmetic comparisons as numeric operands, so `IF FUNCTION POS(NEEDLE, HAY) > 0` works without an intermediate `COMPUTE`. The remaining intrinsics return alphanumeric and compare by bytes (rstripped).

`FUNCTION POS` is the substring-search hook used by `runtime/cobol/gusl.cob` to filter the customers file when the operator types a search term at GUST's `S` action.

INSPECT

```
INSPECT subject TALLYING counter FOR (ALL | LEADING | CHARACTERS)
                                     [needle] [BEFORE/AFTER INITIAL delim]
                                     [, ...]
INSPECT subject REPLACING (ALL | LEADING | FIRST | CHARACTERS)
                          [needle] BY replacement
                          [BEFORE/AFTER INITIAL delim]
                          [, ...]
INSPECT subject CONVERTING from TO to
                          [BEFORE/AFTER INITIAL delim]
```

Quantifiers:

- **ALL** — every occurrence of `needle` (TALLYING) or every occurrence rewritten (REPLACING).
- **LEADING** — only successive occurrences at the start of the in-scope region.
- **FIRST** — only the first occurrence (REPLACING only).
- **CHARACTERS** — every character in the in-scope region; for TALLYING this just counts characters, for REPLACING it overwrites each one with the first byte of `replacement`.

Limiters:

- **BEFORE INITIAL delim** narrows the in-scope region to everything before the first occurrence of `delim`.
- **AFTER INITIAL delim** narrows to everything after the first occurrence of `delim`.

Both can be set on the same phrase; AFTER picks the start, BEFORE picks the end.

Multiple phrases can be combined per statement, optionally separated by commas:

```
INSPECT REC REPLACING ALL ',' BY '|' ALL ';' BY '/'.
INSPECT REC TALLYING C FOR LEADING '0', ALL 'X' AFTER INITIAL ' '.
```

Counter behaviour: TALLYING **accumulates** — successive `INSPECT TALLYING` statements add to the existing counter value rather than resetting, matching IBM semantics. REPLACING phrases run left-to- right in declared order, so later phrases see the output of earlier ones.

Converting: `CONVERTING from TO to` builds a positional per-character map and translates every character of the in-scope region through it — `from[k]` becomes `to[k]`, exactly like Unix `tr` or the old COBOL `TRANSFORM`.

This is the verb for case folding:

```
INSPECT VMUSR CONVERTING 'abcdefghijklmnopqrstuvwxyz'
                      TO 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'.
```

If `from` repeats a character, the **leftmost** occurrence defines its mapping and later duplicates are ignored. The sets need not be the same length: only the overlapping `min(len(from), len(to))` positions map and any surplus is silently ignored (no runtime error), matching the REPLACING leniency philosophy. IBM allows only **one** CONVERTING phrase per INSPECT (no comma list), and `BEFORE/AFTER INITIAL delim` narrows the region just like REPLACING.

Subject, counter, and operands all accept qualified data-names (`INSPECT REC OF DET ...`). Trailing PIC X padding is rstripped from both the subject and the needle before matching, so a 30-byte filter containing `FOO` plus trailing spaces matches against the meaningful content of the subject rather than against the padding.

IBM's CONVERTING form is now supported (see the Converting paragraph above).

EVALUATE

```
EVALUATE subject-1 [ALSO subject-2 ...]
   {WHEN object-1 [ALSO object-2 ...]} ...
       statements
   [WHEN OTHER statements]
[END-EVALUATE]
```

A subject is a data item, literal, arithmetic expression, a condition, `TRUE` or `FALSE`. Each `WHEN` must list exactly one object per subject, separated by `ALSO`; a mismatch is a parse error (`WHEN has 1 object(s) but EVALUATE has 2 subject(s)`). Objects pair with their subject like this:

Subject	Allowed objects
item / literal / arithmetic expression	`[NOT] value`, `[NOT] value THRU value`, `ANY`
`TRUE`, `FALSE`, or a condition	a condition (88-level names, `NOT`, `AND` / `OR`, parentheses), `TRUE`, `FALSE`, `ANY`

`THRU` (or `THROUGH`) is an inclusive range: `WHEN 1 THRU 5` means `subject >= 1 AND subject <= 5`, compared with the same rules as `IF`. Several `WHEN` phrases in a row share one body. The first matching `WHEN` runs, with no fall-through; `WHEN OTHER` runs when none matched. Every subject is evaluated once. `END-EVALUATE` may be left out when a period ends the statement.

A `WHEN` takes one value. `WHEN 'A' OR 'B'` is rejected with `WHEN takes one value – write alternatives as separate WHEN lines`; stack the `WHEN`s instead (`WHEN 'A'` / `WHEN 'B'` sharing one body). This does not apply to a condition object (under `EVALUATE TRUE`, `FALSE` or a condition subject), where `OR` is part of the condition: `WHEN A = 1 OR B = 2` and the abbreviated `WHEN A = 1 OR 3` both work.

```
EVALUATE TRUE
    WHEN WS-QTY = 0          MOVE 'EMPTY' TO WS-STATE
    WHEN WS-QTY < 10
    WHEN RUSH-ORDER          MOVE 'LOW'   TO WS-STATE
    WHEN OTHER               MOVE 'OK'    TO WS-STATE
END-EVALUATE

EVALUATE WS-ACTION ALSO WS-LEVEL
    WHEN 'A' ALSO 1 THRU 3   PERFORM ADD-BASIC
    WHEN 'A' ALSO ANY        PERFORM ADD-FULL
    WHEN NOT 'Q' ALSO ANY    PERFORM OTHER-ACTION
END-EVALUATE

EVALUATE WS-QTY > 0 ALSO RUSH-ORDER
    WHEN TRUE  ALSO TRUE     PERFORM SHIP-NOW
    WHEN TRUE  ALSO FALSE    PERFORM SHIP-LATER
    WHEN FALSE ALSO ANY      PERFORM BACKORDER
END-EVALUATE
```

STRING and UNSTRING

STRING joins the sending items into the receiver. `DELIMITED BY SIZE` moves the whole item — a data item at its full PIC width (trailing spaces included), a literal exactly as written; `DELIMITED BY x` stops at the first `x`. Several sending items may share one `DELIMITED BY`. bricks never trims a sending identifier: to drop its padding, use `DELIMITED BY SPACE`. The receiver must not be reference-modified (parse error `STRING INTO receiver must not be reference-modified`, as IBM requires).

- Without `POINTER` the receiver is space-filled first, then written (bricks behaviour; IBM leaves unwritten positions alone).
- With `[WITH] POINTER ptr` the IBM rules apply: writing starts at position `ptr`, `ptr` goes up by one per character, and the rest of the receiver keeps its contents — so successive `STRING`s can build a line piece by piece.
- Overflow: the data did not fit, or `ptr` was < 1 or past the end at the start (then nothing moves). `ON OVERFLOW` runs on overflow, `NOT ON OVERFLOW` otherwise.

```
01 WS-NAME  PIC X(10) VALUE 'JOE SMITH'.
01 WS-LINE  PIC X(12) VALUE '............'.
01 WS-PTR   PIC 99.
01 WS-TRUNC PIC X VALUE 'N'.
...
MOVE 1 TO WS-PTR
STRING 'Name: ' DELIMITED BY SIZE
       WS-NAME  DELIMITED BY SPACE
    INTO WS-LINE WITH POINTER WS-PTR
    ON OVERFLOW MOVE 'Y' TO WS-TRUNC
END-STRING
DISPLAY WS-LINE '|' WS-PTR
```

Output: `Name: JOE...|10` — the literal keeps its trailing space, the last three dots are untouched, and `WS-PTR` points at the next free position.

UNSTRING splits the sending item, scanning from `POINTER` (default 1) over its full length. The sending item must be a data item, as in IBM; a literal, figurative constant or function is rejected with `UNSTRING source must be a data item, got "..."`.

- each receiver gets the characters up to the next delimiter; with several delimiters (`OR`) the first one found wins, and on a tie the first one listed; `ALL` treats a run of the same delimiter as one; two

delimiters in a row (without `ALL`) give an empty field, i.e. spaces or zeros;

- `DELIMITER IN` receives the delimiter that ended the field (spaces at the end of the data), `COUNT IN` the field's length; both need `DELIMITED BY` (else `DELIMITER IN / COUNT IN need DELIMITED BY`);
- without `DELIMITED BY` each receiver takes as many characters as its own size;
- `TALLYING IN` is increased by the number of receivers filled; `POINTER` ends one past the last character examined;
- receivers that are not reached keep their contents. When the data is used up exactly on a delimiter (`'a|b|'` in a 4-byte field with three receivers), the third receiver is not reached and keeps its old value, as IBM specifies. Earlier bricks versions set it to spaces;
- overflow: `POINTER` < 1 or past the end at the start, or characters are left over when every receiver is filled. `END-UNSTRING` is optional.

```
01 WS-REC    PIC X(20) VALUE 'AB   CD,EF'.
01 WS-F1     PIC X(5).
01 WS-F2     PIC X(5).
01 WS-D1     PIC X.
01 WS-N1     PIC 99.
01 WS-PTR    PIC 99 VALUE 1.
01 WS-FIELDS PIC 99 VALUE 0.
...
UNSTRING WS-REC DELIMITED BY ALL SPACES OR ','
    INTO WS-F1 DELIMITER IN WS-D1 COUNT IN WS-N1
         WS-F2
    WITH POINTER WS-PTR TALLYING IN WS-FIELDS
    ON OVERFLOW DISPLAY 'MORE DATA'
END-UNSTRING
DISPLAY WS-F1 '|' WS-D1 '|' WS-N1 '|' WS-F2 '|' WS-PTR '|' WS-FIELDS
```

Output: `MORE DATA`, then `AB | |02|CD |09|02`. The run of spaces after `AB` is one delimiter (`ALL`), `WS-F2` stops at the comma, and `EF` plus the padding are left over, which is the overflow. `POINTER` 9 is where a second `UNSTRING` would carry on.

> **Changed behaviour.** Earlier bricks releases read `INTO B COUNT IN P` as a second receiver (the qualified name `COUNT IN P`); the program compiled and then failed at run time. `DELIMITER IN`, `COUNT IN`, `WITH POINTER` and `TALLYING IN` are now recognised wherever IBM allows them.

JSON GENERATE and JSON PARSE

The IBM Enterprise COBOL 6.4 statements turn a record into JSON text and back, so a web API program no longer has to build or pick apart JSON by hand. Both work on plain alphanumeric fields, which is exactly what `EXEC CICS WEB RECEIVE INTO` and `WEB SEND FROM` use.

```
01 LST-REC.
   05 LST-KEY.
      10 LST-USER   PIC X(16).
      10 LST-ID     PIC 9(6).
   05 LST-NEXTITEM  PIC 9(8).
   05 LST-COUNT     PIC 9(5).
   05 LST-NAME      PIC X(80).
01 JSON-BUF         PIC X(4000).
01 JS-LEN           PIC 9(5).

    JSON GENERATE JSON-BUF FROM LST-REC COUNT IN JS-LEN
        NAME OF LST-REC IS OMITTED LST-ID IS 'id' LST-NAME IS 'name'
        SUPPRESS LST-USER LST-NEXTITEM
        ON EXCEPTION PERFORM FAIL-JSON
    END-JSON
    EXEC CICS WEB SEND FROM(JSON-BUF) LENGTH(JS-LEN) END-EXEC
```

gives `{"LST-KEY":{"id":12},"LST-COUNT":3,"name":"Buy milk"}`.

Special registers

Register	Shape	Meaning
`JSON-CODE`	`PIC S9(9) BINARY`	0 after a successful statement, else the exception code (tables below).
`JSON-STATUS`	`PIC 9(9) BINARY`	Set by `JSON PARSE`: the sum of the non-exception reason codes.

Both are declared automatically, like `EIBRESP` and `SQLCODE`. JSON-STATUS is additive (1 + 2 = 3 means "an item had no member and a member had no item"); `IF JSON-STATUS = 0` tests for a perfect match.

JSON GENERATE

```
JSON GENERATE identifier-1 FROM identifier-2
   [COUNT [IN] identifier-3]
   [NAME [OF] identifier-4 [IS] {literal | OMITTED} ...]
   [SUPPRESS {identifier-5 [WHEN fig [OR fig] ...]
             | EVERY [NUMERIC | NONNUMERIC] WHEN fig [OR fig] ...} ...]
   [CONVERTING {identifier-6 TO JSON BOOLEAN USING {condition-name | literal}
              | identifier-6 TO JSON NULL USING fig} ...]
   [[ON] EXCEPTION imperative] [NOT [ON] EXCEPTION imperative]
[END-JSON]
```

`fig` is `ZERO`, `SPACE`, `LOW-VALUE` or `HIGH-VALUE` (singular or plural). The phrases must appear in the order shown (a phrase out of order is a parse error, as in IBM's syntax diagram); entries of `SUPPRESS` and `CONVERTING` may be separated by `ALSO`.

- **Shape.** `{"REC":{...}}` names the `FROM` item. `NAME OF rec IS OMITTED` drops that wrapper, which is what web clients want. The `FROM` item may be a group, an elementary item, a subscripted occurrence, or a whole `OCCURS` table — with `OMITTED` the table becomes an anonymous array `[{...},{...}]`, and an elementary item becomes just its value (`1234`, `"BOB"`; IBM APAR PH58384).
- **Names.** A member's name is its data name; `NAME OF item IS 'lit'` replaces it. bricks upper-cases data names when it reads the source, so `05 Cust-Name` comes out as `"CUST-NAME"` (IBM keeps the source spelling). Use `NAME` for lower-case JSON names.
- **Groups** become objects, `OCCURS` items arrays. `FILLER`, `REDEFINES` items (and everything below them) and `USAGE POINTER` items are left out. A group whose members are all left out or suppressed disappears; the outermost item is always written, even as `{"REC":{}}`.

- **Numbers** are written as JSON numbers without leading zeros, with a – sign and a decimal point from the PIC's `V`: `PIC 9(6)` 12 gives `12`, `PIC S9(3)V999` gives `-63.435`, `PIC 9V99` 1.5 gives `1.50` (fraction zeros stay), zero gives `0`. COMP and COMP-3 work the same. Numeric-edited items are strings, trimmed of leading and trailing spaces.
- **Text** loses its trailing spaces; leading spaces stay. A field of only spaces is written as `" "` (IBM). `"` and `\` are escaped, as are control characters (`\t \n \r \b \f`, others below `X'20'` as `\u00hh`). Every byte from `X'80'` up (Latin-1, the graphic plane, HIGH-VALUES `X'FF'`) is written raw: the JSON text is SBCS like the receiving `PIC X` item, and the web edge converts it to UTF-8 (see COBOL character set).
- **SUPPRESS** `item` leaves that item (or group) out. `item WHEN fig` and `EVERY [NUMERIC | NONNUMERIC] WHEN fig` leave out an elementary item whose value matches: `ZERO` matches numbers equal to zero and text of only `0` characters; `SPACE`, `LOW-VALUE`, `HIGH-VALUE` match DISPLAY items only. Elements of an elementary `OCCURS` table are not suppressed one by one (that would shift the positions of the rest).
- **CONVERTING** `item TO JSON BOOLEAN USING cond-name | literal` writes `true` when the item satisfies the condition-name (which must be an 88 of that item) or equals the literal, otherwise `false`. `item TO JSON NULL USING fig` writes `null` when the item is all `fig`.
- **COUNT IN** receives the number of characters generated. The rest of the receiver is filled with spaces. The receiver may be subscripted or reference-modified (`JSON GENERATE BUF(8:30) ...`); then only that window is written and space-filled, and the too-small test and `COUNT IN` are relative to it.
- **Exception.** If the receiver is too small, `JSON-CODE` is 1, the receiver holds what fit, `COUNT IN` holds the receiver's length and `ON EXCEPTION` runs. Without `ON EXCEPTION` the program just continues. Otherwise `JSON-CODE` is 0 and `NOT ON EXCEPTION` runs.

JSON PARSE

```
JSON PARSE identifier-1 INTO identifier-2
   [WITH DETAIL]
   [NAME [OF] identifier-3 [IS] {literal | OMITTED} ...]
   [SUPPRESS identifier-4 ...]
   [CONVERTING {identifier-5 FROM JSON BOOLEAN USING
               {cond-1 [AND cond-2] | literal-1 [AND literal-2]}} ...]
   [[ON] EXCEPTION imperative] [NOT [ON] EXCEPTION imperative]
[END-JSON]
```

```
01 IN-REC.
   05 IN-TITLE PIC X(80).
   05 IN-PRIO  PIC X.
   05 IN-DONE  PIC X.

*> body: {"title":"Buy milk","priority":"H","done":true}
    JSON PARSE WS-BODY INTO IN-REC
        NAME OF IN-REC IS OMITTED IN-TITLE IS 'title'
             IN-PRIO IS 'priority' IN-DONE IS 'done'
        CONVERTING IN-DONE FROM JSON BOOLEAN USING 'Y' AND 'N'
        ON EXCEPTION PERFORM BAD-REQUEST
    END-JSON
```

- identifier-1 holds the text (it may be reference-modified); trailing spaces and LOW-VALUES are ignored.
- **Matching.** Without `OMITTED` the text must be `{"IN-REC":{...}}`. With `NAME OF IN-REC IS OMITTED` the outermost object's members map straight onto IN-REC's subordinates (or, for a whole `OCCURS` table, the outermost array onto its occurrences). Data names match case-insensitively; a `NAME` literal replaces the data name and matches case-sensitively. `FILLER` and `REDEFINES` items never match; `SUPPRESS` items are skipped.

- **Values** are assigned with `MOVE` rules: a number (exponent allowed) or a string holding a number goes to numeric and numeric-edited items, aligned on the decimal point (extra fraction digits are truncated); strings and numbers go to alphanumeric items (padded / truncated on the right). Objects go to groups, arrays to `OCCURS` items. `true` / `false` need a `CONVERTING` entry: the item gets literal-1 / cond-1's value for true and literal-2 / cond-2's value for false. `null` leaves the item unchanged. A `\uXXXX` escape is stored as its single SBCS byte (`\u00e9` → é, `\u20ac` → €), or `?` when the code point has none (CJK, emoji, C1 controls). Raw bytes of the JSON text are copied as they are.
- Members with no matching item are ignored and items with no member keep their contents; `JSON-STATUS` reports it:

JSON-STATUS	Meaning (IBM)
1	An item had no matching name, so it was not changed.
2	A JSON name matched no item.
4	An item had duplicate names with the same value.
8	A table had more elements than the JSON array.
16	A JSON array had more values than the table (extras ignored).
32	An item was not changed because the value was `null`.
64	A table element was not changed because the value was `null`.
128	SIZE ERROR in a numeric assignment (stored anyway, high digits lost).
256	Text was truncated in an alphanumeric assignment (stored anyway).

JSON-CODE	Exception (IBM)
100	The JSON text is invalid (also: nesting deeper than 64 levels, top level not an object).
101	The JSON text is empty or only whitespace.
102	Extra characters after the outermost value.
103	Duplicate names with different values; the leftmost was kept.
104	A value does not fit its item (object into an elementary item, text into a numeric item, ...).
106	No JSON name matched any item. In the `{"REC":{...}}` form the outer name matching the `INTO` item already counts as a match, so `{"REC":{"foo":1}}` is code 0 with JSON-STATUS 3.
107	`true` / `false` for an item without `CONVERTING`.
108	A top-level array without `NAME ... OMITTED` on an `OCCURS` item.

An exception stops the statement and runs `ON EXCEPTION`. The whole text is checked before anything is stored, so invalid JSON (100-102) leaves the record untouched; a later exception (103, 104, 107) leaves the members before it assigned. `WITH DETAIL` writes one log line per statement with `JSON-CODE`, `JSON-STATUS` and the reason.

Differences from IBM: names come out upper-case (see above); the single condition-name form `USING cond-1` cannot set "false" because bricks has no `88 ... WHEN SET TO FALSE` — JSON `false` gives code 104, so use `USING cond-1 AND cond-2`. Invalid JSON text leaves the whole record untouched, where IBM may already have updated part of it. `ENCODING`, `IGNORING`, `INDICATING`, `CONVERTING ... FROM JSON NULL` and `OCCURS DEPENDING ON` are not supported.

COBOL pointer model: `USAGE POINTER` / `SET ADDRESS OF`

Bricks implements the IBM COBOL/CICS pointer surface so a program can address dynamically allocated or CICS-owned storage. **Pointers are opaque handles, not machine addresses** — there is no pointer arithmetic and they are not displayable numbers.

`USAGE POINTER` items

Declared with no `PIC` clause:

```
01 WS-PTR  USAGE POINTER.        *> 'USAGE IS POINTER' also accepted
```

A pointer occupies a 4-byte fullword slot and starts **NULL**. Its value comes from `EXEC CICS ADDRESS` / `GETMAIN`, from `ADDRESS OF identifier`, or from another pointer; it can be compared against `NULL` and against `ADDRESS OF X`, and moved like any other item. (Internally a pointer's value is a small integer handle; the program never sees or depends on the integer.)

`ADDRESS OF identifier`

`ADDRESS OF X` is the address of data item `X` — usable on the right of `SET`, and in an equality test:

```
SET WS-PTR TO ADDRESS OF WS-REC.
IF WS-PTR = ADDRESS OF WS-REC THEN DISPLAY 'same' END-IF.
```

`ADDRESS OF` of the same item always compares equal (handles are interned), so pointer equality is meaningful.

`NULL` / `NULLS`

The figurative `NULL` (synonym `NULLS`) is the null pointer:

```
SET WS-PTR TO NULL.
IF WS-PTR = NULL THEN DISPLAY 'unset' END-IF.
```

A freshly declared `USAGE POINTER` item, and any based 01 that has not been pointed anywhere, are NULL.

`SET P TO {ADDRESS OF X | NULL | ptr}`

Assigns a pointer variable: to the address of an item, to NULL, or to the value of another pointer.

`SET ADDRESS OF L TO {ptr | ADDRESS OF X | NULL}`

Redirects a **based** LINKAGE 01 onto bytes. After the SET, every reference to `L`'s children reads / writes the addressed storage:

```
LINKAGE SECTION.
01 GM-AREA BASED.
   05 GM-TEXT PIC X(16).
PROCEDURE DIVISION.
    EXEC CICS GETMAIN SET(WS-PTR) FLENGTH(16) END-EXEC.
    SET ADDRESS OF GM-AREA TO WS-PTR.    *> map the record onto the buffer
    MOVE 'POINTER-OK' TO GM-TEXT.        *> writes into the GETMAIN bytes
```

`SET ADDRESS OF L TO NULL` un-addresses `L`; a subsequent read or write through `L` raises a **clean NULL-pointer abend** — the task ends with a diagnostic, the server never crashes. The same applies to a based 01 that was never SET, and to dereferencing through a pointer whose GETMAIN buffer was already `FREEMAIN`-ed.

Based LINKAGE 01s, `DFHEIB`, `DFHCOMMAREA`

`DFHCOMMAREA` and `DFHEIB` are themselves based 01s. `DFHCOMMAREA` ships with a default buffer (so a plain `MOVE … TO DFHCOMMAREA` works without any SET) yet can be redirected: `SET ADDRESS OF DFHCOMMAREA TO ADDRESS OF WS-NEW`. `DFHEIB`'s children keep the bare IBM names so `EIBRESP`, `EIBCALEN`, etc. are referenced unqualified. See LINKAGE SECTION in Chapter 21.

`SET ptr UP BY n` / `SET ptr DOWN BY n` — pointer arithmetic

`SET ptr UP BY n` advances a `USAGE POINTER` item by `n` bytes; `SET ptr DOWN BY n` moves it back. `n` is a **non-negative integer** displacement — a literal, a data item, or an arithmetic expression that evaluates to a whole number (the same evaluator `COMPUTE` uses). A non-integer (e.g. `1.5`) or a negative amount is rejected at `SET` time; `UP`/`DOWN` carry the direction, so use `DOWN BY n` rather than `UP BY -n`:

```
01 WS-REC PIC X(10) VALUE 'AAAAAAAAAA'.
01 P      USAGE POINTER.
01 L BASED.
   05 L-FLD PIC X(6).
...
    SET P TO ADDRESS OF WS-REC.
    SET P UP BY 4.                *> P now addresses byte 4 of WS-REC
    SET ADDRESS OF L TO P.
    MOVE 'XXXXXX' TO L-FLD.       *> writes WS-REC bytes 4..9
    SET P DOWN BY 4.              *> back to the start of WS-REC
```

Nuance — arithmetic stays within the pointer's current target buffer. Bricks pointers are interned `(buffer, offset)` handles, not flat region addresses, so `UP`/`DOWN BY` adjusts the offset *inside the same buffer* the pointer already names; it does not let you walk across unrelated allocations. Moving a pointer past the end of its buffer is allowed at `SET` time (the address is interned, never validated), but a **dereference** through it raises a clean bounds abend — exactly as with any out-of-range based access. Moving below offset `0`, or arithmetic on a NULL/never-set pointer, raises a clean diagnostic at `SET` time. To walk an array inside a GETMAIN buffer you may still prefer a based 01 with an `OCCURS` table and subscript it, which keeps the bounds check per element.

See `EXEC CICS ADDRESS` and `GETMAIN` / `FREEMAIN` in Chapter 6 for the verbs that produce pointer handles, and `runtime/cobol/gmpt.cob` (TRANSID `GMPT`) for a worked example.

Chapter 23. The EIB block in COBOL

`EIBRESP`, `EIBRESP2`, `EIBAID`, `EIBCPOSN`, `EIBCALEN`, `EIBTRMID`, `RC`, and `DFHCOMMAREA` are auto-injected if not declared, so most programs need no boilerplate. The same goes for the SQLCA fields (`SQLCODE`, `SQLSTATE`, `SQLERRMC`, … — see Chapter 26) and the `JSON-CODE` / `JSON-STATUS` special registers set by `JSON GENERATE` and `JSON PARSE` (see JSON GENERATE and JSON PARSE). After every `EXEC CICS`, `EIBRESP` and `EIBRESP2` are populated by the handler the same way they are for REXX. The legacy "test `EIBRESP` after every verb" pattern is the most common idiom; for non-local error handling without a REXX-style `SIGNAL ON ERROR`, use `EXEC CICS HANDLE CONDITION` / `EXEC CICS HANDLE ABEND` (Chapter 10).

See Chapter 11 for the field meanings.

Chapter 24. EXEC CICS in COBOL

The COBOL parser collects every token between `EXEC CICS` and `END-EXEC` and reconstructs the body verbatim for the same `cics.ParseCommand` REXX uses. Three consequences:

- **Map-field names must match group-child names exactly.** When `EXEC CICS SEND MAP('CUST1') FROM(SCR)` fires, the CICS handler asks the frame for `SCR.INFOLINE`, `SCR.MSG`, etc. — so SCR must have children spelled exactly as the map declares fields. Real COBOL solves this with BMS-generated copybooks; bricks doesn't ship those, so the operator declares fields by hand.
- `DFHCOMMAREA` **is the COMMAREA marshalling slot.** Caller passes bytes in via the frame; sub-program reads with `MOVE DFHCOMMAREA TO ...`, mutates a working copy, and `MOVE ... TO DFHCOMMAREA`. The caller gets back exactly the length it passed, trailing blanks and binary bytes included.
- **Command-line arguments arrive via** `EXEC CICS RECEIVE INTO(...)`**.** When the operator types `EXAM 1 2 3` at the blank prompt, the dispatcher hands the unedited line (TRANSID prefix included) to the first task in the chain. The program reads it with `EXEC CICS RECEIVE INTO(WS-INPUT) LENGTH(WS-LEN) END-EXEC` and parses with `UNSTRING WS-INPUT DELIMITED BY ' ' INTO TRANSID A B C END-UNSTRING`. Single-shot per task, then `EOC`. See `runtime/cobol/exam.cob`.

`EXEC CICS SEND TEXT FROM(area) [ERASE]` works the same way. Reconstruction routes through the shared `cics.Handler`, so any verb REXX can issue, COBOL can issue too. The body is laid out as a flat row-major buffer (every `cols` bytes = one row, default 80) — 3270 has no LF, so build `area` as a group of `PIC X(80)` children and fill them row by row, or compose a single `PIC X(n*80)` with a matching `MOVE` per row.

The full `EXEC CICS` verb set is documented in Part 4; behaviour is identical between REXX, COBOL and PL/I.

Chapter 25. Copybooks

A copybook is a fragment of COBOL source — typically declarations for constants and groups — kept in a separate file and pulled into a program at parse time with a single line:

```
       WORKING-STORAGE SECTION.
       COPY DFHAID.
       COPY DFHRESP.
       01 OWN-COUNTER PIC 9(4).
```

bricks expands every `COPY name.` directive at the source-text level before the lexer runs, so the rest of the parser sees one unbroken stream of COBOL.

The COPY directive

Syntax:

```
       COPY <name>.
```

- The directive must be on its own line. Anything before or after the `COPY` keyword on the same line is a parse error.
- `<name>` is the basename of a copybook file; leave the extension off. The lookup tries `<name>.cpy`, then `<name>.cbl`, then the exact `<name>`, all case-insensitive.
- The trailing period is required.
- Leading whitespace is tolerated, and an `*>` inline comment on the same line is fine: `COPY DFHAID. *> 3270 AID bytes.`
- The directive is case-insensitive (`COPY`, `Copy`, `copy` all work), and so is the name (`COPY DFHAID.` and `copy dfhaid.` resolve to the same file).

Where bricks searches

The search directory is configured by `copybook_dir=` in `bricks.cnf`. The default is `runtime/cobolcopy/` (auto-derived from `runtime_dir` when that line is set). The standard CICS copybooks ship under this directory, so a fresh install already has `DFHAID.cpy` and `DFHRESP.cpy` available.

Copybook names are restricted to a flat alphanumeric namespace: `A–Z`, `a–z`, `0–9`, `–`, `_`, and `.`. Names containing a path separator, a `..` sequence, or a leading dot are rejected before any filesystem access. A copybook itself may contain `COPY` directives; nesting is capped at depth 5 and cycles are detected explicitly.

Delivered copybook: DFHAID

`COPY DFHAID.` brings in the standard 3270 attention-identifier (AID) byte constants. Every AID is declared **twice**, once under the friendly bricks mnemonic and once under the IBM-traditional `DFH` alias — both names resolve to the same byte, and a program may use either style or mix them.

bricks mnemonic	IBM alias	AID byte	Key
ENTER	DFHENTER	X'7D'	Enter
CLEAR	DFHCLEAR	X'6D'	Clear
PA1	DFHPA1	X'6C'	PA1
PA2	DFHPA2	X'6E'	PA2
PA3	DFHPA3	X'6B'	PA3
PF01	DFHPF1	X'F1'	PF1
PF02	DFHPF2	X'F2'	PF2
PF03	DFHPF3	X'F3'	PF3
PF04	DFHPF4	X'F4'	PF4
PF05	DFHPF5	X'F5'	PF5
PF06	DFHPF6	X'F6'	PF6
PF07	DFHPF7	X'F7'	PF7
PF08	DFHPF8	X'F8'	PF8
PF09	DFHPF9	X'F9'	PF9
PF10	DFHPF10	X'7A'	PF10
PF11	DFHPF11	X'7B'	PF11
PF12	DFHPF12	X'7C'	PF12
PF13	DFHPF13	X'C1'	PF13

bricks mnemonic	IBM alias	AID byte	Key
PF14	DFHPF14	X'C2'	PF14
PF15	DFHPF15	X'C3'	PF15
PF16	DFHPF16	X'C4'	PF16
PF17	DFHPF17	X'C5'	PF17
PF18	DFHPF18	X'C6'	PF18
PF19	DFHPF19	X'C7'	PF19
PF20	DFHPF20	X'C8'	PF20
PF21	DFHPF21	X'C9'	PF21
PF22	DFHPF22	X'4A'	PF22
PF23	DFHPF23	X'4B'	PF23
PF24	DFHPF24	X'4C'	PF24

The bricks mnemonic uses uniform-width `PF01`..`PF24`; the IBM alias keeps the no-leading-zero form (`DFHPF1`..`DFHPF24`) everyone recognises from real CICS code.

Delivered copybook: DFHRESP

`COPY DFHRESP.` brings in the EXEC CICS condition-code constants for `EIBRESP`. Each code is declared twice as well — `RESP-X` mnemonic and `DFHRESP-X` traditional alias. Numeric values match the runtime's emitter table (`cics/resp.go`), so any code bricks returns from a verb has a named constant here.

The most useful entries:

bricks mnemonic	IBM alias	EIBRESP	Condition
RESP-NORMAL	DFHRESP-NORMAL	0	success
RESP-ERROR	DFHRESP-ERROR	1	generic error
RESP-EOC	DFHRESP-EOC	6	end-of-chain / nothing to RECEIVE
RESP-NOTFND	DFHRESP-NOTFND	13	record / item / file not found
RESP-DUPREC	DFHRESP-DUPREC	14	duplicate record on WRITE
RESP-DUPKEY	DFHRESP-DUPKEY	15	duplicate key in browse
RESP-INVREQ	DFHRESP-INVREQ	16	invalid request (bad args, sandbox violation)
RESP-IOERR	DFHRESP-IOERR	17	underlying I/O failure
RESP-NOSPACE	DFHRESP-NOSPACE	18	TS / TD queue full
RESP-NOTOPEN	DFHRESP-NOTOPEN	19	file not open / dataset not enabled
RESP-ENDFILE	DFHRESP-ENDFILE	20	end of browse
RESP-LENGERR	DFHRESP-LENGERR	22	length mismatch
RESP-QZERO	DFHRESP-QZERO	23	TS / TD queue empty (READQ TD returns this)
RESP-ITEMERR	DFHRESP-ITEMERR	26	TS item number out of range
RESP-PGMIDERR	DFHRESP-PGMIDERR	27	LINK / XCTL program not found
RESP-MAPFAIL	DFHRESP-MAPFAIL	36	RECEIVE MAP with no input
RESP-INVMPSZ	DFHRESP-INVMPSZ	38	map size mismatch
RESP-QIDERR	DFHRESP-QIDERR	44	TS queue id unknown

bricks mnemonic	IBM alias	EIBRESP	Condition
RESP-ROLLEDBACK	DFHRESP-ROLLEDBACK	82	unit-of-work rolled back

The full list (every code bricks emits) is in `runtime/cobolcopy/DFHRESP.cpy`. Open the file to see the codes not summarised above (`RESP-ILLOGIC`, `RESP-SIGNAL`, `RESP-QBUSY`, the various `RESP-INV...` codes, `RESP-SYSIDERR`, etc.).

Delivered copybook: SQLCA

`COPY SQLCA.` brings in named constants for `SQLCODE` and the most common `SQLSTATE` values, so embedded-SQL programs can write:

```
       COPY SQLCA.
       ...
       EXEC SQL SELECT name INTO :NM
                FROM customers WHERE id = :CUSTID END-EXEC.
       EVALUATE SQLCODE
           WHEN SQL-OK             PERFORM SHOW-ROW
           WHEN SQL-NODATA         PERFORM SHOW-NOT-FOUND
           WHEN SQL-MULTIPLEROWS   PERFORM SHOW-AMBIGUOUS
           WHEN OTHER              PERFORM SHOW-SQL-ERROR
       END-EVALUATE.
```

The SQLCA fields themselves (`SQLCODE`, `SQLSTATE`, `SQLERRMC`) are auto-injected by the bricks COBOL parser — no `COPY` needed for the data items. SQLCA only adds the constants programs compare against.

bricks mnemonic	IBM alias	SQLCODE	Meaning
SQL-OK	DB2-SUCCESS	0	success
SQL-NODATA	DB2-NOTFOUND	+100	no row / end-of-cursor
SQL-NOTFND	DB2-NOTFOUND	+100	alias
SQL-NOCONFIG	—	-1	SQL not configured in bricks.cnf
SQL-DDLREJECTED	—	-2	DDL rejected (CEDA-only)
SQL-GENERIC	—	-100	generic PG error (see SQLERRMC + log)
SQL-MULTIPLEROWS	DB2-DUPLICATE	-811	SELECT INTO returned >1 row

SQLSTATE constants are PIC X(5) and compare directly:

```
       IF SQLSTATE = SQLSTATE-DUP-KEY THEN
           MOVE 'Already exists.' TO MSG
       END-IF.
```

Constant	SQLSTATE	Class
SQLSTATE-OK	00000	success
SQLSTATE-NODATA	02000	no-data
SQLSTATE-WARNING	01000	warning
SQLSTATE-NOT-NULL	23502	NOT NULL constraint violated
SQLSTATE-FK-VIOL	23503	foreign-key constraint violated
SQLSTATE-UQ-VIOL	23505	UNIQUE constraint violated
SQLSTATE-DUP-KEY	23505	alias

Constant	SQLSTATE	Class
SQLSTATE-CHK-VIOL	23514	CHECK constraint violated
SQLSTATE-CONN-FAIL	08001	unable to connect to PG
SQLSTATE-CONN-LOST	08006	connection dropped mid-flight
SQLSTATE-INV-AUTH	28000	PG authentication failed
SQLSTATE-INV-PWD	28P01	wrong password
SQLSTATE-SYNTAX	42601	SQL syntax error
SQLSTATE-UNDEF-TBL	42P01	table doesn't exist
SQLSTATE-UNDEF-COL	42703	column doesn't exist
SQLSTATE-UNDEF-FN	42883	function doesn't exist
SQLSTATE-AMBIG-COL	42702	ambiguous column reference
SQLSTATE-INSUF-PRIV	42501	permission denied

The bricks SQL executor also logs every non-zero `SQLCODE`'s full PG error text via `brickslog.Sys` (one line on the operator console + the per-run log file). Programs render only the short mnemonic-driven message on the 3270 screen; the diagnostic detail lives in the log so PG hostnames / port numbers / wrapper text don't leak to terminal users.

Idiom: replacing raw literals

Before — opaque hex and bare integers, easy to mis-key:

```
       IF EIBAID = X'F3' THEN
           MOVE 'Y' TO EXIT-FLAG
       END-IF.
       IF EIBRESP = 13 THEN
           MOVE 'Customer not found.' TO MSG
       END-IF.
       IF EIBRESP = 23 THEN          *> TD queue empty
           MOVE 'Y' TO DONE-FLAG
       END-IF.
```

After — names a reader can scan:

```
       WORKING-STORAGE SECTION.
       COPY DFHAID.
       COPY DFHRESP.
       ...
       IF EIBAID = PF03 THEN          *> or DFHPF3 if you prefer
           MOVE 'Y' TO EXIT-FLAG
       END-IF.
       IF EIBRESP = RESP-NOTFND THEN
           MOVE 'Customer not found.' TO MSG
       END-IF.
       IF EIBRESP = RESP-QZERO THEN
           MOVE 'Y' TO DONE-FLAG
       END-IF.
```

Live examples: `qagc.cob` (PF03 cancel branch), `gusl.cob` (EVALUATE EIBAID with PF03 / PF07 / PF08 / PF12), `ordr.cob` (`RESP-QZERO` and `RESP-DUPREC` against READQ TD / WRITE FILE), `exam.cob` (`RESP-EOC` after RECEIVE). All under `runtime/cobol/`.

Restrictions

The bricks COPY preprocessor is deliberately minimal. The following real-CICS extensions are **not** supported:

- `COPY ... REPLACING ==X== BY ==Y==.` — token substitution. Rare in modern code; deferred until a concrete use case appears.
- `COPY xyz OF DFHCOB.` / `COPY xyz IN libname.` — library qualifier. bricks has one search root per process.
- `COPY xyz SUPPRESS.` — suppress copybook listing. bricks produces no listing to suppress.
- Embedded directives: multiple statements on one line including a `COPY` are rejected. The line must be `COPY <name>.` and nothing else.

If a copybook is edited, programs that include it will pick up the new contents only after the program's own source mtime changes (the program cache keys by the program path + mtime, not the copybook's). A trivial way to force a reload is to touch the COBOL file:

```
touch runtime/cobol/qagc.cob
```

Or use `CEMT P C` from a TSO/3270 session to flush the entire program cache.

Part 9. The PL/I language

> **Part 9 begins here.** Bricks ships a third front end beside REXX and COBOL: an interpreter for a business subset of IBM Enterprise PL/I (`pli/`). PL/I programs issue the same `EXEC CICS` and `EXEC SQL` commands, through the same handler, as the other two languages; only the statement terminator differs (`EXEC CICS ... ;`). The chapters that follow describe program structure, declarations, decimal arithmetic, statements, the built-in functions, the abend model that replaces ON-units, the CICS binding (EIB, maps, COMMAREA, handlers), `%INCLUDE`, and the deliberate deviations from Enterprise PL/I. Command-level behaviour is described in **Parts 4**, **5**, **6** and **7**.

The subset is defined so that a mainframe PL/I programmer can read any bricks PL/I program without a manual. Where Enterprise PL/I has a trap that hurts business code (implicit FLOAT declarations, SIZE disabled by default, intermediate decimal overflow, undefined uninitialised storage), the subset takes the safer behaviour and says so. Every such case is listed in Chapter 38.

The shipped sample is `CUSP` (`runtime/pli/cusp.pli`), a one-program PL/I port of the REXX `CUST` application; see Chapter 28 and Porting CUST to PL/I.

Chapter 30. PL/I program structure

A bricks PL/I program is one source file holding exactly one external procedure. A program that bricks dispatches as a transaction (or reaches through `EXEC CICS LINK` / `XCTL`) has `OPTIONS(MAIN)`:

```
/* HELP - minimum PL/I transaction: paint HELO1 and end. */
HELLOP: PROC OPTIONS(MAIN);
   DCL 1 SCR,
         2 INFOLINE CHAR(78),
         2 GREETING CHAR(20),
         2 FOOTER   CHAR(60);
   DCL UID CHAR(8);
   DCL RC  FIXED BIN(31);

   EXEC CICS ASSIGN USERID(UID) RESP(RC);
   SCR = '';
   SCR.GREETING = 'Hello, ' || TRIM(UID);
   SCR.FOOTER   = 'Press any key.';
   EXEC CICS SEND MAP('HELO1') FROM(SCR) ERASE RESP(RC);
   EXEC CICS RETURN;
END HELLOP;
```

The task ends normally (and commits its unit of work) when MAIN executes `RETURN`, falls off its `END`, or issues `EXEC CICS RETURN` or `XCTL`.

Source rules

Item	Rule
Source form	Free form. There are no margins; statements end with `;`.
Case	Keywords and identifiers are case-insensitive.
Identifiers	Up to 31 characters: `A–Z 0–9 _ # @ $`, starting with a letter or `_ # @ $`.
Comments	`/* ... */`, may span lines, **not** nested.

Item	Rule
Character constants	`'O''BRIEN'` (doubled quote as escape). Double quotes (`"ABC"`) are accepted too.
Bit constants	`'1'B`, `'0101'B`
Hex constants	`'C1C2'X` (character), `'00FF'XN` (numeric)
Constant suffixes	Only `B`, `X` or `XN` written directly after the closing quote are a suffix; any other word there is the next token, so `IF S = 'A'THEN` is valid, as in IBM PL/I.
Numeric constants	`123`, `123.45`, `.5`. No exponent form: there is no FLOAT.
Abbreviations	`DCL PROC CHAR DEC BIN PIC VAR INIT AUTO EXT OTHER`

Reserved words. Enterprise PL/I has no reserved words. The bricks subset reserves the statement keywords so the parser never has to guess:

```
IF DO END SELECT WHEN OTHERWISE OTHER CALL RETURN LEAVE ITERATE PUT EXEC DCL DECLARE PROC
PROCEDURE ELSE
```

None of these may name a variable, a procedure or a label (`DCL SELECT CHAR(1);` fails with `RESERVED NAME SELECT`). Every other keyword (`THEN`, `TO`, `BY`, `WHILE`, attribute names) is recognised by context. Names of the form `_<digits>_` (`_1_`, `_42_LEAF`) are reserved for compiler-generated aliases (Chapter 36).

Built-in names. A built-in function name (Chapter 34) cannot be redeclared as a level-1 variable or a procedure: `DCL INDEX FIXED BIN(31);` and `DCL DAYS(12) ...` are compile errors (`BUILTIN NAME DAYS`). A structure member (level 2 or deeper) may share a built-in name, which is what lets a map structure carry a field called `SEARCH` (Maps as structures). `DCL name BUILTIN;` is accepted and documents that `name` means the built-in.

PL/I character set

Program data is one byte per character in the bricks single-byte character set (bricks SBCS), exactly as for REXX and COBOL (see REXX character set and COBOL character set for the code page). The source file is UTF-8 and every quoted constant is converted to SBCS once, at compile time:

- Inside quotes: ASCII, Latin-1 and the bricks graphic plane. Any other character is `CHARACTER U+4E2D NOT IN CODE PAGE`.
- Comments may hold any valid UTF-8 text.
- Outside quotes and comments only ASCII is allowed, plus the PL/I NOT sign ¬ (U+00AC).

The NOT sign. PL/I's native NOT is ¬. It is EBCDIC 037 `X'5F'`, the key a 3270 keyboard has, so a program typed in the ISPF editor (area 4) on a 3270 uses ¬, ¬=, ¬< and ¬>. An ASCII keyboard usually lacks ¬; bricks accepts ^ as the same operator (^=, ^<, ^>), and <> for not-equal. OR is `|` or `!`. The shipped `cusp.pli` uses <> so it reads the same in every editor:

```
IF A ¬= 2 THEN PUT SKIP LIST('not-sign');
IF A ^= 2 THEN PUT SKIP LIST('caret');
IF A <> 2 THEN PUT SKIP LIST('angle');
IF ¬F & (A = B ! A > B) THEN PUT SKIP LIST('or-bang');
```

> **Programming Note.** ^ is not an EBCDIC 037 printable. Use it only in source edited outside bricks. Never put ^ (or |) in a character constant that is painted on a 3270 screen.

Procedures

```
ORDPRICE: PROC OPTIONS(MAIN);
   DCL AMT FIXED DEC(13,2) INIT(199.99);
   PUT SKIP LIST('TAX', CALCTAX(AMT));      /* called before its PROC */
   CALL SHOW('done');

   CALCTAX: PROC(A) RETURNS(FIXED DEC(13,2));
      DCL A FIXED DEC(13,2);
      RETURN (ROUND(A * 0.0825, 2));
   END CALCTAX;

   SHOW: PROC(MSG);
      DCL MSG CHAR(*) VAR;
      PUT SKIP LIST(MSG);
   END SHOW;
END ORDPRICE;
```

- **Internal procedures** may be nested and see their parents' variables through lexical scope. Names resolve over the whole enclosing block, so a procedure may be called before its `PROC` statement and a variable may be declared anywhere in its block. The usual layout, used by `cusp.pli`, is declarations first, then the main line, then the internal procedures.
- **Parameters** are passed by reference. `CHAR(*)` (length taken from the argument) is allowed on parameters only.
- **Function procedures** declare `RETURNS(attrs)` and are used in expressions. `CALL` of a function is a compile error (`CALL OF FUNCTION F`). A function that reaches its `END` without `RETURN(value)` abends `ABEN`.
- `RECURSIVE`**.** A procedure that is already active may call itself, directly or indirectly, only if it is declared `RECURSIVE`; otherwise the call abends `ABEN`. Call nesting is limited to 1000.
- `OPTIONS(MAIN)` procedures take no parameters (compile error `MAIN PROC HAS NO PARAMETERS`). Input arrives through the COMMAREA, `RECEIVE` or `RETRIEVE`.
- An external procedure **without** `OPTIONS(MAIN)` is a callable subprogram, reached through an `ENTRY ... EXTERNAL` declaration (External entries).

Where programs live

Item	Location
Programs	`pli_dir` in `bricks.cnf` (default `runtime/pli`), extension `.pli`
Include members	`pli_include_dir` in `bricks.cnf` (default `runtime/pliinc`), one file per member, `NAME.inc` (Chapter 37)
Transactions	`runtime/transactions.conf`, language `pli`

```
# bricks.cnf
pli_dir=runtime/pli
pli_include_dir=runtime/pliinc
```

Both keys default under `runtime_dir`, like `rexx_dir`, `cobol_dir` and `copybook_dir`, so a fresh install needs neither line.

A transaction names its program with the language token `pli`:

```
CUSP:pli:cusp.pli:PUBLIC
```

The token is exactly `pli` (case-insensitive). The spellings `PL/I` and `PL1` are rejected, so `transactions.conf` keeps one form per language. The optional 5th field (database binding) works as for the other languages (Chapter 26).

Checking and editing programs

- `brickscompile` accepts `.pli` files and directories holding them. It resolves `%INCLUDE` through the `pli_include_dir` of the nearest `bricks.cnf`:

  ```
  $ go run ./cmd/brickscompile runtime/pli/cusp.pli
  runtime/pli/cusp.pli: OK
         lines     463
    procedures      23
    statements     188
     EXEC CICS      22
  ```

  A compile error names the line, the column and a short ASCII message, for example `line 4 col 4: UNDECLARED QTYY`. An error inside an include member is prefixed with the member name (`DFHAID line 3 col 1: ...`).

- `CEDA TRANSACTION` accepts `PLI` in the LANGUAGE field (the hint reads `(REXX, COBOL or PLI)`). `CEDA PROGRAM` and `CEMT PERFORM PROGRAMS` (`CEMT P R P`) list the `.pli` files under `pli_dir` with a SYNTAX verdict (`OK` or `ERR L<n>`); include members are not programs and are not listed.
- **ISPF area 4** (`4 – Edit PL/I programs`) edits `pli_dir` with PL/I highlighting and validates the buffer before save with the same parser, `%INCLUDE` included (see `ISPF_editor.md`).
- `EXEC CICS INQUIRE PROGRAM(name) LANGUAGE(v)` returns `PLI` for a PL/I program.

Caching and storage per task

The compiled program is cached like REXX and COBOL programs. The cache key covers the program file and every include member it uses, so editing a member recompiles every program that includes it (Chapter 37). Each task gets its own storage: there is no shared mutable state between tasks (Storage classes).

Chapter 31. Data declarations

Every variable is declared with `DECLARE` (`DCL`). There are **no implicit declarations**: in Enterprise PL/I an undeclared `QTYY` silently becomes a FLOAT variable, in bricks it is the compile error `UNDECLARED QTYY`.

Data types

Type	Syntax	Limits	Use
Fixed decimal	`FIXED DEC(p,q)`	p <= 31, 0 <= q <= p	Money, quantities, rates. The default for business data.
Fixed binary	`FIXED BIN(p)`	Signed: 15, 31 or 63; other values round up	Counters, subscripts, lengths
64-bit integer	`FIXED BIN(63)`, `UNSIGNED FIXED BIN(64)`	+-9.22x10^18 / 0 to 1.84x10^19	IDs, timestamps, record counts
Character	`CHAR(n)`	1 <= n <= 32767	Fixed-length text, blank padded

Type	Syntax	Limits	Use
Varying character	`CHAR(n) VAR`	n <= 32767	Variable-length text
Bit	`BIT(n)`	1 <= n <= 64	Flags; `BIT(1)` is the boolean type
Numeric picture	`PIC '...'`	<= 31 digit positions	Edited output, zoned input
Character picture	`PIC 'XXX'`, `PIC 'AAA'`		Validated text

- `FIXED` **is required.** `DEC(7,2)` and `BIN(31)` without `FIXED` would mean FLOAT in Enterprise PL/I; here they are the compile error `FIXED REQUIRED`.
- `UNSIGNED` is allowed only with `FIXED BIN`, precisions 8, 16, 32 and 64 (others round up). `SIGNED` is the default and may be written. `FIXED BIN(64)` without `UNSIGNED` is a synonym for `FIXED BIN(63)`.
- **Not available:** `FLOAT`, `COMPLEX`, `POINTER`, `OFFSET`, `AREA`, `LABEL`, `FORMAT`, `FILE`, `HANDLE`, `ORDINAL`, `UNION`, and the storage attributes `BASED`, `CONTROLLED`, `DEFINED`, `ALIGNED` and `EXTERNAL` on variables.

```
DCL PRICE    FIXED DEC(9,2)  INIT(1234.56);
DCL QTY      FIXED BIN(31)   INIT(12);
DCL TXN_ID   FIXED BIN(63)   INIT(9000000000);
DCL BYTES    UNSIGNED FIXED BIN(64);
DCL NAME     CHAR(20)        INIT('ACME');
DCL NOTE     CHAR(100) VAR   INIT('short');
DCL FLAGS    BIT(8)          INIT('10100000'B);
DCL ZIP      PIC '99999'     INIT(1234);
DCL MONEY    PIC '$$$,$$9V.99' INIT(1234.5);
```

`ZIP` holds `01234` and `MONEY` holds `$1,234.50`. `HUGE(QTY)` is `2147483647`; `HUGE(BYTES)` is `18446744073709551615`.

Picture characters

Char	Meaning
`9`	Digit
`Z`	Digit; a leading zero becomes a blank
`*`	Digit; a leading zero becomes *
`V`	Implied decimal point. Aligns the value; not printed.
`. , / B`	Insertion characters (`B` inserts a blank); suppressed inside a zero-suppressed field
`$ S + –`	Sign or currency: static once, drifting when repeated
`CR DB`	Trailing credit / debit marker, shown only for a negative value
`X A`	Any character / alphabetic or blank (character pictures)

- A `.` written without `V` is only an insertion character, as in Enterprise PL/I: `PIC '999.99'` has scale 0. Write `'999V.99'`.
- Repetition factors are accepted: `PIC '(5)9'` is `'99999'`.
- Assigning a negative value to a picture with no sign position (`S + – CR DB`) is the `ABSZ` abend; the sign is never dropped silently. A `+` sign position shows a blank for a negative value.

Structures and arrays

```
DCL MAX_LINES FIXED BIN(31) VALUE(3);
DCL 1 ORDER,
      2 CUST_ID    CHAR(8)       INIT('C0042'),
      2 LINE_COUNT FIXED BIN(15) INIT(2),
      2 LINE(MAX_LINES),
        3 SKU      CHAR(12),
        3 QTY      PIC '99999',
        3 PRICE    FIXED DEC(9,2);
DCL 1 SAVE_ORDER LIKE ORDER;
DCL MDAYS(12) FIXED BIN(15) INIT(31,28,31,30,31,30,31,31,30,31,30,31);
DCL TABLE(5) FIXED DEC(5,2) INIT((5)0);
DCL GRID(0:2, 1:4) CHAR(4);
DCL (A, B, C) FIXED DEC(9,2) INIT(1.5);
DCL BAR CHAR(6) INIT((3)'AB');
```

- **Levels** run from 1 to 15; arrays of structures are allowed.
- **Arrays** have up to 3 dimensions, counting dimensions inherited from enclosing structure arrays. Bounds are constants or `VALUE` names; the default lower bound is 1 (`GRID(0:2, 1:4)` overrides it).
- **Qualified references.** `ORDER.LINE(1).QTY` is the full form. A subscript may sit at any level, so `ORDER.LINE.QTY(1)` is the same element. Partial qualification is allowed while it stays unambiguous: `ORDER.QTY(2)` works, but a bare `QTY(2)` here is `AMBIGUOUS QTY` because `SAVE_ORDER` has a `QTY` too.
- `LIKE` copies the layout of a structure, every member attribute and `INIT` included, but not the storage class. A parameter declared with `LIKE` ignores the copied `INIT`: its storage is the argument's.
- **Factored attributes**: `DCL (A, B, C) FIXED DEC(9,2) INIT(1.5);` gives all three names the same attributes.

INIT, VALUE and default values

- `INIT(...)` sets the initial value. A factor before a constant is an iteration factor for an array (`INIT((5)0)` is five zeros) and a string repetition for a scalar (`INIT((3)'AB')` is `'ABABAB'`). A repetition longer than the target (`CHAR(n)`: n characters, otherwise 32767) is `INIT VALUE TOO LONG FOR name`; more values than the array has elements is `TOO MANY INIT VALUES`.

- `VALUE(expr)` declares a **named constant**. It cannot be assigned, it may be used as an array bound, and inside `EXEC CICS` it is folded into the command text. The shipped `DFHAID` and `DFHRESP` members are made of `VALUE` declarations.

- **Deterministic defaults.** Enterprise PL/I leaves uninitialised storage undefined. Bricks gives every variable without `INIT` a fixed starting value each time its storage is allocated:

Type	Initial value
`CHAR(n)`, character `PIC`	all blanks
`FIXED DEC`, `FIXED BIN`, numeric `PIC`	0
`BIT(n)`	all `'0'B`
`CHAR(n) VAR`	length 0

Storage classes: AUTOMATIC and STATIC

Class	Lifetime
`AUTOMATIC` (`AUTO`, the default)	One procedure invocation. `INIT` (or the default value) is applied again on every entry.
`STATIC`	**One task.** Initialised the first time the program runs in the task, then shared by every activation of that program in the task, including activations reached by `EXEC CICS LINK` and by a `CALL` to an external entry. Never shared between tasks.

```
COUNTER: PROC;
   DCL CALLS FIXED BIN(31) STATIC INIT(0);    /* once per task  */
   DCL LOCAL FIXED BIN(31) INIT(0);           /* every entry    */
   CALLS = CALLS + 1;
   LOCAL = LOCAL + 1;
   PUT SKIP LIST('CALLS', CALLS, 'LOCAL', LOCAL);
END COUNTER;
```

Called three times in one task the PUT lines read `CALLS 1 LOCAL 1`, `CALLS 2 LOCAL 1`, `CALLS 3 LOCAL 1`. The next task starts again at `CALLS 1`.

Storage limits. bricks runs in a fixed memory budget:

- One program may declare at most **1 MiB** of `AUTOMATIC` storage (all procedures together) plus `STATIC` storage. More is the compile error `STORAGE LIMIT EXCEEDED`.
- One task may hold at most **8 MiB** of live activations. Every active call holds its `AUTOMATIC` storage until it returns, so deep recursion over a large `AUTOMATIC` area counts each level; a call to an external procedure also counts its `STATIC`. The call that would pass the limit abends `ABST`.

  Programming Note. STATIC is per task, not per region. State that must outlive the task or be seen by other tasks belongs in a KSDS file, a TS queue or the database, never in program storage.

Storage layout and COMMAREA interop with COBOL

All data is `UNALIGNED`: there is no padding between structure members, and the byte image of every type is fixed. A structure passed as a COMMAREA, a map area or a file record therefore has the same bytes in PL/I and COBOL. `STG(x)` (`STORAGE`) returns the declared size, `CSTG(x)` (`CURRENTSTORAGE`) the current size.

PL/I declaration	Bytes	Encoding	COBOL equivalent
`CHAR(n)`	n	characters	`PIC X(n)`
`CHAR(n) VAR`	2+n	2-byte big-endian length, then n bytes (tail written as blanks)	`05 L PIC S9(4) COMP.` + `05 T PIC X(n).`
`FIXED DEC(p,q)`	p/2+1	packed decimal, sign nibble C/D; even p has a leading zero nibble	`PIC S9(p-q)V9(q) COMP-3`
`FIXED BIN(15)`	2	big-endian two's complement	`PIC S9(4) COMP`
`FIXED BIN(31)`	4	big-endian two's complement	`PIC S9(9) COMP`
`FIXED BIN(63)`	8	big-endian two's complement	`PIC S9(18) COMP`
`UNSIGNED FIXED BIN(8/16/32/64)`	1/2/4/8	big-endian unsigned	`PIC 9(n) COMP` of the same width
`BIT(n)`	ceil(n/8)	bits from the high-order end, unused low bits zero	`PIC X(ceil(n/8))`

PL/I declaration	Bytes	Encoding	COBOL equivalent
`PIC '...'`	one per picture character (`V` excluded)	character image as displayed	the matching `PIC 9...` / edited `PIC` (DISPLAY)

Reads of packed data also accept the sign nibbles `F`, `A`, `E` (plus) and `B` (minus), so a COBOL unsigned `COMP-3` item reads correctly. A structure's size is the sum of its members' sizes, an array's the element size times the element count. Example:

```
DCL 1 CUST_AREA,
      2 CA_KEY      CHAR(8),
      2 CA_BALANCE  FIXED DEC(11,2),
      2 CA_COUNT    FIXED BIN(31),
      2 CA_FLAGS    BIT(8),
      2 CA_NOTE     CHAR(20) VAR,
      2 CA_ZONED    PIC '9999';
```

`STG(CUST_AREA)` is 45 (8 + 6 + 4 + 1 + 22 + 4). The COBOL program on the other side of the `LINK` declares the same 45 bytes:

```
       01 CUST-AREA.
          05 CA-KEY      PIC X(8).
          05 CA-BALANCE  PIC S9(9)V99 COMP-3.
          05 CA-COUNT    PIC S9(9) COMP.
          05 CA-FLAGS    PIC X.
          05 CA-NOTE-LEN PIC S9(4) COMP.
          05 CA-NOTE     PIC X(20).
          05 CA-ZONED    PIC 9(4).
```

> **Programming Note.** A PL/I numeric picture is its character image, one byte per picture character: `PIC '9999'` holding 42 is the four characters `0042`, the same bytes as a COBOL `PIC 9(4)` DISPLAY item. A signed picture (`PIC 'S9999'`) puts the sign in its own leading byte (`+0042`, five bytes), which is not the COBOL zoned overpunch. Carry signed values as `FIXED DEC` / `COMP-3`.

Chapter 32. Expressions, conversions and decimal arithmetic

Operator precedence

Highest first:

Level	Operators	Associativity
1	**, prefix + - ¬	right
2	* /	left
3	+ -	left
4	||	left
5	= ¬= < > <= >= ¬< ¬>	non-chaining
6	&	left, short-circuit
7	|	left, short-circuit

- `=` **is both assignment and comparison.** At statement level the first `=` assigns, so `F = I = 0;` stores the result of the comparison `I = 0` (a `BIT(1)`) in `F`.

- **Comparisons do not chain.** `A < B < C` is the compile error `CHAINED COMPARISON`; write `A < B & B < C`.
- **Conditions** (`IF`, `WHILE`, `UNTIL`, boolean `WHEN`) need a `BIT` expression; a non-zero bit string is true.
- **Short-circuit.** When both operands of `&` or `|` are `BIT(1)`, the right operand is evaluated only if needed. Otherwise both are evaluated and the operation is bitwise.
- `**` takes a non-negative integer constant exponent and is evaluated as repeated multiplication. Prefix minus binds tighter than `*` but `**` is applied first: `-2 ** 2` is −4.

```
IF I > 0 & TAB(I) = 5 THEN PUT SKIP LIST('hit');
ELSE PUT SKIP LIST('no hit, no ABSR');
```

With `I = 0` the subscript `TAB(I)` is never evaluated, so the statement does not abend `ABSR`.

Fixed-decimal precision

Every arithmetic result has a precision `(p,q)` computed at compile time from its operands, with N = 31:

Op	Integer digits	Scale q
+ −	max(p1-q1, p2-q2) + 1	max(q1, q2)
*	(p1-q1) + (p2-q2) + 1	q1 + q2
/	(p1-q1) + q2	31 - integer digits

The result is then clamped:

```
int' = min(int, 31)
q'   = min(q, 31 − int')
p'   = int' + q'
```

The exact value is truncated toward zero to scale `q'`. Only if the integer part still needs more than `int'` digits does `ABFO` (FIXEDOVERFLOW) fire.

The bricks rule. When a result would need more than 31 digits, scale digits are dropped to make room; an intermediate result never overflows on the right. This removes the classic Enterprise PL/I trap where `25 + 1/3` raises FIXEDOVERFLOW:

- `1/3`: both constants are `DEC(1,0)`. Integer digits (1-0)+0 = 1, scale 31-1 = 30: `DEC(31,30)`, `0.333...3`.
- `25 + 1/3`: integer digits max(2, 1)+1 = 3, scale max(0, 30) = 30; the clamp gives q' = min(30, 31-3) = 28: `25.3333333333333333333333333333` (28 decimals). No overflow.

Worked example: an invoice line.

```
DCL PRICE FIXED DEC(7,2) INIT(19.99);
DCL QTY   FIXED DEC(5,0) INIT(3);
DCL RATE  FIXED DEC(5,4) INIT(0.0825);
DCL TOTAL FIXED DEC(9,2);
DCL TAX   FIXED DEC(9,2);

TOTAL = PRICE * QTY;              /* DEC(13,2): 59.97            */
TAX   = TOTAL * RATE;             /* 4.947525 truncated to 4.94  */
TAX   = ROUND(TOTAL * RATE, 2);   /* 4.95                        */
```

- `PRICE * QTY`: integer digits (7-2)+(5-0)+1 = 11, scale 2+0 = 2, so `DEC(13,2)`; the value 59.97 fits `TOTAL`.
- `TOTAL * RATE`: integer digits (9-2)+(5-4)+1 = 9, scale 2+4 = 6: `DEC(15,6)` = 4.947525. Assigning it to `DEC(9,2)` **truncates** to 4.94.

- `ROUND(x, 2)` rounds half away from zero: 4.95. Use `ROUND` wherever money is rounded.

Division is always decimal. `/` never produces a binary result. `FIXED BIN` operands of `/` convert to `FIXED DEC` first, so `7/2` is `3.5` (shown with its full scale, `3.50000000000000000000000000000000`), not `3`. To drop the fraction use `TRUNC`, an integer target, or `DIVIDE` with an explicit precision:

```
DCL N     FIXED BIN(31)  INIT(7);
DCL R     FIXED DEC(9,2);
R = N / 2;
PUT SKIP LIST(R, TRUNC(N / 2), DIVIDE(N, 2, 5, 1));
```

The PUT line reads `3.50 3 3.5`.

Explicit precision. `ADD(x, y, p, q)`, `MULTIPLY(x, y, p, q)` and `DIVIDE(x, y, p, q)` compute the result at exactly `(p,q)`.

Mixed binary and decimal

- **BIN op DEC.** The binary operand converts to decimal, not the other way round: `BIN(15)` becomes `DEC(5,0)`, `BIN(31)` `DEC(10,0)`, `BIN(63)` `DEC(19,0)`, `UNSIGNED BIN(64)` `DEC(20,0)`. Decimal constants (`1`, `0.5`) are decimal, so `N + 1` with a `FIXED BIN(31) N` is a decimal addition.
- **BIN op BIN.** +, – and ∗ are computed as `BIN(63)` (int64); an overflow is `ABFO`, never a wraparound. Any `UNSIGNED` operand makes the operation unsigned 64-bit; going below 0 or above 2^64-1 is `ABFO`.
- **Pictures** compute as the `FIXED DEC(p,q)` their digit positions and `V` imply.
- A `CHAR` operand of an arithmetic operator converts to `FIXED DEC(31,8)`.

Truncation versus ROUND

Assignment between arithmetic types **truncates** the scale to the target's scale; it never rounds. If the integer part does not fit the target, the task abends `ABSZ` (SIZE is always enabled):

```
DCL X  FIXED DEC(7,3) INIT(-2.675);
DCL Y  FIXED DEC(7,2);
DCL I  FIXED BIN(31);
Y = X;
I = X;
PUT SKIP LIST(Y, ROUND(X, 2), I, CEIL(X), FLOOR(X), TRUNC(X));
```

The PUT line reads `-2.67 -2.68 -2 -2 -3 -2`. Assigning `12345.67` to a `FIXED DEC(5,2)` abends `ABSZ` instead of keeping `345.67`.

Conversions

From -> To	Rule
arithmetic -> arithmetic	Scale truncated to the target; ABSZ if the integer part does not fit.
CHAR -> arithmetic	Optional blanks, optional sign, digits, optional `.`. An all-blank string is 0. Anything else abends ABCV.
arithmetic -> CHAR	Minimal form: no leading blanks, – only when negative, every scale digit kept (`12.50`, `-3.00`). Use `EDIT` or a `PIC` for formatted output.
CHAR -> CHAR	Padded with blanks or truncated on the right; truncation is silent.
CHAR <-> BIT	Only `'0'` and `'1'` characters; anything else is ABCV.

From -> To	Rule
arithmetic -> BIT	Compile error (ARITHMETIC TO BIT). Use BIT(x, n).

```
AMT = '  -12.5 ';                /* -12.50                    */
N   = '';                        /* 0                         */
S   = AMT;                       /* '-12.50'                  */
C   = 'ABCDEFG';                 /* CHAR(5): 'ABCDE'          */
PUT SKIP LIST(CHAR(42), EDIT(AMT, 'ZZZ9V.99-'));
```

`CHAR(42)` is `'42'` (never a right-justified field) and the `EDIT` result is `' 12.50-'`.

- **Comparisons.** Two `CHAR` operands compare after padding the shorter with blanks, in SBCS byte order (`'AB' = 'AB '` is true). A `CHAR` compared with a number is converted to the number exactly (`'42' = 42.0` is true). Comparing a `BIT` with a number is a compile error; `BIT` against `CHAR` compares as characters.
- **Pictures** give their character image where a string is expected (concatenation, `CHAR` target, `PUT LIST`) and their value where a number is expected. Picture data that does not match its picture abends `ABCV` only when used as a number.
- `||` of two bit strings with at most 64 bits in total is a bit string; every other concatenation is character.

Validate first. There is no `ON CONVERSION`. Check data before converting it:

```
IF VERIFY(TXT, '0123456789') = 0 THEN QTY = TXT;
ELSE PUT SKIP LIST('not numeric at', VERIFY(TXT, '0123456789'));
IF VALIDDATE('20260230', 'YYYYMMDD') THEN PUT SKIP LIST('date ok');
ELSE PUT SKIP LIST('bad date');
```

`VALID(picvar)` tests a picture variable, which is how map input received into a numeric `PIC` member is checked (Maps as structures).

Chapter 33. Statements and control flow

Assignment

```
A, B = 1.5;                      /* several targets           */
MONTHLY = 0;                     /* scalar to every element   */
OUT_REC = '';                    /* every CHAR blank, numbers 0 */
OUT_REC = IN_REC, BY NAME;       /* members with matching names */
SUBSTR(LINE, 5, 5) = 'TOTAL';    /* SUBSTR pseudovariable     */
```

- A scalar assigned to an array or structure is assigned to every element, each with its own conversion. `REC = '';` is the idiom to clear a whole map structure: `''` blanks the `CHAR` members and converts to 0 for the numeric ones.
- `BY NAME` copies only the members whose names match, whatever their order, converting each. Members without a partner are left alone.
- `SUBSTR(s, i, n) = v` replaces `n` characters of `s` from position `i`. A position outside the string is `ABSR`.

IF, DO groups and loops

```
IF BAL < 0 THEN FLAG = '1'B;
ELSE DO;
   ...
END;

DO I = 1 TO N;               ... END;
DO I = N TO 1 BY -1;         ... END;
DO I = 1 TO N WHILE(¬FOUND); ... END;
DO WHILE(MORE);              ... END;
DO UNTIL(DONE);              ... END;   /* tested after the body */
DO FOREVER;                  ... END;   /* DO LOOP also accepted */
```

- `DO; ... END;` is both the block and the loop construct. There are no `BEGIN` blocks.
- `TO` and `BY` are evaluated once, at loop entry; `BY` may precede `TO`. `DO I = X;` with no `TO`, `BY`, `WHILE` or `UNTIL` runs once.
- The control variable is reloaded at every iteration. Do not change it in the body. An overflow while incrementing it is `ABSZ`.

SELECT

```
SELECT (SHIP);
   WHEN ('EXP')        DAYS_ = 1;
   WHEN ('STD', 'GRD') DAYS_ = 5;
   OTHERWISE           DAYS_ = 0;
END;
SELECT;
   WHEN (DAYS_ > 3) PUT SKIP LIST('slow', DAYS_);
   WHEN (DAYS_ > 0) PUT SKIP LIST('fast', DAYS_);
   OTHERWISE;
END;
```

If no `WHEN` matches and there is no `OTHERWISE`, the task abends `ABSL`. Write `OTHERWISE;` when no action is intended.

LEAVE and ITERATE

```
OUTER: DO I = 1 TO 3;
   DO J = 1 TO 3;
      IF J = 2 THEN ITERATE;
      IF I = 3 THEN LEAVE OUTER;
      S = S || CHAR(I) || CHAR(J) || ' ';
   END;
END OUTER;
```

`S` ends as `'11 13 21 23 '`.

- A label is allowed only on a `PROC` or a `DO`; it names the target of `LEAVE`, `ITERATE` and `END name`.
- `LEAVE` and `ITERATE` **without a label** apply to the innermost *iterative* `DO` (one with a control variable, `WHILE`, `UNTIL`, `FOREVER` or `LOOP`), never to an enclosing simple `DO;` group. So `IF X THEN DO; ...; LEAVE; END;` inside a loop leaves the loop. A labelled `LEAVE` may leave a simple group.

CALL, RETURN and arguments

```
   CALL VALIDATE(ORDER, RC);
   RETURN;                     /* from a subroutine, or from MAIN  */
   RETURN (TOTAL);             /* from a function                  */
```

`RETURN` in MAIN ends the transaction normally and commits.

Arguments are passed by reference. If the argument is an expression or a constant, or its attributes differ from the parameter's, a **dummy argument** (a temporary) is passed instead, and the procedure's writes to it are not seen by the caller:

```
   DCL N  FIXED BIN(31)  INIT(1);
   DCL D  FIXED DEC(5,0) INIT(1);
   CALL BUMP(N);              /* same attributes: by reference   */
   CALL BUMP(D);              /* DEC vs BIN: a dummy is passed    */
   CALL BUMP(N + 0);          /* an expression: a dummy           */
   PUT SKIP LIST(N, D);       /* 11 1                             */

   BUMP: PROC(X);
      DCL X FIXED BIN(31);
      X = X + 10;
   END BUMP;
```

Declare a parameter `CHAR(*)` or `CHAR(*) VAR` to accept strings of any length by reference, as `cusp.pli`'s `VALIDATE_KEY` does to normalise the caller's key in place.

PUT LIST

`PUT [SKIP] LIST(expr, ...)` is accepted, evaluates its items, and **discards** the line, exactly as REXX `SAY` and COBOL `DISPLAY` are discarded in a running transaction: it reaches neither the terminal nor the bricks log (at high transaction rates every line would otherwise hit the operator console). The line it would form separates items by a blank, shows strings without quotes, numbers in minimal form and bits as `0`/`1`:

```
   PUT SKIP LIST('READ', KEY, 'BAL', BAL, OK);
```

would form `READ C0042 BAL -12.50 1`. The examples in this part quote PUT LIST lines this way, to show values. To see values while debugging a transaction, paint them with `EXEC CICS SEND TEXT` or write them to a TS queue (`EXEC CICS WRITEQ TS QUEUE('DEBUG') FROM(LINE)`) and browse it with CECI.

Stream I/O (`GET`, `PUT EDIT`, `FORMAT`) and record I/O are not available: the terminal is reached through `EXEC CICS SEND`, files through `EXEC CICS READ` / `WRITE`.

External entries

A call to another program is declared with `ENTRY ... EXTERNAL`:

```
ORDTOT: PROC OPTIONS(MAIN);
   DCL CALCDISC ENTRY(FIXED DEC(13,2), CHAR(8))
                RETURNS(FIXED DEC(13,2)) EXTERNAL;
   DCL GROSS FIXED DEC(13,2) INIT(250.00);
   DCL CUST  CHAR(8)         INIT('C0042');
   GROSS = GROSS - CALCDISC(GROSS, CUST);
   PUT SKIP LIST('NET', GROSS);
END ORDTOT;
```

```
/* CALCDISC - external subprogram (no OPTIONS(MAIN)) */
CALCDISC: PROC(AMT, CUST) RETURNS(FIXED DEC(13,2));
   DCL AMT  FIXED DEC(13,2);
   DCL CUST CHAR(8);
   IF SUBSTR(CUST, 1, 1) = 'C' THEN RETURN(ROUND(AMT * 0.10, 2));
   RETURN(0);
END CALCDISC;
```

- The entry name is resolved as the member `<name>.pli` in `pli_dir` (case-insensitive), through the program cache. It needs no `transactions.conf` row.
- The callee runs in the caller's task and unit of work, and shares the task's STATIC storage.
- If the member is missing, is an `OPTIONS(MAIN)` program, or its parameter list or `RETURNS` does not match the `ENTRY` declaration, the call abends `ABEN`.

`CALL` reaches PL/I subprograms only. To call a REXX or COBOL program, or a PL/I `OPTIONS(MAIN)` program, use `EXEC CICS LINK` (LINK, XCTL and RETURN).

Chapter 34. Built-in functions

All built-ins below are available without declaration. Arguments in `[ ]` are optional.

String functions

Function	Result
`SUBSTR(s, i [, n])`	Substring; also an assignment target. A position or length outside `1 .. LENGTH(s)+1` is ABSR (the check cannot overflow: `SUBSTR(s, 2, HUGE(n))` is ABSR too); `SUBSTR(s, LENGTH(s)+1)` is `''`.
`INDEX(s, t [, start])`	Position of `t` in `s`, or 0
`LENGTH(s)`	Current length
`MAXLENGTH(s)`	Declared length of a VAR string
`TRIM(s [, lead [, trail]])`	Blanks trimmed from both ends; `TRIM(s, lead)` trims `lead` characters on the left and blanks on the right
`LEFT(s, n [, pad])` / `RIGHT(s, n [, pad])`	Left- / right-justify into `n` characters
`COPY(s, n)`	`s` repeated `n` times; a result longer than 32767 characters is ABSR
`TRANSLATE(s, to [, from])`	Character mapping; the first occurrence in `from` wins; `to` is blank padded
`VERIFY(s, set [, start])`	Position of the first character of `s` not in `set`, or 0
`SEARCH(s, set [, start])`	Position of the first character of `s` in `set`, or 0
`UPPERCASE(s)` / `LOWERCASE(s)`	Changes a–z / A–Z only, as in Enterprise PL/I
`REVERSE(s)`	Characters in reverse order

```
   T2 = TRIM(S);                        /* 'Hello, World'          */
   PUT SKIP LIST(INDEX(T2, 'o'), INDEX(T2, 'o', 6));    /* 5 9    */
   PUT SKIP LIST(LEFT('AB', 5, '*'), RIGHT('AB', 5, '0'));
   PUT SKIP LIST(TRANSLATE('A-B-C', ' ', '-'), TRIM('000120', '0'));
```

The last two lines form `AB*** 000AB` and `A B C 120`.

Arithmetic functions

Function	Result
`ABS(x)`, `SIGN(x)`	Absolute value; -1, 0 or 1
`MOD(x, y)`	Remainder, always >= 0 (`MOD(-7, 3)` = 2)
`REM(x, y)`	Remainder with the sign of x (`REM(-7, 3)` = -1); exact for decimals: `x - y * TRUNC(x / y)`
`MAX(x, y, ...)`, `MIN(x, y, ...)`	Largest / smallest argument
`ROUND(x, n)`	Rounded half away from zero at n decimal places
`TRUNC(x)`, `CEIL(x)`, `FLOOR(x)`	Toward zero / up / down to an integer
`ADD(x, y, p, q)`, `MULTIPLY(x, y, p, q)`, `DIVIDE(x, y, p, q)`	Result at the explicit precision `(p,q)`
`HUGE(x)` / `TINY(x)`	Largest / smallest value of x's type (0 for `UNSIGNED`)

Conversion and validation functions

Function	Result
`CHAR(x [, n])`	Minimal character form of a number, for every type (`CHAR(42)` is `'42'`); n pads or truncates
`DEC(x, p [, q])`, `FIXED(x, p [, q])`	Fixed decimal at `(p,q)`
`BIN(x, p)`	Fixed binary; `BIN(b)` of a `BIT(64)` reads two's complement, a shorter bit string as unsigned
`BIT(x [, n])`	Bit string; of a number, the rightmost n bits of its 64-bit two's-complement integer part, so `BIN(BIT(x, 64), 63) = x`; of a character variable without n, the variable's declared length (at most 64)
`EDIT(x, 'picture')`	x formatted through a picture, as `CHAR`
`VALID(picvar)`	`'1'B` if a picture variable holds data matching its picture
`STRING(struct)`	The structure's `CHAR` and `PIC` members concatenated

Aggregate and storage functions

Function	Result
`HBOUND(a [, d])`, `LBOUND(a [, d])`, `DIM(a [, d])`	Upper bound, lower bound, extent of dimension d (default 1)
`SUM(a)`	Sum of all elements
`STORAGE(x)` / `STG(x)`	Declared byte size (see the layout table)
`CURRENTSTORAGE(x)` / `CSTG(x)`	Current byte size; for a `VAR` string 2 + current length

Date and time functions

Patterns use `YYYY YY MM DD DDD HH MI SS 999` plus literal separators. `YY` uses the window 1950-2049 and `DDD` is the day of the year.

Function	Result
`DATETIME([pat])`	Current task-clock time; default pattern `'YYYYMMDDHHMISS999'`
`DATE()`	`'YYMMDD'`; kept for compatibility, prefer `DATETIME`

Function	Result
`TIME()`	`'HHMISS999'`
`DAYS(d [, pat])`	Date to day number (Lilian: 1 = 15 Oct 1582)
`DAYSTODATE(n [, pat])`	Day number to date; default pattern `'YYYYMMDD'`
`VALIDDATE(d [, pat])`	`'1'B` if d is a real calendar date

Day numbers run from 1 (15 Oct 1582) to 3074324 (31 Dec 9999); a date outside that range or a malformed date string abends `ABCV`. `DDD` runs from 1 to 365 (366 in a leap year); `000` is not a date, so `VALIDDATE('2026000', 'YYYYDDD')` is `'0'B`. A letter or digit in a pattern that is not part of a field makes the pattern invalid: `ABCV` for `DATETIME`, `DAYS` and `DAYSTODATE`, `'0'B` for `VALIDDATE`. Date arithmetic goes through day numbers:

```
   D = DAYSTODATE(DAYS('20260131', 'YYYYMMDD') + 30, 'YYYYMMDD');
```

gives `'20260302'`. The clock is the region's NTP-corrected clock, the same one `EXEC CICS ASKTIME` reads.

CICS constants and the ABEND routine

Name	Result
`DFHRESP(cond)`	Compile-time `EIBRESP` value of a condition: `DFHRESP(NOTFND)` is 13. An unknown name is a compile error (`UNKNOWN CONDITION NOTFOUND`).
`DFHVALUE(cvda)`	Compile-time CVDA value: `OPEN` 18, `CLOSED` 19, `ENABLED` 23, `DISABLED` 24, `READABLE` .. `NOTALTERABLE` 50 .. 57
`CALL ABEND('code')`	Abends the task with the given 1-4 character code (Chapter 35)

`DFHRESP` uses the same single table as COBOL's `DFHRESP(name)` and the shipped `DFHRESP.cpy` / `DFHRESP.inc` members, so the numbers cannot drift between languages.

Chapter 35. Runtime abends and the unit of work

Enterprise PL/I handles runtime conditions with ON-units (`ON CONVERSION`, `ON SIZE`, `SIGNAL`, `REVERT`). Bricks PL/I has none: `ON` is the compile error `ON NOT SUPPORTED`. Instead every runtime condition is **deterministic and fatal**. It:

1. abends the transaction with a short code,
2. rolls back the task's unit of work (every abend rolls back, whatever raised it), and
3. logs program, procedure, file, source line and statement text.

The operator sees only the short form, for example `ABSZ CUSP.ADD_CUSTOMER L42` (`ABSZ CUSP L42` for a statement in MAIN itself). The bricks log carries the detail on one line (shown here wrapped):

```
term=T0001 transid=CALC: PL/I ABEND ABZD (ZERODIVIDE) PROGRAM CALCP
  PROC TOTALS FILE calcp.pli LINE 6: ZERODIVIDE | X = 10 / Y ;
```

Abend codes

Code	Condition	Raised when
ABCV	CONVERSION	An invalid character string is converted to a number or a bit string; a bad date
ABSZ	SIZE	The integer part does not fit the target of an assignment; a negative value to UNSIGNED or to a picture without a sign. Always enabled.
ABFO	FIXEDOVERFLOW	An intermediate result needs more than 31 integer digits, or a 64-bit binary operation overflows
ABZD	ZERODIVIDE	Division, MOD or REM by zero
ABSR	SUBSCRIPTRANGE / STRINGRANGE	A subscript or SUBSTR position is out of bounds. Always checked.
ABSL	SELECT	No WHEN matched and there is no OTHERWISE
ABEN	ENTRY	An external entry is missing or does not match its ENTRY declaration; a non-RECURSIVE procedure re-entered; call depth over 1000; a function without RETURN(value)
ABHB	HANDLER	A CICS condition, AID or abend handler fired while a handler procedure was already running, or named a procedure that is not a handler of this program
AICA	RUNAWAY	The interpreter's statement limit was exceeded (a loop that never ends)
AEY9	CICS NOT AVAILABLE	EXEC CICS ran with no CICS handler attached (offline test harness)
ABST	STORAGE	The task's live activations would hold more than 8 MiB: each active call holds its AUTOMATIC storage until it returns (recursion included), and an external procedure call also counts its STATIC
APLI	INTERNAL	The interpreter caught an internal failure; the operator sees the code, the detail goes to the bricks log line only
any	program-raised	CALL ABEND('CU01') or EXEC CICS ABEND ABCODE('CU01'): the given code, for deliberate business-rule aborts

```
POST: PROC(B);
   DCL B FIXED DEC(9,2);
   IF B < 0 THEN CALL ABEND('CU01');
END POST;
```

This ends the task with CU01 T.POST L7. The code must be 1-4 printable ASCII characters that also exist in EBCDIC code page 037 (no brackets, braces, tilde, backslash, backtick, vertical bar or caret): a constant that breaks the rule is the compile error BAD ABEND CODE, a variable holding one abends ABEN.

Compile-time limits and errors

Some mistakes that Enterprise PL/I reports late, or not at all, are compile errors in bricks, so they show up in brickscompile, the CEDA / CEMT SYNTAX column and the ISPF save check:

Message	Cause
STORAGE LIMIT EXCEEDED	The program declares more than 1 MiB of AUTOMATIC (all procedures together) plus STATIC storage
NESTING TOO DEEP	More than 256 nested parentheses, prefix operators, statements (IF, DO, SELECT) or procedures, or an expression tree deeper than 4096 (long + / || / & chains count)
UNKNOWN CICS COMMAND x	EXEC CICS with a verb bricks does not dispatch, for example the typo SENDD; caught at compile time instead of INVREQ at run time
BAD ABEND CODE	A CALL ABEND constant that is empty, longer than 4 characters, or not EBCDIC 037 printable

Message	Cause
`INIT VALUE TOO LONG FOR x`	A string repetition longer than the target (`DCL R CHAR(4) INIT((3)'AB')`)
`TOO MANY INIT VALUES`	`INIT` values (iteration factors included) for more elements than the array has

Not an error: a `CHAR` value assigned to a shorter target is truncated silently, as in Enterprise PL/I with STRINGSIZE disabled.

The unit of work

Task end	Unit of work
`RETURN` from MAIN, or falling off its `END`	committed
`EXEC CICS RETURN` (with or without `TRANSID`)	committed
`EXEC CICS XCTL`	carried into the target program, committed when it ends normally
a handler procedure that returns (HANDLE CONDITION, AID and ABEND)	committed
any abend: the table above, `CALL ABEND`, `EXEC CICS ABEND`, an interpreter failure	rolled back

`EXEC CICS SYNCPOINT` and `SYNCPOINT ROLLBACK` work as in the other languages (Chapter 10). `EIBABCODE` (`CHAR(4)`) holds the abend code for `ASSIGN ABCODE` and a `HANDLE ABEND PROGRAM`.

HANDLE ABEND in PL/I

`EXEC CICS HANDLE ABEND LABEL(proc)` catches `EXEC CICS ABEND` only. The runtime conditions in the table above always abend and roll back; a PL/I program cannot intercept them.

Why there are no ON-units, and what to use instead

ON-units turn every statement into a possible jump target, which is hard to read, hard to cache and easy to get wrong in money code (`ON SIZE` disabled by default loses digits silently). Bricks programs **prevent** conditions instead of catching them:

Instead of	Write
`ON CONVERSION`	`VERIFY(s, '0123456789')`, `VALID(picvar)`, `VALIDDATE(d)` before converting
`ON SIZE`	a target wide enough, or `ROUND` / `TRUNC` / `DIVIDE(x, y, p, q)` to the precision you mean
`ON ZERODIVIDE`	`IF Y ¬= 0 THEN ...`
`ON SUBSCRIPTRANGE`	`HBOUND` / `LBOUND` / `LENGTH` checks
`ON ERROR` around `EXEC CICS`	`RESP(RC)` on the command and a test of `RC` (Chapter 36)

CICS conditions (`NOTFND`, `DUPREC`, `MAPFAIL`, ...) are **not** runtime errors: they never abend a PL/I task (Chapter 36).

Chapter 36. EXEC CICS in PL/I

Statement form

PL/I uses the IBM-canonical Enterprise PL/I form: `EXEC CICS`, the command, and a `;`. There is no `END-EXEC`.

```
EXEC CICS RECEIVE MAP('ORDMAP') MAPSET('ORDSET') INTO(ORDER_IN) RESP(RC);
EXEC CICS SEND MAP('ORDMAP') FROM(ORDER_OUT) ERASE;
EXEC CICS READ FILE('customers') INTO(REC) RIDFLD(KEY) RESP(RC);
EXEC CICS LINK PROGRAM('CALC') COMMAREA(CALC_AREA);
EXEC CICS RETURN TRANSID('ORDR') COMMAREA(STATE);
```

- Every command, option and clause IBM documents is accepted with the same spelling and meaning as in COBOL and REXX. The command set is the one in **Parts 4**, **5** and **7**; options bricks cannot act on are accepted and ignored.
- The statement runs to the first `;` at parenthesis depth 0 outside quotes, so a command may span lines freely (unlike the REXX one-line rule).
- Host variables are plain PL/I references with **no `:`** prefix: `INTO(ORDER_IN)`, `RIDFLD(CUST.KEY)`, `FROM(LST.ROW(3))`.
- A verb bricks does not dispatch is a compile error: `EXEC CICS SENDD MAP('X');` fails with `UNKNOWN CICS COMMAND SENDD` instead of answering `INVREQ` at run time.

Options are PL/I expressions

Option arguments are PL/I expressions: `RIDFLD(KEY)`, `LENGTH(STG(REC))`, `ITEM(I + 1)`, `MAP('CUST' || SUFFIX)`. The compiler rewrites each command once:

- An argument that folds to a constant is inlined: `MAP('CU' || 'ST1')` reaches CICS as `MAP('CUST1')`, `LENGTH(STG(REC))` as `LENGTH(25)`. `DFHRESP`, `DFHVALUE` and `VALUE` names fold too.
- Every other reference becomes a compiler-generated alias bound to its storage at each execution: `READ FILE('F') INTO(_1_REC) RIDFLD(_2_KEY) RESP(_3_RC)`. A computed argument is bound as `_<n>_`. These names show up in error messages; they are why `_<digits>_` names are reserved.
- Qualified names (with the usual partial qualification) and subscripted elements are allowed wherever an area is expected.
- The CICS handler reads a fixed `CHAR` value without its trailing blanks, as for COBOL; data areas (`COMMAREA`, `FROM` of `LINK`) travel with all their bytes, packed and binary fields included.
- Data moved into a structure or array (`INTO`, `COMMAREA`) overlays its first bytes. When the data is shorter (a `READ ... INTO` of a short record), the bytes past its end **keep their previous value**, as in IBM CICS. Clear the area first (`REC = '';`) when stale bytes matter.

RESP, RESP2 and DFHRESP

```
T: PROC OPTIONS(MAIN);
   %INCLUDE DFHRESP;
   DCL REC  CHAR(200) VAR;
   DCL KEY  CHAR(8) INIT('C0042');
   DCL MSG  CHAR(60) VAR;
   DCL (RC, RC2) FIXED BIN(31);

   EXEC CICS READ FILE('customers') INTO(REC) RIDFLD(KEY)
             RESP(RC) RESP2(RC2);
   SELECT (RC);
      WHEN (DFHRESP(NORMAL)) MSG = 'Found.';
      WHEN (DFHRESP(NOTFND)) MSG = 'Customer ' || KEY || ' not found.';
      OTHERWISE MSG = 'READ failed RESP=' || CHAR(RC)
                    || ' RESP2=' || CHAR(RC2);
   END;
   IF RC = RESP_NOTFND | RC = DFHRESP_NOTFND THEN PUT SKIP LIST(MSG);

   EXEC CICS DELETEQ TS QUEUE('SCRATCH') NOHANDLE;
   IF EIBRESP ¬= DFHRESP(NORMAL) THEN PUT SKIP LIST('EIBRESP', EIBRESP);
END T;
```

- `RESP(var)` and `RESP2(var)` are accepted on every command. After the command `var` holds the response; `FIXED BIN(31)` is the natural type. PL/I has no predeclared `RC`: declare the variable you name.
- `DFHRESP(cond)` is a compile-time constant; no include is needed. The `DFHRESP` member adds the same values as named constants (`RESP_NOTFND` and `DFHRESP_NOTFND`, see Chapter 37).
- Coding `RESP` or `NOHANDLE` makes the command's conditions non-fatal and suppresses `HANDLE CONDITION` and `HANDLE AID` for that one command, as in IBM CICS.
- Writes the runtime makes to `EIBRESP`, `EIBRESP2` or a `RESP` target never touch any other program variable.

Conditions never abend a PL/I task. A non-`NORMAL` response (`NOTFND`, `DUPREC`, `MAPFAIL`, `PGMIDERR`, ...) never ends a PL/I program, even with no `RESP`, no `NOHANDLE` and no handler: the program continues and reads the outcome from `EIBRESP`. This is a deliberate deviation from IBM's default action (which abends). Code `RESP(RC)` on every command anyway, as `cusp.pli` does: it documents that the result is tested, and it suppresses any `HANDLE CONDITION` left armed.

Some outcomes are **not** CICS conditions, and end the task whatever the program codes:

- the terminal connection is lost during `SEND` / `RECEIVE`;
- `EXEC CICS RETURN`, `XCTL` and `ABEND`;
- a `HANDLE CONDITION` / `HANDLE AID` branch that fires (when the command codes neither `RESP` nor `NOHANDLE`);
- a `LINK`ed program that fails while it runs (abend or runtime error). A missing `LINK` target is `PGMIDERR`, an ordinary condition;
- a blocking `SEND` / `RECEIVE` in a scheduler-fired background task.

> **Programming Note.** The same `RESP` / `NOHANDLE` rule now holds for REXX and COBOL: with either option coded, a CICS condition never ends the task (Response-code handling). What differs is the case with neither option: an error-class condition (such as `MAPFAIL` for a map that is not installed) ends a REXX or COBOL task, while a PL/I task continues.

The EIB and other predeclared names

Every PL/I program sees these names without declaring them.

`DFHEIBLK`**, the EIB.** Its fields are referenced unqualified:

Field	PL/I type	Meaning
`EIBAID`	`CHAR(1)`	AID byte of the last terminal input; compare with `DFHAID` names
`EIBRESP`	`FIXED BIN(31)`	Response of the last `EXEC CICS` command
`EIBRESP2`	`FIXED BIN(31)`	Response detail (RESP2)
`EIBCALEN`	`FIXED BIN(15)`	Length of the COMMAREA passed in
`EIBTRMID`	`CHAR(4)`	Terminal id, 4 characters as in IBM `DFHEIBLK` and the COBOL EIB. bricks ids have 5 (`T0001`), so `EIBTRMID` reads `T000`; use `EXEC CICS ASSIGN TERMID(x)` into a `CHAR(8)` for the full id
`EIBTRNID`	`CHAR(4)`	Transaction id
`EIBTASKN`	`FIXED DEC(7)`	Task number
`EIBDATE`	`FIXED DEC(7)`	Task start date, `0CYYDDD`
`EIBTIME`	`FIXED DEC(7)`	Task start time, `0HHMMSS`
`EIBCPOSN`	`FIXED BIN(15)`	Cursor offset at the last input
`EIBFN`	`CHAR(2)`	Function code of the last command
`EIBABCODE`	`CHAR(4)`	Abend code (`HANDLE ABEND`, `ASSIGN ABCODE`)

The EIB is a real data structure in PL/I: `EIBTRNID`, `EIBTASKN`, `EIBDATE` and `EIBTIME` are filled in when the program starts, not only on `ASSIGN`. Field meanings are in Chapter 11.

`SQLCA`, the SQL communication area: `SQLCODE FIXED BIN(31)`, `SQLSTATE CHAR(5)`, `SQLERRMC CHAR(70) VAR`, `SQLERRP CHAR(8)`, `SQLERRD(6) FIXED BIN(31)` (`SQLERRD(3)` = rows affected) and the flags `SQLWARN0` .. `SQLWARN9`, `SQLWARNA` (`CHAR(1)`), as in the COBOL SQLCA (Chapter 26).

`DFHCOMMAREA` (COMMAREA).

Maps as structures

`SEND MAP ... FROM(struct)` and `RECEIVE MAP ... INTO(struct)` move map fields to and from the members of a structure **by name**, exactly as the COBOL group and the REXX stem do (Chapter 3). Declare one member per map field, with the field's length:

```
CUSPDEMO: PROC OPTIONS(MAIN);
   %INCLUDE DFHAID;
   DCL 1 MENU,                       /* map CUST1                */
         2 INFOLINE CHAR(78),
         2 ACTION   CHAR(1),
         2 CUSTNO   CHAR(8),
         2 SEARCH   CHAR(30),        /* a builtin name: legal here */
         2 MSG      CHAR(60),
         2 MSG_C    CHAR(9);         /* runtime colour override    */
   DCL 1 MENU_IN LIKE MENU;
   DCL 1 LST,                        /* map CUSTL: ROW1 .. ROW15   */
         2 INFOLINE CHAR(78),
         2 ROW(15)  CHAR(71);
   DCL RC FIXED BIN(31);
   DCL P  FIXED BIN(31);

   MENU = '';
   MENU.MSG   = 'Type an action and press ENTER.';
   MENU.MSG_C = 'YELLOW';
   EXEC CICS SEND MAP('CUST1') FROM(MENU) ERASE RESP(RC);
   MENU_IN = '';
   EXEC CICS RECEIVE MAP('CUST1') INTO(MENU_IN) RESP(RC);
   IF EIBAID = PF03 | RC = DFHRESP(MAPFAIL) THEN EXEC CICS RETURN;

   P = SEARCH(MENU_IN.CUSTNO, ' ');  /* with arguments: builtin    */
   LST = '';
   LST.INFOLINE = 'Search: ' || TRIM(MENU_IN.SEARCH);
   LST.ROW(1)   = 'first row';
   EXEC CICS SEND MAP('CUSTL') FROM(LST) ERASE RESP(RC);
   EXEC CICS RETURN;
END CUSPDEMO;
```

Frame naming. A map field name is looked up inside the structure in this order:

1. an exact member path (`INFOLINE`, or a deeper qualified path);
2. a unique member of that name anywhere below the structure (not through an array);
3. a field named `ROWn` (as in `CUSTL`'s `ROW1` .. `ROW15`) maps to element `n` of a unique one-dimensional array member `ROW`. The PL/I side declares `2 ROW(15) CHAR(71)` instead of fifteen scalars; `CUSTLL`'s 35 rows fit `2 ROW(35)`, and a map with fewer rows simply ignores the extra elements.

- **Colour overrides** use the REXX spelling: a member `FIELD_C` set to a colour mnemonic (`MSG_C = 'YELLOW'`) recolours `FIELD` for that `SEND` (Runtime field attribute overrides).
- **Built-in names as members.** A member may be called `SEARCH`, `INDEX`, `LENGTH`, ... because map fields often are. `MENU_IN.SEARCH` (or an unambiguous bare `SEARCH`) means the member; `SEARCH(s, set)` with arguments is still the built-in.
- **Separate IN and OUT copies.** `cusp.pli` receives into `MENU_IN LIKE MENU` so the next `SEND` of `MENU` never repaints what the operator typed.
- **Numeric input.** `RECEIVE MAP` into a numeric `PIC` member stores the characters as typed (zoned input), so `VALID` can reject bad data. Into a `FIXED DEC` or `FIXED BIN` member the text is converted, and non-numeric input abends `ABCV`. Receive numbers into `PIC` or `CHAR` members and validate them.
- **Sized maps.** Choose the map name at run time (`MAP(M_MENU)`, `M_MENU = 'CUST1' || SUFFIX`) and fall back when `SEND` answers `MAPFAIL` (Appendix A).

COMMAREA and DFHCOMMAREA

`DFHCOMMAREA` holds the COMMAREA passed to the program by `RETURN TRANSID ... COMMAREA`, `XCTL` or `LINK`; `EIBCALEN` is its length.

- **Declared in MAIN.** If the MAIN procedure declares a variable named `DFHCOMMAREA` (usually a structure), the passed bytes are mapped onto it with the storage layout, packed and binary members included. A shorter structure leaves the rest of a `LINK` caller's area untouched.
- **Default.** Otherwise `DFHCOMMAREA` is predeclared as `CHAR(32767) VAR` whose current length is `EIBCALEN`. Assigning to it in a `LINK`ed program replaces the reply: a shorter value is blank padded, a longer one cut.

The reply to a `LINK` is always exactly the caller's COMMAREA length. A pseudo-conversational state block:

```
ORDC: PROC OPTIONS(MAIN);
   DCL 1 DFHCOMMAREA,
         2 STEP     CHAR(1),
         2 CUST_ID  CHAR(8),
         2 TOTAL    FIXED DEC(11,2);
   DCL RC FIXED BIN(31);

   IF EIBCALEN = 0 THEN DO;          /* first entry               */
      DFHCOMMAREA = '';
      DFHCOMMAREA.STEP = '1';
   END;
   ELSE DO;
      DFHCOMMAREA.TOTAL = DFHCOMMAREA.TOTAL + 10;
      DFHCOMMAREA.STEP  = '2';
   END;
   EXEC CICS RETURN TRANSID('ORDC') COMMAREA(DFHCOMMAREA)
             LENGTH(STG(DFHCOMMAREA)) RESP(RC);
END ORDC;
```

A `LINK` target that works on the default area:

```
CUSVP: PROC OPTIONS(MAIN);
   DCL KEY CHAR(8) VAR;
   IF EIBCALEN = 0 THEN RETURN;
   KEY = UPPERCASE(TRIM(SUBSTR(DFHCOMMAREA, 1, MIN(EIBCALEN, 8))));
   DFHCOMMAREA = KEY;                /* blank-padded to EIBCALEN  */
END CUSVP;
```

LINK, XCTL and RETURN across languages

PL/I, REXX and COBOL programs `LINK` and `XCTL` to each other freely; the COMMAREA is opaque bytes. With a PL/I structure on one side and a COBOL group on the other, give both the same byte layout (the layout table):

```
   DCL 1 CALC_AREA,
         2 CA_KEY      CHAR(8),
         2 CA_BALANCE  FIXED DEC(11,2),
         2 CA_COUNT    FIXED BIN(31),
         2 CA_RC       CHAR(2);
   DCL RC FIXED BIN(31);
   CALC_AREA = '';
   CALC_AREA.CA_KEY = 'C0042';
   EXEC CICS LINK PROGRAM('GUSV') COMMAREA(CALC_AREA)
             LENGTH(STG(CALC_AREA)) RESP(RC);
   IF RC = DFHRESP(PGMIDERR) THEN PUT SKIP LIST('GUSV not available');
```

- A `LINK`ed PL/I program runs in the caller's task and unit of work and shares the task's STATIC storage with every other PL/I program of the task.

- A REXX target sees the COMMAREA as a string, a COBOL target maps it onto its `DFHCOMMAREA`; changes flow back to the caller.
- Use `EXEC CICS LINK` to reach REXX, COBOL and other `OPTIONS(MAIN)` PL/I programs; use an `ENTRY ... EXTERNAL CALL` for PL/I subprograms (External entries).
- PL/I transactions also run on an MRO peer region exactly as REXX and COBOL ones do (see `MRO.md`).

START and RETRIEVE

`EXEC CICS START` schedules a PL/I transaction like any other. When the timer fires while the terminal's current task is waiting for input, bricks runs the started task inline as a **background task** (REXX or PL/I programs only). A background PL/I task has no COMMAREA (`EIBCALEN` 0), reads the `START` payload with `RETRIEVE`, may use only write-only verbs (`SEND MAP ... DATAONLY`, file, TS, `ASSIGN`, `START`), and has no SQL connection (`SQLCODE` -1). It commits its unit of work at normal end and backs it out on an abend, like a terminal task (the same holds for REXX background tasks):

```
TICK: PROC OPTIONS(MAIN);
   DCL BUF CHAR(4);
   DCL LEN FIXED BIN(15) INIT(4);
   DCL RC  FIXED BIN(31);
   DCL 1 CLK, 2 CLOCK CHAR(8);

   EXEC CICS RETRIEVE INTO(BUF) LENGTH(LEN) RESP(RC);
   IF RC = DFHRESP(NORMAL) & BUF = 'TICK' THEN DO;
      /* scheduler-fired: write-only verbs only */
      CLK.CLOCK = DATETIME('HH:MI:SS');
      EXEC CICS SEND MAP('CLOCK1') FROM(CLK) DATAONLY RESP(RC);
   END;
   EXEC CICS START TRANSID('TICK') AFTER SECONDS(5) FROM('TICK')
             LENGTH(4) RESP(RC);
END TICK;
```

See `DATAONLY` - background partial paint and Chapter 5.

HANDLE CONDITION, HANDLE AID and HANDLE ABEND

```
   EXEC CICS HANDLE AID PF3(EXIT_PROC) CLEAR(EXIT_PROC);
   EXEC CICS HANDLE CONDITION MAPFAIL(NO_INPUT);
   EXEC CICS HANDLE ABEND LABEL(CLEANUP);
   ...
   EXIT_PROC: PROC;
      EXEC CICS SEND TEXT FROM('Goodbye.') ERASE;
      EXEC CICS RETURN;
   END EXIT_PROC;
```

The subset has no statement labels and no `GO TO`, so a handler target is a procedure:

- It must be a **parameterless internal procedure directly nested in MAIN**; anything else is a compile error (`HANDLER INNER MUST BE A PARAMETERLESS PROC IN T`).
- **Terminal-handler semantics.** When the handler fires, the current call stack is unwound and the handler procedure runs. When it returns, the task ends as if MAIN had executed `RETURN`, and the unit of work commits. The handler may itself issue `EXEC CICS RETURN TRANSID(...)`, `XCTL` or `ABEND`, which then ends the task that way.
- A handler triggered again while a handler procedure is running abends `ABHB`.
- `IGNORE CONDITION`, `HANDLE CONDITION cond` with no target (to disarm), `PUSH HANDLE` and `POP HANDLE` behave as in IBM CICS.

- `RESP` or `NOHANDLE` on a command suppresses `HANDLE CONDITION` and `HANDLE AID` for that command.
- `HANDLE ABEND LABEL(proc)` catches `EXEC CICS ABEND` only (Chapter 35).

  Programming Note. Prefer `RESP(RC)` tests to `HANDLE CONDITION`. Because a handler ends the task, it fits "show a message and exit" paths (PF3, CLEAR, a fatal `MAPFAIL`), not retry loops.

Pointer commands are unavailable

There are no pointers in the subset, so the CICS commands that hand out addresses are compile errors: `ADDRESS` (`ADDRESS NOT SUPPORTED (NO POINTERS)`), `GETMAIN`, `FREEMAIN` and every `SET(ptr)` option. Use the `INTO(area)` form of each command instead (`READ ... INTO`, `RECEIVE ... INTO`, `READQ TS ... INTO`).

EXEC SQL in PL/I

`EXEC SQL ... ;` works like `EXEC CICS`, with IBM PL/I's `:` prefix on host variables:

```
   EXEC SQL INCLUDE SQLCA;
   EXEC SQL WHENEVER SQLERROR DO CALL DB_FAILED;

   EXEC SQL SELECT name INTO :CUSTNM :NM_IND
            FROM customers_sql WHERE id = :CUSTID;
   IF SQLCODE = 100 THEN PUT SKIP LIST('not found');
   ELSE IF NM_IND < 0 THEN PUT SKIP LIST('name is NULL');
   ELSE PUT SKIP LIST('customer', CUSTNM);
```

- The SQLCA is predeclared; `EXEC SQL INCLUDE SQLCA;` and `BEGIN` / `END DECLARE SECTION` are accepted and do nothing.
- A structure host variable expands to its members: `FETCH C1 INTO :ROW` binds `:NAME, :CITY`. A host **array** is a compile error (`ARRAY HOST VARIABLE`).
- The host-variable scan skips quoted strings, `--` comments and Postgres `::` casts, so `'a:b'` or `x::text` are not host variables.
- `WHENEVER SQLERROR | SQLWARNING | NOT FOUND` accepts `CONTINUE`, `DO CALL proc` and `GO TO proc` (treated as `DO CALL`). The procedure follows the handler rules above: parameterless, nested in MAIN, and the task ends when it returns.
- The verbs, cursors, null indicators, `CONNECT TO` and the `SQLCODE` catalogue are those of Chapter 26.

Chapter 37. %INCLUDE

`%INCLUDE` is the only preprocessor statement:

```
CUSTX: PROC OPTIONS(MAIN);
   %INCLUDE DFHAID;
   %INCLUDE DFHRESP;
   %INCLUDE CUSTREC;
```

- `%INCLUDE NAME;` inserts the member `NAME.inc` from `pli_include_dir` (default `runtime/pliinc`; the file name is matched case-insensitively) at token level, before parsing.
- Member names are plain identifiers: no paths, no extension, no directory escape (`%INCLUDE ../DFHAID;` is `INCLUDE NAME EXPECTED`). A missing member is `INCLUDE NOSUCH NOT FOUND`.
- Members may include other members up to **8 levels** deep; a 9th level or an include cycle is a compile error.
- A compile error inside a member names the member and its own line: `DFHAID line 3 col 1: ...`.

- **Cache invalidation.** Every member a program includes is a dependency of its cache entry. Editing or deleting a member recompiles every program that includes it on its next dispatch. (COBOL copybooks, by contrast, need the program to be touched; see Chapter 25.)

A user member is ordinary PL/I text, typically declarations shared by the programs of one application:

```
/* CUSTREC - customer COMMAREA, shared by every CU* program. */
DCL 1 CUST_AREA,
      2 CA_KEY      CHAR(8),
      2 CA_BALANCE  FIXED DEC(11,2),
      2 CA_STATUS   CHAR(1);
```

Shipped members: DFHAID and DFHRESP

Two members ship in `runtime/pliinc`. Like the COBOL copybooks, each declares every name **twice** with the same `VALUE`: a bricks mnemonic and the IBM-traditional alias. Use either style, or mix them.

```
DCL PF03     CHAR(1) VALUE('F3'X);       /* in DFHAID.inc */
DCL DFHPF3   CHAR(1) VALUE('F3'X);
DCL RESP_NOTFND    FIXED BIN(31) VALUE(13);   /* in DFHRESP.inc */
DCL DFHRESP_NOTFND FIXED BIN(31) VALUE(13);
```

Member	bricks mnemonic	IBM alias	Contents
`DFHAID`	`ENTER`, `CLEAR`, `PA1`..`PA3`, `PF01`..`PF24`	`DFHENTER`, `DFHCLEAR`, `DFHPA1`..`DFHPA3`, `DFHPF1`..`DFHPF24`	`CHAR(1)` AID bytes, compared with `EIBAID`
`DFHRESP`	`RESP_NORMAL`, `RESP_NOTFND`, ...	`DFHRESP_NORMAL`, `DFHRESP_NOTFND`, ...	`FIXED BIN(31)` response codes, compared with `EIBRESP` or a `RESP` variable

The byte values are those of the COBOL `DFHAID` table in Chapter 25; the response codes are those of `DFHRESP.cpy` (the PL/I names use _ where COBOL uses –). The `DFHRESP(cond)` built-in gives the same numbers with no include at all.

> **Programming Note.** `ENTER`, `CLEAR` and `PF03` become program names once `DFHAID` is included. Do not declare variables with those names in the same program.

Chapter 38. Restrictions and deviations

Deviations from Enterprise PL/I

Each of these is a deliberate choice for safer business code.

Area	Bricks behaviour	Enterprise PL/I
Declarations	Every variable must be declared (`UNDECLARED name`)	Implicit declaration, usually FLOAT
`DEC` / `BIN`	`FIXED` is required	Bare `DEC` / `BIN` means FLOAT
Reserved words	Statement keywords are reserved	No reserved words
Initial values	Every variable starts blank / 0 / `'0'B` / empty	Undefined unless `INIT`
`STATIC`	Per task, shared by the task's activations	Per program load
Storage	Always `UNALIGNED`, fixed byte layout	`ALIGNED` default for some types
Decimal precision	Scale digits dropped to fit 31 digits; overflow only on the integer part	FIXEDOVERFLOW on intermediate results (`25 + 1/3`)

Area	Bricks behaviour	Enterprise PL/I
`/`	Always decimal; BIN operands convert to DEC	Binary division for BIN operands
BIN op DEC	The BIN operand converts to DEC	The DEC operand converts to BIN
Comparisons	Non-chaining	Chaining allowed
`&` / `|`	Short-circuit for `BIT(1)` operands	Evaluation order not guaranteed
`CHAR(x)` of a number	Minimal form (`'42'`, `'-3.00'`)	Right-justified with leading blanks
SIZE	Always enabled: `ABSZ`	Disabled by default
Runtime conditions	Deterministic abend + rollback, no ON-units	ON-units, `SIGNAL`, `REVERT`
CICS conditions	Never abend; test `RESP` / `EIBRESP`	Default action abends the task
CICS handlers	Parameterless procedures nested in MAIN; the task ends when the handler returns	Labels, `GO TO` semantics
`FIXED BIN(64)`	Accepted as `FIXED BIN(63)`	An error without `UNSIGNED`

Not supported

Feature	Why / instead
`ON`, `SIGNAL`, `REVERT`, `ONCODE`, `ONSOURCE`	Validate first; abends are deterministic (Chapter 35)
`GO TO`, statement labels, label variables	`LEAVE`, `ITERATE`, `RETURN`; handler targets are procedures
`BEGIN` blocks	`DO` groups and internal procedures
`FLOAT`, `COMPLEX`	Not for money; use `FIXED DEC`
`POINTER`, `BASED`, `ALLOCATE`, `FREE`, `AREA`, `OFFSET`, `ADDR`, `UNSPEC`	No raw storage; tasks stay isolated. CICS `ADDRESS`, `GETMAIN`, `FREEMAIN`, `SET()` are compile errors.
`ALIGNED`	One byte layout for every type
`CONTROLLED`, `DEFINED`, `EXTERNAL` variables	Overlays and shared mutable state; use files, TS queues or SQL
Stream and record I/O (`GET`, `PUT EDIT`, `FORMAT`, `READ`, `WRITE`, `OPEN`)	`EXEC CICS` and `EXEC SQL`; `PUT LIST` is accepted and discarded, like REXX `SAY`
Macro preprocessor (`%IF`, `%DO`, `%DCL`, `%REPLACE`)	Only `%INCLUDE`
Multitasking, `ENTRY` variables, generic entries	Not needed in the transaction model
SQL host arrays	One row per `FETCH`
CICS pointer verbs	`INTO(area)` forms

Errors bricks reports at compile time

`STORAGE LIMIT EXCEEDED`, `NESTING TOO DEEP`, `UNKNOWN CICS COMMAND x`, `BAD ABEND CODE`, `INIT VALUE TOO LONG` and `TOO MANY INIT VALUES` catch at compile time what Enterprise PL/I reports late or not at all; see Compile-time limits and errors.

Limits

Limit	Value
Identifier length	31
`FIXED DEC` precision	31 digits

Limit	Value
`CHAR` / `CHAR VAR` length	32767
`BIT` length	64
Structure levels	15
Array dimensions per reference	3
`%INCLUDE` nesting	8
Call nesting	1000 (then `ABEN`)
Storage per program (`AUTOMATIC` of all procedures + `STATIC`)	1 MiB (`STORAGE LIMIT EXCEEDED`)
Live activation storage per task	8 MiB (then `ABST`)
Nesting of parentheses, prefix operators, statements, procedures	256 (`NESTING TOO DEEP`)
Expression tree depth	4096 (`NESTING TOO DEEP`)

Part 6. EXEC SQL command reference

> **Part 6 begins here.** The chapter that follows is the complete `EXEC SQL` surface — identical for COBOL, REXX and PL/I. It covers every embedded-SQL verb Bricks supports, the SQLCA layout and SQLCODE catalogue, the `WHENEVER` declarative error handler, cursor lifecycle, per-task `CONNECT TO`, null indicators, and SYNCPOINT integration with the bricks unit of work.

Chapter 26. Embedded SQL (COBOL, REXX and PL/I)

> **Programming Note.** Although this chapter was originally written against COBOL, every `EXEC SQL` shape and behaviour described here applies identically to REXX and PL/I. The language-specific notes appear later in the chapter under *REXX equivalent* and *PL/I equivalent*. Programs in every language read and write the SQLCA fields (`SQLCODE`, `SQLSTATE`, `SQLERRMC`) the same way the COBOL samples do.

bricks accepts the classic IBM-style `EXEC SQL … END-EXEC` embedded-SQL surface and runs it against the Postgres connection configured via `db_*` in `bricks.cnf` (see SQL support — Phase 1). Programs read and write rows through prepared statements; the SQLCA fields `SQLCODE`, `SQLSTATE`, `SQLERRMC` are auto-injected into every program's DATA DIVISION so the program can test them without declaring them.

Scope: single-row CRUD (`SELECT INTO`, `INSERT`, `UPDATE`, `DELETE`, `COMMIT`, `ROLLBACK`), the four-verb cursor lifecycle (`DECLARE` / `OPEN` / `FETCH` / `CLOSE`), per-task `CONNECT TO 'name'` database switching, null indicators (`:hostvar :indicator`), and the `WHENEVER` declarative error-handling directive. The same surface works identically in COBOL, REXX and PL/I.

Format

```
        EXEC SQL <statement> END-EXEC.
```

Where `<statement>` is any of the supported verbs (below). Host variables are written with a leading colon: `:CUSTID`, `:NM`, `:BALANCE`. The variable name must already exist in DATA DIVISION — the SQL executor reads its current value (for INPUT bindings) and writes back (for SELECT INTO targets).

Supported verbs

Verb	Notes
`SELECT cols INTO :v, :v FROM …`	Single-row read. Returns +100 (no-data) when 0 rows, -811 when >1 row.
`INSERT INTO t (…) VALUES (…)`	Host-variable references bound as $N placeholders.
`UPDATE t SET … WHERE …`	Returns RESP-NORMAL even when 0 rows match (matches DB2).
`DELETE FROM t WHERE …`	Same row-count semantics as UPDATE.
`COMMIT WORK`	Commits the per-task PG transaction. Also fired by `EXEC CICS SYNCPOINT`.
`ROLLBACK WORK`	Rolls back. Also fired by `EXEC CICS SYNCPOINT ROLLBACK`.

DDL (`CREATE DATABASE`, `DROP DATABASE`, `CREATE USER`, `DROP USER`, `ALTER ROLE`, `CREATE TABLESPACE`, …) is **rejected** with `SQLCODE = -2` and `SQLSTATE = 42501`. CEDA DATABASE owns those operations and audits each one through `brickslog.Audit`. `CREATE TABLE` / `ALTER TABLE` / similar non-privileged DDL is not blocked

but is also unsupported in the executor's statement classification — porters should run schema migrations with `psql`.

SQLCODE catalog

bricks adopts the IBM DB2 convention adapted to Postgres. The catalog below covers what programs typically test:

SQLCODE	SQLSTATE	Meaning
0	00000	Success.
+100	02000	No data: SELECT INTO returned 0 rows, or an update/delete affected 0 rows.
−1	empty	No database configured (no db_* lines in bricks.cnf).
−2	42501	DDL rejected (CEDA owns CREATE/DROP DATABASE
−100	varies	Generic SQL failure — PG returned an error not specifically mapped.
−811	21000	SELECT INTO returned more than one row.

`SQLERRMC` always carries the human-readable message from Postgres (truncated to 70 chars to match the PIC X(70) auto- injection). When the error path includes a specific SQLSTATE, that 5-char code lands in `SQLSTATE`; otherwise `SQLSTATE` is the relevant standard code (`00000` on success, `02000` on no-data, `HV000` when the failure didn't carry a state).

SYNCPOINT integration

Each task lazily begins a Postgres transaction on its first `EXEC SQL` that touches data. The transaction commits when:

- `EXEC CICS SYNCPOINT` runs.
- `EXEC SQL COMMIT WORK` runs.
- The task ends cleanly (RETURN, GOBACK, STOP RUN).

And rolls back when:

- `EXEC CICS SYNCPOINT ROLLBACK` runs.
- `EXEC SQL ROLLBACK WORK` runs.
- The task abends via `EXEC CICS ABEND`.

Postgres-side commit fires *before* the bricks-side bbolt journal commit. That ordering avoids a hung PG commit leaving the bricks journal already finalised; failures during the SQL commit surface as `ROLLEDBACK` from SYNCPOINT, and the bricks journal then rolls back too.

Idiomatic test

```
       EXEC SQL
           SELECT name INTO :NM
           FROM customers_sql
           WHERE id = :CUSTID
       END-EXEC.

       EVALUATE SQLCODE
           WHEN RESP-NORMAL   PERFORM SHOW-NAME       *> +0
           WHEN RESP-NOTFND   PERFORM SHOW-NOTFOUND   *> wait -- that's an EIBRESP code; SQLCODE
uses RESP-NORMAL/+100/etc.
           WHEN OTHER         PERFORM SHOW-SQL-ERR
       END-EVALUATE.
```

The `RESP-NORMAL` constant (from `COPY DFHRESP`) is value `0`, the same as the SQLCODE-OK value, so it conveniently doubles as the success test for both EIBRESP (EXEC CICS) and SQLCODE (EXEC SQL). For the no-data case, test `SQLCODE = +100` directly — there's no DFHSQLCODE copybook yet.

Worked example: SQLD

`runtime/cobol/sqld.cob` (TRANSID `SQLD`) is the bundled demonstration. It reads a single `customers_sql` row by id and renders the name + SQLCA verdict on screen `SQLD1`. The expected schema is documented in the program header. The operator runs psql once to seed the table:

```
CREATE TABLE customers_sql (
    id   text PRIMARY KEY,
    name text NOT NULL
);
INSERT INTO customers_sql VALUES
    ('K001', 'Alice'),
    ('K002', 'Bob'),
    ('K003', 'Carol');
```

Then `bricks` → sign on → `SQLD` → type `K001` → ENTER. The operator sees `Alice` in the NM cell with SQLCODE = 0 and SQLSTATE = 00000.

REXX equivalent

REXX programs use the same `EXEC SQL` shape as COBOL, with `:name` binding REXX variables. There's no DATA DIVISION analog -- REXX is dynamically typed -- so SQLCODE, SQLSTATE, SQLERRMC arrive as ordinary REXX variables the program reads after each statement:

```
ADDRESS CICS

CUSTID = 'K001'
EXEC SQL
    SELECT name INTO :NM
    FROM customers_sql
    WHERE id = :CUSTID
END-EXEC

IF SQLCODE = 0 THEN
    SAY 'Customer:' NM
ELSE IF SQLCODE = 100 THEN
    SAY 'Not found'
ELSE
    SAY 'SQL error' SQLCODE ':' SQLERRMC
```

Behind the scenes the REXX preprocessor (`rexx.PreprocessExecCICS`) rewrites the EXEC SQL block into a sentinel bare-string that `runCommand` recognises and routes to the same per-task SQL executor COBOL uses. The two language surfaces are parity-clean: the executor doesn't know or care which language drove it. Live sample: `runtime/rexx/sqlr.rexx`, TRANSID `SQLR`.

REXX stem-tail gotcha. Classic REXX resolves `STEM.NAME` by looking up the simple symbol `NAME` and using its value as the tail. If a program does `EXEC SQL ... INTO :NM` then writes `SCR.NM = NM`, the LHS resolves to `SCR.<value-of-NM> = SCR.Alice` — the value lands in the wrong slot. Workaround: pick a host-var name that is NOT also a stem tail. The bundled SQLR uses `CUSTNM` instead of `NM` for exactly this reason.

PL/I equivalent

PL/I programs write `EXEC SQL ... ;` (no `END-EXEC`) with IBM PL/I's `:` prefix on host variables. The SQLCA is predeclared, with `SQLCODE FIXED BIN(31)`, `SQLSTATE CHAR(5)`, `SQLERRMC CHAR(70) VAR` and `SQLERRD(6) FIXED BIN(31)`:

```
SQLP: PROC OPTIONS(MAIN);
   DCL CUSTID CHAR(8) INIT('K001');
   DCL CUSTNM CHAR(40) VAR;
   DCL 1 ROW,
         2 NAME CHAR(40) VAR,
         2 CITY CHAR(20) VAR;
   DCL NM_IND FIXED BIN(15);

   EXEC SQL INCLUDE SQLCA;
   EXEC SQL WHENEVER SQLERROR DO CALL DB_FAILED;

   EXEC SQL SELECT name INTO :CUSTNM :NM_IND
            FROM customers_sql WHERE id = :CUSTID;
   IF SQLCODE = 100 THEN PUT SKIP LIST('not found');
   ELSE IF NM_IND < 0 THEN PUT SKIP LIST('name is NULL');
   ELSE PUT SKIP LIST('customer', CUSTNM);

   EXEC SQL DECLARE C1 CURSOR FOR
            SELECT name, city FROM customers_sql WHERE id >= :CUSTID;
   EXEC SQL OPEN C1;
   DO WHILE ('1'B);
      EXEC SQL FETCH C1 INTO :ROW;    /* expands to :NAME, :CITY */
      IF SQLCODE ¬= 0 THEN LEAVE;
      PUT SKIP LIST(ROW.NAME, ROW.CITY);
   END;
   EXEC SQL CLOSE C1;

   DB_FAILED: PROC;
      PUT SKIP LIST('SQL failed', SQLCODE, SQLSTATE, SQLERRMC);
      EXEC CICS SYNCPOINT ROLLBACK;
      EXEC CICS ABEND ABCODE('SQLF');
   END DB_FAILED;
END SQLP;
```

- **Host structures** expand to their members, so `INTO :ROW` binds `:NAME, :CITY` in declaration order. A host array is a compile error.
- `EXEC SQL INCLUDE SQLCA;` and `BEGIN` / `END DECLARE SECTION` are accepted and do nothing.
- `WHENEVER` takes `CONTINUE`, `DO CALL proc` or `GO TO proc` (treated as `DO CALL`). The target is a parameterless procedure nested directly in MAIN. It runs as a terminal handler: when it returns the task ends and commits, so issue `SYNCPOINT ROLLBACK` or `ABEND` in it when the work must not be kept. See HANDLE CONDITION, HANDLE AID and HANDLE ABEND.
- Host variables of a `DECLARE ... CURSOR` are bound again when `OPEN` runs, so the cursor sees their values at `OPEN` time. This holds wherever the `DECLARE` sits in the source, even after the `OPEN` or in a branch that never runs.
- The host-variable scan skips quoted strings, `--` comments and Postgres `::` casts.
- A scheduler-fired background PL/I task has no database connection; every statement answers `SQLCODE = -1`.

WHENEVER (declarative error handling)

Instead of testing `SQLCODE` after every statement, a program can declare a standing rule with `EXEC SQL WHENEVER`:

```
EXEC SQL WHENEVER SQLERROR  GO TO SQL-ERROR-EXIT END-EXEC.
EXEC SQL WHENEVER NOT FOUND GO TO NO-MORE-ROWS  END-EXEC.

EXEC SQL SELECT name INTO :NM FROM customers_sql
         WHERE id = :CUSTID END-EXEC.
*> no SQLCODE test here -- WHENEVER handles it
```

Three conditions, checked after every subsequent `EXEC SQL`:

Condition	Fires when
`SQLERROR`	`SQLCODE` is negative
`NOT FOUND`	`SQLCODE` = +100
`SQLWARNING`	`SQLWARN0` = `'W'` (bricks sets it when a result lost characters to the SBCS conversion, see `SQLWARN8`; PG itself errors rather than warns)

Each condition takes one of two actions:

- `CONTINUE` — ignore the condition, fall through (the default for any condition never declared).
- `GO TO label` / `GOTO label` — branch to the paragraph (COBOL) or label (REXX). The branch unwinds any enclosing `PERFORM` exactly as an explicit `GO TO` would.

A later `WHENEVER` for the same condition replaces the earlier one (e.g. declare `GO TO` for a block of statements, then `CONTINUE` to resume manual handling).

REXX uses the identical syntax; the branch is a `SIGNAL` to the named label:

```
EXEC SQL WHENEVER SQLERROR GOTO SQLERR END-EXEC
EXEC SQL SELECT name INTO :CUSTNM FROM customers_sql
         WHERE id = :CUSTID END-EXEC
SAY 'customer:' CUSTNM
EXIT
SQLERR:
SAY 'SQL failed, SQLCODE' SQLCODE
EXIT
```

PL/I has no statement labels: its `WHENEVER` names a procedure (`EXEC SQL WHENEVER SQLERROR DO CALL DB_FAILED;`, with `GO TO` accepted as a synonym), and the task ends when the procedure returns. See PL/I equivalent.

Scoping caveat. bricks's `WHENEVER` is *execution-order* scoped: the directive set by the most recently **executed** `EXEC SQL WHENEVER` governs later statements. Real DB2 scopes *lexically* (by source position) because its precompiler inlines a `SQLCODE` test after every statement. For the dominant pattern — one `WHENEVER` near the top of the program — the two are identical. Programs that re-declare `WHENEVER` inside conditional branches see execution-order semantics; keep WHENEVER declarations at the top of a paragraph to avoid surprises.

Cursors (DECLARE / OPEN / FETCH / CLOSE)

Multi-row result sets use the four-verb cursor lifecycle. The executor parks one `*sql.Rows` per cursor per task; cursors are closed automatically on `COMMIT`, `ROLLBACK`, or task end so a program that forgets `CLOSE` doesn't pin Postgres MVCC.

```
EXEC SQL DECLARE C1 CURSOR FOR
    SELECT id, name FROM customers_sql ORDER BY id
END-EXEC.

EXEC SQL OPEN C1 END-EXEC.

PERFORM UNTIL SQLCODE = 100
    EXEC SQL FETCH C1 INTO :ID, :NM END-EXEC
    IF SQLCODE = 0 THEN
        DISPLAY 'row: ' ID ' ' NM
    END-IF
END-PERFORM.

EXEC SQL CLOSE C1 END-EXEC.
```

REXX uses the same four verbs (the preprocessor routes them all through the same executor):

```
EXEC SQL DECLARE C1 CURSOR FOR SELECT id, name FROM customers_sql END-EXEC
EXEC SQL OPEN C1 END-EXEC
DO FOREVER
    EXEC SQL FETCH C1 INTO :ID, :CUSTNM END-EXEC
    IF SQLCODE = 100 THEN LEAVE
    SAY 'row:' ID CUSTNM
END
EXEC SQL CLOSE C1 END-EXEC
```

End-of-data is `SQLCODE = +100`. A closed cursor stays in the registry, so `OPEN c1` can rewind it without re-`DECLARE`.

Database binding (`transactions.conf` 5th field)

Every transaction starts on exactly one database. Which one is fixed by the 5th colon-separated field of its row in `runtime/transactions.conf`:

```
BANK:cobol:bank.cob:public,users,admin:bank
```

The grammar of the file is therefore:

```
transid:lang:program[:groups[:database]]
```

- `database` -- name of a row in `runtime/databases.conf`. The match is **case-insensitive**: both sides are folded to lower case, the way Postgres folds unquoted identifiers, so `:BANK` and `:bank` reach the same database. (They used to be two different records opening two pools onto one physical database, and a `:BANK` binding against a `bank` row silently failed at run time with `SQLCODE = -1`.) Whitespace around the name is trimmed, and the name may not contain `:`, a newline or a whitespace-preceded `#` -- `CEDA TRANSACTION` refuses those, because they produce a line the parser reads back as a different record.
- **Absent 5th field** -- the transaction binds to the **first** row of `databases.conf`. The first row is the deployment-wide default; CEDA DATABASE labels it `(def)` and refuses `Delete` on it.
- **Unknown name** -- the dispatcher logs the misbinding at task start and the first `EXEC SQL` returns `SQLCODE = -1` with an empty `SQLSTATE`.

A transaction whose program never runs `EXEC SQL` may leave the field empty -- the implicit default never gets touched. A transaction whose program **does** run `EXEC SQL` against a non-default database **must** list

the binding explicitly: omitting it sends every statement to the default pool, which typically does not host the requested table. The operator-visible symptom is a flood of paired errors:

```
SQL error SQLCODE=-204 SQLSTATE=42P01: relation "accounts" does not exist
SQL error SQLCODE=-100 SQLSTATE=25P02: current transaction is aborted, commands ignored until en
d of transaction block
```

The `-204` is the first statement landing on the wrong pool; the `-100 / 25P02` cascade is Postgres marking the per-task PG transaction aborted, so every follow-up statement under a `WHENEVER SQLERROR CONTINUE` declaration logs the same pair until SYNCPOINT runs at task end. Fix: add the correct 5th field and restart the task.

`CEDA TRANSACTION` (A / U actions) and operator-`vi` edits of `transactions.conf` both honour the field; the file is hot- reloaded on mtime change.

CONNECT TO 'name' (per-task database switch)

A program that needs to touch a second database during a task issues `EXEC SQL CONNECT TO 'name'`. The named database must exist in `databases.conf` (the CEDA DATABASE catalogue); an unknown name returns `SQLCODE = -1`. The name is matched case-insensitively — it is folded to lower case, like every other database-name path in bricks — so `CONNECT TO 'ORDERS'` finds the `orders` row. An in-flight PG transaction on the previous database is committed implicitly (matches DB2's behaviour) -- programs that want to discard the previous work should `EXEC SQL ROLLBACK` before connecting.

```
EXEC SQL CONNECT TO 'orders' END-EXEC.
EXEC SQL INSERT INTO order_lines VALUES (:ID, :QTY) END-EXEC.
EXEC SQL COMMIT END-EXEC.
EXEC SQL CONNECT TO 'customers' END-EXEC.
EXEC SQL SELECT email INTO :EM FROM customers WHERE id = :CID END-EXEC.
```

Implicit binding still applies to the first statement of every task -- the transaction's `Database` column in `transactions.conf` picks the starting pool. `CONNECT TO` is only needed when the program needs to move between databases inside a single task.

Null indicators (`:hostvar :indicator`)

DB2 programs use a second host var beside the value to signal NULL. Bricks supports the same pair syntax in both COBOL and REXX:

```
EXEC SQL
    SELECT name INTO :NM :NIND
    FROM customers_sql WHERE id = :CUSTID
END-EXEC.

EVALUATE NIND
    WHEN 0   DISPLAY 'Name: ' NM
    WHEN -1  DISPLAY 'Name is NULL'
END-EVALUATE.
```

Output side (SELECT INTO / FETCH INTO):

- Column was non-NULL → indicator = `0`, value holds the result.
- Column was NULL → indicator = `-1`, value is cleared to the empty string so a program that ignored its indicator can't read a stale value from a previous statement.

Input side (WHERE / VALUES / SET clauses):

```
MOVE -1 TO NIND.
EXEC SQL
    INSERT INTO customers_sql (id, name) VALUES (:K, :NM :NIND)
END-EXEC.
```

- Indicator = `-1` at bind time → bricks passes NULL to PG, regardless of what `NM` currently holds.
- Indicator = `0` (or unset) → the value var's frame contents bind normally.

Pair detection: a value/indicator pair is two `:name` tokens separated only by whitespace. `:NM, :NIND` (with a comma) means two independent host vars, not a pair. The `INDICATOR` keyword form (`:NM INDICATOR :NIND`) is not currently supported -- use the juxtaposed form.

Column-value coercion

PG's wire format doesn't always match what COBOL or REXX want to read. Bricks normalises three cases before the value reaches the program's host variable:

- `bool` **columns** -- PG returns `"t"` / `"f"` (or `"true"` / `"false"`). Bricks converts to `"1"` / `"0"` so COBOL PIC X(1) flags and REXX `IF VAR = 1` idioms work the way they do in DB2.
- `NUMERIC(p, s)` **columns** -- PG drops trailing fractional zeros for computed expressions (`SELECT 1.5` returns `"1.5"` even though the result type has scale 2). Bricks pads the value to the column's declared scale, so a COBOL `PIC 9(3)V99` target receives `"1.50"` → digits `"150"` → stored as `001.50`, not `000.15`.
- `CHAR(N)` **columns** -- PG returns space-padded values. Bricks trims trailing spaces so a REXX literal compare (`IF VAR = 'X'`) doesn't fail because the CHAR(5) value was `"X "`. VARCHAR is left alone (PG never pads it; its trailing spaces are real).

Other types (INT, BIGINT, TEXT, DATE, TIMESTAMP, JSON, …) flow through verbatim. COBOL's MOVE semantics handle integer sign and zero-pad automatically; REXX is dynamically typed so the string form works for both display and arithmetic.

Restrictions

Out of scope	Why
Stored procedure calls (`EXEC SQL CALL`)	Out of scope — porters use plain SELECT-from-function.
Multi-row INSERT VALUES (…), (…)	Out of scope; rewrite as one statement per row.
`CREATE` / `DROP DATABASE`	USER`
Long-running cursors across `EXEC CICS RETURN` chains	Out of scope; each task gets a fresh PG connection.

Chapter 27. Restrictions and deferred features

This chapter is the canonical list of language features bricks does **not** accept. Anything not listed here can be assumed implemented; refer back to the relevant Chapter 14–26 section for the supported syntax. The PL/I subset has its own list in Chapter 38.

COBOL — disallowed at parse time

- **Calculated GOTOs.** `GO TO ... DEPENDING ON expr` is rejected. Every task has its own heap and stack with no static control-block aliasing; supporting calculated GOTOs would require a per-program label-table the parser deliberately doesn't build.
- **Nested `OCCURS`.** An array of arrays is rejected at parse time. One level of `OCCURS` is fine.
- `EXIT SECTION`**.** bricks has no PROCEDURE DIVISION sections.
- **Forms IBM itself forbids.** These are rejected with a short message rather than run with a guessed meaning: a literal, figurative constant or function as the `UNSTRING` source; a reference-modified `STRING INTO` receiver; `DELIMITER IN` / `COUNT IN` without `DELIMITED BY`; `WHEN 'A' OR 'B'` (stack two `WHEN`s); an `EVALUATE WHEN` whose `ALSO` count differs from the subjects; `NOT NOT ...`; and an abbreviated relation that opens a parenthesised group (`A = B AND (< C OR D)`). See Chapter 22 for each.

COBOL — deferred (would be implementable, not yet wired)

Area	Specifics
DATA DIVISION	`USAGE COMP-1` (single float), `USAGE COMP-2` (double float), `SYNC`, `INDEXED BY`, `OCCURS DEPENDING ON`, 66 / 78 levels, `BLANK WHEN ZERO`, and IBM's extension allowing a level-01 redefiner *larger* than its target (bricks requires the redefining item to fit at every level). `USAGE COMP` / `COMP-4` / `COMPUTATIONAL` / `BINARY`, `USAGE COMP-3` / `COMPUTATIONAL-3` / `PACKED-DECIMAL`, and `REDEFINES` ARE supported — see Chapter 21.
Edited PIC	Edit characters +, −, `CR`, `DB`, `B`, `0`, `/`. (Z, `.`, `,`, `$`, ∗ are supported.)
PROCEDURE DIVISION	`CALL` (external program), `SEARCH` / `SEARCH ALL`, `ALTER`, `INITIALIZE`, `ACCEPT`, `GO TO ... DEPENDING ON`, `DIVIDE … REMAINDER`. In-line `PERFORM ... END-PERFORM`, `PERFORM ... THRU`, `WITH TEST BEFORE/AFTER`, `VARYING ... AFTER`, `EXIT PERFORM [CYCLE]`, `EXIT PARAGRAPH`, full `EVALUATE`, parenthesised and abbreviated combined conditions, the full IBM `STRING` / `UNSTRING` (`POINTER`, `OVERFLOW`, `DELIMITER IN`, `COUNT IN`, `TALLYING IN`) and `JSON GENERATE` / `JSON PARSE` ARE supported — see Chapter 22.
Reference modification	Two forms are rejected at parse time: a slice on an `OF` / `IN` qualifier (`K(1:2) OF ROW`), and a subscript combined with a slice in one reference (`TBL(idx)(start:length)`). Slice the whole reference, or `MOVE` the OCCURS slot to a work field first. Plain `identifier(start:length)` on either side of a statement IS supported — see Chapter 22.
Intrinsic functions	Only `UPPER-CASE`, `LOWER-CASE`, `LENGTH`, `BYTE-LENGTH`, `ORD`, `CHAR`, `NUMVAL`, `TRIM`, `REVERSE`, `POS` are implemented. `PIC N` / `NATIONAL` and `NATIONAL-OF` / `DISPLAY-OF` are not supported. The statistical / date / time families are not. `FUNCTION TRIM` does NOT accept the `LEADING` / `TRAILING` modifier.
JSON	`JSON GENERATE` / `JSON PARSE` phrases `ENCODING`, `IGNORING`, `INDICATING` and `CONVERTING ... FROM JSON NULL`; the single condition-name form of `JSON PARSE ... CONVERTING` cannot set "false" (no 88 `... WHEN SET TO FALSE`). The statements themselves ARE supported — see JSON GENERATE and JSON PARSE.
Copybooks	`COPY ... REPLACING ==X== BY ==Y==.`, library qualifiers (`COPY name OF lib`), `SUPPRESS`.
EXEC SQL	Multi-row `INSERT VALUES (...), (...)`. Stored-procedure calls (`EXEC SQL CALL`). DDL inside the executor — use `psql` or CEDA DATABASE.
EXEC CICS	`SCREENHT`-based map family suffix (e.g. `CUST1L` on a mod-4 screen) is used by the REXX `CUST` family and the PL/I `CUSP`; the COBOL twins always render the unsuffixed mod-2 map.

REXX — deferred / accepted but inert

Area	Specifics
Tracing	`TRACE` is parsed but emits no trace output.

Area	Specifics
Settings	`OPTIONS …` parsed and accepted; recognised options are none. `NUMERIC FORM SCIENTIFIC` / `ENGINEERING` accepted syntactically, rendering is always plain decimal.
External routines	No dynamic linking — every callable is either a `PROCEDURE` in the program or a built-in.
Address environments	Only `ADDRESS CICS` is wired for host-command dispatch. `ADDRESS COMMAND` / `ADDRESS SYSTEM` / `ADDRESS TSO` do parse, and `ADDRESS()` reports whichever name is current, but the first host command issued under one fails with `ADDRESS handler not registered`.
Terminal queue	`PUSH`, `QUEUE`, `PULL`, `PARSE PULL` are parsed; the queue is always empty. Use `EXEC CICS RECEIVE` for terminal input.
PARSE	`PARSE SOURCE`, `PARSE VERSION`, `PARSE NUMERIC` are not wired. `PARSE LOWER` is not parsed (only `UPPER`).
Conditions	`FAILURE`, `NOTREADY`, `LOSTDIGITS` are not implemented. `HALT` parses but never fires (bricks has no external-halt fan-in).
Numeric	Arithmetic is float64 internally, then rounded to `NUMERIC DIGITS` significant figures — strict IEEE behaviour is not preserved.
Built-ins	`ERRORTEXT(code)` echoes the code rather than returning the standard REXX error-text string.
Variables / `DROP`	Two divergences, both visible through `SYMBOL()`. First, `DROP A.J` does not derive the tail: the DROP parser consumes a single identifier and `.` is an ordinary symbol character, so the literal tail `J` is dropped where TRL2 drops `A.3` — clear the slot with `A.J = ''` if you need the derived name. Second, after `S. = 'd'` a `DROP S.1` removes the tail but leaves the stem default in force, so `S.1` reads back as `d` and `SYMBOL('S.1')` answers `VAR` where TRL2 answers `LIT`.

Part 8. Sample programs

Part 8 begins here. Bricks ships a curated set of REXX, COBOL and PL/I programs that exercise every command family in the earlier parts. Each transaction has been built to be operator- reachable from the blank prompt and self-explaining when run — reading the source is the fastest way to internalise the bricks programming model.

Chapter 28. Pre-installed sample transactions

COBOL

Transid	File	Notes
HELC	runtime/cobol/hello.cob	Hello-world; SEND MAP('HELO1') FROM(SCR), RETURN. Smallest end-to-end demo.
QAGC	runtime/cobol/qagc.cob	COBOL twin of QAGR (REXX). Validates the QAGE1 birthdate, computes age in years and approximate days, sends QAGR1. Pseudo-conversational redisplay of QAGE1 on validation errors.
GUST	runtime/cobol/gust.cob	COBOL CUST. A=Add, Q=Query, U=Update, D=Delete, L=List, S=Search; the S action LINKs to GUSL with the search term in COMMAREA and renders the match count returned.
GUSV	runtime/cobol/gusv.cob	COBOL twin of CUSV. Validates the customer-number COMMAREA (LINK target).
GUSL	runtime/cobol/gusl.cob	COBOL twin of CUSL. Renders the customers file via STARTBR / READNEXT / ENDBR. Blank inbound COMMAREA = paginated all-records list (PF7/PF8). Non-blank inbound COMMAREA = filtered single-page browse: every record is FUNCTION POS-tested against the upper-cased filter, the first 15 matches populate ROW1..ROW15, and the total match count flows back through DFHCOMMAREA so the caller (GUST) can render a summary.
EXAM	runtime/cobol/exam.cob	Worked example of reading the operator's command-line arguments. Type EXAM 1 2 3 at the blank prompt.
ORDR	runtime/cobol/ordr.cob	Conversational import: reads runtime/tmp/orders.sample.txt via READQ TD, parses pipe-delimited rows, and WRITE FILE('ORDERS') keyed on customer-id. Tolerates duplicates (EIBRESP = RESP-DUPREC). Summary screen shows counts. See worked example E.
TIMC	runtime/cobol/timc.cob	Timed-reminder demo: exercises CONVERSE, START (with INTERVAL + FROM), and RETRIEVE end-to-end. Shares tim1.map + tim2.map with TIMR.
SQLD	runtime/cobol/sqld.cob	Embedded-SQL demo: EXEC SQL SELECT name INTO :NM FROM customers_sql WHERE id = :CUSTID. Renders the row + SQLCODE / SQLSTATE / SQLERRMC. Requires Postgres seeded with the schema shown in Chapter 26.
SQLR	runtime/rexx/sqlr.rexx	REXX twin of SQLD. Shares the customers_sql table; demonstrates how REXX accesses the same EXEC SQL surface (SQLCODE / SQLSTATE / SQLERRMC arrive as plain REXX variables).
GMPT	runtime/cobol/gmpt.cob	COBOL pointer / storage demo: EXEC CICS GETMAIN SET(WS-PTR) FLENGTH(16), SET ADDRESS OF a based LINKAGE 01 onto the buffer, write-and-read through it, then EXEC CICS ADDRESS COMMAREA / ADDRESS EIB. Result painted with SEND TEXT (no map). See GETMAIN/FREEMAIN and the COBOL pointer model.

Transid	File	Notes
TODO	runtime/cobol/todo.cob	First server-side COBOL WAPI sample: a multi-user to-do REST API under /api/todo/ over the VSAM files TODOUSR / TODOLST / TODOITM. Uses JSON GENERATE / JSON PARSE, EVALUATE ... ALSO, EXIT PARAGRAPH and SYNCPOINT ROLLBACK units of work. Browser front end in runtime/web/todo/, test script scripts/todo_api_test.bash. See COBOL server-side sample.

All five non-trivial COBOL samples (QAGC, GUST, GUSL, ORDR, EXAM) COPY DFHAID and/or COPY DFHRESP instead of hard-coding hex AID bytes or numeric EIBRESP codes. Skim any of them as living examples of the named-constant idiom from Chapter 25.

REXX (selected)

Transid	File	Notes
HELO	runtime/rexx/hello.rexx	Hello-world with system-info pane on mod 4.
CUST	runtime/rexx/cust.rexx	Full customer maintenance — Add / Query / Update / Delete / List / Search; LINKs to CUSV for validation.
CUSV	runtime/rexx/cusv.rexx	Validation sub-program; called by CUST via EXEC CICS LINK.
CUSL	runtime/rexx/cusl.rexx	Paginated customer list using STARTBR / READNEXT / ENDBR; adapts paging to mod 2 vs mod 4.
QAGE / QAGR	runtime/rexx/qage.rexx / qagr.rexx	Pseudo-conversational chain; QAGE prompts for a birthdate and chains to QAGR to render the result.
PROD / CONS	runtime/rexx/prod.rexx / cons.rexx	TS queue producer / consumer pair. Conversational; PF3 to exit.
GETC	runtime/rexx/getc.rexx	RECEIVE of command-line + READ FILE + SEND TEXT (no map).
INQR	runtime/rexx/inqr.rexx	EXEC CICS INQUIRE demo: direct INQUIRE TASK on this task plus a STARTBROWSE FILE / INQUIRE FILE NEXT / ENDBROWSE loop listing every file's OPEN/ENABLE status and record/key sizes. Result painted with SEND TEXT (no map). See INQUIRE / SET resources.
TIMR	runtime/rexx/timr.rexx	REXX twin of TIMC: START schedules a reminder; RETRIEVE discriminates cold vs scheduled entry. Shares tim1.map + tim2.map with TIMC.
SCHD	runtime/rexx/schd.rexx	Demonstrates three features in one transaction: START ... AFTER SECONDS(n), ADDRESS() and SYMBOL(). Paints one PASS/FAIL row per check, then schedules itself with AFTER SECONDS(10); the timer-fire path reports the payload and issues no further START, so the chain ends on its own. Contrast the plain AFTER SECONDS(...) here with the manual HH/MM/SS packing TIMR needs for INTERVAL(hhmmss).
CHAT	runtime/rexx/chat.rexx	Real-time multi-user chat. Self-refreshes every 2 seconds via EXEC CICS START TRANSID('CHAT') INTERVAL(000002); the tick handler does SEND MAP ... DATAONLY so the operator's in-progress typing at the bottom of the screen is **not** clobbered by the refresh. Messages persist in the auto-created KSDS file CHATLOG (one record per message, key shape YYYYMMDDHHMMSS–NNNN–TTTT for lexicographic / chronological sort). Adapts between Model 2 (16 history rows, chatm2.map) and Model 4 (35 history rows, chatm4.map) via EXEC CICS ASSIGN SCREENHT. F3 exits cleanly — no further tick is scheduled, the self-refresh chain dies on its own. Colours mirror the original tsu/chat.go palette: BLUE/BRIGHT title, TURQUOISE clock + footer, YELLOW topic, RED status line, GREEN history rows + prompt, WHITE underscored input.

PL/I

Transid	File	Notes
CUSP	runtime/pli/cusp.pli	PL/I port of CUST in a single program: A=Add, Q=Query, U=Update, D=Delete, L=List, S=Search on the same CUST1 / CUST2 / CUSTL maps (CUST1L / CUST2L / CUSTLL on a mod-4 screen, 35 list rows per page). Key validation (CUSV) and the paged list / search (CUSL) are internal procedures (VALIDATE_KEY, LIST_CUSTOMERS) instead of LINK targets. Every command codes RESP(RC) and tests it with DFHRESP; %INCLUDE DFHAID and DFHRESP. Shares the customers file with CUST and GUST. Alias custpli in aliases.conf. See Porting CUST to PL/I.

PL/I sources live under pli_dir (runtime/pli) and are edited in ISPF **area 4**; the include members DFHAID.inc and DFHRESP.inc ship under pli_include_dir (runtime/pliinc).

Run any TRANSID by typing it at the blank prompt after CSSN sign-on. Refer to runtime/transactions.conf for the full list and ACL configuration.

> **Bank-domain bindings.** BANK, BALC, BALR, and BRDS carry an explicit :bank 5th field in transactions.conf so their EXEC SQL statements land on the retail-banking pool rather than the default bricks database. BALC / BALR / BRDS do not currently issue SQL, but they share the bank-domain map set and are pinned to the same pool for forward compatibility — adding an EXEC SQL later will Just Work without re-binding. See Chapter 26 — Database binding.

Built-in transactions (no entry in transactions.conf)

The bricks core dispatches a handful of TRANSIDs directly, without consulting transactions.conf. They take precedence over a same-name entry in the table.

Transid	Purpose	Notes
CSSN	Sign-on	Default authentication flow. Configurable via secure_login_transacton in bricks.cnf.
CSSF	Sign-off	CSSF LOGOFF clears the session's identity; bare CSSF is a no-op.
CEMT	Master-operator	INQUIRE / MONITOR / PERFORM trees; CONTROLBLOCKS sub-tree and PERFORM gated on the admin group.
CEDA	Resource definitions	TRANSACTION / PROGRAM / USER screens; admin-only.
CECI	Command-level interpreter	Type one EXEC CICS / EXEC SQL / EXEC WEB command (up to 5 input rows), press PF5, see RESPONSE / RESP2 / LEN / ELAPSED plus any variables the handler set or INTO buffers it filled. The TxCB and cics.Handler are session-scoped — built once on CECI entry, torn down on PF3 / CLEAR / PA1 / disconnect. Each PF5 press is a per-PF5 commit boundary via h.CommitImplicit() on success / h.RollbackImplicit() on failure; SYNCPOINT in bricks does not touch cursors, so the per-task TS read cursor, all open browses, TD handles, WEB sessions, and DOCUMENT tokens survive across PF5 presses (a STARTBR on PF5 #1 drives READNEXT on PF5 #2). Variable frame is also session-scoped. READ UPDATE + REWRITE still cannot span PF5 — the per-PF5 implicit SYNCPOINT releases non-HOLD record locks; see the REWRITE deviation note for the recommended workaround. Verbs that would unwind the CECI task itself (RETURN, XCTL, LINK, ABEND), repaint the screen (SEND MAP, SEND TEXT, RECEIVE MAP, CONVERSE), schedule deferred work (START, RETRIEVE, CANCEL, DELAY), or set per-task trap tables (HANDLE, IGNORE, WHENEVER) are refused with a short red-status message. Every command is wrapped in a 7-second wall-clock cap; a cap-hit *poisons* the session (every subsequent PF5 short-circuits with TIMEOUT, and the session-end defer skips the handler-owned closers to avoid racing the abandoned goroutine — PF3 is the only exit). Gated on the dev group — and the same gate applies to EXEC CICS LINK PROGRAM('CECI') / XCTL PROGRAM('CECI'), so a non-DEV caller

Transid	Purpose	Notes
		sees PGMIDERR (LINK) or a 403 task-error screen (XCTL) without the built-in's own Run ever being reached. **Operator manual:** the CECI section in README.md covers layout, key bindings, verb policy, the response-line state mapping, and the EBCDIC-037 input rules.
ISPF	Source editor	Browse and edit the REXX, COBOL, BMS-map and PL/I source trees (menu areas 1-4; area 4 is PL/I). Gated on the dev group. **Operator manual:** ISPF_editor.md — covers every PF key, every command-line word, every line-prefix command (D / I / C / M / R / U / L /) / (/ X / O / A / B plus the doubled block forms), the file browser, the warn-then-save flow, multi-file editing, and edit locks.

Every built-in TRANSID above can be reached via `EXEC CICS LINK PROGRAM(name)` and `EXEC CICS XCTL PROGRAM(name)` as well as by typing it at the blank prompt. The two paths share one dispatch helper, so the per-built-in auth gate (`dev` for ISPF/CECI, `admin` for CEDA/IDCA) fires uniformly regardless of how the operator reached it. CEMT carries a mixed model — the outer menu is open to any signed-on user (INQUIRE TRANSACTION, MONITOR), and the `CONTROLBLOCKS` / `PERFORM` sub-trees gate on `admin` inside `walk()`. The ACL check at the LINK / XCTL boundary refuses a denied caller before the built-in's `Run` is reached: the LINK returns the access-denied error to `programControl` which surfaces `PGMIDERR` via `EIBRESP`, and the XCTL renders the familiar 403 task-error screen — no internal refusal paint hijacks the caller's session. Built-ins do not have a COMMAREA-style interface; `LINK PROGRAM('CEMT') COMMAREA(VAR)` returns an empty COMMAREA (`VAR` is set to `""` on return) and the caller's `sess.Commarea` is restored unchanged. `CSSN` is the one exception: it refuses LINK/XCTL with a short message because the auth-prompt loop cannot be wedged into a caller's task stack sanely; operators sign on at a blank prompt.

Chapter 29. Worked examples

A. Producer / consumer over a TS queue

`PROD` writes one item per ENTER from an interactive map; `CONS` reads with the implicit per-task cursor. PF4 in `CONS` deletes the queue; PF5 rewinds the cursor by chaining back to itself (`RETURN TRANSID('CONS')`), so the dispatcher's task-end hook clears the cursor.

```
/* PROD: write one item per ENTER */
ADDRESS CICS
DO FOREVER
  EXEC CICS SEND    MAP('PROD1') FROM(SCR.) ERASE END-EXEC
  EXEC CICS RECEIVE MAP('PROD1')                END-EXEC
  IF C2X(EIBAID) = 'F3' THEN EXEC CICS RETURN END-EXEC
  EXEC CICS WRITEQ TS QUEUE(MAP.QNAME) FROM(MAP.PAYLOAD) END-EXEC
END
```

```
/* CONS: cursor-advancing read */
ADDRESS CICS
DO FOREVER
  EXEC CICS SEND    MAP('CONS1') FROM(SCR.) ERASE END-EXEC
  EXEC CICS RECEIVE MAP('CONS1')                END-EXEC
  IF C2X(EIBAID) = 'F3' THEN EXEC CICS RETURN END-EXEC
  EXEC CICS READQ TS QUEUE(QNM) INTO(REC) ITEM(GOTI) END-EXEC
  IF EIBRESP = 26 THEN /* ITEMERR - end of queue */ NOP
END
```

B. SEND TEXT + RECEIVE — no BMS map

`GETC` takes a customer number from the operator's command line (`GETC 100`), reads the `customers` KSDS, and paints the record as free-form text. Because **3270 has no LF**, the body is built as a flat row-major buffer — each logical line padded to the 80-column screen width with `LEFT(s,80)`.

```
/* GETC: lookup-and-display, no map */
ADDRESS CICS
EXEC CICS RECEIVE INTO(BUF) END-EXEC          /* "GETC 100" */
PARSE VAR BUF TID CKEY .
EXEC CICS READ FILE('customers') INTO(REC) RIDFLD(CKEY) END-EXEC
PARSE VAR REC NM '|' AD '|' CY '|' PH
TXT = LEFT('Customer #' || CKEY, 80)          /* row 0 */
TXT = TXT || LEFT('', 80)                     /* row 1 (blank) */
TXT = TXT || LEFT('Name:    ' || NM, 80)      /* row 2 */
TXT = TXT || LEFT('Address: ' || AD, 80)      /* row 3 */
EXEC CICS SEND TEXT FROM(TXT) ERASE END-EXEC
EXEC CICS RETURN END-EXEC
```

The COBOL companion `runtime/cobol/exam.cob` shows the same `RECEIVE INTO` pattern with `UNSTRING ... DELIMITED BY ' '` doing the tokenisation:

```
EXEC CICS RECEIVE INTO(WS-INPUT) LENGTH(WS-LEN) END-EXEC
UNSTRING WS-INPUT DELIMITED BY ' '
   INTO WS-TID WS-A WS-B WS-C
END-UNSTRING.
```

C. Browse with GENERIC prefix

A paginated customer list filtered by a 3-character key prefix:

```
EXEC CICS STARTBR FILE('CUSTOMERS')
                  RIDFLD('NY-')
                  GENERIC KEYLENGTH(3)
END-EXEC
DO FOREVER
  EXEC CICS READNEXT FILE('CUSTOMERS') INTO(REC) RIDFLD(K) END-EXEC
  IF EIBRESP = 20 THEN LEAVE          /* ENDFILE */
  CALL ADD_ROW K, REC
END
EXEC CICS ENDBR FILE('CUSTOMERS') END-EXEC
```

D. Synchronous LINK with COMMAREA

`CUST` LINKs to `CUSV` to validate a customer number:

```
SAV = CKEY                                    /* in: number to validate */
EXEC CICS LINK PROGRAM('CUSV') COMMAREA(SAV) END-EXEC
PARSE VAR SAV STATUS '|' MSG                  /* out: status, message  */
IF STATUS <> 'OK' THEN ...
```

`CUSV` reads `DFHCOMMAREA`, runs its check, and writes the result back into `DFHCOMMAREA` before `RETURN`. The dispatcher hands the final value back to the caller's `SAV`.

E. Sequential import via READQ TD + WRITE FILE

The `ORDR` transaction (`runtime/cobol/ordr.cob`) demonstrates the "text file → VSAM" pattern that the `tmp_dir` sandbox is built for. The sample data ships in `runtime/tmp/orders.sample.txt`:

```
C0000001|WIDGET-A|10|19.95
C0000002|WIDGET-B|2|249.00
...
```

The loop body is just two EXEC CICS calls and an `UNSTRING`:

```
IMPORT-ONE.
    MOVE SPACES TO REC.
    EXEC CICS READQ TD QUEUE('orders.sample.txt') INTO(REC) END-EXEC.
    IF EIBRESP = 12 THEN
        MOVE 'Y' TO DONE-FLAG       *> QZERO -- end of file
    END-IF.
    IF EIBRESP = 0 THEN
        COMPUTE N-READ = N-READ + 1
        MOVE SPACES TO CUST-ID PRODUCT QTY PRICE
        UNSTRING REC DELIMITED BY '|'
            INTO CUST-ID PRODUCT QTY PRICE
        END-UNSTRING
        PERFORM WRITE-ORDER
    END-IF.

WRITE-ORDER.
    MOVE SPACES TO OREC.
    STRING PRODUCT DELIMITED BY SIZE
           '|' DELIMITED BY SIZE
           QTY DELIMITED BY SIZE
           '|' DELIMITED BY SIZE
           PRICE DELIMITED BY SIZE
        INTO OREC
    END-STRING.
    EXEC CICS WRITE FILE('ORDERS') FROM(OREC) RIDFLD(CUST-ID) END-EXEC.
    IF EIBRESP = 0  THEN COMPUTE N-WRITE = N-WRITE + 1 END-IF.
    IF EIBRESP = 14 THEN COMPUTE N-DUP   = N-DUP   + 1 END-IF.
```

Notes:

- The READQ loop tracks EOF with `EIBRESP = 12` (`QZERO`) — the CICS-canonical "queue empty / no more records" signal. The handle is closed automatically at task end.
- `EIBRESP = 14` (`DUPREC`) is treated as a counted no-op, not an error: `WRITE FILE` against an existing key fails atomically without overwriting, so a re-run of `ORDR` on the same file is idempotent on the customer-id set.
- `ORDR` ships `public` in `runtime/transactions.conf` so the sample runs out of the box. In a production deployment that imports real data, restrict it to a privileged group (`ORDR:cobol:ordr.cob:admin`) so a casual operator can't replay the import.
- A REXX equivalent reads the same file via `LINEIN` and writes via `EXEC CICS WRITE`. Because both languages share the `tmp_dir` backend, the sample file works untouched from either side.

F. Menu items gated by `QUERY SECURITY`

The shipped `GUST` transaction (`runtime/cobol/gust.cob`) is a small CRUD UI: A=Add / Q=Query / U=Update / D=Delete / L=List / S=Search. Delete is destructive and only the admin role should be allowed to invoke it. Before 2.7.x the operator had to hard-code `IF EIBUSER = 'ADMIN'` or `IF EIBTRMID = ...`, which bypasses

the ACL gate that already governs sign-on. `QUERY SECURITY` lets the program ask the dispatcher's `SecurityChecker` directly:

```
       WORKING-STORAGE SECTION.
       COPY DFHAID.
       COPY DFHRESP.
       COPY DFHVALUE.
       ...
       01 WS-CACR-UP PIC S9(8) VALUE 0.
       ...
       DISPATCH.
           ...
      *> EXEC CICS QUERY SECURITY: refuse D=Delete to callers who
      *> are not authorised to UPDATE the GUST resource.
           IF VALID-ACTION = 'Y' AND ACTION = 'D' THEN
               EXEC CICS QUERY SECURITY RESOURCE('GUST')
                                        UPDATE(WS-CACR-UP) END-EXEC
               IF WS-CACR-UP NOT = DFHVALUE-UPDATABLE THEN
                   MOVE 'Delete not permitted for this user.' TO MSG
                   MOVE 'N' TO VALID-ACTION
               END-IF
           END-IF.
```

The gate runs once per dispatch (the verb is stateless and cheap) and lands before the `EVALUATE ACTION` switch. Non-admins see the `MSG` line on the next menu paint; admins fall through to `ACT-DELETE` unchanged. The same pattern transports to any program with a destructive action — replace `'GUST'` with the gated transaction's TRANSID and reuse the four-line block.

See the in-tree `runtime/cobolcopy/DFHVALUE.cpy` for the full constant set, and the security-inquiry sub-chapter of Chapter 6 for the bricks-flavoured access mapping that decides the `WS-CACR-UP` value.

G. Porting CUST to PL/I

The REXX `CUST` application is three programs: `cust.rexx` (the menu and the single-record actions), `cusv.rexx` (key validation, reached by `EXEC CICS LINK`) and `cusl.rexx` (list and search, reached by `LINK`). `CUSP` (`runtime/pli/cusp.pli`) ports all three into **one** PL/I program. The operator sees the same `CUST1`, `CUST2` and `CUSTL` screens and messages, and both transactions share the `customers` file, so a record added under one is found under the other.

```
CUSP:pli:cusp.pli:PUBLIC        (runtime/transactions.conf)
custpli:CUSP                    (runtime/aliases.conf)
```

Program shape. Declarations come first (map structures, then the request and list state), then a conversational main line, then the internal procedures:

```
   DO FOREVER;
      CALL SEND_MENU;
      MENU_IN = '';
      EXEC CICS RECEIVE MAP(M_MENU) INTO(MENU_IN) RESP(RC);

      /* PF12 from the menu ends the transaction.                    */
      IF EIBAID = PF12 THEN EXEC CICS RETURN;

      AKT  = UPPERCASE(TRIM(MENU_IN.ACTION));
      CKEY = UPPERCASE(TRIM(MENU_IN.CUSTNO));
      SRCH = TRIM(MENU_IN.SEARCH);
      CALL DISPATCH;
   END;
```

LINK becomes an internal CALL. `CUSV` did nothing but normalise the key in `DFHCOMMAREA`. In PL/I that is a procedure with a by-reference `CHAR(*) VAR` parameter:

```
VALIDATE_KEY: PROC(K);
   DCL K CHAR(*) VAR;
   K = UPPERCASE(TRIM(K));
   PUT SKIP LIST('CUSV: validated customer number =', K);
END VALIDATE_KEY;
```

`CUSL` becomes `LIST_CUSTOMERS(TERM, COUNT)`, which returns the match count through its second parameter instead of the COMMAREA.

RESP on every command. Where `cust.rexx` tested `EIBRESP` against bare numbers (`IF EIBRESP = 13`), `CUSP` codes `RESP(RC)` and compares with `DFHRESP`:

```
QUERY_CUSTOMER: PROC;
   EXEC CICS READ FILE('customers') INTO(REC) RIDFLD(CKEY) RESP(RC);
   IF RC = DFHRESP(NOTFND) THEN
      MENU.MSG = 'Customer ' || CKEY || ' not found.';
   ELSE IF RC <> DFHRESP(NORMAL) THEN
      MENU.MSG = 'READ failed RESP=' || CHAR(RC);
```

Stems become structures and arrays. The REXX list built map field names at run time (`CALL VALUE 'SCR.ROW' || J, LINE`). The PL/I list structure declares `2 ROW(35) CHAR(71)` once, and the map's `ROW1` .. `ROW15` (or `CUSTLL`'s 35 rows) land in its elements (Maps as structures). `cusl.rexx` also read every record into the stems `KEYS.` / `RECS.` before paging; `CUSP` keeps storage fixed by counting matches in one browse and building each page with a fresh `STARTBR` / `READNEXT` / `ENDBR` that skips the earlier pages' matches.

PARSE VAR becomes INDEX and SUBSTR. The record body is `name|address|city|phone`. `PARSE VAR R NM '|' AD '|' CY '|' PH` becomes a small field cutter called four times:

```
CUT_FIELD: PROC(S, F);
   DCL (S, F) CHAR(*) VAR;
   DCL P FIXED BIN(31);
   P = INDEX(S, '|');
   IF P = 0 THEN DO;
      F = S;
      S = '';
   END;
   ELSE DO;
      F = SUBSTR(S, 1, P - 1);
      S = SUBSTR(S, P + 1);
   END;
END CUT_FIELD;
```

Idiom table.

REXX (`CUST`)	PL/I (`CUSP`)
`PARSE VAR R NM '|' AD '|' CY '|' PH`	`P = INDEX(S, '|'); F = SUBSTR(S, 1, P - 1); S = SUBSTR(S, P + 1);`
`POS(FILTER, HAY) > 0`	`INDEX(UPPERCASE(K || ' ' || R), FILTER) > 0`
`UPPER(STRIP(X))`	`UPPERCASE(TRIM(X))`
`LEFT(K, 8) LEFT(NM, 28)` (blank join)	`LEFT(K, 8) || ' ' || LEFT(FNM, 28)`
`SCR. = ''` then `SCR.MSG = ...`	`DCL 1 MENU, 2 MSG CHAR(60), ...;` then `MENU = '';`
`CALL VALUE 'SCR.ROW' || J, LINE`	`LST.ROW(J) = LINE;`

REXX (CUST)	PL/I (CUSP)
KEYS.IDX, RECS.IDX buffers	re-browse per page, fixed storage
SAY 'n =' N (discarded)	PUT SKIP LIST('n =', N); (discarded too; use SEND TEXT or a TS queue to debug)
EXEC CICS LINK PROGRAM('CUSV') COMMAREA(CKEY)	CALL VALIDATE_KEY(CKEY);
EXEC CICS LINK PROGRAM('CUSL') COMMAREA(CMSG)	CALL LIST_CUSTOMERS(SRCH, N);
... NOHANDLE + IF EIBRESP = 36	... RESP(RC); + IF RC = DFHRESP(MAPFAIL)
IF EIBRESP = 13	IF RC = DFHRESP(NOTFND)
IF C2X(EIBAID) = '7C'	%INCLUDE DFHAID; then IF EIBAID = PF12
IF SCRH >= 43 THEN SUFFIX = 'L'	the same, with DCL SUFFIX CHAR(1) VAR
'Customer' CKEY 'added.' (blank join)	'Customer ' || CKEY || ' added.'
NPAGES = (TOTAL + ROWS_PER_PAGE - 1) % ROWS_PER_PAGE	NPAGES = (TOTAL + ROWS - 1) / ROWS; into a FIXED BIN target (the fraction is dropped on assignment)

Map fallback and the CUSM abend. Each SEND tries the sized map first and falls back to the 24x80 map on MAPFAIL. If even that fails, the map catalogue is broken and CUSP abends CUSM rather than loop on a screen it cannot show:

```
   SEND_MENU: PROC;
      EXEC CICS SEND MAP(M_MENU) FROM(MENU) ERASE RESP(RC);
      IF RC = DFHRESP(MAPFAIL) & M_MENU <> 'CUST1' THEN DO;
         M_MENU = 'CUST1';
         EXEC CICS SEND MAP(M_MENU) FROM(MENU) ERASE RESP(RC);
      END;
      IF RC <> DFHRESP(NORMAL) THEN EXEC CICS ABEND ABCODE('CUSM');
   END SEND_MENU;
```

Run it by typing CUSP (or the alias custpli) at the blank prompt. Edit it under ISPF area 4 and check it with go run ./cmd/brickscompile runtime/pli/cusp.pli.

Appendix A. Adapting to terminal size (mod 2 vs mod 4)

A 3270 connection negotiates one of several screen models — typically mod 2 (24 - 80) or mod 4 (43 - 80). Bricks captures the size from the telnet/3270 handshake into `session.TCB.Rows/Cols`, and exposes it to programs via `EXEC CICS ASSIGN`:

```
EXEC CICS ASSIGN SCREENHT(SCRH) SCREENWD(SCRW)
                 ALTSCRNHT(AH)  ALTSCRNWD(AW)  END-EXEC
```

`SEND MAP` always passes the negotiated `DevInfo` through to `go3270.ScreenOpts.AltScreen`, so the underlying datastream uses Erase/Write Alternate (`0x7e`) and the terminal clears its full buffer — but a 24-row map painted on a 43-row screen leaves rows 24-42 blank. To use the extra real estate the program has to dispatch to a sized map variant.

Convention

Author one map per model. The mod-2 map keeps its bare name; bigger models add a single-letter suffix:

Model	Suffix	Example map names
mod 2	(none)	`HELO1`, `CUST1`, `CUST2`, `CUSTL`
mod 3	`M`	`HELO1M`, `CUST1M`, …
mod 4	`L`	`HELO1L`, `CUST1L`, `CUST2L`, `CUSTLL`
mod 5	`W`	`HELO1W`, …

The REXX program builds the suffixed name once and dispatches:

```
EXEC CICS ASSIGN SCREENHT(SCRH) END-EXEC
SUFFIX = ''
IF SCRH >= 43 THEN SUFFIX = 'L'
ELSE IF SCRH >= 32 THEN SUFFIX = 'M'

EXEC CICS SEND MAP('HELO1' || SUFFIX) FROM(SCR.) ERASE END-EXEC
IF EIBRESP = 36 THEN DO            /* MAPFAIL — sized variant missing */
  EXEC CICS SEND MAP('HELO1') FROM(SCR.) ERASE END-EXEC
END
```

The PL/I sample `cusp.pli` uses the same convention with a `VAR` suffix variable and a structure for the map:

```
   EXEC CICS ASSIGN USERID(UID) TERMID(TID) SCREENHT(SCRH) RESP(RC);

   /* A model 4 screen (43 rows) gets the L variants of every map.     */
   IF SCRH >= 43 THEN SUFFIX = 'L';
   M_MENU   = 'CUST1' || SUFFIX;
   M_DETAIL = 'CUST2' || SUFFIX;
```

and falls back with `IF RC = DFHRESP(MAPFAIL) & M_MENU <> 'CUST1'` (see Porting CUST to PL/I). Declare the list structure's `ROW` array with the largest row count of the family (`2 ROW(35)` for `CUSTLL`); the 15-row `CUSTL` ignores the extra elements.

Three properties make this work without any DSL changes:

1. **Same field names across the family.** `helo1.map` and `helo1l.map` both declare `INFOLINE`, `GREETING`, `FOOTER`. The same `SCR.` stem feeds either one. Bonus tails (e.g. `INFO1` / `INFO2` / `ACT1` …) are silently ignored on the smaller map (the renderer only writes values for fields that the map declares).
2. **MAPFAIL fallback.** If the suffixed map isn't on disk, the `SEND` returns `EIBRESP = 36` and the program retries with the bare name — so an operator who deletes `helo1l.map` doesn't break mod-4 connections; they just see the 24-80 layout.
3. **Paging arithmetic adapts at runtime.** `cusl.rexx` reads `SCREENHT` and uses `ROWS_PER_PAGE = 35` on mod 4 vs `15` on mod 2, then picks `CUSTLL` vs `CUSTL` accordingly (`cusp.pli` does the same with `ROWS` and `L_MAP`).

Bricks ships sized variants for every map the demo transactions use:

Mod-2 map (24-80)	Mod-4 sibling (43-80)
`runtime/map/helo1.map`	`runtime/map/helo1l.map` — adds system-information + recent-activity panes
`runtime/map/cust1.map` (menu)	`runtime/map/cust1l.map` — adds recent-activity history
`runtime/map/cust2.map` (detail)	`runtime/map/cust2l.map` — adds an audit-log pane
`runtime/map/custl.map` (15-row list)	`runtime/map/custll.map` (35-row list)

To see the difference: connect with `c3270 -model 2 localhost 2300` vs. `c3270 -model 4 localhost 2300`, sign on, and run `HELO`, `CUST` or `CUSP`. The mod-4 view fills the bottom three-quarters of the screen with extra panels.

Appendix B. Pitfalls and idioms

REXX compound-symbol pitfall

`STEM.tail` with `tail` an *unset* symbol resolves to `STEM.<TAIL>` (a literal tail). With `tail` a *set* symbol it resolves to `STEM.<value-of-tail>` — so reusing a map field name (`OUT.BIRTH = …` when `BIRTH` is also a local variable) silently writes the wrong tail.

The convention used by `runtime/rexx/cust.rexx` is to give locals distinct names from map fields (`AKT` vs `ACTION`, `CKEY` vs `CUSTNO`, `BSTR` / `NDAYS` vs `BIRTH` / `DAYS`).

COBOL data names across groups

A child name reused across groups is allowed; bare references that match more than one declaration must be qualified with `OF` / `IN` (`MOVE x OF DET TO y OF SCR`). Bare names that are unique across the program continue to work without qualification. Sibling duplicates within a single group are still rejected at parse time because no qualifier can disambiguate them.

COBOL conditions follow IBM, not intuition

Three IBM rules that surprise people, all honoured by bricks (see Combined conditions and parentheses):

- `IF A / 2 > 2` with integer operands truncates the quotient: for `A = 5` it compares `2 > 2` and is false. When the fraction matters, give the relation a decimal place outside the divisor: a `V` field as dividend, or `IF A / 2 > 2.0` (true). A decimal divisor does not help — `IF A / 2.0 > 2` is still false, because divisors don't count towards `dmax`.
- `IF A NOT = 1 AND 2` means `A NOT = 1 AND A NOT = 2` — the `NOT` carries to the omitted operator.
- `STRING ... DELIMITED BY SIZE` moves an identifier at its full PIC width, trailing spaces included. Use `DELIMITED BY SPACE` to stop at the first blank.

3270 has no LF...obviously.

`SEND TEXT` lays its body out as a flat row-major buffer: every `cols` characters (cells) start a new row, and only the first `cols-1` of them are painted because column 0 holds the attribute byte. Programs that expect newline semantics must pad each logical line to the column width (`LEFT(s,80)` in REXX, `PIC X(80)` group children in COBOL) and concatenate. `LENGTH(n)` is a byte count, which under bricks SBCS is exactly `n` characters.

Kinda obvious, but just making sure it's clear to folks who are new to 3270.

Reset stems before paginated reads

A REXX `STEM.` accumulator that's reused across pages will leak values from the previous page if not dropped. Use `DROP RECS.` (with the trailing dot) at the top of each pagination loop.

EIBAID comparison

In REXX, compare `C2X(EIBAID)` to a hex string: `IF C2X(EIBAID) = 'F3' THEN ...` (PF3). In COBOL, compare `EIBAID` directly to a hex literal: `IF EIBAID = X'F3' ...`. In PL/I, `%INCLUDE DFHAID;` and compare with the

name (`IF EIBAID = PF03 THEN ...`), or with a hex constant (`IF EIBAID = 'F3'X THEN ...`).

PL/I: declare everything, mind the builtin names

Every PL/I variable must be declared (`UNDECLARED QTYY`), and `DEC` / `BIN` need `FIXED`. A built-in name cannot be a level-1 variable: `DCL DAYS(12) FIXED BIN(15);` fails with `BUILTIN NAME DAYS`, and so do `INDEX`, `LENGTH`, `DATE`, `TIME`, `SUM`, ... Pick another name (`MDAYS`). Structure members may use built-in names, which is why a map structure can have a `SEARCH` field. Once `%INCLUDE DFHAID;` is in, `ENTER`, `CLEAR` and `PF01` .. `PF24` are taken too.

PL/I: assignment truncates, ROUND rounds

Assigning `4.947525` to a `FIXED DEC(9,2)` gives `4.94`. Write `TAX = ROUND(TOTAL * RATE, 2);` wherever money is rounded. A value too large for its target abends `ABSZ` instead of losing high-order digits.

PL/I: `/` is always decimal

`7 / 2` is `3.5`, also for `FIXED BIN` operands. Assign to an integer target, or use `TRUNC` or `DIVIDE(x, y, p, q)`, when you want the integer quotient. `CHAR(N / 2)` keeps the full scale (`3.500000000000000000`).

PL/I: CHAR() of a number is minimal

`CHAR(42)` is `'42'` and `CHAR(-3)` is `'-3'`, so `'Total: ' || CHAR(N)` needs no `TRIM`. For aligned columns use `EDIT(x, 'ZZZ9V.99')` or a `PIC` variable.

PL/I: CICS conditions do not stop the program

A `NOTFND` or `MAPFAIL` never abends a PL/I task, even without `RESP`. A program that does not test the response simply carries on with stale data. Code `RESP(RC)` on every command and test `RC` against `DFHRESP(...)`.

PL/I: handlers end the task

A `HANDLE CONDITION`, `HANDLE AID` or `WHENEVER` target procedure does not return to the failing statement: when it returns, the task ends and commits. Use handlers for exit paths only, and `SYNCPOINT ROLLBACK` in a handler whose work must not be kept.

PL/I: receive numbers into PIC or CHAR members

`RECEIVE MAP` into a `FIXED DEC` / `FIXED BIN` member converts the typed text, and an operator who types `12A` abends the task `ABCV`. Receive into a numeric `PIC` (then `VALID`) or a `CHAR` member (then `VERIFY`), and convert after the check.

COBOL and PL/I: EIBTRMID holds 4 characters

bricks terminal ids are 5 characters (`T0001`), but `EIBTRMID` is a 4-character field in the COBOL and PL/I EIB, as in IBM's `DFHEIBLK`, so it reads `T000` and two terminals can look alike. For the full id use `EXEC CICS ASSIGN TERMID(TID)` with an 8-character target (`DCL TID CHAR(8);` in PL/I, `PIC X(8)` in COBOL), as `cusp.pli` does. REXX variables have no fixed length, so REXX `EIBTRMID` holds the full id.

PL/I: unlabelled LEAVE skips DO groups

`IF X THEN DO; ...; LEAVE; END;` inside a loop leaves the **loop**: an unlabelled `LEAVE` / `ITERATE` targets the innermost iterative `DO`, never a plain `DO;` group.

Appendix C. Quick command reference card

A one-page cheat-sheet of every command family. Use the chapter reference for full syntax, options, and condition codes.

EXEC CICS — terminal I/O (Chapter 4)

Command	Purpose
`SEND MAP(name) [FROM(stem.)] [ERASE] [CURSOR(p)]`	Paint a BMS map; wait for an AID.
`RECEIVE MAP(name) [INTO(stem.)]`	Pull the operator's response.
`CONVERSE MAP(name) FROM(s) INTO(s) [ERASE]`	Fused `SEND MAP+RECEIVE MAP`.
`SEND TEXT FROM(buf) [LENGTH(n)] [ERASE]`	Free-form row-major output.
`RECEIVE INTO(buf) [LENGTH(v)]`	Read the operator's command-line tail.

EXEC CICS — program control (Chapter 5)

Command	Purpose
`RETURN [TRANSID(id)] [COMMAREA(d)]`	End task; optional chain.
`XCTL PROGRAM(name) [COMMAREA(d)]`	Transfer control.
`LINK PROGRAM(name) [COMMAREA(v)]`	Synchronous sub-program call.
`ABEND [ABCODE(c)]`	Abnormal task termination.
`START TRANSID(id) [INTERVAL/TIME] [FROM(d)] [TERMID(t)]`	Schedule a deferred fire.
`RETRIEVE INTO(v) [LENGTH(v)]`	Pull the `START` payload.

EXEC CICS — system services (Chapter 6)

Command	Purpose
`ASSIGN <FIELD>(v) ...`	Read EIB / environment fields.
`ASKTIME [ABSTIME(v)]`	Refresh `EIBDATE` / `EIBTIME` (+ optional ABSTIME).
`FORMATTIME ABSTIME(s) [DATE / TIME / YYYY... / DAYOFWEEK ...]`	Decode an ABSTIME.
`QUERY SECURITY RESOURCE(name) [RESCLASS(c)] [READ/UPDATE/CONTROL/ALTER(v)]`	Four-axis access inquiry.
`VERIFY PASSWORD USERID(u) PASSWORD(p)`	Re-authenticate credentials.
`INQUIRE SYSTEM [GMMTEXT(v)]`	SIT-level inquiry (v1 honours GMMTEXT only).

EXEC CICS — KSDS files (Chapter 7-8)

Command	Purpose
`READ FILE(f) INTO(v) RIDFLD(k) [UPDATE]`	Exact-key lookup.
`WRITE FILE(f) FROM(d) RIDFLD(k)`	Insert record.

Command	Purpose
`REWRITE FILE(f) FROM(d)`	Replace the `READ … UPDATE` record.
`DELETE FILE(f) [RIDFLD(k)]`	Drop record.
`STARTBR FILE(f) [RIDFLD(k)] [GTEQ/EQUAL] [GENERIC KEYLENGTH(n)]`	Open browse.
`READNEXT FILE(f) INTO(v) [RIDFLD(v)]`	Forward one record.
`READPREV FILE(f) INTO(v) [RIDFLD(v)]`	Backward one record.
`RESETBR FILE(f) RIDFLD(k)`	Re-position within an open browse.
`ENDBR FILE(f)`	Close browse.

EXEC CICS — temporary storage / transient data (Chapter 9)

Command	Purpose
`READQ TS QUEUE(q) INTO(v) [ITEM(n)	NEXT]`
`WRITEQ TS QUEUE(q) FROM(d) [ITEM(n) REWRITE]`	Append or rewrite TS item.
`DELETEQ TS QUEUE(q)`	Drop a TS queue.
`READQ TD QUEUE(q) INTO(v)`	Read next line from `tmp_dir/q`.
`WRITEQ TD QUEUE(q) FROM(d)`	Append line to `tmp_dir/q`.
`DELETEQ TD QUEUE(q)`	Delete a TD file.

EXEC CICS — recovery + condition (Chapter 10)

Command	Purpose
`SYNCPOINT`	Commit pending unit of work.
`SYNCPOINT ROLLBACK`	Undo pending unit of work.
`HANDLE CONDITION cond(label) ...`	Arm condition trap.
`IGNORE CONDITION cond ...`	Suppress condition trap.
`HANDLE AID key(label) ...`	Arm AID-key trap.
`HANDLE ABEND LABEL(l)	PROGRAM(p)`

EXEC SQL (Chapter 26)

Command	Purpose
`EXEC SQL SELECT cols INTO :v, :v FROM ... END-EXEC`	Single-row read; `+100` = no-data, `−811` = >1 row.
`EXEC SQL INSERT INTO t (...) VALUES (...) END-EXEC`	Insert.
`EXEC SQL UPDATE t SET ... WHERE ... END-EXEC`	Update; 0 rows is still NORMAL.
`EXEC SQL DELETE FROM t WHERE ... END-EXEC`	Delete.
`EXEC SQL DECLARE c CURSOR FOR <select> END-EXEC`	Declare cursor.
`EXEC SQL OPEN c END-EXEC`	Open cursor.
`EXEC SQL FETCH c INTO :v, :v END-EXEC`	Next row; `+100` at end.
`EXEC SQL CLOSE c END-EXEC`	Close cursor.
`EXEC SQL COMMIT WORK END-EXEC` / `ROLLBACK WORK`	Transaction control.

Command	Purpose
`EXEC SQL CONNECT TO 'db' END-EXEC`	Per-task database switch.
`EXEC SQL WHENEVER {SQLERROR`	NOT FOUND

EXEC CICS WEB — server side (Phase 1)

Command	Purpose
`WEB EXTRACT METHOD(v) PATH(v) HOST(v) PORT(v) SCHEME(v) QUERYSTRING(v) ...`	Inbound request meta.
`WEB EXTRACT CERTIFICATE COMMONNAME(v) ORGANISATION(v) COUNTRY(v) SERIALNUM(v) ISSUER(v)`	mTLS peer-cert fields.
`WEB EXTRACT TCPIPSERVICE(v) PORTNUMBER(v) IPADDRESS(v) CLIENT(v) AUTHENTICATE(v)`	Listener inspection.
`WEB READ HTTPHEADER(name) VALUE(v) [LENGTH(v)]`	Inbound header by name.
`WEB STARTBROWSE / READNEXT / ENDBROWSE HTTPHEADER`	Walk inbound headers.
`WEB READ QUERYPARM(name) VALUE(v)`	Query parameter / path capture, percent-decoded.
`WEB STARTBROWSE / READNEXT / ENDBROWSE QUERYPARM`	Walk parameters.
`WEB READ FORMFIELD(name) VALUE(v)`	Form-encoded field.
`WEB STARTBROWSE / READNEXT / ENDBROWSE FORMFIELD`	Walk form fields.
`WEB RECEIVE INTO(v) [MAXLENGTH(n)] [LENGTH(v)] [MEDIATYPE(v)] [TYPE(v)]`	Read raw request body.
`WEB WRITE HTTPHEADER(name) VALUE(v)`	Set outbound response header.
`WEB SEND FROM(buf) [MEDIATYPE(s)] [STATUSCODE(n)]`	Emit response.
`WEB PARSE URL URL(s) SCHEMENAME(v) HOST(v) PORT(v) PATH(v) QUERYSTRING(v)`	URL splitter.
`WEB CONVERTTIME DATESTRING(s) ABSTIME(v)`	RFC 1123 / 850 / asctime → ABSTIME.
`WEB RETRIEVE DOCTOKEN(v)`	Inbound body as DOCUMENT.

EXEC CICS WEB — client side (Phase 2)

Command	Purpose
`WEB OPEN HOST(s) [PORT(n)] [SCHEME(s)] SESSTOKEN(v)`	Open client session.
`WEB OPEN URIMAP(name) SESSTOKEN(v)`	Open via URIMAP.
`WEB CONVERSE SESSTOKEN(t) METHOD(m) PATH(p) [FROM(b)] INTO(v) [STATUSCODE(v)]`	One-shot send + receive.
`CONVERSE WEB ...`	Alias for `WEB CONVERSE`.
`WEB SEND SESSTOKEN(t) METHOD(m) PATH(p) [FROM(b)] [QUERYSTRING(s)] [MEDIATYPE(s)]`	Stage request.
`WEB SEND URIMAP(name) METHOD(m) PATH(p) ... INTO(v)`	One-shot via URIMAP.
`WEB RECEIVE SESSTOKEN(t) INTO(v) [STATUSCODE(v)] [MEDIATYPE(v)] [MAXLENGTH(n)]`	Fire staged request.
`WEB CLOSE SESSTOKEN(t)`	Release session.

Command	Purpose
`WEB WRITE HTTPHEADER(name) VALUE(v) SESSTOKEN(t)`	Set outbound *request* header.
`WEB READ HTTPHEADER(name) VALUE(v) [LENGTH(v)] SESSTOKEN(t)`	Read response header.
`WEB STARTBROWSE / READNEXT / ENDBROWSE HTTPHEADER SESSTOKEN(t)`	Walk response headers.
`WEB EXTRACT SESSTOKEN(t) [SCHEME(v)] [HOST(v)] [PORT(v)] [PATH(v)]`	Inspect session endpoint.
`WEB EXTRACT URIMAP(name) [SCHEME(v)] [HOST(v)] [PORT(v)] [PATH(v)]`	Inspect URIMAP entry.

EXEC CICS DOCUMENT

Command	Purpose
`DOCUMENT CREATE DOCTOKEN(v) [SYMBOLLIST(s) [LISTLENGTH(n)]] [DELIMITER(s)]`	New empty document.
`DOCUMENT INSERT DOCTOKEN(t) FROM(b) [LENGTH(n)] [AT(pos)]`	Append / splice bytes.
`DOCUMENT INSERT DOCTOKEN(t) SYMBOL(name)`	Substitute from symbol list.
`DOCUMENT INSERT DOCTOKEN(t) DOCUMENT(other-tok) [AT(pos)]`	Splice another document.
`DOCUMENT SET DOCTOKEN(t) SYMBOLLIST(s) [DELIMITER(d)]`	Re-bind symbol list.
`DOCUMENT RETRIEVE DOCTOKEN(t) INTO(b) LENGTH(v)`	Read assembled body.
`DOCUMENT DELETE DOCTOKEN(t)`	Release document.
`WEB SEND DOCTOKEN(t) [STATUSCODE(n)] [MEDIATYPE(s)]`	Emit document as response.

Standard copybooks

Copybook	Brings in
`DFHAID`	AID byte constants (`ENTER`, `PF03`, `PA1`, …) and their `DFH...` aliases.
`DFHRESP`	`EIBRESP` condition constants (`RESP-NORMAL` / `DFHRESP-NORMAL`, etc.).
`SQLCA`	SQLCODE + SQLSTATE constants (`SQL-OK`, `SQLSTATE-DUP-KEY`, …).
`DFHWBSC`	HTTP status codes (`DFHRESP-WB-OK`, …).
`DFHWBUH`	Common header-name literals (`WB-CONTENT-TYPE`, …).
`DFHWBMT`	MIME-type literals (`WB-MT-JSON`, …).
`DFHWBMETH`	HTTP method literals (`WB-GET`, `WB-POST`, …).
`DFHWBSI`	Session-token + body buffer templates.
`DFHWBUM`	URIMAP record fields.
`DFHWBHB`	Header-browse helpers (`DFH-WB-HDR-NAME`, …).
`DFHWBCC`	TLS / mTLS cert-extract pack.
`DFHDCDOC`	DOCUMENT token + delimiter constants.

PL/I quick card (Part 9)

Construct	Form
Program	`NAME: PROC OPTIONS(MAIN); ... END NAME;`

Construct	Form
Internal procedure	`P: PROC(A, B) [RETURNS(attrs)] [RECURSIVE]; ... END P;`
Declarations	`DCL X FIXED DEC(p,q) [INIT(v)] [STATIC];` / `FIXED BIN(15|31|63)` / `CHAR(n) [VAR]` / `BIT(n)` / `PIC '...'`
Structure	`DCL 1 S, 2 A CHAR(8), 2 R(15) CHAR(71);` / `DCL 1 T LIKE S;`
Named constant	`DCL MAX FIXED BIN(31) VALUE(50);`
External entry	`DCL F ENTRY(attrs, ...) RETURNS(attrs) EXTERNAL;`
Assignment	`A, B = 0;` / `S = '';` / `T = S, BY NAME;` / `SUBSTR(L, 5, 3) = 'ABC';`
Conditional	`IF c THEN s; ELSE s;` / `SELECT (x); WHEN (v1, v2) s; OTHERWISE s; END;`
Loops	`DO I = 1 TO N [BY k] [WHILE(c)];` / `DO WHILE(c);` / `DO UNTIL(c);` / `DO FOREVER;`
Loop control	`LEAVE [label];` / `ITERATE [label];`
Calls	`CALL P(args);` / `RETURN;` / `RETURN (v);`
Debug line (discarded)	`PUT SKIP LIST(a, b, ...);`
Abend	`CALL ABEND('CU01');`
CICS	`EXEC CICS READ FILE('f') INTO(REC) RIDFLD(KEY) RESP(RC);`
SQL	`EXEC SQL SELECT c INTO :V FROM t WHERE k = :K;`
Include	`%INCLUDE DFHAID;`

Abend	Cause
`ABCV`	bad character-to-number / bit conversion, bad date
`ABSZ`	value does not fit the target
`ABF0`	intermediate over 31 integer digits, 64-bit overflow
`ABZD`	division by zero
`ABSR`	subscript or `SUBSTR` out of range
`ABSL`	`SELECT` with no match and no `OTHERWISE`
`ABEN`	external entry mismatch, recursion, call depth
`ABHB`	handler loop
`AICA`	runaway task
`ABST`	task storage over 8 MiB
`APLI`	internal interpreter error (detail in the log)

Standard PL/I include members

Member	Brings in
`DFHAID`	AID byte constants (`ENTER`, `PF03`, `PA1`, ...) and their `DFH...` aliases, `CHAR(1) VALUE`s.
`DFHRESP`	`EIBRESP` codes (`RESP_NOTFND` / `DFHRESP_NOTFND`, ...), `FIXED BIN(31) VALUE`s.

`DFHRESP(cond)` and `DFHVALUE(cvda)` are compile-time built-ins and need no include. The SQLCA and the EIB are predeclared.

EIB fields (Chapter 11)

Field	Set by	Meaning
EIBAID	SEND MAP / RECEIVE MAP	AID byte.
EIBCPOSN	SEND MAP / RECEIVE MAP	1-based cursor position.
EIBCALEN	dispatcher entry	DFHCOMMAREA length.
EIBTRMID	dispatcher	Terminal ID (4 characters in COBOL / PL/I: T000; full id via ASSIGN TERMID).
EIBRESP / EIBRESP2	every EXEC CICS	Response code + secondary.
EIBDATE / EIBTIME	ASKTIME	0CYYDDD / HHMMSS.
EIBABCODE	abend trap	4-char abend code.
RC	every EXEC CICS	Mirror of EIBRESP (REXX and COBOL; a PL/I program declares its own RESP variable).

SQLCA fields (Chapter 26)

Field	Type	Meaning
SQLCODE	PIC S9(9)	0 = OK, +100 = no-data, -N = error.
SQLSTATE	PIC X(5)	5-character ISO state code.
SQLERRMC	PIC X(70)	Human-readable PG error text.
SQLERRD3	PIC S9(9)	Rows-affected count after INSERT / UPDATE / DELETE.
SQLWARN0	PIC X(1)	'W' if any warning fired (bricks sets it together with SQLWARN8).
SQLWARN8	PIC X(1)	'W' when a result character had no bricks SBCS byte and was replaced by ? (SQLCODE stays 0).

End of publication.